POLYFACE

V11.0

Jude Katsch @ ANON
373 Redwood Road
Felton, CA 95018
jjk@netcom.com
(831)335-3284

11/04/1999

Contents

1 EPITOMIZED DOCUMENTATION **6**

1.1 Polyface (POLYFACE) . . . 7

1.2 Command/Data Protocol (PROTOCOL) . . . 8

1.3 Pinout (PINOUT) . . . 9

1.4 Memory Map (MEMMAP) . . . 10

1.5 Memory Timing (MEMORY) . . . 11

1.6 Read Data Timing (RDDATA) . . . 12

1.7 Read Null Timing (RDNULL) . . . 13

1.8 Write Data Timing (WRDATA) . . . 14

1.9 Write Command Timing (WRCMD) . . . 15

1.10 Misc (internal) Timing (MISC) . . . 16

1.11 Configure Info (CONFIG) . . . 17

1.12 Synchronous Serial Timing (SSERTIM) . . . 18

1.13 Asynchronous Serial Timing (ASERTIM) . . . 19

2 SPECIFICATIONS **20**

2.1 ROHM . . . 20

2.2 ORBIT . . . 20

2.2.1 ABSOLUTE RATINGS . . . 20

2.2.2 RECOMMENDED OPERATING CONDITIONS . . . 20

2.2.3 DC CHARACTERISTICS . . . 20

2.3 ALTERA . . . 21

3 COMMAND FORMAT **22**

4 CONFIGURATION **23**
4.1 CONFIGURE COMMANDS . . . 23
4.1.1 RESET . . . 23
4.1.2 TAG . . . 24
4.1.3 FOCUS . . . 24
4.1.4 BLUR . . . 24
4.1.5 UNBRAND . . . 25
4.1.6 BRAND . . . 25
4.1.7 PROBE . . . 25
4.1.8 MAGIC . . . 26
4.1.9 ALIVE . . . 26
4.1.10 REQUEST . . . 26
4.1.11 ERROR . . . 27
4.2 CONFIGURE STATES . . . 27
4.2.1 TAGGED (PROBE[7]) . . . 27
4.2.2 FOCUSED . . . 27
4.2.3 BRANDED (PROBE[6]) . . . 28
4.3 CONFIGURE REGISTERS (DEFAULT CONFIG) . . . 28
4.3.1 STATE (0x99) (read/write) . . . 28
4.3.2 ERROR (0x98) (read/write clear) . . . 28

5 CRC **30**

6 POLYFACE REGISTERS **31**
6.1 MEMORY (0x1?) . . . 31
6.1.1 DATA (0010_xx00b) (read/write) . . . 32
6.1.2 DATA++ (0010_xx01b) (read/write) . . . 32
6.1.3 PROG (0010_xx10b) (write only) . . . 32
6.1.4 PROG_ADD (0010_xx11b) (write only) . . . 32
6.1.5 STROBES (ALE, BLE, CLE, /ADSTB0, /ADSTB1, and /ADSTB2) . . . 32
6.2 ADDRESS (0x2?) . . . 34
6.2.1 A0 - A[7:0] (0x20) (write only) . . . 35
6.2.2 A1 - A[15:8] (0x21) (write only) . . . 35
6.2.3 A2 - STICKY_CS,A[22:16] (0x22) (write only) . . . 35
6.2.4 STROBE (0x23) (write only) . . . 36
6.2.5 PINOUT (0x24) (write only) . . . 36
6.2.6 CONFIG - CFG[7:0] (0x25) (read only) . . . 38

6.2.7 INPUTA - IA[7:0] (0x26) (read only) . . . 38
6.2.8 REQUEST (0x27) (read/write) . . . 38
6.3 INPUTB - IB[2:0] (0x28) (read only) . . . 39
6.4 JOYSTICK (0x3?) . . . 39
6.4.1 S0 - SWITCH[8:1] (0x30) (read only) . . . 39
6.4.2 S1 - SWITCH[16:9] (0x31) (read only) . . . 39
6.4.3 QUADX (0x32) (read only) . . . 39
6.4.4 QUADY (0x33) (read only) . . . 40
6.4.5 CHANNEL (0x34) (write only) . . . 40
6.4.6 ANALOG (0x35) (read only) . . . 40
6.5 SERIAL (0x4?) . . . 40
6.5.1 BAUD (0x40) (write only) . . . 41
6.5.2 FLAGS0 (0x41) (write only) . . . 41
6.5.3 FLAGS1 (0x42) (write only) . . . 41
6.5.4 SDATA (0x43) (read/write) . . . 43
6.5.5 SSTATUS (0x44) (read/write-clear) . . . 43
6.5.6 RSTATUS (0x45) (read/write-clear) . . . 43

7 ATOD CONVERTER 44

8 REFERENCE SCHEMATICS 45
8.1 Overview (MPI) . . . 46
8.2 Chassis (CHASSIS) . . . 47
8.3 Splitter (SPLITTER) . . . 48
8.4 Joystick (JOYSTICK) . . . 49
8.5 Parallel Memory (PMEMORY) . . . 50
8.6 Serial Memory (SMEMORY) . . . 51
8.7 Latched Memory (LMEMORY) . . . 52
8.8 Rumble (RUMBLE) . . . 53
8.9 Simple Rumble (SRUMBLE) . . . 54
8.10 Micro Controller (UCONT) . . . 55
8.11 Thru (THRU) . . . 56
8.12 Modem (MODEM) . . . 57

9 POLYFACE VERILOG 58

10 PPTEST **59**
10.1 OVERVIEW . . . 59
10.2 VERILOG . . . 59
10.2.1 Peripheral Port TEST (PPTEST) . . . 59
10.2.2 Serial Data Bus (SDB) . . . 66
10.2.3 Serial Data Bus Data IN (SDB_DIN) . . . 72
10.2.4 FIFO 4 x 32bits (SCFIFO_4X32_SH) . . . 73
10.2.5 FIFO 32 x 8bits (SCFIFO_32X8_NS) . . . 75
10.3 SCHEMATIC . . . 77
10.3.1 Peripheral Port TEST (PPTEST) . . . 78

11 TEST BOARD **79**
11.1 MAIN TEST BOARD SCHEMATICS V1.0 . . . 79
11.2 MAIN TEST BOARD HACKS V1.0 . . . 90
11.3 MAIN TEST BOARD PCB V1.0 . . . 90
11.4 MAIN TEST BOARD SCHEMATICS V2.0 . . . 90
11.5 FLASH ADAPTER SCHEMATICS V1.0 . . . 101
11.5.1 FLASH ADAPTER (FLASHA) . . . 102
11.6 FLASH ADAPTER PCB V1.0 . . . 103
11.7 FLASH ADAPTER SCHEMATICS V2.0 . . . 103
11.7.1 FLASH ADAPTER (FLASHA) . . . 104
11.8 SERIAL ADAPTER . . . 105
11.8.1 SERIAL DRIVERS (SERIAL) . . . 106
11.8.2 SERIAL MAXIM (MAXIM) . . . 107

12 SOFTWARE TEST ENVIRONMENT **108**
12.1 Peripheral Port main include (PPORT_H) . . . 109
12.2 main() loop to interface with pptrace (TEST_C) . . . 113
12.3 Configure (map/add/remove) (CONFIG_C) . . . 117
12.4 DEFCFG includes (DEFAULT_CONFIG_H) . . . 122
12.5 DEFCFG routines (DEFAULT_CONFIG_C) . . . 122
12.6 Peripheral Port includes (PPTEST_H) . . . 125
12.7 Peripheral Port Command constructor (PPORT_C) . . . 125
12.8 FIFO include (FIFO_H) . . . 134
12.9 FIFO interface to/from pptrace (FIFO_C) . . . 134
12.10 Interface between pport and epp (CTRL_C) . . . 137
12.11 Socket EPP (enhanced parallel port) include (SEPP_H) . . . 139

12.12Socket EPP (enhanced parallel port) interface (for verilog environment) (SEPP_C) 140
12.13EPP (enhanced parallel port) (for linux environment) (EPP_C) 143
12.14CRC Look Up Table (pre-constructed) (CRC_LUT_H) 145
12.15CRC routine (CRC_C) 149
12.16CRC Look Up Table generation routine (CRC_LUT_C) 149
12.17Polyface includes (POLYFACE_H) 150
12.18Polyface Driver (POLYFACE_DRIVER_C) 153
12.19Polyface Test includes (POLYFACE_TEST_H) 159
12.20Polyface Test routines (POLYFACE_TEST_C) 160
12.21Polyface Joystick Driver (POLYFACE_JOYSTICK_C) 168
12.22Polyface Serial Driver (POLYFACE_SERIAL_C) 172
12.23Polyface Parallel Memory Driver (POLYFACE_MEMORY_PARALLEL_C) 176
12.24Generic Memory Driver include (MEMORY_H) 180
12.25Generic Memory Driver (MEMORY_C) 183
12.26Generic FLASH CFI interface (FLASH_CFI_C) 190
12.27AMD FLASH interface (FLASH_AMD_SCS_C) 192
12.28PPTRACE GUI (PPTRACE_TCL) 196
12.29DIRECT GUI (enter commands directly to polyface) (DIRECT_TCL) 204
12.30SERIAL GUI (SERIAL_TCL) 212

13 CONFIGURATION 218

14 Nintendo 64 info 219

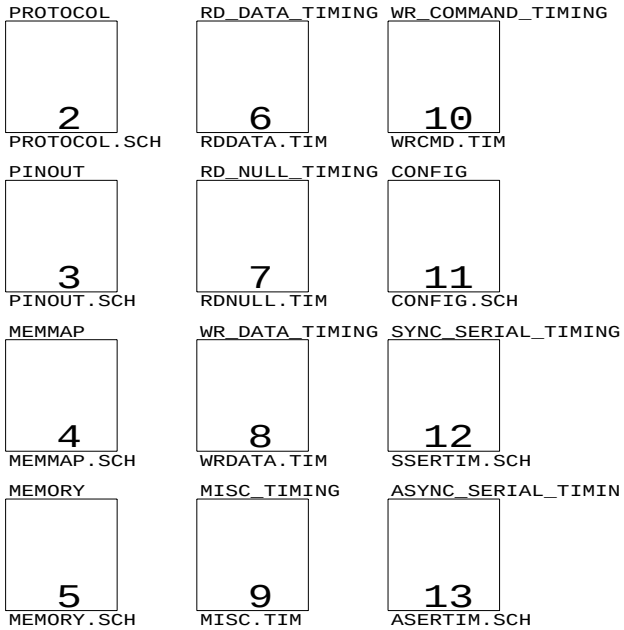
Chapter 1

EPITOMIZED DOCUMENTATION

The following pages contain my reference documentation. These are the gold standard from which the English documents were created.

Description	Section	Page
Polyface	1.1	7
Command/Data Protocol	1.2	8
Pinout	1.3	9
Memory Map	1.4	10
Memory Timing	1.5	11
Read Data Timing	1.6	12
Read Null Timing	1.7	13
Write Data Timing	1.8	14
Write Command Timing	1.9	15
Misc (internal) Timing	1.10	16
Configure Info	1.11	17
Synchronous Serial Timing	1.12	18
Asynchronous Serial Timing	1.13	19

1.1 Polyface (POLYFACE)



Copyright (C) 1998,1999,

ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
POLYFACE		
Size A	Document Number POLYFACE.SCH	REV 1
Date: April 14, 1999	Sheet 1 of 13	

1.2 Command/Data Protocol (PROTOCOL)

DATA FORMAT

(BIG END IN)
DATA IS SENT MSB TO LSB

eparity is the xor of all the other bits (even parity)

		DATA READ								
		POLL	PROBE	MAGIC	data		DIRECT	INDIRECT		
0	LSB	uncfged	eparity	1	byte3_0	eparity	eparity	eparity	0	31
1		device1	id0	0	byte3_1	C0	D0	C0	1	30
2		device2	id1	1	byte3_2	C1	D1	C1	2	29
3		device3	id2	0	byte3_3	C2	D2	C2	3	28
4		device4	id3	0	byte3_4	C3	D3	C3	4	27
5		device5	id4	0	byte3_5	C4	D4	C4	5	26
6		device6	branded	1	byte3_6	C5	D5	C5	6	25
7		device7	tagged	0	byte3_7	C6	D6	C6	7	24
8		device8	mfg0	0	byte2_0	C7	D7	CRC	8	23
9		device9	mfg1	0	byte2_1	B0	S0	S0	9	22
10		device10	mfg2	1	byte2_2	B1	S1	S1	10	21
11		device11	mfg3	0	byte2_3	B2	S2	S2	11	20
12		device12	mfg4	0	byte2_4	B3	S3	S3	12	19
13		device13	mfg5	0	byte2_5	B4	S4	S4	13	18
14		device14	mfg6	1	byte2_6	B5	S5	S5	14	17
15		device15	mfg7	0	byte2_7	B6	S6	S6	15	16
16		device16	type0	1	byte1_0	B7	IA	IA	16	15
17		device17	type1	0	byte1_1	A0	A0	A0	17	14
18		device18	type2	1	byte1_2	A1	A1	A1	18	13
19		device19	type3	0	byte1_3	A2	A2	A2	19	12
20		device20	type4	1	byte1_4	A3	A3	A3	20	11
21		device21	type5	0	byte1_5	A4	A4	A4	21	10
22		device22	type6	1	byte1_6	A5	A5	A5	22	9
23		device23	type7	0	byte1_7	A6	A6	A6	23	8
24		device24	version0	0	byte0_0	A7	1=CMD/0=DATA	1=CMD/0=DATA	24	7
25		device25	version1	1	byte0_1	TYPE0	0=WRITE	1=READ/0=WRITE	25	6
26		device26	version2	0	byte0_2	TYPE1	1=DIRECT	0=INDIR	26	5
27		device27	version3	1	byte0_3	ID0	ID0	ID0	27	4
28		device28	version4	0	byte0_4	ID1	ID1	ID1	28	3
29		device29	version5	0	byte0_5	ID2	ID2	ID2	29	2
30		device30	version6	1	byte0_6	ID3	ID3	ID3	30	1
31	MSB	device31	defcfg	0	byte0_7	ID4	ID4	ID4	31	0
		0	0	0	0	1	1	1		1
		1	1	1	1	1	1	1		0

defcfg: default config
altera version= 0x00
rohm version= 0x01

=0	>0	size	RD	WR	SCMD	CMD	C*
BC	BC	4	0x80		POLL	ALIVE	0x40
BC	BC	4	0x84		POLL	REQUEST	
BC	BC	4	0x88		POLL	ERROR	
BC	BC	0		0xb0	FOCUS	FOCUS	0x01=FOCUS 0x02=BLUR
BC	DE	0		0xb1	EXEC	RESET	
BC	DE	0		0xb2	EXEC	TAG	
BC	DE	0		0xb3	EXEC	UNBRAND	
FO	DE	4	0x94		FREAD	PROBE	
FO	DE	4	0x90		FREAD	MAGIC	
FO	DE	0		0xb4	EXEC	BRAND	ID
BC	BC	0			NOP		
XX	DE	0	0x9A		REG	CRC	0x80
XX	DE	1	0x98	0x98	REG	ERROR	0xnn
XX	DE	1	0x99	0x99	REG	STATE	

A = BYTE ADDRESS
D = DATA
S = SIZE in BYTES
IA : 1=increment address

C* set unused bits to zero
FO=SELECT FOCUSED DEVICE
DE=SELECT DEVICE #ID
BC=BROADCAST TO ALL DEVICES
XX=IGNORE!

byte = 8 bits
word = 16 bits
long = 32 bits
WR = to peripheral
RD = to console

data order

FEDCBA9876543210FEDCBA9876543210
31 0

0	1	2	3	1
4	5	6	7	2
8	9	10	11	3

.
.

ERROR

0	COLLISION0
1	COLLISION1
2	COLLISION2
3	COLLISION3
4	WRITE_OVERFLOW
5	READ_UNDERFLOW
6	INVALID_COMMAND
7	DEVICE_ERROR

STATE

0	VACANT0
1	VACANT1
2	VACANT2
3	VACANT3
4	SELECT0
5	SELECT1
6	ROOT
7	ENABLE

Copyright (C) 1998,1999,

ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title
PERIPHERAL PORT PROTOCOL

Size	Document Number	REV
A	PROTOCOL.SCH	1

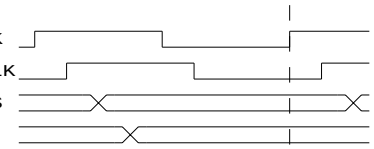
Date: August 11, 1999 Sheet 2 of 13

1.3 Pinout (PINOUT)

	PINS	0		1		2		3			IN	OUT	PULL	PIN	PIN	POWER UP
1	A0	SW01	I ↗	A0	O						C	TS	DN	1		I
2	A1	SW02	I ↗	A1	O						C	TS	DN	2		I
3	A2	SW03	I ↗	A2	O						C	TS	DN	3		I
4	A3	SW04	I ↗	A3	O						C	TS	DN	4		I
5	A4	SW05	I ↗	A4	O						C	TS	DN	24		I
6	A5	SW06	I ↗	A5	O						C	TS	DN	25		I
7	A6	SW07	I ↗	A6	O					A	C	TS	DN	32		I
8	A7	SW08	I ↗	A7	O						C	TS	DN	33		I
9	A8	SW09	I ↗	A8	O						C	TS	DN	34		I
10	A9	SW10	I ↗	A9	O						C	TS	DN	35		I
11	A10	SW11	I ↗	A10	O						C	TS	DN	36		I
12	A11	SW12	I ↗	A11	O						C	TS	DN	37		I
13	A12	SW13	I ↗	A12	O						C	TS	DN	40		I
14	A13	SW14	I ↗	A13	O						C	TS	DN	41		I
15	A14	SW15	I ↗	A14	O					B	C	TS	DN	42		I
16	A15	SW16	I ↗	A15	O						C	TS	DN	43		I
17	D0	ATOD_X1A	T	D0	B						C	TS	-	9		I CFG0
18	D1	ATOD_X1B	T	D1	B						C	TS	-	10		I CFG1
19	D2	ATOD_Y1A	T	D2	B						C	TS	-	11		I CFG2
20	D3	ATOD_Y1B	T	D3	B						C	TS	-	12		I CFG3
21	D4	ATOD_X2A	T	D4	B						C	TS	-	28		I CFG4
22	D5	ATOD_X2B	T	D5	B						C	TS	-	29		I CFG5
23	D6	ATOD_Y2A	T	D6	B					C	C	TS	-	30		I CFG6
24	D7	ATOD_Y2B	T	D7	B						C	TS	-	31		I CFG7
25	ATOD_MA		T								-	SR	-	15		O
26	ATOD_MB		T							G	-	SR	-	16		O
27	ATOD_OUT										-	SK	-	13		O
28	ATOD_IN										ST	-	-	14	-	I
29	GPIO0	QUAD_X	I	BUSY	I	PPO2_DATA	B	BUSY	I		C	TS	DN	5		I IA0
30	GPIO1	QUAD_XQ	I	/OE	O	PPO2_CLK	O	SER_CTS	I		C	TS	UP	6		I IA1
31	GPIO2	QUAD_Y	I	/WE	O	PPO3_DATA	B	SER_RTS	O	E	C	TS	DN	7		I IA2
32	GPIO3	QUAD_YQ	I	/CE	O	PPO3_CLK	O	SER_DIN	I		C	TS	UP	8		I IA3
33	GPIO4	IN4	I ↗	AA0	O ↘	SER_DIO	O/B	PWM_OUT	O		C	TS	UP	21		I IA4
34	GPIO5	IN5	I ↗	AA1	O ↘	SER_CLK	O	IN5	I	D	C	TS	UP	22		I IA5
35	GPIO6	IN6	I ↗	AA2	O ↘	SER_CS	O	IN6	I		C	TS	UP	23		I IA6
36	/RESET										ST	-	UP	46	-	I
37	GPIO7	IN7	I ↗	AB0	O ↘	PPO1_DATA	B	INT	I		C	TS	DN	20		I IA7
38	GPIO8	IN8	I ↗	AB1	O ↘	PPO1_CLK	O	AB1	O		C	TS	UP	19		I IB0
39	GPIO9	IN9	I ↗	AB2	O ↘	PPO0_DATA	B	PPO0_DATA	B	F	C	TS	DN	48	-	I IB1
40	GPIO10	IN10	I ↗	AB3	O ↘	PPO0_CLK	O	PPO0_CLK	O		C	TS	DN	47	-	I IB2
41	PPI_DATA										C	TS	DN	39		I
42	PPI_CLK										ST	-	-	38		I
43	VCC											POWER		17		
44	VCC											POWER		26		
45	VCC											POWER		44		
46	GND											POWER		18		
47	GND											POWER		27		
48	GND											POWER		45		

PINOUT	
0	A
1	B
2	D
3	
4	E
5	
6	F
7	

POLYFACE TEST BOARD
1 JOYSTICK A/B/C/E/F
4 PARALLEL MEM A/B/C/D/E/F
5 LATCHED MEM C/E/F
6 SERIAL MEM D/E
7 QUAD E
7 PWM D
8 JS/MIDI D/E
9 THRU D/E



PULL-UP (UP)
PULL-DOWN (DN)
=230uA

sample outputs

C=(E==1)

C=CMOS
SR=SOURCE
SK=SINK
TS=TRI-STATE
ST=SCHMITT

PINOUT FOR 48 PIN TQFP
FROM DAN KRAMER AT NYKO

Copyright (C) 1998,1999,

ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
POLYFACE PINOUT		
Size A	Document Number PINOUT.SCH	REV 1
Date: October 13, 1999	Sheet 3 of 13	

1.4 Memory Map (MEMMAP)

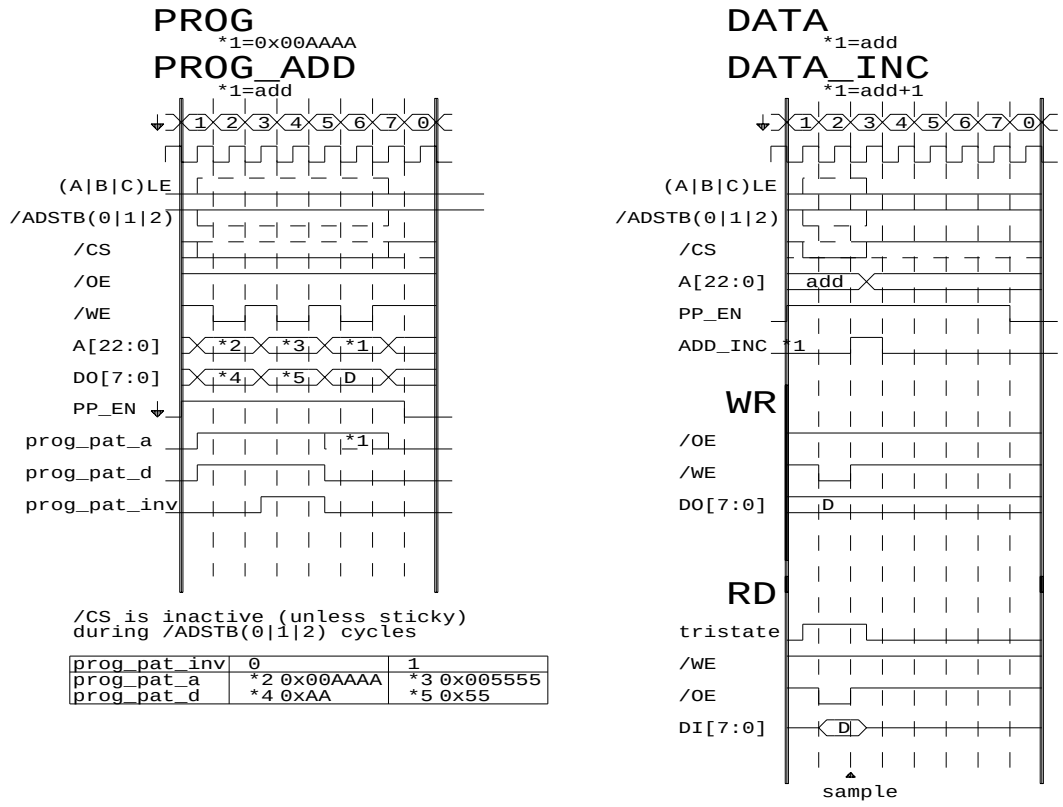
		READ	WRITE			READ	WRITE
	0x000			SERIAL	0x040		BAUD
	0x001				0x041		FLAGS0
	0x002				0x042		FLAGS1
	0x003				0x043	DATA	DATA
	0x004				0x044	SSTATUS	SSTATUS-ECLR
	0x005				0x045	RSTATUS	RSTATUS-ECLR
	0x006				0x046		
	0x007				0x047		
	0x008				0x048		
	0x009				0x049		
	0x00A				0x04A		
	0x00B				0x04B		
	0x00C				0x04C		
	0x00D				0x04D		
	0x00E				0x04E		
	0x00F				0x04F		
MEMORY	0x010	DATA	DATA		0x050		
	0x011	DATA_INC	DATA_INC		0x051		
	0x012		PROG		0x052		
	0x013		PROG_ADD		0x053		
STB_A	0x014	DATA	DATA		0x054		
STB_A	0x015	DATA_INC	DATA_INC		0x055		
STB_A	0x016		PROG		0x056		
STB_A	0x017		PROG_ADD		0x057		
STB_B	0x018	DATA	DATA		0x058		
STB_B	0x019	DATA_INC	DATA_INC		0x059		
STB_B	0x01A		PROG		0x05A		
STB_B	0x01B		PROG_ADD		0x05B		
STB_C	0x01C	DATA	DATA		0x05C		
STB_C	0x01D	DATA_INC	DATA_INC		0x05D		
STB_C	0x01E		PROG		0x05E		
STB_C	0x01F		PROG_ADD		0x05F		
ADDRESS	0x020		A[7:0] A0		0x060		
	0x021		A[15:8] A1		0x061		
	0x022		A[23:16] A2		0x062		
	0x023		STROBE		0x063		
	0x024		PINOUT		0x064		
	0x025	CONFIG			0x065		
	0x026	INPUTA			0x066		
	0x027	REQUEST	REQUEST		0x067		
	0x028	INPUTB			0x068		
	0x029				0x069		
	0x02A				0x06A		
	0x02B				0x06B		
	0x02C				0x06C		
	0x02D				0x06D		
	0x02E				0x06E		
	0x02F				0x06F		
JOYSTICK	0x030	SWITCH[8:1]			0x070		
	0x031	SWITCH[16:9]			0x071		
	0x032	QUADX			0x072		
	0x033	QUADY			0x073		
	0x034		CHANNEL		0x074		
	0x035	ANALOG			0x075		
	0x036				0x076		
	0x037				0x077		
	0x038				0x078		
	0x039				0x079		
	0x03A				0x07A		
	0x03B				0x07B		
	0x03C				0x07C		
	0x03D				0x07D		
	0x03E				0x07E		
	0x03F				0x07F		

A[23] is sticky CS

Copyright (C) 1998,1999,
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
POLYFACE MEMMAP		
Size A	Document Number MEMMAP.SCH	REV 1
Date: September 27, 1999	Sheet 4 of 13	

1.5 Memory Timing (MEMORY)



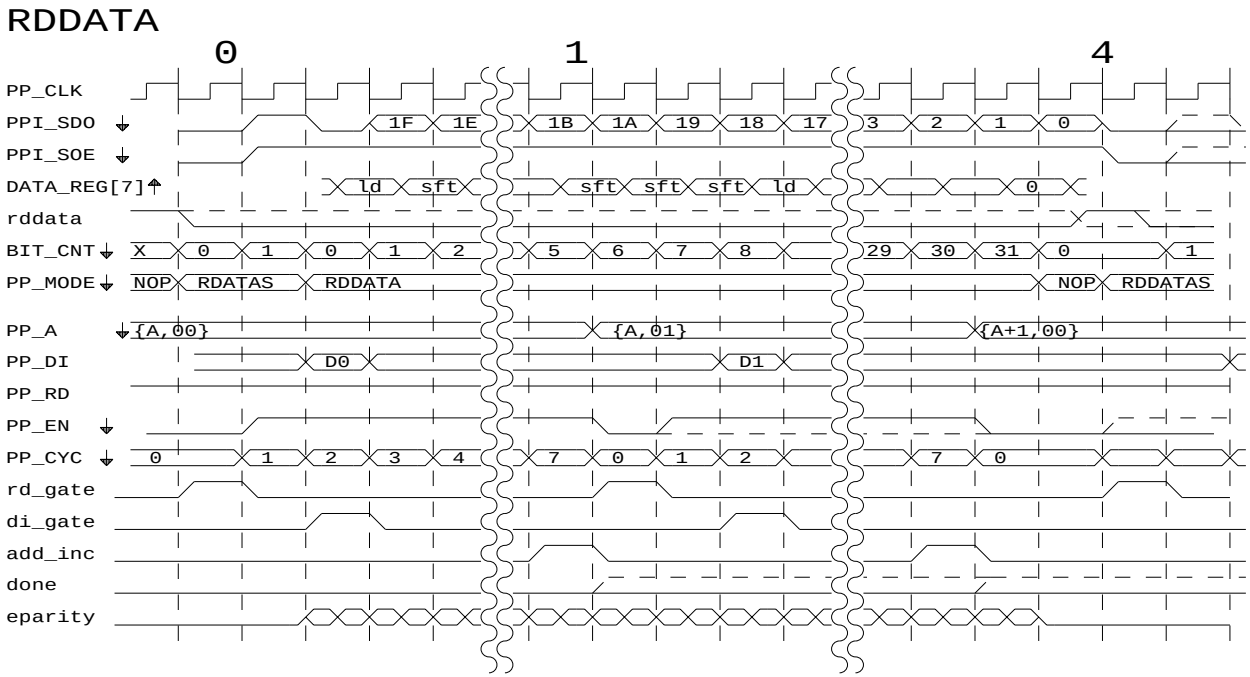
/CS is inactive (unless sticky)
during /ADSTB(0|1|2) cycles

prog_pat_inv	0	1
prog_pat_a	*2 0x00AAAA	*3 0x005555
prog_pat_d	*4 0xAA	*5 0x55

Copyright (C) 1998,1999,
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title	MEMORY PROTOCOL	
Size A	Document Number MEMORY.SCH	REV 1
Date: March 25, 1999	Sheet 5 of 13	

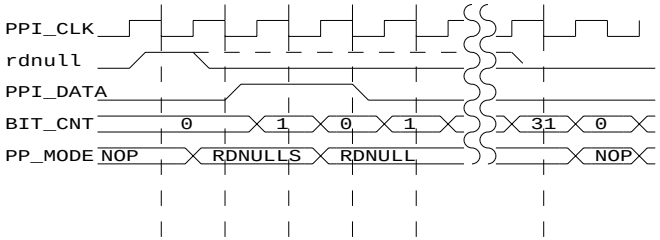
1.6 Read Data Timing (RDDATA)



Copyright (C) 1998,1999,
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
READ DATA TIMING		
Size A	Document Number RDDATA.TIM	REV 1
Date: March 3, 1999	Sheet 6 of 13	

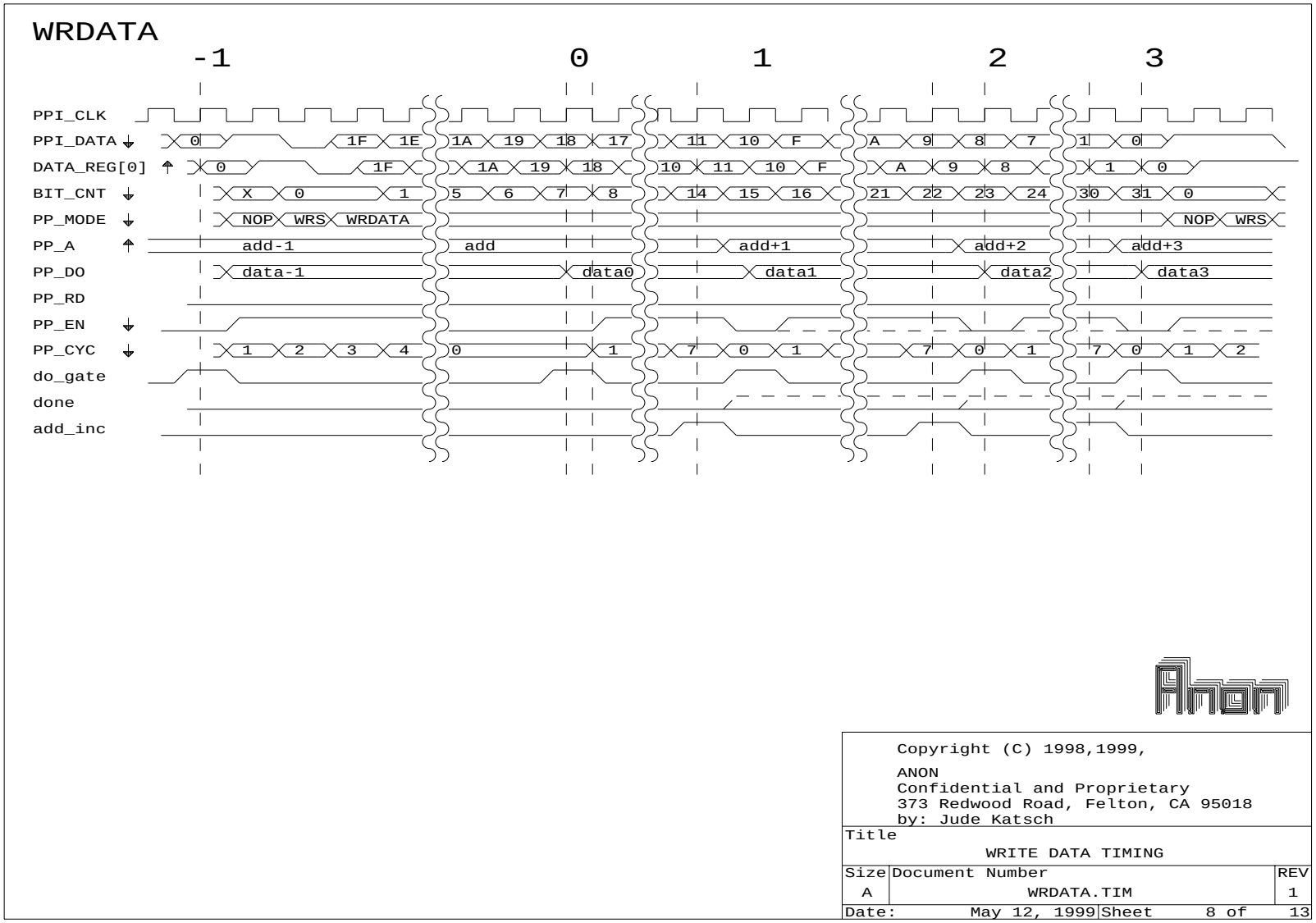
1.7 Read Null Timing (RDNULL)



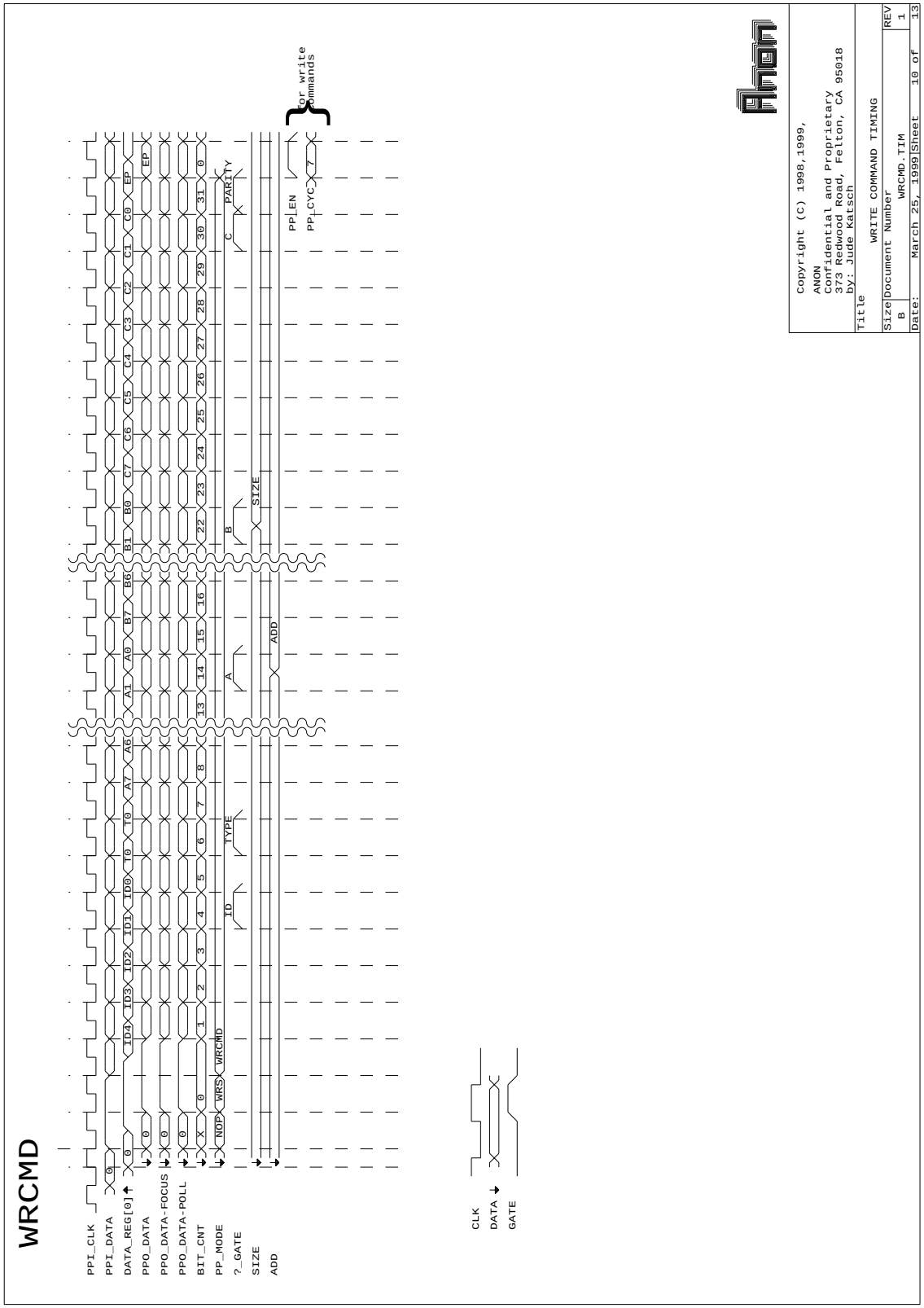
Copyright (C) 1998,1999,
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
READ NULL TIMING		
Size A	Document Number RDNULL.TIM	REV 1
Date: March 24, 1999	Sheet 7 of 13	

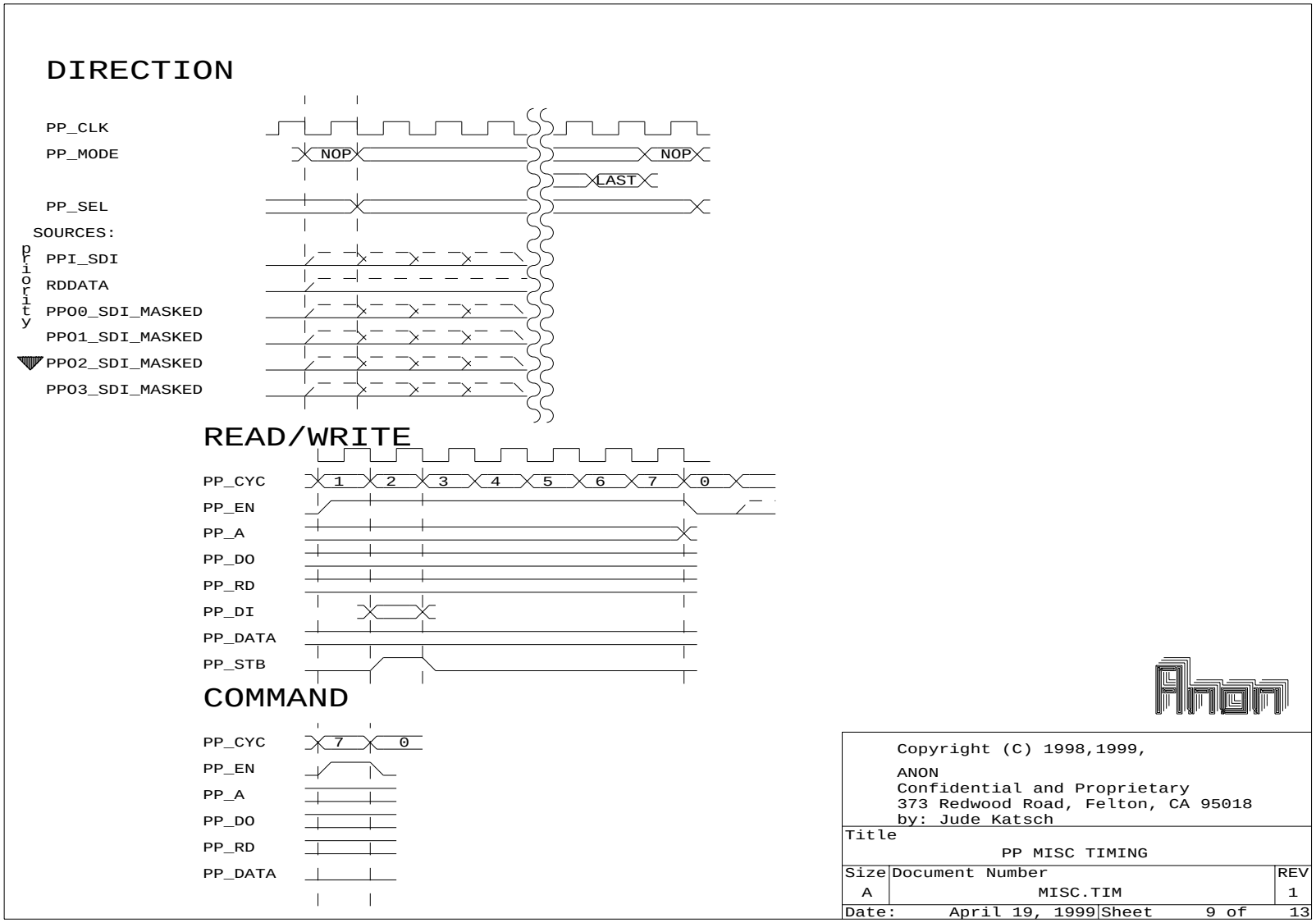
1.8 Write Data Timing (WRDATA)



1.9 Write Command Timing (WRCMD)



1.10 Misc (internal) Timing (MISC)



1.11 Configure Info (CONFIG)

MODE	FUNCTION	CONFIG	AMA	AMB	POS
1	JOYSTICK		1.82K	8.25K	18
2	WHEEL		2.74K	7.32K	27
3			3.65K	6.34K	36
4			4.53K	5.49K	45
5			5.49K	4.53K	54
6			6.34K	3.65K	63
7	CONFIG		7.32K	2.74K	72
8	CONFIG-UPPER4		8.25K	1.82K	81
7	SPLITTER	00000000			
7	RUMBLE	00000001			
7	THRU	00000010			
7	UCONT	00000011			
7	RS232	00000100			
7	OZJS	00000101			
7		00000110			
7		00000111			
7	MEM-SERIAL	0001xxxx			
7	MEM-PARALLEL	0010xxxx			
7	MEM-LATCH	0011xxxx			

xxxx=0000 - part can be determined by JEDEC,CFI, OR BLOCK

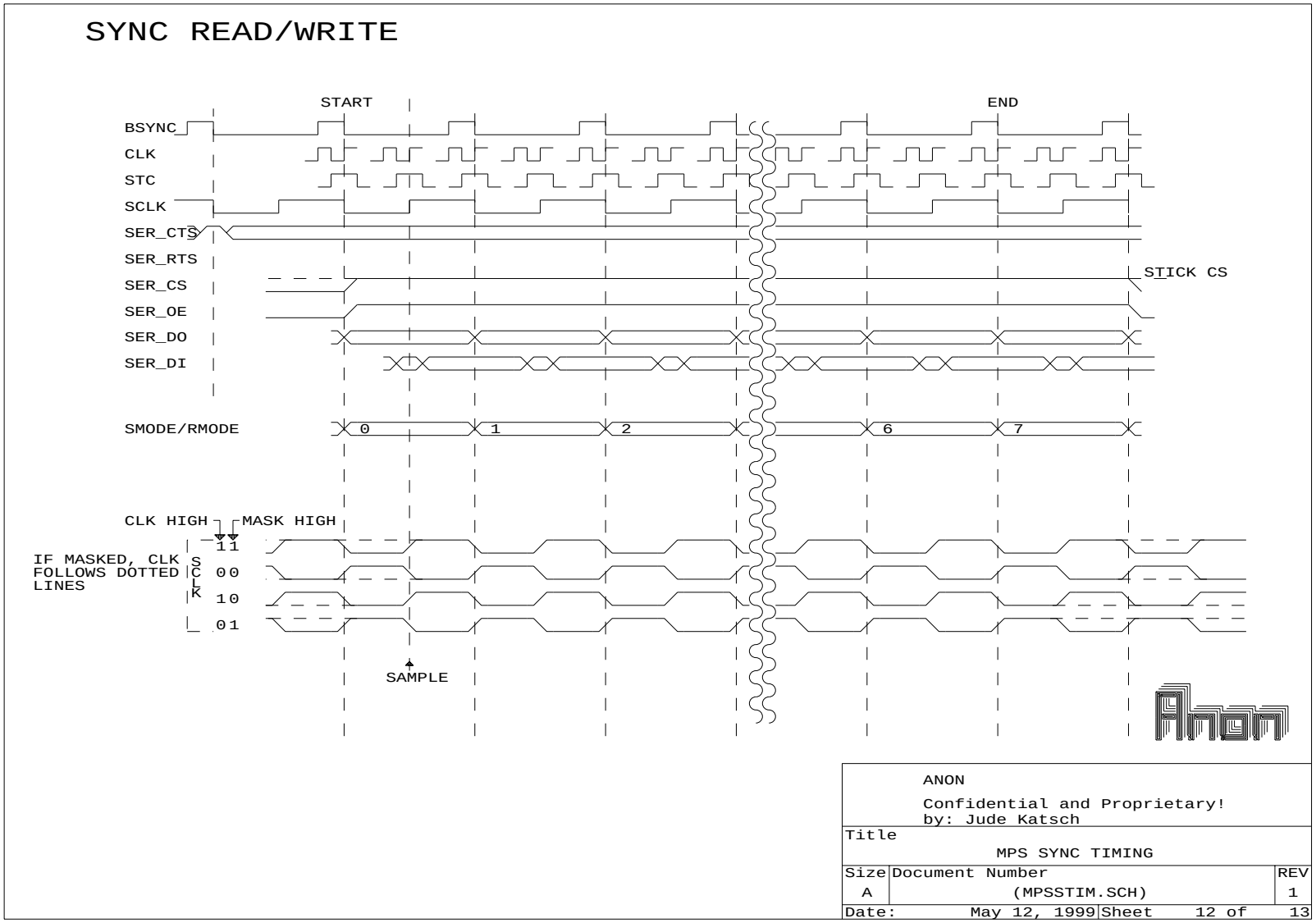
Copyright (C) 1998,1999,
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title
CONFIG PROTOCOL

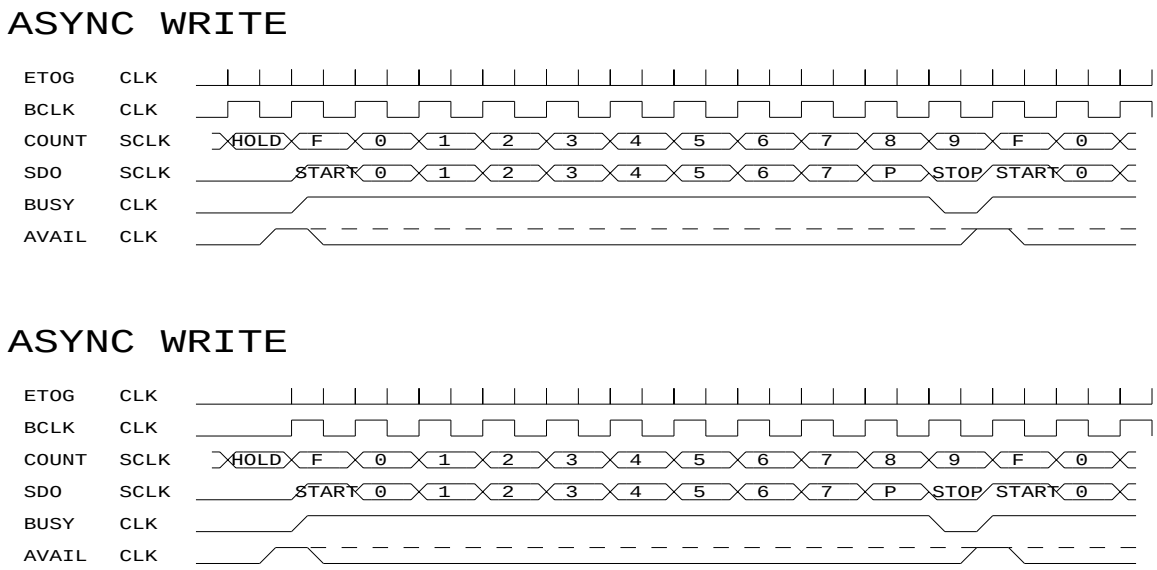
Size	Document Number	REV
A	CONFIG.SCH	1

Date: August 11, 1999 Sheet 11 of 13

1.12 Synchronous Serial Timing (SSERTIM)



1.13 Asynchronous Serial Timing (ASERTIM)



ANON Confidential and Proprietary! by: Jude Katsch		
Title MPS ASYNC TIMING		
Size A	Document Number (MPSATIM.SCH)	REV 1
Date: May 12, 1999	Sheet 13 of 13	

Chapter 2

SPECIFICATIONS

2.1 ROHM

2.2 ORBIT

2.2.1 ABSOLUTE RATINGS

PARAMETER	SYMBOL	LIMITS	UNIT
DC Supply Voltage	VDD	-0.3 to 7.0	V
Input Voltage	VIN	VDD+0.3	V
Input Current	IIN	10	uA
Storage Temperature	TSTG	-50 to +150	C

2.2.2 RECOMMENDED OPERATING CONDITIONS

PARAMETER	SYMBOL	LIMITS	UNIT
DC Supply Voltage	VDD	+3 to 5.5	V
Operating Temperature	TA	0 to +70	C

2.2.3 DC CHARACTERISTICS

VDD=+3.3V +=10%

SYMBOL	PARAMETER	CONDITION	MIN	TYP	MAX	UNIT
VIL	Voltage Input LOW	TTL	-	-	0.8	V
	Voltage Input LOW	CMOS	-	-	0.8	V
	Voltage Input LOW	SCHMITT		1.45		V
VIH	Voltage Input HIGH	TTL	2.0	-	-	V
	Voltage Input HIGH	CMOS	2.00	-	-	V
	Voltage Input HIGH	SCHMITT		2.13		V
IIN	Current Input	TTL,VIN=VSS or VDD	-10	1	10	uA
	Current Input	CMOS,VIN=VSS or VDD	-10	1	10	uA
VOL	Voltage Output LOW	IOL=Rated current	-	-	0.4	V
VOH	Voltage Output HIGH	IOH=Rated current	VDD-0.6	-	-	V
IOZ	TriState Output Leakage	VOH=VSS or VDD	-10	1	10	uA

2.3 ALTERA

Chapter 3

COMMAND FORMAT

Chapter 4

CONFIGURATION

Peripheral port commands are sent using the 32 bit command format using the ctrlCmd1 and ctrlCmd2 comm bus registers. The peripheral port divider should be programmed to produce a 1MHz PP_CLK.

4.1 CONFIGURE COMMANDS

All peripherals must execute the following configure commands:

COMMAND	ID=0	ID>0	TYPE
RESET	BROADCAST	DEVICE	EXECUTE
TAG	BROADCAST	DEVICE	EXECUTE
FOCUS	BROADCAST	BROADCAST	EXECUTE
BLUR	BROADCAST	BROADCAST	EXECUTE
UNBRAND	BROADCAST	DEVICE	EXECUTE
BRAND	FOCUS	FOCUS	EXECUTE
PROBE	FOCUS	DEVICE	READ
MAGIC	FOCUS	DEVICE	READ
ALIVE	BROADCAST	BROADCAST	POLL
REQUEST	BROADCAST	BROADCAST	POLL
ERROR	BROADCAST	BROADCAST	POLL

BROADCAST	Command is sent to all devices
FOCUS	Command is sent to device with FOCUS
DEVICE	Command is sent to device ID (1-31)

4.1.1 RESET

Identical to a power up reset. See specific registers for power up conditions.

RESET	
ID	=0: BROADCAST >0: DEVICE
READ	0
DIRECT	0
A[7:0]	0xb1
S[7:0]	0x00
C[7:0]	0x00

4.1.2 TAG

TAGS a DEVICE. Used during mapping to mutually exclusively set FOCUSED in an unmapped DEVICE.

TAG	
ID	=0: UNTAG BROADCAST >0: TAG DEVICE
READ	0
DIRECT	0
A[7:0]	0xb2
S[7:0]	0x00
C[7:0]	0x00

4.1.3 FOCUS

All DEVICES convert a FOCUS command to a BLUR command as it propagates up the tree structure.

FOCUS	
ID	0x00
READ	0
DIRECT	0
A[7:0]	0xb0
S[7:0]	0x00
C[7:0]	0x01

4.1.4 BLUR

Any DEVICE that receives a BLUR command becomes UNFOCUSED.

Only a selected port on a TAGGED device converts a BLUR command to a FOCUS command.

BLUR	
ID	0x00.
READ	0
DIRECT	0
A[7:0]	0xb0
S[7:0]	0x00
C[7:0]	0x02

4.1.5 UNBRAND

Clear the BRANDED bit.

UNBRAND	
ID	=0: BROADCAST >0: DEVICE
READ	0
DIRECT	0
A[7:0]	0xb3
S[7:0]	0x00
C[7:0]	0x00

4.1.6 BRAND

Set the BRANDED bit and write ID to the FOCUSED device.

BRAND	
ID	0x00
READ	0
DIRECT	0
A[7:0]	0xb4
S[7:0]	0x00
C[7:0]	=0: UNBRAND >0: BRAND

4.1.7 PROBE

Read (probe) configuration data during mapping.

PROBE	
ID	=0: FOCUSED >0: DEVICE
READ	1
DIRECT	0
A[7:0]	0x94
S[7:0]	0x04
C[7:0]	0x00

Returns the following probe information:

FUNCTION	BITS	
DEFCFG	PROBE[31]	Use DEFault ConFiG
VERSION[6:0]	PROBE[30:24]	10
TYPE[7:0]	PROBE[23:16]	0x01 = ALTERA 0x02 = ROHM 0x03 = ORBIT
MFG[7:0]	PROBE[15:8]	0x00 = VMLABS
TAGGED	PROBE[7]	Device is TAGGED
BRANDED	PROBE[6]	Device is BRANDED
ID[4:0]	PROBE[5:1]	Only valid if BRANDED==1
EPARITY	PROBE[0]	Even Parity

DEFCFG: Use the default STATE and ERROR registers, see Section 4.3.1 and 4.3.2 respectively.

4.1.8 MAGIC

Read a predefined long (32bits) from a DEVICE. This is just used for sanity checking, always returns 0x4a554445.

MAGIC	
ID	=0: FOCUSED >0: DEVICE
READ	1
DIRECT	0
A[7:0]	0xb1
S[7:0]	0x04
C[7:0]	0x00

4.1.9 ALIVE

All BRANDED DEVICES respond with a "1" in their bit slot. (ID=bit)

All UNBRANDED DEVICES respond with a "1" in bit slot 0.

An ALIVE command returns the necessary information to determine if any devices have been added or removed.

ALIVE	
ID	0x00
READ	1
DIRECT	0
A[7:0]	0x80
S[7:0]	0x04
C[7:0]	0x40

4.1.10 REQUEST

All BRANDED DEVICES respond with a "1" in their bit slot if they request interaction.

This can be used as a polled-interrupt.

UNBRANDED DEVICES should NEVER respond.

REQUEST	
ID	0x00
READ	1
DIRECT	0
A[7:0]	0x84
S[7:0]	0x04
C[7:0]	0x40

4.1.11 ERROR

All BRANDED DEVICES respond with a "1" in their bit slot if an error condition exists.

UNBRANDED DEVICES should NEVER respond.

ERROR	
ID	0x00
READ	1
DIRECT	0
A[7:0]	0x85
S[7:0]	0x04
C[7:0]	0x40

4.2 CONFIGURE STATES

Each device has three state bits TAGGED, FOCUSED, and BRANDED used to configure devices.

4.2.1 TAGGED (PROBE[7])

A TAGGED DEVICE converts a BLUR command to a FOCUS command on a selected port. Only one device can be TAGGED, only one port can be selected. All other DEVICES (UNTAGGED) and all unselected ports convert the FOCUS command to a BLUR command. The ports for a DEVICE are cycled thru using the device_driver->NEXT command in the device driver. When all ports have been exhausted, device_driver->NEXT returns DONE. A DEVICE is UNTAGGED (TAGGED=0) can be cleared by a hardware reset, a software reset, or a broadcasted TAG command (UNTAG). A DEVICE is TAGGED (TAGGED=1) by the TAG command using the DEVICE ID. Only BRANDED DEVICES can be TAGGED. Once a port is selected and a DEVICE is TAGGED, a BLUR command is sent. The DEVICE on the selected port of the TAGGED device will be FOCUSED.

4.2.2 FOCUSED

A FOCUSED DEVICE can be PROBED, the MAGIC register read, and BRANDED. Only ONE device can be FOCUSED at a time. The FOCUSED device will be the one following the TAGGED device on the selected port after the FOCUS/BLUR command is executed. All DEVICES convert a FOCUS command to a BLUR command is it propagates thru. A DEVICE is UNFOCUSED (FOCUSED=0) by a hardware reset, a software reset, by a BLUR command, or by a broadcasted TAG command (UNTAG). A DEVICE is FOCUSED (FOCUSED=1) by a FOCUS command.

4.2.3 BRANDED (PROBE[6])

BRANDED DEVICES have a unique ID between 1 and 31. DEVICES are addressed by their ID, chaos will ensue if two DEVICES are BRANDED with the same ID. The software is responsible to ensure that all DEVICES are BRANDED properly. A DEVICE is UNBRANDED (BRANDED=0) by a hardware reset, a software reset, a BRAND command with an ID of 0 when the DEVICE is FOCUSED, or the UNBRAND command. A DEVICE is BRANDED (BRANDED=1) by, the BRAND command with an ID between 1 and 31 when the DEVICE is FOCUSED. NOTE: The ID returned with the PROBE command is only valid if BRANDED.

4.3 CONFIGURE REGISTERS (DEFAULT CONFIG)

4.3.1 STATE (0x99) (read/write)

A hardware or software reset forces STATE to 0x00.

bit	FUNCTION
0	VACANT0
1	VACANT1
2	VACANT2
3	VACANT3
4	SELECT0
5	SELECT1
6	ROOT
7	ENABLE

VACANT# is set after interrogating the port and determining that no device is attached. This will cause PP#_DATA to be masked, except during bit 0 of a poll command. This is to prevent hot-plugged devices from writing erroneous data.

SELECT[1:0] selects the port to receive FOCUS if, and only if, this device is BRANDED and TAGGED. The hardware will convert a BLUR command to a FOCUS command only on the selected port of a TAGGED device. A FOCUS command is always converted to BLUR command.

A ROOT device responds to a PROBE command sending a RDNULL packet after a timeout without an intervening RDDATA packet. A RDNULL packet is a start bit, set to 1, followed by a control bit, set to 1, followed by 32 data bits set to 0. (11_00000000_00000000_00000000_00000000b) This tries to hide a bug in Aries that may cause the peripheral port module to drop RDDATA packets if there is a collision with other registers in the same module. A missing RDDATA packet forces the peripheral port software to retry after a timeout. Normally, executing a PROBE on a vacant node during configuration would force a retry. If however, a ROOT device is configured, a RDNULL packet will be returned and the peripheral port software will not need to retry. A ROOT device is the first device connected to the peripheral port, depth=1, as shown in the tree configuration in Figure ???. The configuration program must determine which device is the first in the tree structure and configure it as a ROOT device. There is no hardware mechanism that prevents or forces a specific device or a single device to be configured as a ROOT device. In the case where there is no ROOT device, responses from vacant ports will need to be timed out and retried to ensure that the non-response was not a lost RDATA packet. There should NEVER be more than ONE device configured as a ROOT device and that device should ALWAYS be the first device connected to the Peripheral Port.

ENABLE does just what it says, it enables the chip. This must be set after configuration, before using the device.

4.3.2 ERROR (0x98) (read/write clear)

A hardware or software reset forces ERROR to 0x00.

bit	NAME
0	COLLISION0
1	COLLISION1
2	COLLISION2
3	COLLISION3
4	WRITE_OVERFLOW
5	READ_OVERFLOW
6	INVALID_COMMAND
7	DEVICE_ERROR

Writing a “1” to the selected error bit clears that bit.

Chapter 5

CRC

Indirect data transfers (read and write) can optionally return a CRC (cyclic redundancy check) to verify the integrity of the data transfer between Merlin and the Device. CRCs are enabled by setting bit 8 (C7=1) in indirect data transfers (DIRECT=0, CMD=0). The CRC is 16 bits.

For the following tables: Dx=read data, CU=CRC Upper, CL=CRC Lower, 00=0x00

Writes will return the CRC after all the data (as set by S[6:0]) has been received. The CRC will be:

CU	CL	00	00

Reads will return the CRC if there are unused byte slots available even if CRC has not been enabled:

size%4	A0	A1	A2	A3	B0	B1	B2	B3
0	Da	Db	Dc	Dd	CU	CL	00	00
1	Da	Db	Dc	CU	CL	00	00	00
2	Da	Db	CU	CL	not needed			
3	Dc	CU	CL	00	not needed			
	always returned				returned only if CRC is enabled			

See Section 29 on page 149 for an C example that calculates the CRC.

Chapter 6

POLYFACE REGISTERS

Address	Definition
0x0?	not used
0x1?	MEMORY
0x2?	ADDRESS
0x3?	JOYSTICK
0x4?	SERIAL
0x5?	not used
0x6?	not used
0x7?	not used

6.1 MEMORY (0x1?)

ADDRESS	STROBE	READ	WRITE
0x10	none	DATA	DATA
0x11	none	DATA++	DATA++
0x12	none		PROG
0x13	none		PROG_ADD
0x14	STB_A	DATA	DATA
0x15	STB_A	DATA++	DATA++
0x16	STB_A		PROG
0x17	STB_A		PROG_ADD
0x18	STB_B	DATA	DATA
0x19	STB_B	DATA++	DATA++
0x1a	STB_B		PROG
0x1b	STB_B		PROG_ADD
0x1c	STB_C	DATA	DATA
0x1d	STB_C	DATA++	DATA++
0x1e	STB_C		PROG
0x1f	STB_C		PROG_ADD

6.1.1 DATA (0010_xx00b) (read/write)

Transfers will produce the necessary timing on the /we, /oe, /ce, strobes, address, an data lines to read or write parallel interface devices, see Figure 6.1 MEMORY DATA TIMING.

6.1.2 DATA++ (0010_xx01b) (read/write)

Same as DATA (above) but increments lower 8 bits of address bus after transfer, see Figure 6.1 MEMORY DATA TIMING.

6.1.3 PROG (0010_xx10b) (write only)

Write the standard PROGRAM FLASH sequence: (0x00aaaa)=0xaa, (0x005555)=0x55, (0x00aaaa)=data, see Figure 6.2 MEMORY PROGRAM TIMING.

6.1.4 PROG_ADD (0010_xx11b) (write only)

Write the standard PROGRAM ADDRESS FLASH sequence: (0x00aaaa)=0xaa, (0x005555)=0x55, (address)=data, see Figure 6.2 MEMORY PROGRAM TIMING.

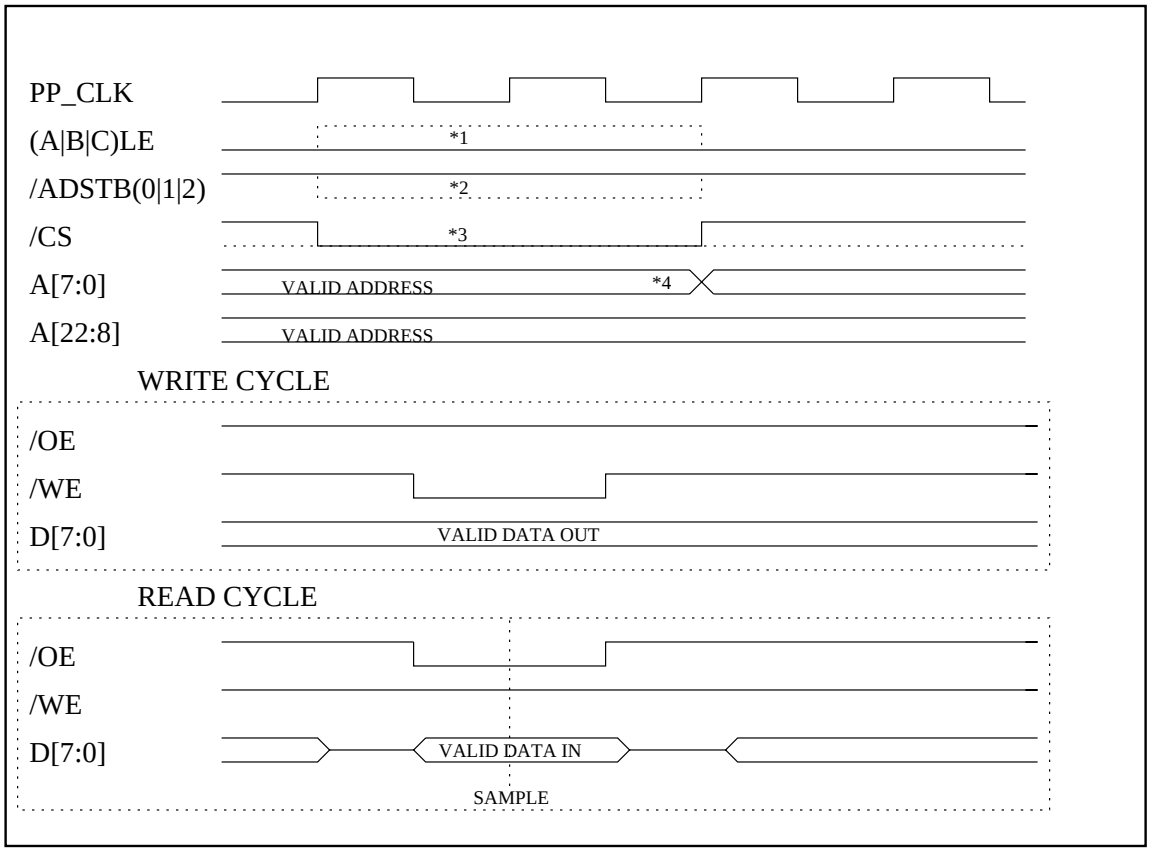
6.1.5 STROBES (ALE, BLE, CLE, /ADSTB0, /ADSTB1, and /ADSTB2)

The active state of the STB_? is determined by PINOUT_F as shown in the following table: (see Section 6.2.5 PINOUT on page 36)

	PINOUT_F ==3	PINOUT_F ==2
STB_A	ALE	/ADSTB0
STB_B	BLE	/ADSTB1
STB_C	CLE	/ADSTB2

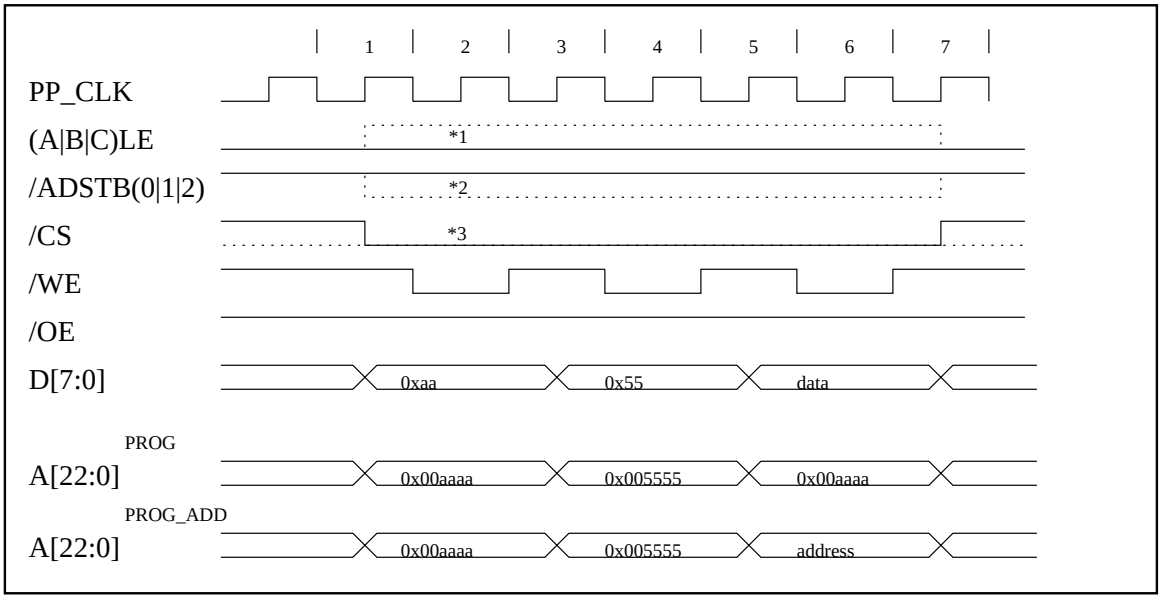
See Figure 6.1 MEMORY DATA TIMING and 6.1 MEMORY PROGRAM TIMING for details.

Figure 6.1: MEMORY DATA TIMING



*1 PINOUT_F==3. (see Section 6.2.5 PINOUT on page 36)
*2 PINOUT_F==2. (see Section 6.2.5 PINOUT on page 36)
*3 /CS can be forced low by setting bit 7 of A2 (STICKY_CS) to 1.
(see Section 6.2.3 A2 on page 35)
*4 Lower 8 bits of address are incremented at end of transfer if DATA++ is read or written.
(see Section 6.1.2 for details on DATA++)

Figure 6.2: MEMORY PROGRAM TIMING



*1 PINOUT_F==3. (see Section 6.2.5 PINOUT on page 36)
*2 PINOUT_F==2. (see Section 6.2.5 PINOUT on page 36)
*3 /CS can be forced low by setting bit 7 of A2 (STICKY_CS) to 1.
(see Section 6.2.3 A2 on page 35)

6.2 ADDRESS (0x2?)

ADDRESS	READ	WRITE
0x20		A0 (A[7:0])
0x21		A1 (A[15:8])
0x22		A2 (sticky_cs,A[23:16])
0x23		STROBE
0x24		PINOUT
0x25	CONFIG	
0x26	INPUTA	
0x27	REQUEST	REQUEST_ENABLE
0x28	INPUTB	
0x29		
0x2a		
0x2b		
0x2c		
0x2d		
0x2e		
0x2f		

6.2.1 A0 - A[7:0] (0x20) (write only)

This is the least significant byte of the address bus for use when interfacing to parallel devices. It is incremented after data is read or written to DATA++, see Section 6.1.2 DATA++. Outputs are enabled when PINOUT_A is 1, see Section 6.2.5 PINOUT. Reading S0 (SWITCH[8:1]), see Section 6.4.1 S0, returns the current value on the pins. This can also be used as eight general purpose outputs. A0 is undefined after reset.

bit	A	S0
0	0	1
1	1	2
2	2	3
3	3	4
4	4	5
5	5	6
6	6	7
7	7	8

6.2.2 A1 - A[15:8] (0x21) (write only)

This byte is output on pins A[15:8] when PINOUT_B is 1, see Section 6.2.5 PINOUT. Reading S1 (SWITCH[16:9]), see Section 6.4.2 S1, returns the current value on the pins. This can also be used as eight general purpose outputs. A1 is undefined after reset.

bit	A	S1
0	8	9
1	9	10
2	10	11
3	11	12
4	12	13
5	13	14
6	14	15
7	15	16

6.2.3 A2 - STICKY_CS,A[22:16] (0x22) (write only)

Bits A[22:16] are output on the GPIO pins shown in the table below. Outputs are enabled by setting the appropriate pinout group as shown in the table below, see Section 6.2.5 PINOUT. Reading INPUT, see Section 6.2.7 INPUT, returns the current value on the pins shown in the table below. A[22:20] and A[19:16] can be used as three and four general purpose outputs respectively. STICKY_CS when “1” forces /CE low. A2 is undefined after reset.

bit	A	IN	GPIO	PINOUT
0	16		7	F==1
1	17		8	F==1
2	18		9	F==1
3	19	7	10	F==1
4	20	4	4	D==1
5	21	5	5	D==1
6	22	6	6	D==1
7	STICKY_CS	3	3	E==1

6.2.4 STROBE (0x23) (write only)

STROBE is undefined after reset.

bit[2:0]	AA2	AA1	AA0	AB3	AB2	AB1	AB0
GPIO	6	5	4	10	9	8	7
000	A22	A21	A20	A19	A18	A17	A16
001	A22	A21	A20	A19	A18	A17	A16
010	A22	A21	A20	PROG	/ASTB0	/ASTB1	/ASTB2
011	A22	A21	A20	PROG	CLE	BLE	ALE
100	A22	A21	A20	A19	A18	A17	A16
101	A22	A21	A20	A19	A18	A17	A16
110	/ASTB0	/ASTB1	/ADSTB2	A19	A18	A17	A16
111	CLE	BLE	ALE	A19	A18	A17	A16

6.2.5 PINOUT (0x24) (write only)

PINOUT is 0x00 after a software or hardware reset.

PINOUT bit	FUNCTION	PIN GROUP
0	PINOUT_A	A[7:0]
1	PINOUT_B	B[7:0]
2	PINOUT_D0	GPIO[6:4]
3	PINOUT_D1	
4	PINOUT_E0	GPIO[3:0]
5	PINOUT_E1	
6	PINOUT_F0	GPIO[10:7]
7	PINOUT_F1	
	PINOUT_C	D[7:0]

NAME	TQFP48	QFP44	GROUP	0	1	2	3	
A0	1		A=PINOUT[0]	SW01	A0			
A1	2			SW02	A1			
A2	3			SW03	A2			
A3	4			SW04	A3			
A4	24			SW05	A4			
A5	25			SW06	A5			
A6	32			SW07	A6			
A7	33			SW08	A7			
A8	34		B=PINOUT[1]	SW09	A8			
A9	35			SW10	A9			
A10	36			SW11	A10			
A11	37			SW12	A11			
A12	40			SW13	A12			
A13	41			SW14	A13			
A14	42			SW15	A14			
A15	43			SW16	A15			
D0	9		C=(PINOUT[5:4]==1)	CFG0/ATOD_X1A	D0			
D1	10			CFG1/ATOD_X1B	D1			
D2	11			CFG2/ATOD_Y1A	D2			
D3	12			CFG3/ATOD_Y1B	D3			
D4	28			CFG4/ATOD_X2A	D4			
D5	29			CFG5/ATOD_X2B	D5			
D6	30			CFG6/ATOD_Y2A	D6			
D7	31			CFG/7ATOD_Y2B	D7			
ATOD_MA	15							
ATOD_MB	16							
ATOD_OUT	13							
ATOD_IN	14							
GPIO0	5		E=PINOUT[5:4]	IA0/QUAD_X	BUSY	PP02_DATA	BUSY	
GPIO1	6			IA1/QUAD_XQ	/OE	PPO2_CLK	SER_CTS	
GPIO2	7			IA2/QUAD_Y	/WE	PPO3_DATA	SER_RTS	
GPIO3	8			IA3/QUAD_YQ	/CE	PPO3_CLK	SER_DIN	
GPIO4	21		D=PINOUT[3:2]	IA4	AA0	SER_DIO	PWM_OUT	
GPIO5	22			IA5	AA1	SER_CLK	IA5	
GPIO6	23			IA6	AA2	SER_CS	IA6	
/RESET	46							
GPIO7	20		F=PINOUT[7:6]	IA7/INT	AB0	PPO1_DATA	IA7/INT	
GPIO8	19			IB0	AB1	PPO1_CLK	AB1	
GPIO9	48			IB1	AB2	PPO0_DATA	PPO0_DATA	
GPIO10	47			IB2	AB3	PPO0_CLK	PPO0_CLK	
PPI_DATA PPI_CLK	39 38							
VCC	17							
VCC	26							
VCC	44							
GND	18							
GND	27							
GND	45							

6.2.6 CONFIG - CFG[7:0] (0x25) (read only)

The CONFIG register reads the data on the D[7:0] pins. This is intended to be used in non-AtoD modes. These bits can be tied high or low in configurations that don't used the data port such as splitters. They can also be pulled high or low thru pull-up/pull-down resistors in configurations that use the data port such as parallel or latched memories. With the information from the CONFIG register, the driver can then configure the chip.

6.2.7 INPUTA - IA[7:0] (0x26) (read only)

Returns the current values on the GPIO pins:

bit	GPIO
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7

6.2.8 REQUEST (0x27) (read/write)

bit	NAME	FUNCTION
0	BUSY_REQUEST	Current status of BUSY pin. (GPIO0)
1	INT_REQUEST	INTerrupt (rising edge on GPIO7) cleared by REQUEST read.
2	ATOD_REQUEST	The AtoD conversion is complete.
3	ATOD_ERROR	An error occurred during the last AtoD conversion.
4	SER_RECV_FOUR	There are at least four bytes available for reading.
5	SER_RECV_ANY	There is at least one byte available for reading.
6	SER_SEND_FOUR	There are at least four empty bytes available for writing.
7	SER_SEND_ANY	There is at least one empty byte available for writing.

The ENABLE bit, see table below, is ANDed with the respective request. The ENABLED requests are then ORed and returned with a REQUEST command. If more than one ENABLE bit is "1", REQUEST must be read to determine which request, see table above, is active. REQUEST_ENABLE is 0x00 after a hardware or software reset.

bit	REQUEST_ENABLE
0	BUSY
1	~BUSY
2	ATOD
3	INT
4	SER_RECV_FOUR
5	SER_RECV_ANY
6	SER_SEND_FOUR
7	SER_SEND_ANY

6.3 INPUTB - IB[2:0] (0x28) (read only)

Returns the current values on the pins:

bit	PIN
0	GPIO8
1	GPIO9
2	GPIO10
3	**0**
4	**0**
5	**0**
6	**0**
7	**0**

6.4 JOYSTICK (0x3?)

ADDRESS	READ	WRITE
0x30	S0-SWITCH[8:1]	
0x30	S1-SWITCH[16:9]	
0x32	QUADX	
0x33	QUADY	
0x34		CHANNEL
0x35	ANALOG	
0x36		
0x37		
0x38		
0x39		
0x3a		
0x3b		
0x3c		
0x3d		
0x3e		
0x3f		

6.4.1 S0 - SWITCH[8:1] (0x30) (read only)

Returns the data on pins A[7:0]. These pins have built in pull-down resistors. Closed switches should pull the pin high “1”.

6.4.2 S1 - SWITCH[16:9] (0x31) (read only)

Returns the data on pins A[15:8]. These pins have built in pull-down resistors. Closed switches should pull the pin high “1”.

6.4.3 QUADX (0x32) (read only)

Returns the current X quadrature count (0x00-0xff). NOTE: This is an 8 bit absolute count. See the code snippet below to convert to an integer count.

```
delta = ((int)data-previous); // data is the value returned when QUAD# is read.
position+=delta;
previous=data;
```

6.4.4 QUADY (0x33) (read only)

Returns the current Y quadrature count. See Section 6.4.3 QUADX for details.

6.4.5 CHANNEL (0x34) (write only)

Selects the AtoD channel and starts a conversion. CHANNEL is 0x00 after a hardware or software reset

data	FUNCTION
0x00	DISABLED
0x01	MODE
0x02	X1
0x03	Y1
0x04	X2
0x05	Y2

6.4.6 ANALOG (0x35) (read only)

Returns the position of the pot where 0=0% and 0xff=100%. This register is only valid if ATOD_REQUEST, see Section 6.2.8 REQUEST, is “1”.

position=(data*100)/255; // data is the value returned when ANALOG is read.

6.5 SERIAL (0x4?)

ADDRESS	READ	WRITE
0x40		BAUD
0x41		FLAGS0
0x42		FLAGS1
0x43	SDATA	SDATA
0x44	SSTATUS	ERROR_CLR
0x45	RSTATUS	ERROR_CLR
0x46		
0x47		
0x48		
0x49		
0x4a		
0x4b		
0x4c		
0x4d		
0x4e		
0x4f		

6.5.1 BAUD (0x40) (write only)

The baud rate generator uses an LFSR as counter. The lfsr value loaded into the register is determined by looking up the divider in the LFSR look-up-table, see Table 6.1.

To determine tha value for a spacific baud rate, calculate the baud rate divider:

$$divider = ((\frac{pp_clk_{frequency}}{baud}) + 1)\&0x1fe$$

Then enter the lfsr numbr from the LFSR look-up-table, see Table 6.1.

For example: The $divider$ for a $baud$ of 9600 with a $pp_clk_{frequency}$ of 500K would be 52. A value of 52 would translate into a lfsr of 0xaa from the LFSR LUT. Loading 0xaa into BAUD would generate a baud of 9615.38.

The LFSR look-up-table, see Table 6.1, was generated with the following C code:

```
main()
{
int lfsr,i;
lfsr = 0xff;
(i=0;i<256;i++)
   {
   lfsr = ((lfsr<<1)| ( ((lfsr>>7)&1) ^ ((lfsr>>5)&1) ^ ((lfsr>>4)&1) ^ ((lfsr>>3)&1))&0xff;
   printf("%03d\t0x%02x\r\n",(255-i)*2,lfsr);
   }
}
```

6.5.2 FLAGS0 (0x41) (write only)

bit	NAME	FUNCTION
0	LSB_FIRST	“0”=76543210, “1”=01234567
1	SYNC_BITS	Send start and stop bits. (if PARITY_BIT==”1” SYNC_BITS=ODD_PARITY)
2	PARITY_BIT	Send start/parity/stop bits.
3	MASKED_CLK	ser_clk is only active during data transfers.
4	CLK_HIGH	Polarity of ser_clk.
5	LEVEL_HIGH	Inactive state of ser_clk.
6	CS_NEG	“0”=ser_cs is active high, “1”=ser_cs is active low.
7	ASYNC	Enable asynchronous receive

6.5.3 FLAGS1 (0x42) (write only)

bit	NAME	FUNCTION
0	RECV	Enable receive channel.
1	SEND	Enable send channel.
2	STICKY_CS	Force chip select.
3	RUN	Enable serial. When disabled, FIFOs are flushed and baud generator is cleared.
4	UNUSED	
5	IO_FLIP	Invert ser_dout, ser_din, ser_rts, and ser_cts.
6	DIN_FLIP	Invert ser_din. This is necessary for the THRU interface.
7	MASK_CTS	Ignore ser_cts (clear to send)

Table 6.1: LFSR look-up-table

/	lfsr	/	lfsr	/	lfsr	/	lfsr	/	lfsr	/	lfsr	/	lfsr	/	lfsr
510	0xfe	446	0x1c	382	0x6e	318	0x61	254	0x0d	190	0x48	126	0xf7	062	0x1d
508	0xfc	444	0x38	380	0xdc	316	0xc3	252	0x1b	188	0x91	124	0xef	060	0x3a
506	0xf8	442	0x71	378	0xb9	314	0x87	250	0x36	186	0x22	122	0xdf	058	0x75
504	0xf0	440	0xe2	376	0x72	312	0x0f	248	0x6c	184	0x45	120	0xbf	056	0xea
502	0xe1	438	0xc4	374	0xe4	310	0x1f	246	0xd8	182	0x8a	118	0x7e	054	0xd5
500	0xc2	436	0x89	372	0xc8	308	0x3e	244	0xb1	180	0x14	116	0xfd	052	0xaa
498	0x85	434	0x12	370	0x90	306	0x7d	242	0x63	178	0x29	114	0xfa	050	0x55
496	0x0b	432	0x25	368	0x20	304	0xfb	240	0xc7	176	0x52	112	0xf4	048	0xab
494	0x17	430	0x4b	366	0x41	302	0xf6	238	0x8f	174	0xa5	110	0xe9	046	0x57
492	0x2f	428	0x97	364	0x82	300	0xed	236	0x1e	172	0x4a	108	0xd3	044	0xaf
490	0x5e	426	0x2e	362	0x05	298	0xdb	234	0x3c	170	0x95	106	0xa6	042	0x5f
488	0xbc	424	0x5c	360	0x0a	296	0xb7	232	0x79	168	0x2a	104	0x4c	040	0xbe
486	0x78	422	0xb8	358	0x15	294	0x6f	230	0xf3	166	0x54	102	0x99	038	0x7c
484	0xf1	420	0x70	356	0x2b	292	0xde	228	0xe7	164	0xa9	100	0x33	036	0xf9
482	0xe3	418	0xe0	354	0x56	290	0xbd	226	0xce	162	0x53	098	0x66	034	0xf2
480	0xc6	416	0xc0	352	0xad	288	0x7a	224	0x9c	160	0xa7	096	0xcd	032	0xe5
478	0x8d	414	0x81	350	0x5b	286	0xf5	222	0x39	158	0x4e	094	0x9a	030	0xca
476	0x1a	412	0x03	348	0xb6	284	0xeb	220	0x73	156	0x9d	092	0x35	028	0x94
474	0x34	410	0x06	346	0x6d	282	0xd7	218	0xe6	154	0x3b	090	0x6a	026	0x28
472	0x68	408	0x0c	344	0xda	280	0xae	216	0xcc	152	0x77	088	0xd4	024	0x50
470	0xd0	406	0x19	342	0xb5	278	0x5d	214	0x98	150	0xee	086	0xa8	022	0xa1
468	0xa0	404	0x32	340	0x6b	276	0xba	212	0x31	148	0xdd	084	0x51	020	0x42
466	0x40	402	0x64	338	0xd6	274	0x74	210	0x62	146	0xbb	082	0xa3	018	0x84
464	0x80	400	0xc9	336	0xac	272	0xe8	208	0xc5	144	0x76	080	0x46	016	0x09
462	0x01	398	0x92	334	0x59	270	0xd1	206	0x8b	142	0xec	078	0x8c	014	0x13
460	0x02	396	0x24	332	0xb2	268	0xa2	204	0x16	140	0xd9	076	0x18	012	0x27
458	0x04	394	0x49	330	0x65	266	0x44	202	0x2d	138	0xb3	074	0x30	010	0x4f
456	0x08	392	0x93	328	0xcb	264	0x88	200	0x5a	136	0x67	072	0x60	008	0x9f
454	0x11	390	0x26	326	0x96	262	0x10	198	0xb4	134	0xcf	070	0xc1	006	0x3f
452	0x23	388	0x4d	324	0x2c	260	0x21	196	0x69	132	0x9e	068	0x83	004	0x7f
450	0x47	386	0x9b	322	0x58	258	0x43	194	0xd2	130	0x3d	066	0x07	002	0xff
448	0x8e	384	0x37	320	0xb0	256	0x86	192	0xa4	128	0x7b	064	0x0e	000	0xfe

6.5.4 SDATA (0x43) (read/write)

SDATA (Serial DATA) read data from the receive FIFO or write data to the send FIFO. If the receive FIFO is empty RUF_ERROR is set. If the send FIFO is full SOF_ERROR is set.

6.5.5 SSTATUS (0x44) (read/write-clear)

bit	NAME	FUNCTION
0	AVAIL0	Number of empty bytes available for writing.
1	AVAIL1	
2	AVAIL2	
3	AVAIL3	
4	AVAIL4	
5	UNUSED	
6	UNUSED	
7	SOF_ERROR	Send overflow error. Writing 1 to this bit clears it

6.5.6 RSTATUS (0x45) (read/write-clear)

bit	NAME	FUNCTION
0	AVAIL0	Number of data bytes available for reading.
1	AVAIL1	
2	AVAIL2	
3	AVAIL3	
4	AVAIL4	
5	ROF_ERROR	Receive overflow error. Writing 1 to this bit clears it.
6	RUF_ERROR	Receive underflow error. Writing 1 to this bit clears it.
7	RP_ERROR	Receive parity error. Writing 1 to this bit clears it.

Chapter 7

ATOD CONVERTER

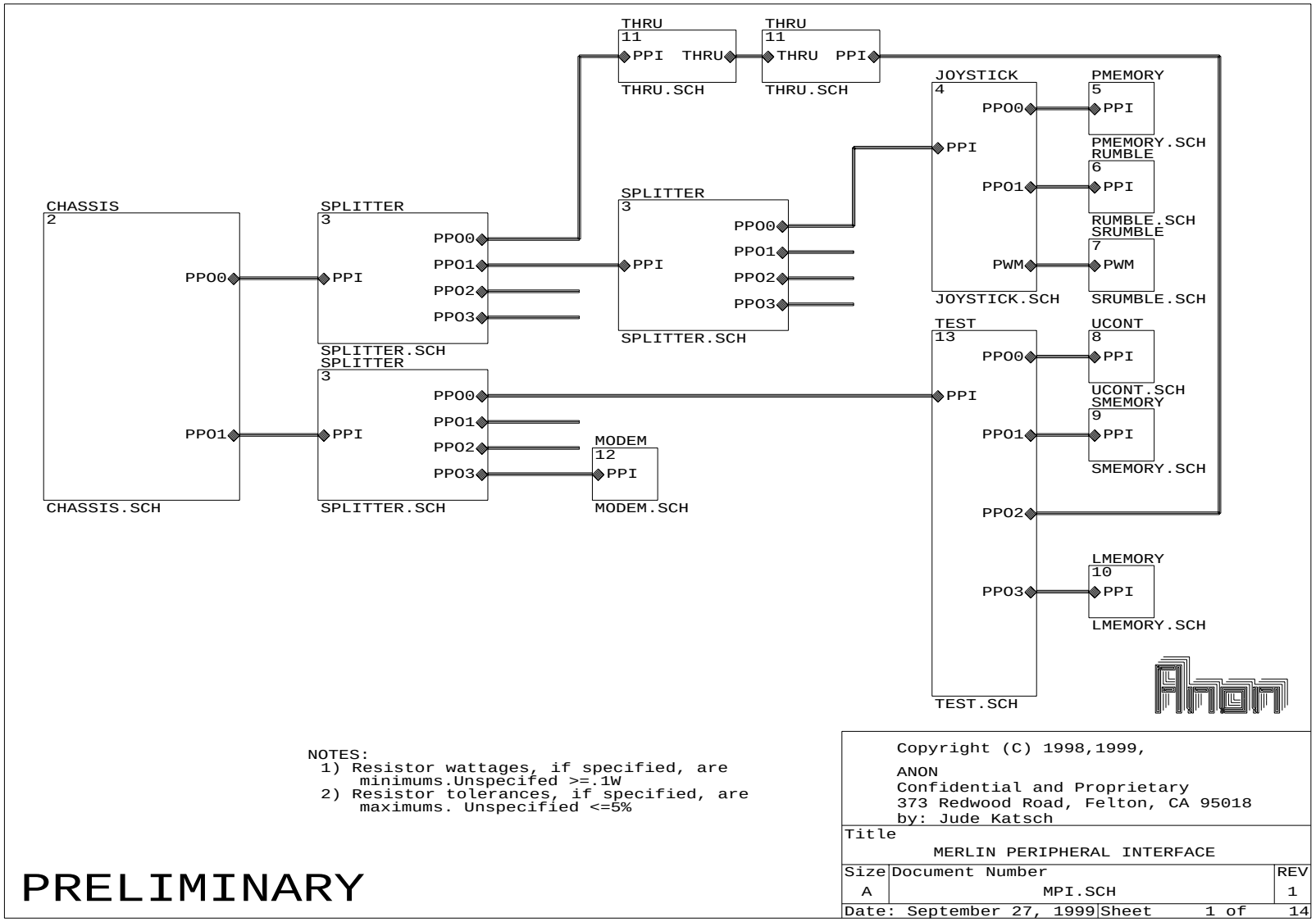
The ATOD converter

Chapter 8

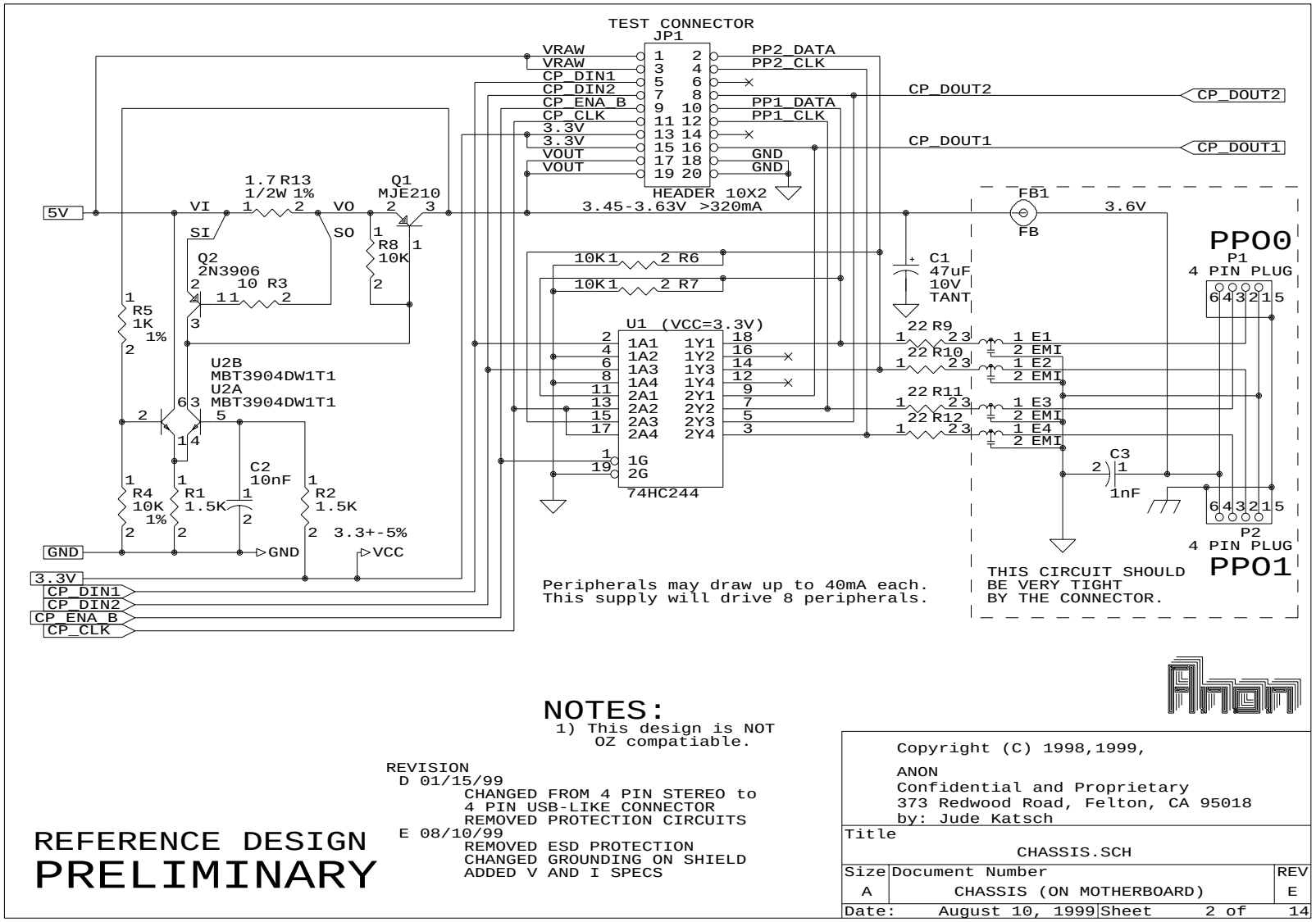
REFERENCE SCHEMATICS

Description	Section	Page
Overview	8.1	46
Chassis	8.2	47
Splitter	8.3	48
Joystick	8.4	49
Parallel Memory	8.5	50
Serial Memory	8.6	51
Latched Memory	8.7	52
Rumble	8.8	53
Simple Rumble	8.9	54
Micro Controller	8.10	55
Thru	8.11	56
Modem	8.12	57

8.1 Overview (MPI)



8.2 Chassis (CHASSIS)



NOTES:
1) This design is NOT OZ compatiable.

REVISION
D 01/15/99
CHANGED FROM 4 PIN STEREO to 4 PIN USB-LIKE CONNECTOR
REMOVED PROTECTION CIRCUITS
E 08/10/99
REMOVED ESD PROTECTION
CHANGED GROUNDING ON SHIELD
ADDED V AND I SPECS

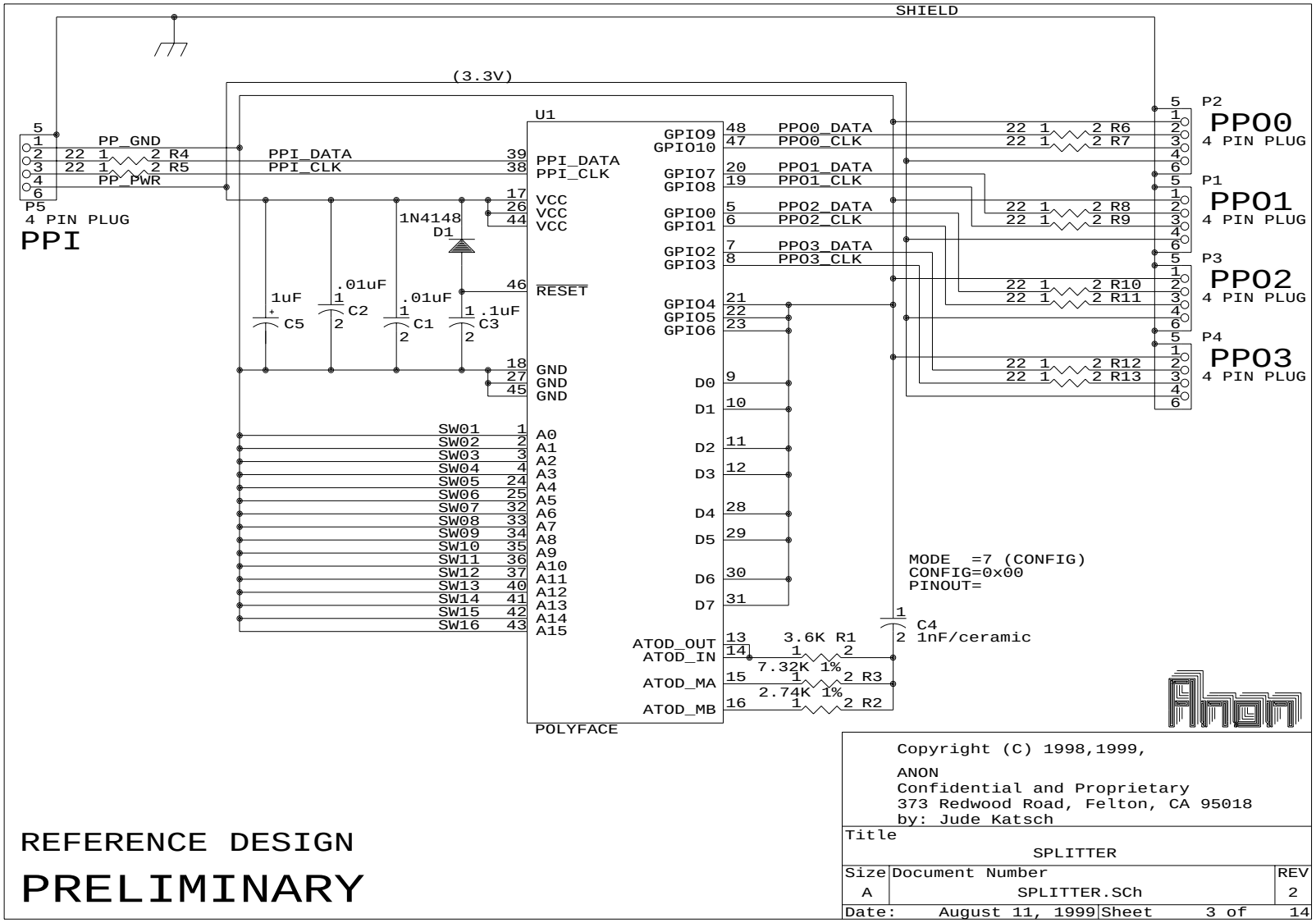
REFERENCE DESIGN
PRELIMINARY

Copyright (C) 1998,1999,

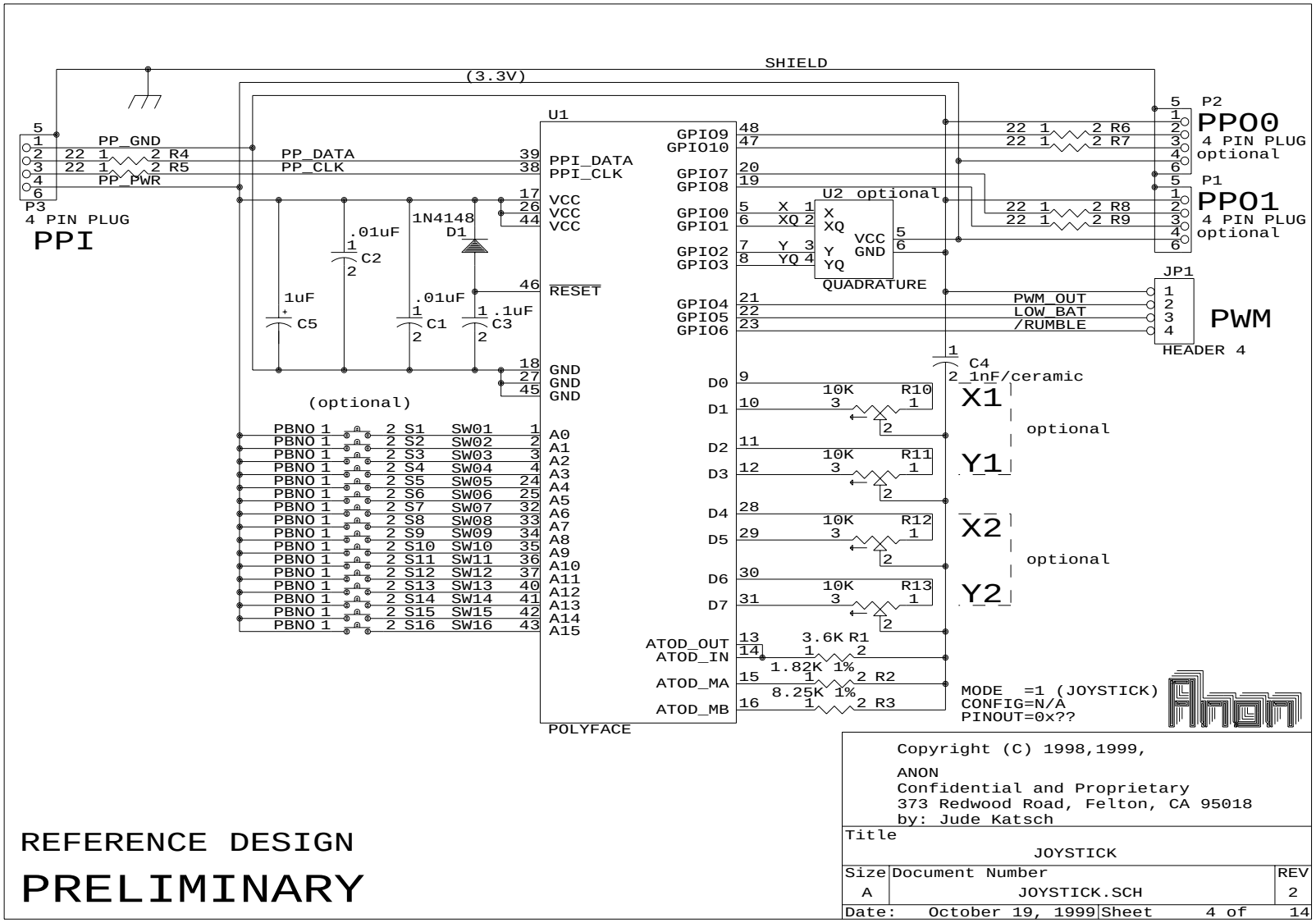
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
	CHASSIS.SCH	
Size	Document Number	REV
A	CHASSIS (ON MOTHERBOARD)	E
Date: August 10, 1999	Sheet 2 of 14	

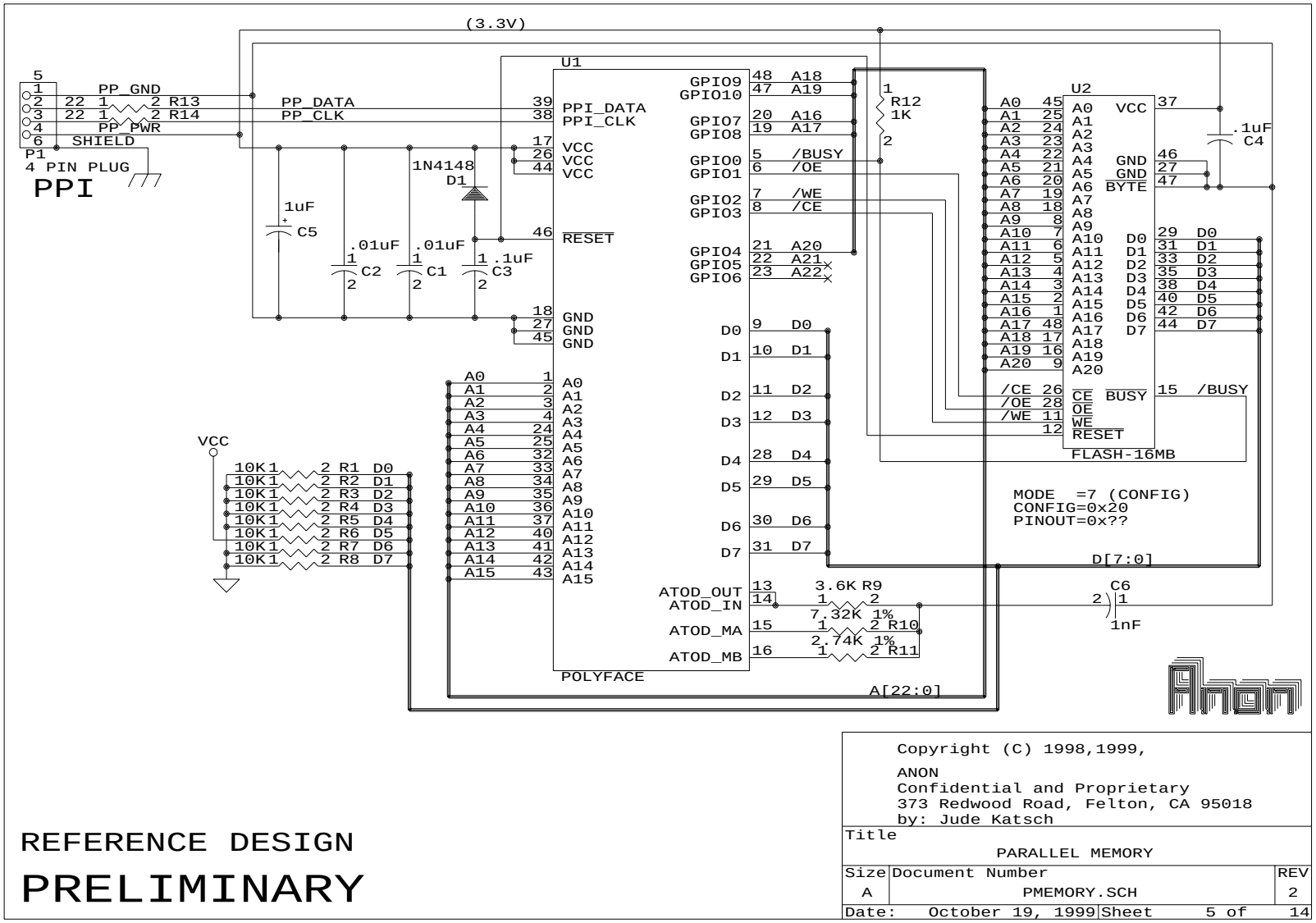
8.3 Splitter (SPLITTER)



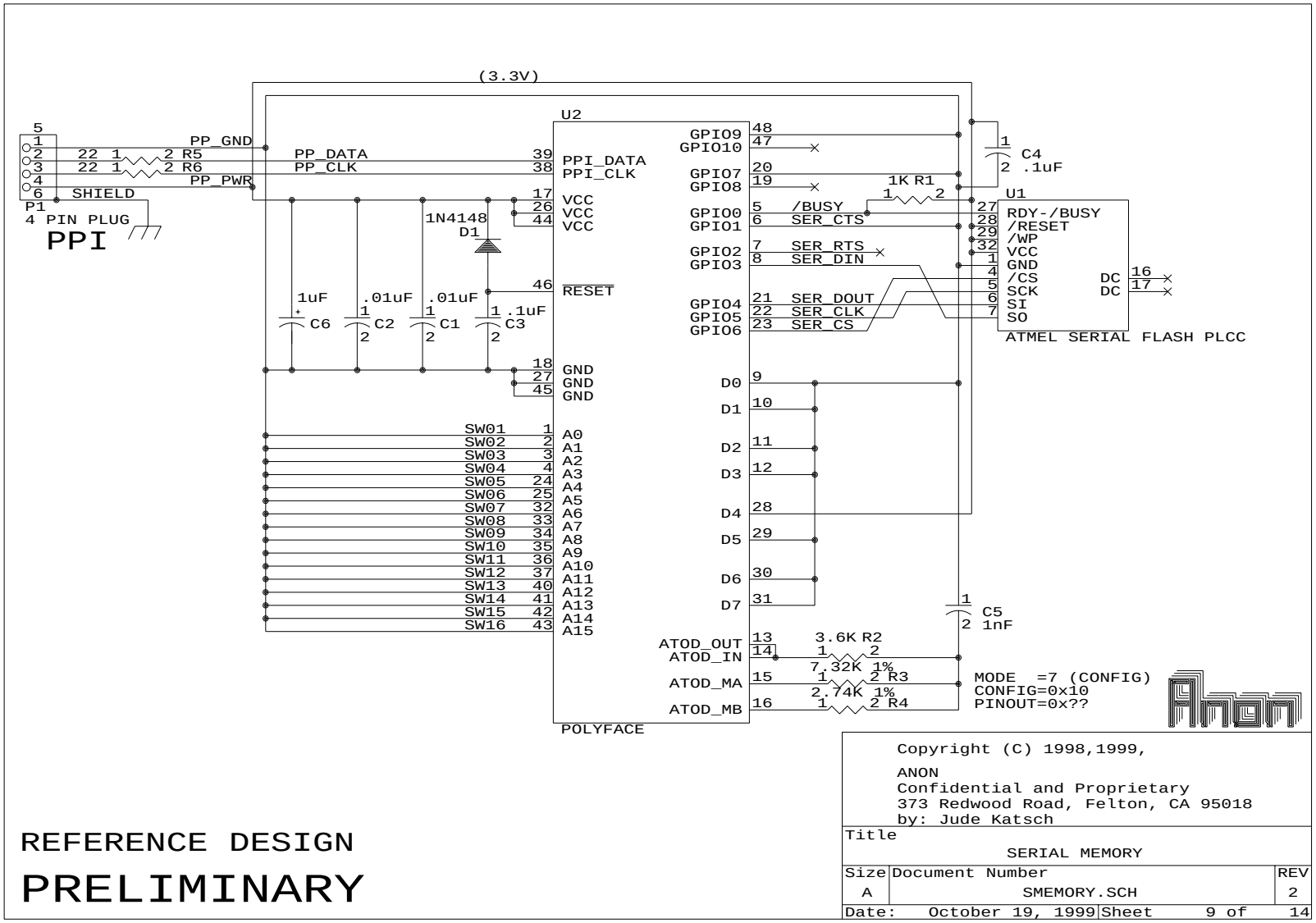
8.4 Joystick (JOYSTICK)



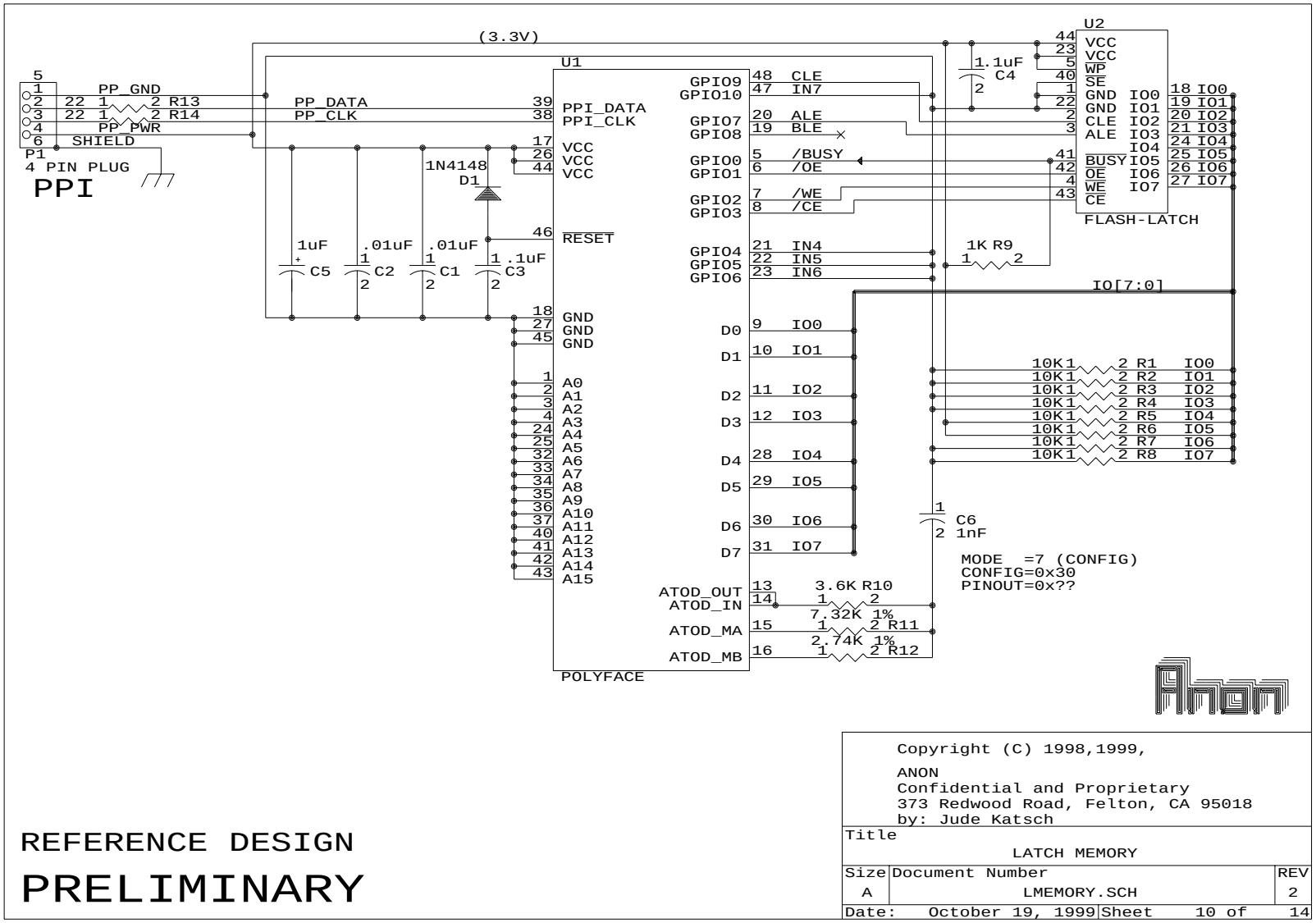
8.5 Parallel Memory (PMEMORY)



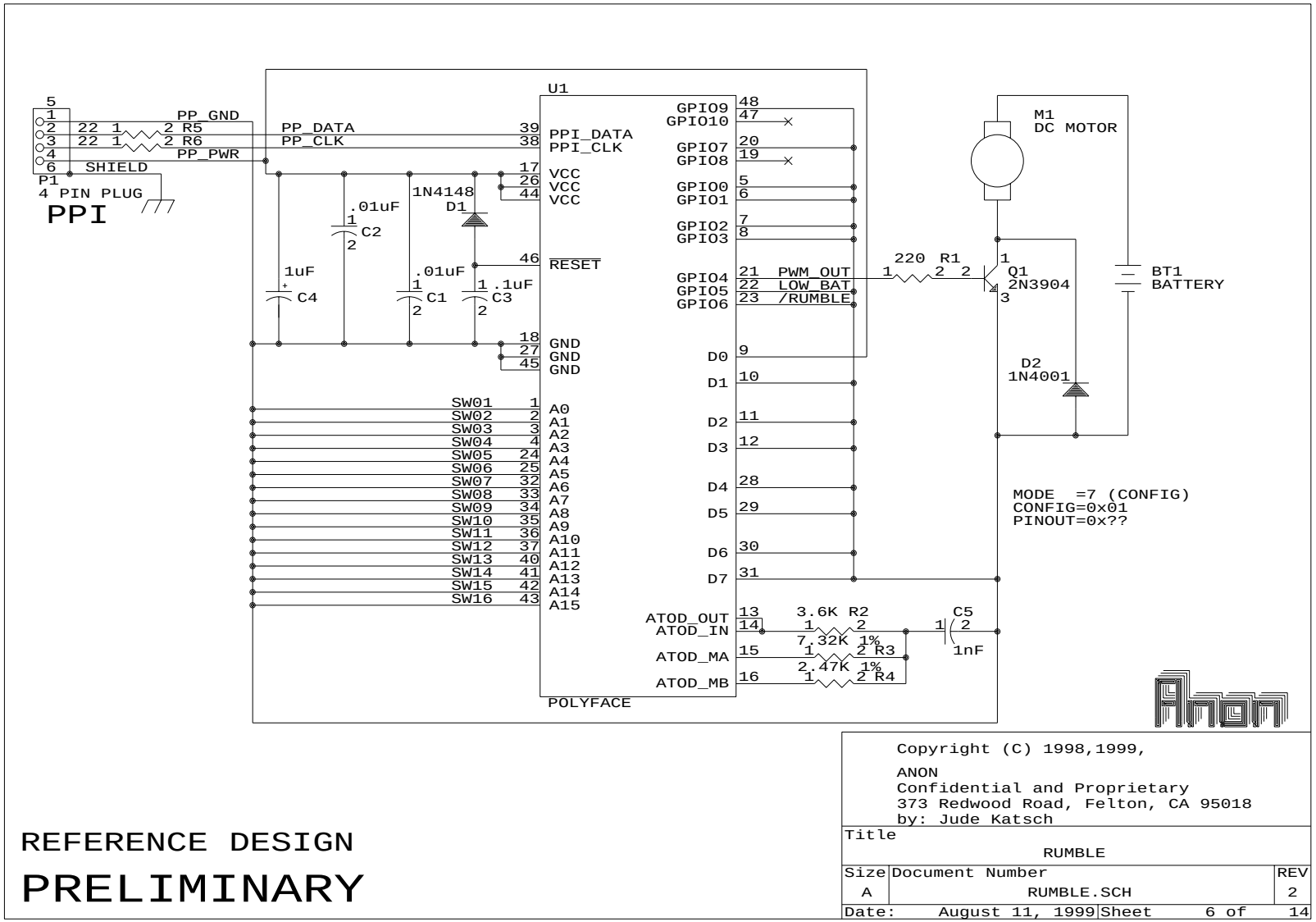
8.6 Serial Memory (SMEMORY)



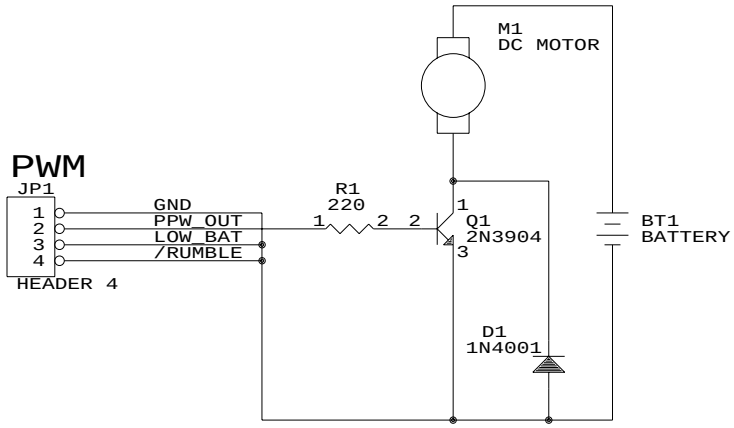
8.7 Latched Memory (LMEMORY)



8.8 Rumble (RUMBLE)



8.9 Simple Rumble (SRUMBLE)



REFERENCE DESIGN

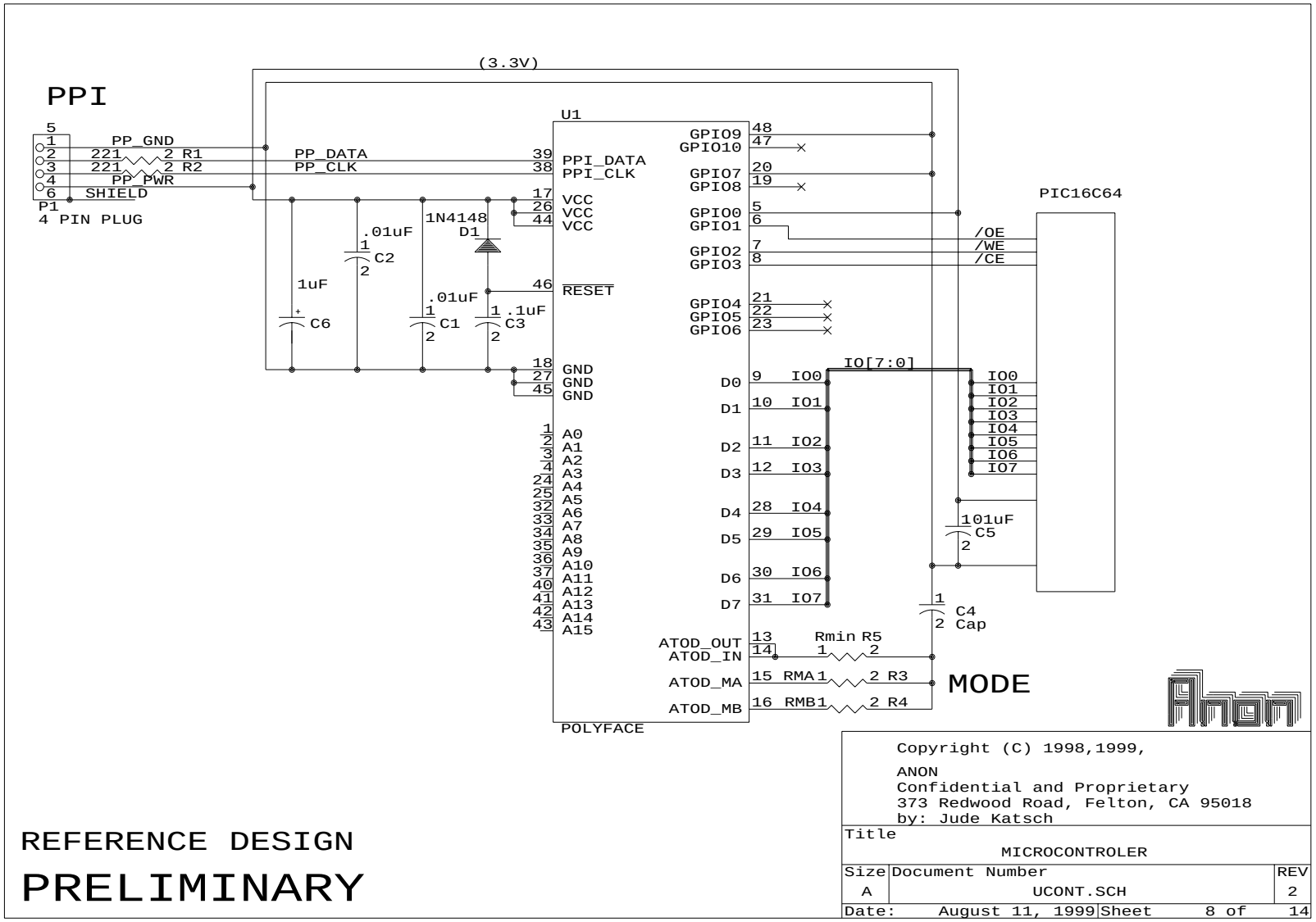
PRELIMINARY

(tie extra pins low?)

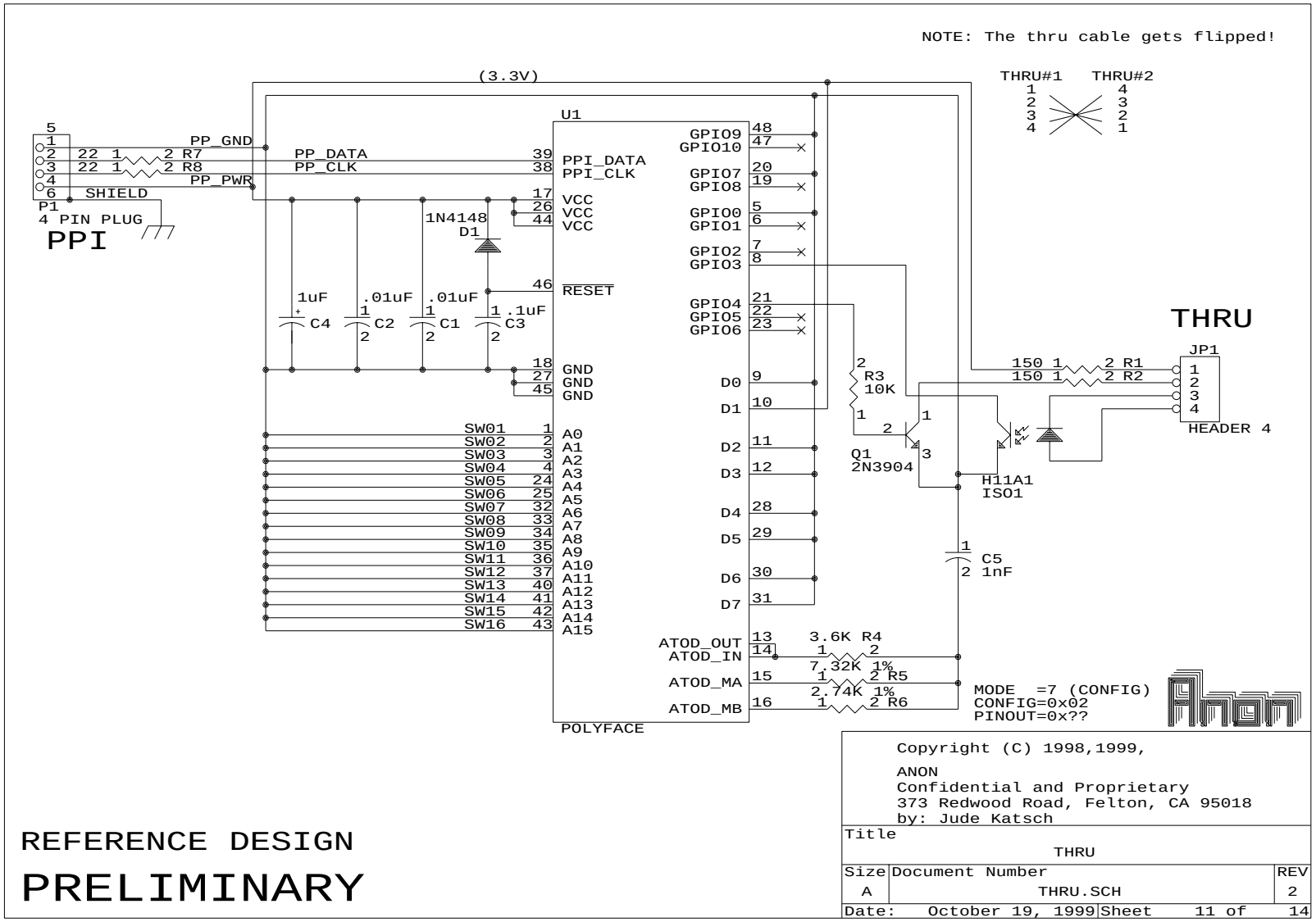
Copyright (C) 1998,1999,
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
	SIMPLE RUMBLE	
Size A	Document Number SRUMBLE.SCH	REV 2
Date: April 25, 1999	Sheet 7 of 14	

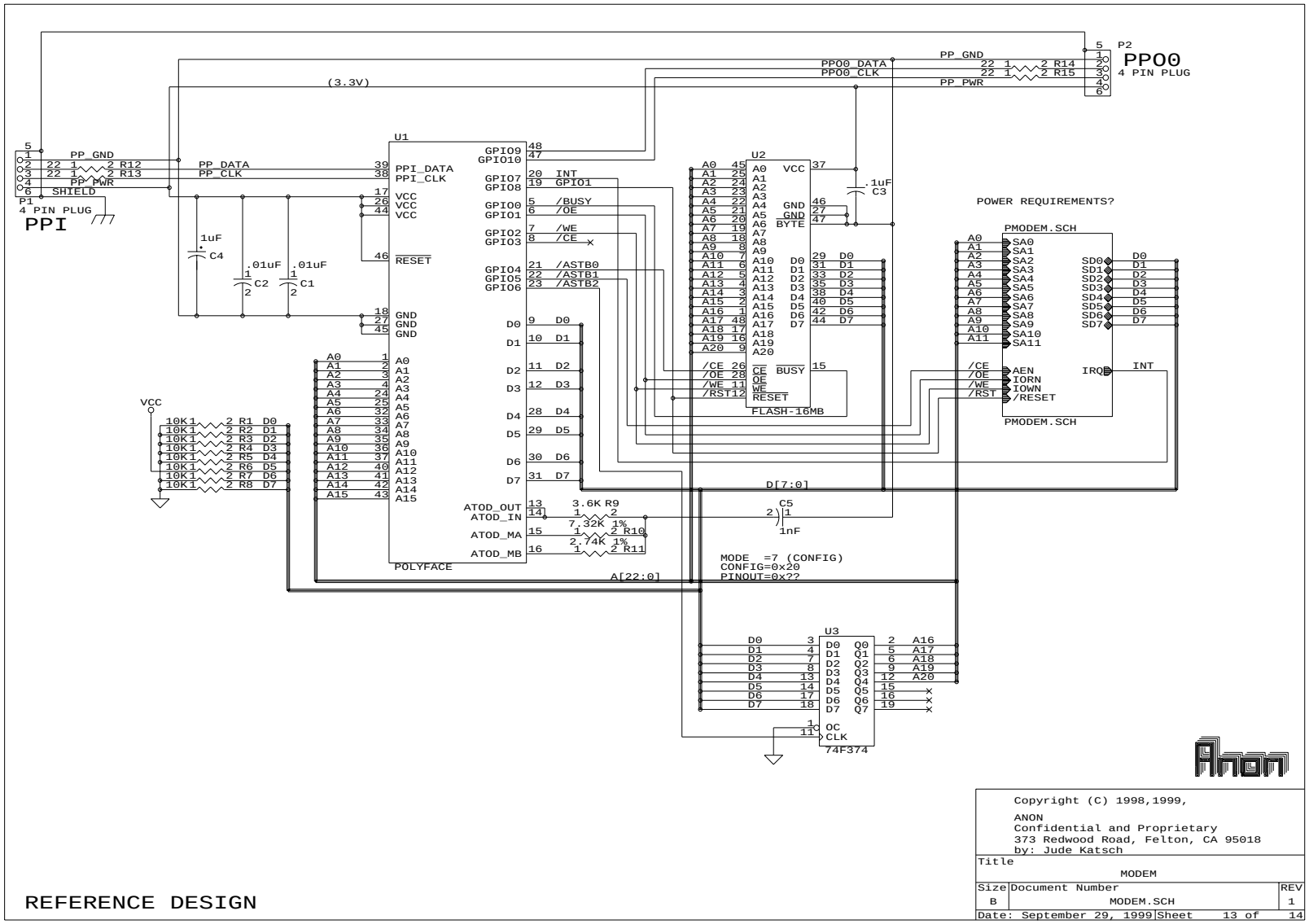
8.10 Micro Controller (UCONT)



8.11 Thru (THRU)



8.12 Modem (MODEM)



Chapter 9

POLYFACE VERILOG

Chapter 10

PPTEST

10.1 OVERVIEW

PPTEST is a board that interfaces between an EPP (Enhanced Parallel Port) and the Merlin Peripheral Port. It allows a PC to interface to Merlin Peripherals for development and testing.

10.2 VERILOG

Description	Section	Page
Peripheral Port TEST	10.2.1	59
Serial Data Bus	10.2.2	66
Serial Data Bus Data IN	10.2.3	72
FIFO 4 x 32bits	10.2.4	73
FIFO 32 x 8bits	10.2.5	75

10.2.1 Peripheral Port TEST (PPTEST)

```
// **********************************
// pptest (sdb wrapper)
// Copyright (c) 1999, Anon
// Confidential and Proprietary
// Written by Jude Katsch 01/20/99
// **********************************

module pptest(
input   clk_in;
output  clk_div2;
output  clk_div4;

input   clk_54; // in aries this is 54Mhz
input   clk_108;// in aries this is 108Mhz
input   reset_b;

// epp i/o
input   epp_write_n;    // write enable
input   epp_datastb_n;  // data strobe
input   epp_addrstb_n;  // address strobe
```

```
21 input   epp_reset_n;    // reset epp port
22 inout   epp_ad1;// epp address/data
23 inout   epp_ad2;
24 inout   epp_ad3;
25 inout   epp_ad4;
26 inout   epp_ad5;
27 inout   epp_ad6;
28 inout   epp_ad7;
29 inout   epp_ad8;
30 output  epp_wait_n;     // hold the epp port
31 output  epp_intr_n;     // interrupt
32
33 // controller port i/o
34 output  cp_clk; //controller port clock
35 output  cp_dout1;       //controller port 1 data out
36 output  cp_dout2;       //controller port 2 data out
37 input   cp_din1;//controller port 1 data in
38 input   cp_din2;//controller port 2 data in
39 output  cp_din_en;      //controller port data input enable
40
41 output  wr_data;
42 output  rd_data;
43 output  wr_add;
44 output  rd_add;
45 output  oen;
46
47 output  data_check;
48 output  data_strobe;
49
50 input   pp_data;
51 input   pp_dir;
52 input   pp_clk;
53 );
54
55 ‘include "pptest.io"
56
57 #define ANALYZER
58
59 reg pp_data_reg;
60 always @ (posedge pp_clk)
61    { : reg_pp_data_reg
62    pp_data_reg <= <DLY> pp_data;
63    }
64
65 reg pp_dir_reg;
66 always @ (posedge pp_clk)
67    { : reg_pp_dir_reg
68    pp_dir_reg <= <DLY> pp_dir;
69    }
70
71 reg pp_clk_buf1,pp_clk_buf2,pre_pp_wr,pp_wr;
72 always @ (posedge clk_54)
73    { : reg_pp_wr
74    pp_clk_buf1 <= <DLY> pp_clk;
75    pp_clk_buf2 <= <DLY> pp_clk_buf1;
76    pre_pp_wr   <= <DLY> pp_clk_buf1 & ~pp_clk_buf2;
77    pp_wr       <= <DLY> pre_pp_wr;
78    }
79
80 reg [1:0] pp_count;
81 always @ (posedge clk_54 or posedge reset_54)
82    { : reg_pp_count
83    if(reset_54)
84       pp_count <= <DLY> 0;
85    else if (pp_wr)
86       pp_count <= <DLY> pp_count+1;
```

```
   else
      pp_count <= <DLY> pp_count;
   }

wire [7:0] pre_pp_sr = {|pp_sr[5:0],pp_data_reg,pp_dir_reg|};
reg [7:0] pp_sr;
always @ (posedge clk_54)
   { : reg_pp_sr
   if(pp_wr)
      pp_sr <= <DLY> pre_pp_sr;
   else
      pp_sr <= <DLY> pp_sr;
   }

#ifdef ANALYZER
wire fifo_wr = pp_wr & &pp_count;

wire pre_fifo_full;
wire pre_fifo_empty;
wire [7:0] pre_fifo_q;
wire [4:0] pre_fifo_depth;

wire [5:0] <DLY> fifo_depth = {|pre_fifo_full,pre_fifo_depth|};
wire [7:0] <DLY> fifo_q     = pre_fifo_q;
wire       <DLY> fifo_full  = pre_fifo_full;
wire       <DLY> fifo_empty = pre_fifo_empty;

reg fifo_of;
always @ (posedge clk_54 or posedge reset_54)
   { : reg_fifo_of
   if(reset_54)
      fifo_of <= <DLY> 0;
   else
      fifo_of <= <DLY> (fifo_full & fifo_wr) | (fifo_of & ~info_rd);
   }

reg fifo_uf;
always @ (posedge clk_54 or posedge reset_54)
   { : reg_fifo_uf
   if(reset_54)
      fifo_uf <= <DLY> 0;
   else
      fifo_uf <= <DLY> (fifo_empty & fifo_rd) | (fifo_uf & ~info_rd);
   }

scfifo_32x8_ns fifo (
.Data(pre_pp_sr),
.Clock(clk_54),
.WrReq(fifo_wr),
.RdReq(fifo_rd),
.Aclr(1#b0),
.Sclr(reset_54),
.Full(pre_fifo_full),
.Empty(pre_fifo_empty),
.Q(pre_fifo_q),
.UsedW(pre_fifo_depth)
   );

wire epp_intr_n = ~(fifo_depth>16);

#endif

reg clk_div2;
reg clk_div4;

// synopsys translate_off
```

```
153 initial
154    {
155    clk_div2 =0;
156    clk_div4 =0;
157    }
158 // synopsys translate_on
159
160 always @ (posedge clk_in)
161    {
162    clk_div2 <= <DLY> ~clk_div2;
163    clk_div4 <= <DLY> (clk_div2) ? ~clk_div4 : clk_div4;
164    }
165
166 reg reset_54;
167 always @ (posedge clk_54 or negedge reset_b)
168    {
169    if(~reset_b)
170       reset_54 <= <DLY> 1;
171    else
172       reset_54 <= <DLY> 0;
173    }
174
175 reg reset_108;
176 always @ (posedge clk_108)
177    reset_108 <= <DLY> reset_54;
178
179 wire epp_reset   = ~epp_reset_n;
180 wire epp_datastb = ~epp_datastb_n;
181 wire epp_addrstb  = ~epp_addrstb_n;
182 wire epp_write   = ~epp_write_n;
183
184 assign {|epp_ad8,epp_ad7,epp_ad6,epp_ad5,epp_ad4,epp_ad3,epp_ad2,epp_ad1|} = (oen) ? data_out : 8#bzz;
185 //wire [8:1] epp_ad = {|epp_ad8,epp_ad7,epp_ad6,epp_ad5,epp_ad4,epp_ad3,epp_ad2,epp_ad1|};
186 wire [7:0] epp_ad = {|epp_ad8,epp_ad7,epp_ad6,epp_ad5,epp_ad4,epp_ad3,epp_ad2,epp_ad1|};
187
188 reg epp_addrstb_dly;
189 reg epp_datastb_dly;
190 always @ (posedge clk_54)
191    {
192    epp_addrstb_dly <= <DLY> epp_addrstb;
193    epp_datastb_dly <= <DLY> epp_datastb;
194    }
195
196 reg rd_data;
197 reg wr_data;
198 reg rd_add ;
199 reg wr_add ;
200 reg done   ;
201 reg oen    ;
202
203 always @ (posedge clk_54 or posedge reset_54)
204    {
205    if(reset_54)
206       {
207       rd_data <= <DLY> 0;
208       wr_data <= <DLY> 0;
209       rd_add  <= <DLY> 0;
210       wr_add  <= <DLY> 0;
211       done    <= <DLY> 0;
212       oen     <= <DLY> 0;
213       }
214    else
215       {
216       rd_data <= <DLY> 0;
217       wr_data <= <DLY> 0;
218       rd_add  <= <DLY> 0;
```

```
219        wr_add  <= <DLY> 0;
220        oen     <= <DLY> oen;
221        done    <= <DLY> done;
222        casex({|epp_reset,epp_write,epp_datastb_dly,epp_addrstb_dly,done,cycle,pre_pp_wr|})
223           7#b 0101_00_x:  wr_add  <= <DLY> 1;
224           7#b 0001_00_0: {rd_add  <= <DLY> 1; oen <= <DLY> 1;}
225           7#b 0110_00_x:  wr_data <= <DLY> 1;
226           7#b 0010_00_x: {rd_data <= <DLY> 1; oen <= <DLY> 1;}
227           7#b 0xxx_01_x:  done    <= <DLY> 1;
228           7#b 1xxx_xx_x,
229           7#b 0x00_10_x:  {done    <= <DLY> 0; oen <= <DLY> 0;}
230        endcase;
231        }
232     }
233
234 wire fifo_rd = rd_add;
235 wire cycle = rd_data | wr_data | rd_add | wr_add;
236
237 reg epp_wait_n;
238 always @ (posedge clk_54)
239     epp_wait_n <= <DLY> done;
240
241 reg [31:0] wdata;
242 always @ (posedge clk_54)
243     {
244     wdata <= <DLY> wdata;
245     if(wr_data)
246        {
247        wdata [31:24] <= <DLY> epp_ad;
248        wdata [23:16] <= <DLY> wdata [31:24];
249        wdata [15: 8] <= <DLY> wdata [23:16];
250        wdata [ 7: 0] <= <DLY> wdata [15: 8];
251        }
252     }
253
254 reg ctrl_cmd1_wr;
255 reg ctrl_data1_wr;
256 reg ctrl_stat1_wr;
257 reg ctrl_data1_rd;
258 reg ctrl_stat1_rd;
259 reg clr_int1;
260 reg ctrl_cmd2_wr;
261 reg ctrl_data2_wr;
262 reg ctrl_stat2_wr;
263 reg ctrl_data2_rd;
264 reg ctrl_stat2_rd;
265 reg clr_int2;
266 reg test_rd;
267 reg info_rd;
268
269 always @ (posedge clk_54)
270     {
271     ctrl_cmd1_wr    <= <DLY> 0;
272     ctrl_data1_wr   <= <DLY> 0;
273     ctrl_stat1_wr   <= <DLY> 0;
274     ctrl_data1_rd   <= <DLY> 0;
275     ctrl_stat1_rd   <= <DLY> 0;
276     ctrl_cmd2_wr    <= <DLY> 0;
277     ctrl_data2_wr   <= <DLY> 0;
278     ctrl_stat2_wr   <= <DLY> 0;
279     ctrl_data2_rd   <= <DLY> 0;
280     ctrl_stat2_rd   <= <DLY> 0;
281     test_rd         <= <DLY> 0;
282     info_rd         <= <DLY> 0;
283
284     case({|wr_add,epp_ad|})
```

```
      9#b 1_00_0_1_0000:  ctrl_cmd1_wr    <= <DLY> 1;
      9#b 1_00_0_1_0001:  ctrl_data1_wr   <= <DLY> 1;
      9#b 1_00_0_1_0010:  ctrl_stat1_wr   <= <DLY> 1;

      9#b 1_00_0_0_0001:  ctrl_data1_rd   <= <DLY> 1;
      9#b 1_00_0_0_0010:  ctrl_stat1_rd   <= <DLY> 1;
      9#b 1_00_0_0_0011:  test_rd         <= <DLY> 1;

      9#b 1_00_0_1_0011:  clr_int1        <= <DLY> 1;

      9#b 1_00_1_1_0000:  ctrl_cmd2_wr    <= <DLY> 1;
      9#b 1_00_1_1_0001:  ctrl_data2_wr   <= <DLY> 1;
      9#b 1_00_1_1_0010:  ctrl_stat2_wr   <= <DLY> 1;

      9#b 1_00_1_0_0001:  ctrl_data2_rd   <= <DLY> 1;
      9#b 1_00_1_0_0010:  ctrl_stat2_rd   <= <DLY> 1;
      9#b 1_00_1_0_0011:  info_rd         <= <DLY> 1;

      9#b 1_00_1_1_0011:  clr_int2        <= <DLY> 1;
   endcase
   }

wire [31:0] ctrl_stat1_data;
wire [31:0] ctrl_rdata1;
wire [31:0] ctrl_stat2_data;
wire [31:0] ctrl_rdata2;
reg [31:0] rdata;
reg [7:0] data_out;

always @ (posedge clk_54)
   {
   rdata <= <DLY> rdata;
   data_out <= <DLY> data_out;
   case(1#b1)
      rd_data:       { data_out <= <DLY> rdata[7:0]; rdata <= <DLY> {|rdata[7:0],rdata[31:8]|}; }
      ctrl_stat1_rd: { rdata <= <DLY> ctrl_stat1_data | (int1<<30); }
      ctrl_data1_rd: { rdata <= <DLY> fifo_rdata1; }
      ctrl_stat2_rd: { rdata <= <DLY> ctrl_stat2_data | (int2<<30); }
      ctrl_data2_rd: { rdata <= <DLY> fifo_rdata2; }
      test_rd      : { rdata <= <DLY> wdata; }
#ifdef ANALYZER
      fifo_rd      : { data_out <= <DLY> fifo_q; }
      info_rd      : { rdata <= <DLY> 32#h00000000 | (fifo_of<<17) | (fifo_uf<<16) | fifo_depth; }
#endif
   endcase;
   }

wire [31:0] fifo_rdata1;
scfifo_4x32_sh rdata1 (
.Data(ctrl_rdata1),
.Clock(clk_54),
.WrReq(ctrl_rdata1_rd),
.RdReq(ctrl_data1_rd),
.Aclr(1#b0),
.Sclr(reset_54),
.Full(fifo_rdata1_full),
.Empty(fifo_rdata1_empty),
.Q(fifo_rdata1)
//      .UsedW(pre_fifo_depth)
   );

wire int1 = ~fifo_rdata1_empty;

reg ctrl_rdata1_rd;
always @ (posedge clk_54)
   {
```

```
351    ctrl_rdata1_rd <= <DLY> 0;
352    if(ctrl_stat1_data[30] & ~fifo_rdata1_full & ~ctrl_rdata1_rd)
353       ctrl_rdata1_rd <= <DLY> 1;
354    }
355
356
357 #ifdef CHAN2
358 wire [31:0] fifo_rdata2;
359 scfifo_4x32_sh rdata2 (
360 .Data(ctrl_rdata2),
361 .Clock(clk_54),
362 .WrReq(ctrl_rdata2_rd),
363 .RdReq(ctrl_data2_rd),
364 .Aclr(1#b0),
365 .Sclr(reset_54),
366 .Full(fifo_rdata2_full),
367 .Empty(fifo_rdata2_empty),
368 .Q(fifo_rdata2)
369 //      .UsedW(pre_fifo_depth)
370    );
371 #else
372 wire [31:0] fifo_rdata2=ctrl_rdata2;
373 wire fifo_rdata2_full=0;
374 wire fifo_rdata2_empty=0;
375 #endif
376
377 wire int2 = ~fifo_rdata2_empty;
378
379 reg ctrl_rdata2_rd;
380 always @ (posedge clk_54)
381    {
382    ctrl_rdata2_rd <= <DLY> 0;
383    if(ctrl_stat1_data[30] & ~fifo_rdata2_full & ~ctrl_rdata2_rd)
384       ctrl_rdata2_rd <= <DLY> 1;
385    }
386
387 //wire epp_intr_n = ~(int1 | int2);
388
389 sdb sdb (
390 .clk(clk_54),
391 .reset_b(~reset_54),
392 .clk_108(clk_108),
393 .reset_108_b(~reset_108),
394
395 .gio_wdata(wdata),
396 .ctrl_cmd1_wr(ctrl_cmd1_wr),
397 .ctrl_sdata1_wr(ctrl_data1_wr),
398 .ctrl_stat1_wr(ctrl_stat1_wr),
399 .ctrl_cmd2_wr(ctrl_cmd2_wr),
400 .ctrl_sdata2_wr(ctrl_data2_wr),
401 .ctrl_stat2_wr(ctrl_stat2_wr),
402
403 .ctrl_stat1_data(ctrl_stat1_data),
404 .ctrl_rdata1(ctrl_rdata1),
405 .ctrl_stat2_data(ctrl_stat2_data),
406 .ctrl_rdata2(ctrl_rdata2),
407
408 .ctrl_rdata1_rd(ctrl_rdata1_rd),
409 .ctrl_rdata2_rd(ctrl_rdata2_rd),
410
411 .cp_clk(cp_clk),
412 .cp_dout1(cp_dout1),
413 .cp_dout2(cp_dout2),
414 .cp_din1(cp_din1),
415 .cp_din2(cp_din2),
416 .sdbdata_enable_b(cp_din_en),
```

```

.data_strobe(data_strobe),
.data_check(data_check)
);

//input [31:0]  gio_wdata;      //write data for the registers
//input clk;    //54MHz system clock
//input clk_108;
//input reset_b;//system reset
//input reset_108_b;    //reset for logic in the 108MHz clock domain
//input ctrl_cmd1_wr;   //write strobe for send-command register port 1
//input ctrl_sdata1_wr; //write strobe for send-data register port 1
//input ctrl_stat1_wr;  //write strobe for status/control register port 1
//input ctrl_cmd2_wr;   //write strobe for send-command register port 2
//input ctrl_sdata2_wr; //write strobe for send-data register port 2
//input ctrl_stat2_wr;  //write strobe for status/control register port 2
//output[31:0]  ctrl_stat1_data;//read data for the registers in this module
//output[31:0]  ctrl_rdata1;    //read data for the registers in this module
//output[31:0]  ctrl_stat2_data;//read data for the registers in this module
//output[31:0]  ctrl_rdata2;    //read data for the registers in this module
//input ctrl_rdata1_rd; //1 tick pulse indicating new data in the register (clear rfull)
//input ctrl_rdata2_rd; //1 tick pulse indicating new data in the register (clear rfull)
//outputcp_clk; //controller port clock
//outputcp_dout1;       //controller port 1 data out
//outputcp_dout2;       //controller port 2 data out
//input cp_din1;//controller port 1 data in
//input cp_din2;//controller port 2 data in
//outputsdbdata_enable_b;       //aries new signal

endmodule
```

10.2.2 Serial Data Bus (SDB)

```
//*****************************************************************************
// Confidential Property of VM Labs
// -------------------------------
// file:sdb.v
// author:      Javier Solis
// created:     19 December, 1996.
// history:
//
// Merlin Serial Device Bus Module
//
// The serial device bus module is the means of connecting to the controller
// ports in the Merlin architecture. The design consists of status/control and
// data holding registers for the two individual ports. There is also the
// circuitry by which the data is serially sent to the controller ports and
// serially received from the ports in accordance with a variable rate clock
// synchronizes the data in both directions.
//
// Aries changes. The aries changes will synchronize the operations of the 2
// ports to each other. Port 1 will be the triggering port. Different data can
// be loaded into port 2 but it will be Port 1 transactions that will initiate
// the operations. Reframing and data transfer can be done at the same time.
// If port 1 is reset, then both ports do reset. Reset to port 2 does nothing.
// A data enable signal, active low, is now generated to indicate that data is
// being transfered out. Data in can only occur when data is not being sent.
// Proper procedure is to load port 2 data then port 1 data to initiate the
// transfer.

‘timescale 1ns/10ps

module  sdb (clk,gio_wdata,reset_b,ctrl_stat1_data,ctrl_rdata1,ctrl_stat2_data,
 ctrl_rdata2,ctrl_cmd1_wr,ctrl_sdata1_wr,ctrl_stat1_wr,
```

```
32  ctrl_cmd2_wr,ctrl_sdata2_wr,ctrl_stat2_wr,
33  ctrl_rdata1_rd,ctrl_rdata2_rd,
34  cp_clk,cp_dout1,cp_dout2,cp_din1,cp_din2,clk_108,reset_108_b,
35  sdbdata_enable_b,data_check,data_strobe);
36
37 // ---------------------------------------------------------------------------
38 // Port Definitions
39 // ---------------------------------------------------------------------------
40 input   [31:0]  gio_wdata; //write data for the registers
41 input   clk;       //54MHz system clock
42 input   clk_108;
43 input   reset_b;   //system reset
44 input   reset_108_b;//reset for logic in the 108MHz clock domain
45 input   ctrl_cmd1_wr; //write strobe for send-command register port 1
46 input   ctrl_sdata1_wr; //write strobe for send-data register port 1
47 input   ctrl_stat1_wr; //write strobe for status/control register port 1
48 input   ctrl_cmd2_wr; //write strobe for send-command register port 2
49 input   ctrl_sdata2_wr; //write strobe for send-data register port 2
50 input   ctrl_stat2_wr; //write strobe for status/control register port 2
51 output  [31:0]  ctrl_stat1_data,ctrl_rdata1,ctrl_stat2_data,ctrl_rdata2;
52 //read data for the registers in this module
53 input   ctrl_rdata1_rd; //1 tick pulse indicating new data in the register
54 input   ctrl_rdata2_rd; //1 tick pulse indicating new data in the register
55 output  cp_clk; //controller port clock
56 output  cp_dout1;//controller port 1 data out
57 output  cp_dout2;//controller port 2 data out
58 input   cp_din1;//controller port 1 data in
59 input   cp_din2;//controller port 2 data in
60 output  sdbdata_enable_b; //aries new signal
61 output  data_check; // ??? judek test
62 output  data_strobe; // ??? jude test
63
64 reg     [31:0]  ctrlcmd1,ctrlsdata1,ctrlrdata1;
65 reg     [31:0]  ctrlcmd2,ctrlsdata2,ctrlrdata2;
66 reg     [31:0]  ctrlstat12;
67 reg     rdata1valid,rdata2valid;
68 reg     [33:0]  shifter1,shifter2;
69 reg     cpdin1cap,cpdin1cap_d1,cpdin2cap,cpdin2cap_d1;
70 reg     [11:0]  clk_count;
71 reg     clk_out,cpclk_rise_d1,cpclk_rise_d2;
72 reg     [31:0]  xmitdata1;
73 reg     wait4fall1,din1_sampled,din1_sampled_d1,txfull1;
74 reg     endofdata1_d1,rdata1null,rxnull1,reframe1,reset1;
75 reg     [5:0]   xmitbitcount1,recbitcount1;
76 reg     [33:0]  inshifter1;
77 reg     [13:0]  resetcount1;
78 reg     [31:0]  xmitdata2;
79 reg     wait4fall2,din2_sampled,din2_sampled_d1,txfull2;
80 reg     endofdata2_d1,rdata2null,rxnull2,reframe2,reset2;
81 reg     [5:0]   xmitbitcount2,recbitcount2;
82 reg     [33:0]  inshifter2;
83 reg     [13:0]  resetcount2;
84 reg     cmdwr1,cmdwr2,rfull1,rfull2,roflow1,roflow2;
85 reg     setrfull1,setrfull2,setfull1_d1,setfull2_d1;
86 reg     catchdata1,catchdata2;
87 // aries change
88 reg     sdbdata_enable;
89
90 wire    [31:0]  ctrl_stat1_data = {txfull1,rfull1,rxnull1,4'h0,ctrlstat12[24],1'b0,
91    ctrlstat12[22:16],roflow1,3'h0,ctrlstat12[11:0]};
92 wire    [31:0]  ctrl_stat2_data = {txfull2,rfull2,rxnull2,4'h0,ctrlstat12[24],1'b0,
93    ctrlstat12[22:16],roflow2,3'h0,ctrlstat12[11:0]};
94 wire    [31:0]  ctrl_rdata1 = ctrlrdata1;
95 wire    [31:0]  ctrl_rdata2 = ctrlrdata2;
96 wire    ctrl_rdata1_valid= rdata1valid;
97 wire    ctrl_rdata2_valid= rdata2valid;
```

```
wire    cp_clk = clk_out;
wire    cp_dout1 = shifter1[33];
wire    cp_dout2 = shifter2[33];
wire    cpclk_fall,cpclk_rise;
wire    loadshifter1,endfull1,din1_rise,endofdata1,endreset1;
wire    loadshifter2,endfull2,din2_rise,endofdata2,endreset2;
wire    catch1,catch2,setfull1,setfull2,count8638_1,count8638_2;
// aries change
wire    sdbdata_enable_b = ~sdbdata_enable;



always @(posedge clk)
begin
 cpdin1cap        <= #1 cp_din1;
 cpdin1cap_d1 <= #1 cpdin1cap;
 cpdin2cap        <= #1 cp_din2;
 cpdin2cap_d1 <= #1 cpdin2cap;
end
always @(posedge clk or negedge reset_b)
if (~reset_b) begin
ctrlstat12[31:12] <= #1 0;
ctrlstat12[11:0] <= #1 12'hfff;
  end
else
 begin
  ctrlstat12[24] <= #1 (ctrl_stat1_wr | ctrl_stat2_wr) ? gio_wdata[24] :
 ctrlstat12[24];
  ctrlstat12[22:16] <= #1 (ctrl_stat1_wr | ctrl_stat2_wr) ? gio_wdata[22:16] :
 ctrlstat12[22:16];
  ctrlstat12[11:0] <= #1 (ctrl_stat1_wr | ctrl_stat2_wr) ? gio_wdata[11:0] :
 ctrlstat12[11:0];
 end
//the rest of the resigters
always @(posedge clk)
begin
 xmitdata1  <= #1 (ctrl_cmd1_wr | ctrl_sdata1_wr) ? gio_wdata : xmitdata1 ;
 xmitdata2  <= #1 (ctrl_cmd2_wr | ctrl_sdata2_wr) ? gio_wdata : xmitdata2 ;
 ctrlrdata1 <= #1 setrfull1 ? inshifter1[31:0] : ctrlrdata1;
 ctrlrdata2 <= #1 setrfull2 ? inshifter2[31:0] : ctrlrdata2;
end
//clock timing
always @(posedge clk or negedge reset_b)
if (~reset_b) begin
clk_out <= #1 0;
clk_count <= #1 0;
  end
else
 begin
  if (ctrl_stat1_wr | ctrl_stat2_wr) clk_count <= #1 12'b0;
  else clk_count <= #1 (clk_count == ctrlstat12[11:0]) ? 0 : clk_count + 1;
  clk_out <= #1 (clk_count == ctrlstat12[11:0]) ? ~clk_out : clk_out;
 end
//detect falling edge and rising edge of clk_out
assign cpclk_fall = clk_out & (clk_count == ctrlstat12[11:0]);
assign cpclk_rise = ~clk_out & (clk_count == ctrlstat12[11:0]);
always @(posedge clk) cpclk_rise_d1 <= #1 cpclk_rise;
always @(posedge clk) cpclk_rise_d2 <= #1 cpclk_rise_d1;
//detect proper time to load xmit data. the data is in a holding register, xmitdata,
//which has to wait for a falling edge of the port clock to be loaded
always @(posedge clk or negedge reset_b)
if (~reset_b) wait4fall1 <= #1 0;
else wait4fall1 <= #1 (wait4fall1 | ctrl_cmd1_wr | ctrl_sdata1_wr |
   ((gio_wdata[28] | gio_wdata[27]) & ctrl_stat1_wr)) &
~loadshifter1;
assign loadshifter1 = wait4fall1 & cpclk_fall;
```

```
//need to save if a command or data was written
always @(posedge clk or negedge reset_b)
if (~reset_b) cmdwr1 <= #1 0;
else cmdwr1 <= #1 ~loadshifter1 & (cmdwr1 | ctrl_cmd1_wr);
//the data in pin is sampled on the rising edge of the clk
//aries change: gate the data in with the sdb data enable to ignore data going out
//that shows up in the input pin. external connection has both data out and data
//in path wired together
always @(posedge clk or negedge reset_b)
if (~reset_b) din1_sampled <= #1 0;
else din1_sampled <= #1 cpclk_rise ? cp_din1 : din1_sampled;
//now the shifter
always @(posedge clk or negedge reset_b)
if (~reset_b) shifter1 <= #1 0;
else if (loadshifter1) shifter1 <= #1 {(reset1 | ~reframe1),
  (reset1 |     (~reframe1 & cmdwr1)),
  ({32{reset1}} | ({32{~reframe1}} & xmitdata1))};
else shifter1 <= #1 cpclk_fall & (txfull1 | reframe1 | reset1) ?
{shifter1[32:0],1'b0}
: shifter1 ;
//this is an indicator that the transmit is busy
always @(posedge clk or negedge reset_b)
if (~reset_b) txfull1 <= #1 0;
else txfull1 <= ((txfull1 | ctrl_cmd1_wr | ctrl_sdata1_wr |
 ((gio_wdata[28] | gio_wdata[27]) & ctrl_stat1_wr)) &
 ~endfull1);
//endfull detects that 34 rising edges of the port clock have occured since the
//loading of the xmit data
always @(posedge clk or negedge reset_b)
if (~reset_b) xmitbitcount1 <= #1 0;
else if (~txfull1) xmitbitcount1 <= #1 0;
 else xmitbitcount1 <= #1 (cpclk_rise & ~wait4fall1) ? xmitbitcount1 + 1 :
        xmitbitcount1;
assign endfull1 = (xmitbitcount1 == 35);
//now for the reframe feature. cause a regular load and gate the data loaded
//to the shifter to be all 0's
always @(posedge clk or negedge reset_b)
if (~reset_b) reframe1 <= #1 0;
else reframe1 <= #1 ((gio_wdata[28] & ctrl_stat1_wr) | reframe1) & ~loadshifter1;
//the reset will also cause an initial load of all 1's and then count 8670 falling
//edges of the port clock
always @(posedge clk or negedge reset_b)
if (~reset_b) reset1 <= #1 0;
else reset1 <= #1 ((gio_wdata[27] & ctrl_stat1_wr) | reset1) & ~endfull1;
//this is the means for counting the 8670 falling edges
//aries change: this is no longer needed since reset is now the same length as all
// transfers
//always @(posedge clk or negedge reset_b)
//      if (~reset_b) resetcount1 <= #1 0;
//      else if (~reset1) resetcount1 <= #1 0;
//       else resetcount1 <= #1 cpclk_rise_d2 ? resetcount1 + 1 : resetcount1;
//assign endreset1 = (resetcount1 == 8672);
//assign count8638_1 = (resetcount1 >= 8638);
//
//receive data stuff
//
//now detect the rising edge of the data in and count 34 falling edges of the
//port clock. this will indicate that data is now ready to load

sdb_din sdbdin1(.clk(clk), .reset_b(reset_108_b), .clk_108(clk_108),
.period(ctrlstat12[11:0]), .din(cp_din1 & ~sdbdata_enable),
.setrfull(setfull1), .catch_data(catch1));

always @(posedge clk)
begin
 setfull1_d1 <= #1 setfull1;
```

```
 setrfull1 <= #1 setfull1_d1;
 catchdata1 <= #1 catch1;
end
//the receive full is set only after the loading the shifted-in data, this is
//marked by the setrfull signal
always @(posedge clk) inshifter1 <= #1 catchdata1 ?
{inshifter1[32:0],cpdin1cap} :
inshifter1;

wire data_check = inshifter1[0]; //???
wire data_strobe=catchdata1;

//this is the 33rd bit that indicates a null or valid data received
always @(posedge clk or negedge reset_b)
if (~reset_b) rdata1null <= #1 0;
else rdata1null <= #1 setrfull1 ? inshifter1[32] : rdata1null;
//here is the 1 tick pulse that indicates valid data
always @(posedge clk) rdata1valid <= #1 setrfull1;
//this next bit decodes if the data received was a null packet
always @(posedge clk or negedge reset_b)
if (~reset_b) rxnull1 <= #1 0;
else rxnull1 <= #1 rdata1valid ? rdata1null & (ctrlrdata1 == 32'h00000000) :
 rxnull1;
//this is the receive full flag
always @(posedge clk or negedge reset_b)
if (~reset_b) rfull1 <= #1 0;
else rfull1 <= (rfull1 | rdata1valid) & ~ctrl_rdata1_rd;
//this is the overrun flag
always @(posedge clk or negedge reset_b)
if (~reset_b) roflow1 <= #1 0;
else roflow1 <= (roflow1 | (rfull1 & rdata1valid)) & ~(gio_wdata[15] & ctrl_stat1_wr);
//************************************
//from here on the circuit is repeated for port 2.
//************************************
//detect proper time to load xmit data. the data is in a holding register, xmitdata,
//which has to wait for a falling edge of the port clock to be loaded
always @(posedge clk or negedge reset_b)
if (~reset_b) wait4fall2 <= #1 0;
else wait4fall2 <= #1 (wait4fall2 | ctrl_cmd2_wr | ctrl_sdata2_wr |
   ((gio_wdata[28] | gio_wdata[27]) & ctrl_stat2_wr)) &
~loadshifter2;
assign loadshifter2 = wait4fall2 & cpclk_fall;
//need to save if a command or data was written
//aries change: clear this signal only after the port 1 load signal has been
//issued.
always @(posedge clk or negedge reset_b)
if (~reset_b) cmdwr2 <= #1 0;
else cmdwr2 <= #1 ~loadshifter1 & (cmdwr2 | ctrl_cmd2_wr);
//the data in pin is sampled on the rising edge of the clk
//aries change: gate the data in to ignore what we are transmiting
always @(posedge clk or negedge reset_b)
if (~reset_b) din2_sampled <= #1 0;
else din2_sampled <= #1 cpclk_rise ? (cp_din2 & ~sdbdata_enable) : din2_sampled;
//now the shifter
//aries change: make most triggering events to the port 1 signals. the reframe2
//signal remains since it can be set independently of port 1 data.
always @(posedge clk or negedge reset_b)
if (~reset_b) shifter2 <= #1 0;
else if (loadshifter1) shifter2 <= #1 {(reset1 | ~reframe2),
  (reset1 |     (~reframe2 & cmdwr2)),
  ({32{reset1}} | ({32{~reframe2}} & xmitdata2))};
else shifter2 <= #1 cpclk_fall & (txfull1 | reframe2 | reset1) ?
{shifter2[32:0],1'b0}
: shifter2 ;
//this is an indicator that the transmit is busy
//aries change: this signal, although modified to use port 1 signals, is not
```

```
296 //used since port 1 semaphores can be used.
297 always @(posedge clk or negedge reset_b)
298 if (~reset_b) txfull2 <= #1 0;
299 else txfull2 <= ((txfull2 | ctrl_cmd2_wr | ctrl_sdata2_wr |
300  ((gio_wdata[28] | gio_wdata[27]) & ctrl_stat2_wr) |
301  (gio_wdata[27] & ctrl_stat1_wr)) &
302  ~endfull1) ;
303 //endfull detects that 34 rising edges of the port clock have occured since the
304 //loading of the xmit data
305 //aries change: this signal, although modified to use port 1 signals, is not
306 //used since port 1 semaphores can be used.
307 always @(posedge clk or negedge reset_b)
308 if (~reset_b) xmitbitcount2 <= #1 0;
309 else if (~txfull1) xmitbitcount2 <= #1 0;
310  else xmitbitcount2 <= #1 (cpclk_rise & ~wait4fall1) ? xmitbitcount2 + 1 :
311         xmitbitcount2;
312 assign endfull2 = (xmitbitcount2 == 35);
313 //now for the reframe feature. cause a regular load and gate the data loaded
314 //to the shifter to be all 0's
315 //aries change: use port 1 load shifter signal to clear this signal.
316 always @(posedge clk or negedge reset_b)
317 if (~reset_b) reframe2 <= #1 0;
318 else reframe2 <= #1 ((gio_wdata[28] & ctrl_stat2_wr) | reframe2) & ~loadshifter1;
319 //the reset will also cause an initial load of all 1's and then count 8670 falling
320 //edges of the port clock
321 //areies change: although not used the reset signal is now port 1's endreset
322 always @(posedge clk or negedge reset_b)
323 if (~reset_b) reset2 <= #1 0;
324 else reset2 <= #1 ((gio_wdata[27] & ctrl_stat2_wr) | reset2) & ~endfull1;
325 //this is the means for counting the 8670 falling edges
326 //aries change: this is no longer needed since reset is now the same length as all
327 // transfers
328 //always @(posedge clk or negedge reset_b)
329 //      if (~reset_b) resetcount2 <= #1 0;
330 //      else if (~reset1) resetcount2 <= #1 0;
331 //       else resetcount2 <= #1 cpclk_rise_d2 ? resetcount2 + 1 : resetcount2;
332 //assign endreset2 = (resetcount2 == 8672);
333 //assign count8638_2 = (resetcount2 >= 8638);
334 //
335 //receive data stuff
336 //
337 //now detect the rising edge of the data in and count 34 falling edges of the
338 //port clock. this will indicate that data is now ready to load
339
340 sdb_din sdbdin2(.clk(clk), .reset_b(reset_108_b), .clk_108(clk_108),
341 .period(ctrlstat12[11:0]), .din(cp_din2 & ~sdbdata_enable),
342 .setrfull(setfull2), .catch_data(catch2));
343 always @(posedge clk)
344 begin
345  setfull2_d1 <= #1 setfull2;
346  setrfull2 <= #1 setfull2_d1;
347  catchdata2 <= #1 catch2;
348 end
349 //the receive full is set only after the loading the shifted-in data, this is
350 //marked by the setrfull signal
351 always @(posedge clk) inshifter2 <= #1 catchdata2 ?
352 {inshifter2[32:0],cpdin2cap} :
353 inshifter2;
354 //this is the 33rd bit that indicates a null or valid data received
355 always @(posedge clk or negedge reset_b)
356 if (~reset_b) rdata2null <= #1 0;
357 else rdata2null <= #1 setrfull2 ? inshifter2[32] : rdata2null;
358 //here is the 1 tick pulse that indicates valid data
359 always @(posedge clk) rdata2valid <= #1 setrfull2;
360 //this next bit decodes if the data received was a null packet
361 always @(posedge clk or negedge reset_b)
```

```
if (~reset_b) rxnull2 <= #1 0;
else rxnull2 <= #1 rdata2valid ? rdata2null & (ctrlrdata2 == 32'h00000000) :
 rxnull2;

//this is the receive full flag
always @(posedge clk or negedge reset_b)
if (~reset_b) rfull2 <= #1 0;
else rfull2 <= (rfull2 | rdata2valid) & ~ctrl_rdata2_rd;
//this is the overrun flag
always @(posedge clk or negedge reset_b)
if (~reset_b) roflow2 <= #1 0;
else roflow2 <= (roflow2 | (rfull2 & rdata2valid)) & ~(gio_wdata[15] & ctrl_stat2_wr);

// aries changes
// this is a new signal that indicates that data is being transfered
always @(posedge clk or negedge reset_b)
if (~reset_b) sdbdata_enable <= #1 0;
else sdbdata_enable <= #1 (loadshifter1 | sdbdata_enable) & ~endfull1;



endmodule
```

10.2.3 Serial Data Bus Data IN (SDB_DIN)

```
//*****************************************************************************
// Confidential Property of VM Labs
// --------------------------------
// file:sdb_din.v
// author:      Javier Solis
// created:     31 December, 1996.
// history:
//
// Merlin Serial Device Bus Data In Module
//
// The receive data for the sdb controller ports will function off a 108 MHz
// clock. When data comes in the rising edge of the start bit will serve as a
// mark to indicate when to start sampling data. Normally one would sample at
// rising edge of the bit clock that is sent out, but due to the edge shifting
// through all the devices that may be out there it is best to sense the start
// and use that as a mark to sample the next bits in the stream

`timescale 1ns/10ps

module  sdb_din (clk,reset_b,clk_108,period,din,setrfull,catch_data);

// ---------------------------------------------------------------------------
// Port Definitions
// ---------------------------------------------------------------------------
input   [11:0]  period; //number of system clocks per half clock of port clock
input   clk;       //54MHz system clock
input   reset_b;   //system reset
input   clk_108;   //108 MHz clock
input   din;       //data in from serial port
output  setrfull;   //signal that indicates that data is to be loaded
output  catch_data; //signal that tells the input shifter to capture data

reg        din_cap,din_cap_d1,clk_cap,get,sampletime;
reg        catchdata,back2back,rfull;
reg [12:0] slotcount;
reg [5:0]  bitslots;
reg [11:0] period_108;

wire       din_fall;
```

```

always @(posedge clk_108)
begin
 din_cap        <= #1 din;
 din_cap_d1 <= #1 din_cap;
 clk_cap        <= #1 clk;
 period_108 <= #1 period;
end
//this detects a rising edge on the data line (start bit)
wire frame_start = din_cap & ~din_cap_d1;
//this signal indicates that data is now being streamed in
always @(posedge clk_108 or negedge reset_b)
if (~reset_b) get <= #1 0;
else get <= (get | frame_start) & ~(back2back & din_fall);
//clock counting is done by a counter that is loaded with 2 at the start. this
//is because by the time we notice a frame start we are 2 ticks into the bit
//slot. from then on every time the count equals the period x2 (54Mhz ticks vs
//108MHz tick) the data in is sampled every other time the equal condition is met.
wire halfclk = (slotcount == {period_108,1'b1});
always @(posedge clk_108 or negedge reset_b)
if (~reset_b) slotcount <= #1 2;
else if (~get) slotcount <= #1 2;
 else if (halfclk) slotcount <= #1 0;
          else slotcount <= #1 slotcount + 1;
always @(posedge clk_108 or negedge reset_b)
if (~reset_b) sampletime <= #1 0;
else if (~get) sampletime <= #1 0;
 else sampletime <= #1 halfclk ? ~sampletime : sampletime;
//data is sampled every other half period and in order to sync to the system clock
//it need to be sampled if the system clock is hi.
wire bit_sampletime = ~sampletime & (slotcount[12:1] == period_108);
always @(posedge clk_108) catchdata <= #1 ~clk_cap ? bit_sampletime : catchdata;
/* put this in other module so that only one clock domain is delt with
//resync to system clock
always @(posedge clk) catchdata_54 <= #1 catchdata;
*/
wire catch_data = catchdata;
//now to keep track of how many bit slots have been done
wire force0 = (bitslots == 33);
always @(posedge clk_108 or negedge reset_b)
if (~reset_b) bitslots <= #1 0;
else if (~get) bitslots <= #1 0;
 else bitslots <= #1 (halfclk & sampletime) ? (force0 ? 0 : bitslots + 1) : bitslots;
wire setb2b = (bitslots == 33) & bit_sampletime;
assign din_fall = ~din_cap & ~sampletime;
wire clearb2b = (bitslots == 0) & bit_sampletime;
always @(posedge clk_108 or negedge reset_b)
if (~reset_b) back2back <= #1 0;
else back2back <= #1 (back2back | setb2b) & ~clearb2b;
//setrfull indicates when to load the received data
always @(posedge clk_108) rfull <= #1 ~clk_cap ? setb2b : rfull ;
/* put this in other module so that only one clock domain is delt with
//now to resync to the system clock
always @(posedge clk) rfull_54 <= #1 rfull;
//and delay one more tick to not race with the catch signal
always @(posedge clk) rfull_d1_54 <= #1 rfull_54;
*/
wire setrfull = rfull;

endmodule
```

10.2.4 FIFO 4 x 32bits (SCFIFO_4X32_SH)

```
/*****************************************************************************
```

```
 2 * Copyright (c) 1996-1998 Altera Corporation.  All rights reserved.  This text     *
 3 * contains proprietary, confidential information of Altera Corporation, and      *
 4 * may be used, copied, and/or disclosed only pursuant to the terms of a          *
 5 * valid software license agreement with Altera Corporation.  This copyright     *
 6 * notice must be retained as part of this text at all times.                     *
 7 * Model generated by Genmem Version 2.7, on Mon Jun 14 08:39:12 1999
 8 ******************************************************************************/
 9 // synopsys translate_off
10 `delay_mode_path
11 // synopsys translate_on
12 `timescale 1ns / 1ns
13 `celldefine
14 module scfifo_4x32_sh (Data, Clock, WrReq, RdReq, Aclr, Sclr, Full, Empty, Q, UsedW);
15
16 parameter Width = 32;
17 parameter WidthAd = 2;
18 parameter NumWords = 4;
19
20 input [Width-1:0] Data;
21 input Clock;
22 input WrReq;
23 input RdReq;
24 input Aclr;
25 input Sclr;
26 output Full;
27 output Empty;
28 output [Width-1:0] Q;
29 output [WidthAd-1:0] UsedW;
30
31
32 // synopsys translate_off
33 // internal reg
34 reg [Width-1:0] mem_data [NumWords-1:0];
35 reg [Width-1:0] tmp_q1;
36 reg [Width-1:0] ZERO, UNKNOWN;
37 reg [WidthAd:0] status_ptr;
38 reg [WidthAd-1:0] read_ptr, write_ptr;
39 integer i;
40 reg empty_tmp, full_tmp;
41
42 initial
43 begin
44    for (i=0; i < Width; i=i+1)
45       begin
46           ZERO[i] = 1'b0;
47           UNKNOWN[i] = 1'bX;
48       end
49
50    for(i = 0; i < NumWords; i=i+1)
51          mem_data[i] = ZERO;
52
53    empty_tmp = 1;
54    full_tmp = 0;
55    read_ptr = 0;
56    write_ptr = 0;
57    status_ptr = 0;
58    tmp_q1 = {Width{1'b0}};
59 end
60
61   always @(posedge Aclr)
62      begin
63         tmp_q1 = UNKNOWN;
64         full_tmp = 0;
65         empty_tmp  = 1;
66         read_ptr = {WidthAd{1'b0}};
67         write_ptr = {WidthAd{1'b0}};
```

```
68          status_ptr = {WidthAd{1'b0}};
69       end
70
71    always @( posedge Clock )
72    begin
73       if ((Aclr || Sclr) !== 'b1)
74       begin
75         if (WrReq && (!full_tmp))
76            begin
77              mem_data[write_ptr] = Data;
78              write_ptr = write_ptr + 1;
79              status_ptr = status_ptr + 1;
80              if (empty_tmp == 'b1)
81                 tmp_q1 <= mem_data[read_ptr];
82            end
83         if (RdReq && (!empty_tmp))
84            begin
85              read_ptr = read_ptr + 1;
86              status_ptr = status_ptr - 1;
87              if (status_ptr == 0)
88                 tmp_q1 <= UNKNOWN;
89              else
90                 tmp_q1 <= mem_data[read_ptr];
91            end
92         if (status_ptr == 0)
93           begin
94             empty_tmp = 1;
95             full_tmp = 0;
96           end else
97         if (status_ptr == NumWords)
98           begin
99             empty_tmp = 0;
100            full_tmp = 1;
101        end else
102          begin
103            empty_tmp = 0;
104            full_tmp = 0;
105          end
106      end else
107      if (Sclr === 'b1)
108      begin
109         tmp_q1 = UNKNOWN;
110         full_tmp = 0;
111         empty_tmp  = 1;
112         read_ptr = {WidthAd{1'b0}};
113         write_ptr = {WidthAd{1'b0}};
114         status_ptr = {WidthAd{1'b0}};
115      end
116   end
117
118
119    assign Q = tmp_q1;
120    assign Full = full_tmp;
121    assign Empty = empty_tmp;
122    assign UsedW = status_ptr;
123 // synopsys translate_on
124 endmodule // scfifo_4x32_sh
125 'endcelldefine
```

10.2.5 FIFO 32 x 8bits (SCFIFO_32X8_NS)

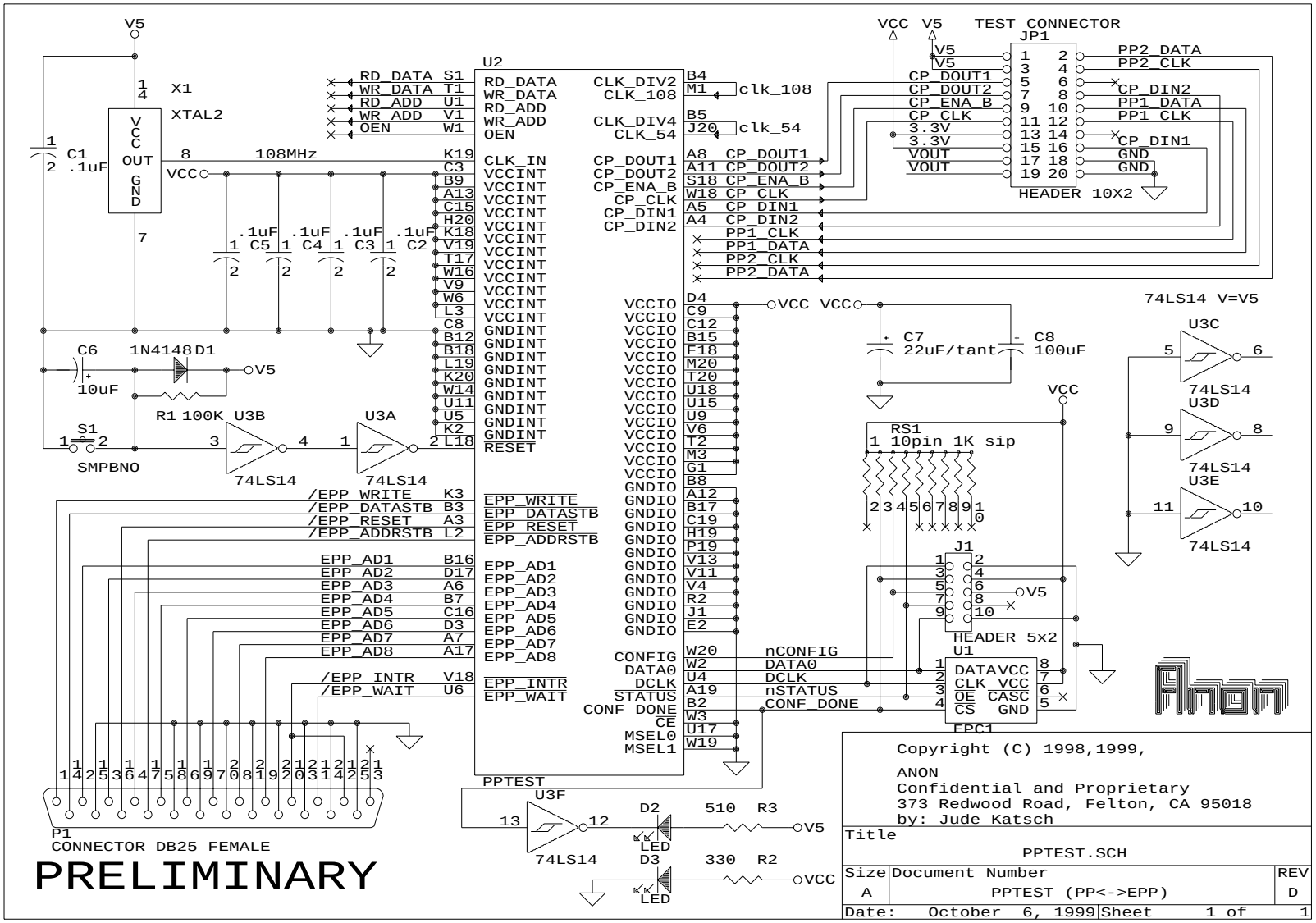
```
1 /*****************************************************************************
2 * Copyright (c) 1996-1998 Altera Corporation.  All rights reserved.  This text     *
3 * contains proprietary, confidential information of Altera Corporation, and      *
```

```
 4 * may be used, copied, and/or disclosed only pursuant to the terms of a          *
 5 * valid software license agreement with Altera Corporation.  This copyright     *
 6 * notice must be retained as part of this text at all times.                     *
 7 * Model generated by Genmem Version 2.8, on Sat Sep  4 18:33:44 1999
 8 ***************************************************************************/
 9 // synopsys translate_off
10 ‘delay_mode_path
11 // synopsys translate_on
12 ‘timescale 1ns / 1ns
13 ‘celldefine
14 module scfifo_32x8_ns (Data, Clock, WrReq, RdReq, Aclr, Sclr, Full, Empty, Q, UsedW);
15
16 parameter Width = 8;
17 parameter WidthAd = 5;
18 parameter NumWords = 32;
19
20 input [Width-1:0] Data;
21 input Clock;
22 input WrReq;
23 input RdReq;
24 input Aclr;
25 input Sclr;
26 output Full;
27 output Empty;
28 output [Width-1:0] Q;
29 output [WidthAd-1:0] UsedW;
30
31
32 // synopsys translate_off
33 // internal reg
34 reg [Width-1:0] mem_data [NumWords-1:0];
35 reg [Width-1:0] tmp_q1;
36 reg [Width-1:0] ZERO, UNKNOWN;
37 reg [WidthAd:0] status_ptr;
38 reg [WidthAd-1:0] read_ptr, write_ptr;
39 integer i;
40 reg empty_tmp, full_tmp;
41
42 initial
43 begin
44    for (i=0; i < Width; i=i+1)
45       begin
46           ZERO[i] = 1’b0;
47           UNKNOWN[i] = 1’bX;
48       end
49
50    for(i = 0; i < NumWords; i=i+1)
51          mem_data[i] = ZERO;
52
53    empty_tmp = 1;
54    full_tmp = 0;
55    read_ptr = 0;
56    write_ptr = 0;
57    status_ptr = 0;
58    tmp_q1 = {Width{1’b0}};
59 end
60
61   always @(posedge Aclr)
62      begin
63         tmp_q1 = ZERO;
64         full_tmp = 0;
65         empty_tmp  = 1;
66         read_ptr = {WidthAd{1’b0}};
67         write_ptr = {WidthAd{1’b0}};
68         status_ptr = {WidthAd{1’b0}};
69      end
```

```
 70
 71   always @( posedge Clock )
 72   begin
 73      if ((Aclr || Sclr) !== 'b1)
 74      begin
 75        if (WrReq && (!full_tmp))
 76           begin
 77             mem_data[write_ptr] = Data;
 78             write_ptr = write_ptr + 1;
 79             status_ptr = status_ptr + 1;
 80           end
 81        if (RdReq && (!empty_tmp))
 82           begin
 83             tmp_q1 <= mem_data[read_ptr];
 84             read_ptr = read_ptr + 1;
 85             status_ptr = status_ptr - 1;
 86           end
 87        if (status_ptr == 0)
 88          begin
 89            empty_tmp = 1;
 90            full_tmp = 0;
 91          end else
 92        if (status_ptr == NumWords)
 93          begin
 94            empty_tmp = 0;
 95            full_tmp = 1;
 96        end else
 97          begin
 98            empty_tmp = 0;
 99            full_tmp = 0;
100          end
101      end else
102      if (Sclr === 'b1)
103      begin
104         tmp_q1 = UNKNOWN;
105         full_tmp = 0;
106         empty_tmp  = 1;
107         read_ptr = {WidthAd{1'b0}};
108         write_ptr = {WidthAd{1'b0}};
109         status_ptr = {WidthAd{1'b0}};
110      end
111   end
112
113
114    assign Q = tmp_q1;
115    assign Full = full_tmp;
116    assign Empty = empty_tmp;
117    assign UsedW = status_ptr;
118 // synopsys translate_on
119 endmodule // scfifo_32x8_ns
120 `endcelldefine
```

10.3 SCHEMATIC

10.3.1 Peripheral Port TEST (PPTEST)



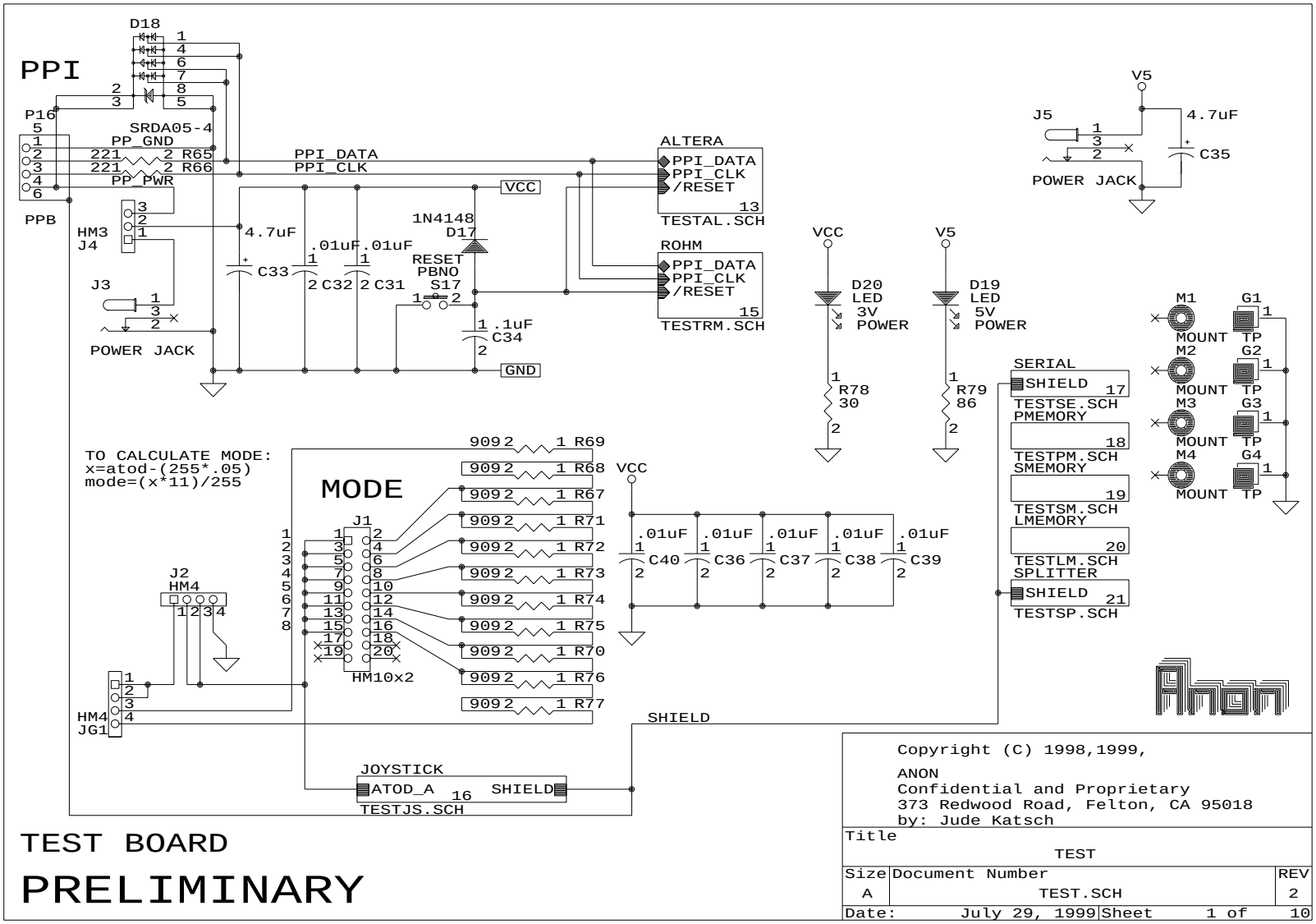
Chapter 11

TEST BOARD

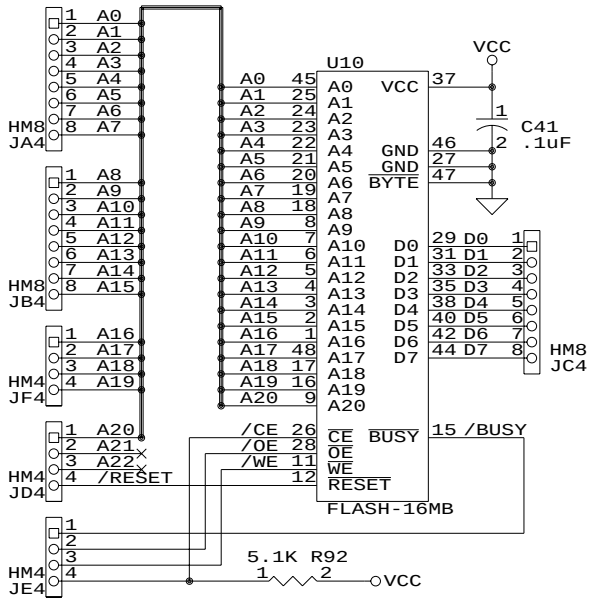
11.1 MAIN TEST BOARD SCHEMATICS V1.0

Description	Section	Page
TOP	11.4	91
Parallel Memory	11.4	92
Serial Memory	11.4	93
Latch Memory	11.4	94
Altera	11.4	95
Altera ROM	11.4	96
Rohm (tqfp) package	11.4	97
Joystick	11.4	98
Serial	11.4	99
Splitter	11.4	100

TOP (TEST)


PPI
D18
SRDA05-4
P16
PP_GND
PP_PWR
PPB
R65
R66
PPI_DATA
PPI_CLK
VCC
GND
HM3
J4
J3
POWER JACK
4.7uF
C33
.01uF
C32
C31
1N4148
D17
RESET
PBNO
S17
.1uF
C34
ALTERA
PPI_DATA
PPI_CLK
/RESET
TESTAL.SCH
ROHM
TESTRM.SCH
V5
J5
C35
D20
LED
3V
POWER
D19
5V
R78
30
R79
86
SERIAL
SHIELD
TESTSE.SCH
PMEMORY
TESTPM.SCH
SMEMORY
TESTSM.SCH
LMEMORY
TESTLM.SCH
SPLITTER
TESTSP.SCH
M1
M2
M3
M4
MOUNT
G1
G2
G3
G4
TP
TO CALCULATE MODE:
x=atod-(255*.05)
mode=(x*11)/255
MODE
J1
HM10x2
J2
HM4
JG1
R69
R68
R67
R71
R72
R73
R74
R75
R70
R76
R77
C40
C36
C37
C38
C39
SHIELD
JOYSTICK
ATOD_A
TESTJS.SCH
TEST BOARD
PRELIMINARY
Copyright (C) 1998,1999,
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch
Title
TEST
Size
A
Document Number
TEST.SCH
REV
2
Date: July 29, 1999
Sheet 1 of 10

Parallel Memory (TESTPM)



TEST BOARD

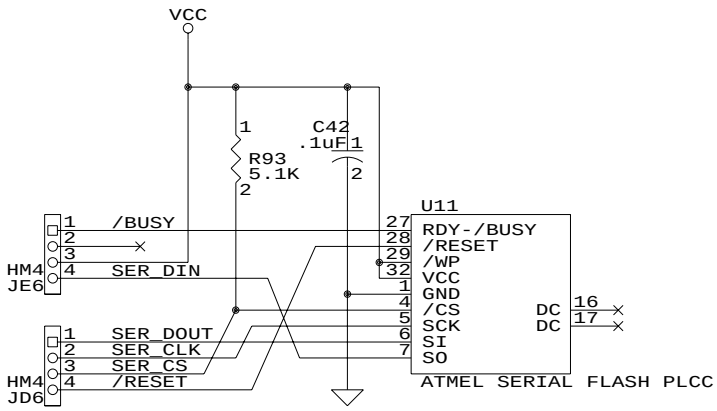
PRELIMINARY

Copyright (C) 1998,1999,

ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
	TEST PARALLEL MEMORY	
Size A	Document Number TESTPM.SCH	REV 2
Date: April 27, 1999	Sheet 2 of 10	

Serial Memory (TESTSM)



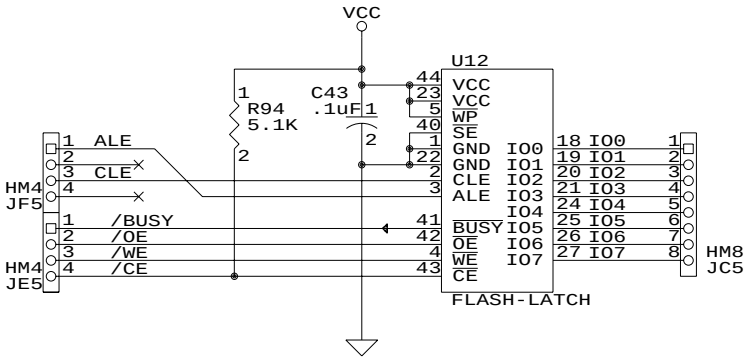
TEST BOARD

PRELIMINARY

Copyright (C) 1998,1999,
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
TEST SERIAL MEMORY		
Size A	Document Number TESTSM.SCH	REV 2
Date: April 28, 1999	Sheet 3 of 10	

Latch Memory (TESTLM)



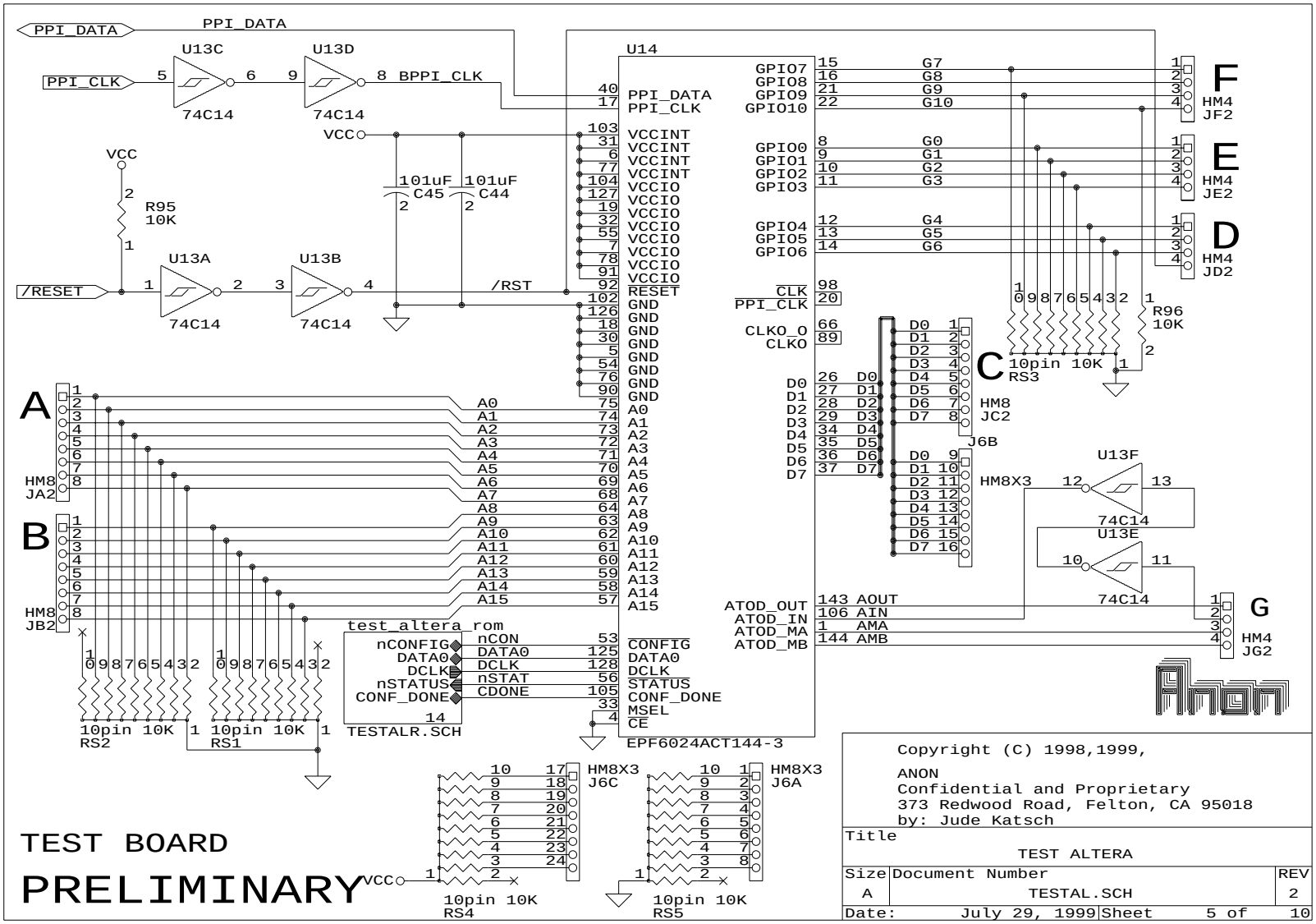
TEST BOARD

PRELIMINARY

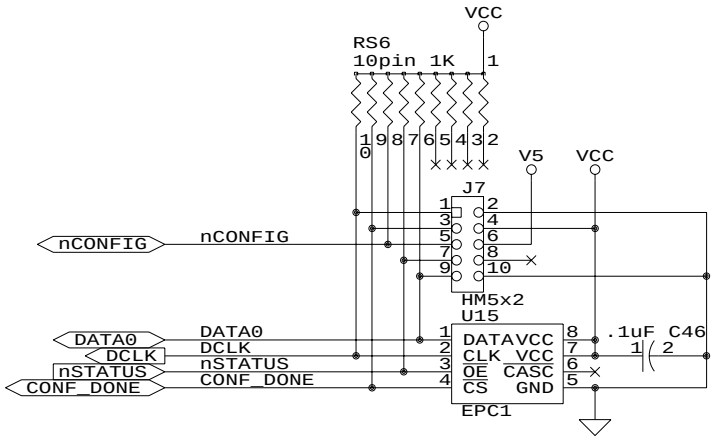
Copyright (C) 1998,1999,
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
TEST LATCH MEMORY		
Size A	Document Number TESTLM.SCH	REV 2
Date: April 25, 1999	Sheet 4 of 10	

Altera (TESTAL)



Altera ROM (TESTALR)



Copyright (C) 1998,1999,
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title
TEST ALTERA ROM

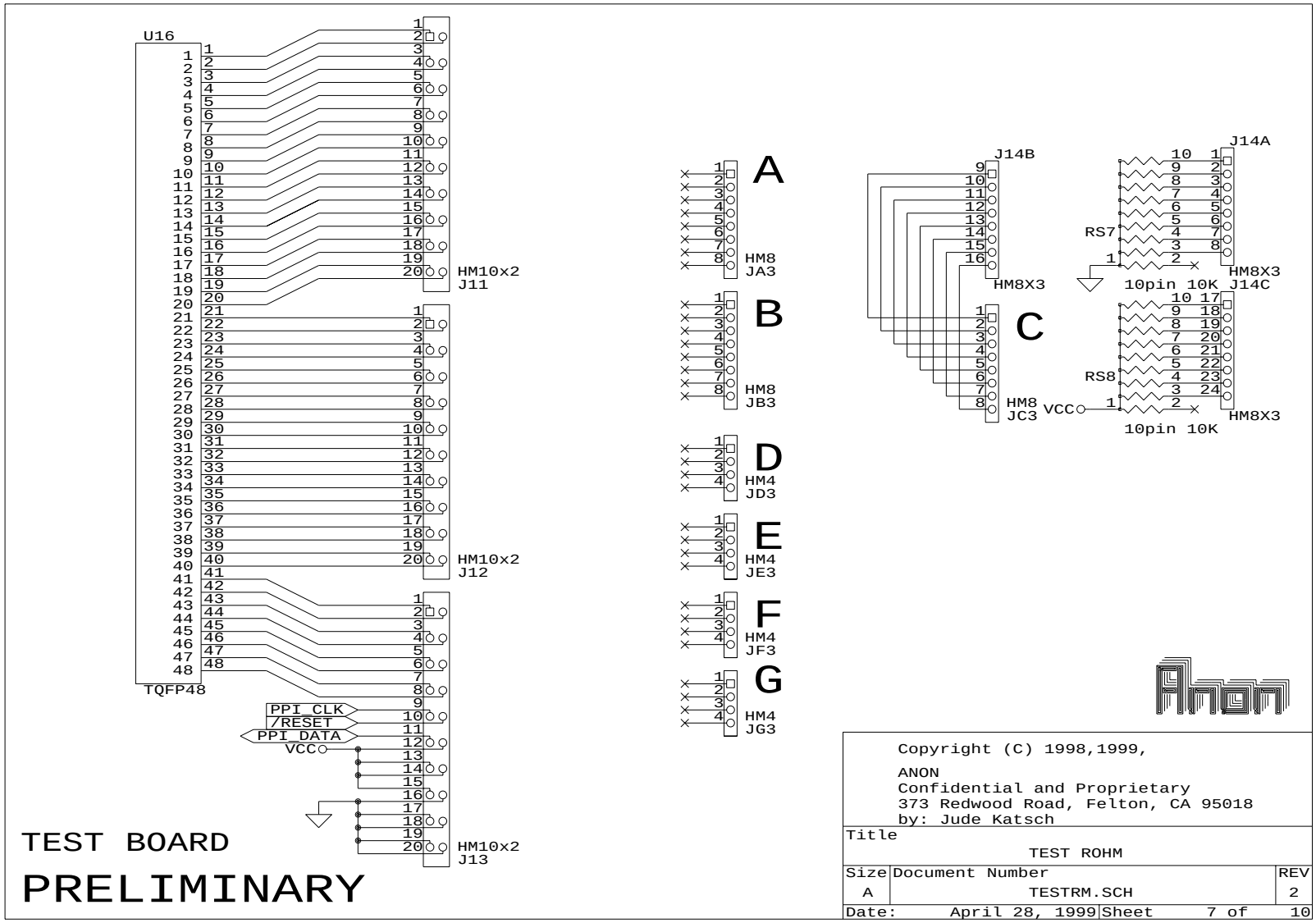
Size	Document Number	REV
A	TESTALR.SCH	2

Date: April 25, 1999 Sheet 6 of 10

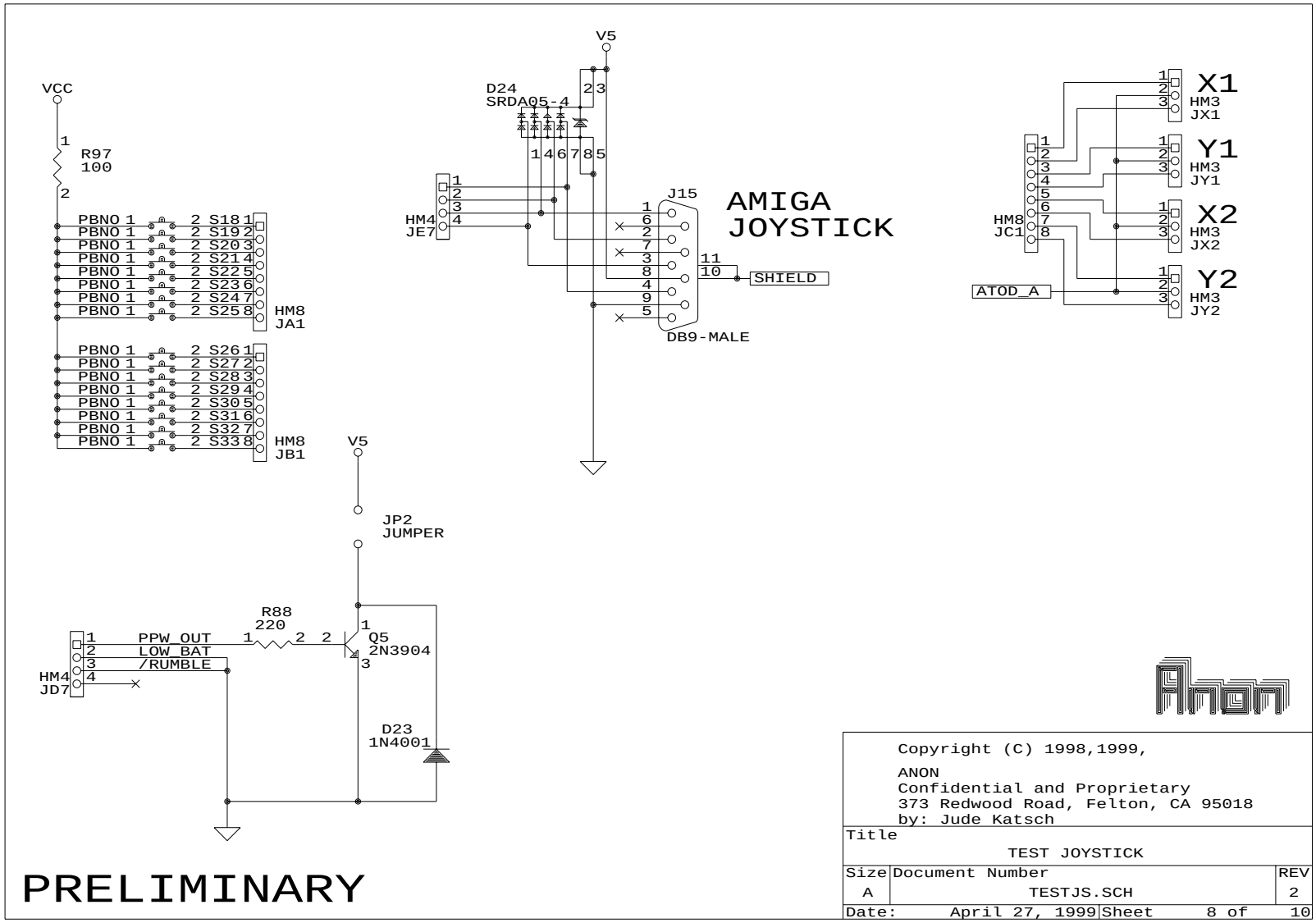
TEST BOARD

PRELIMINARY

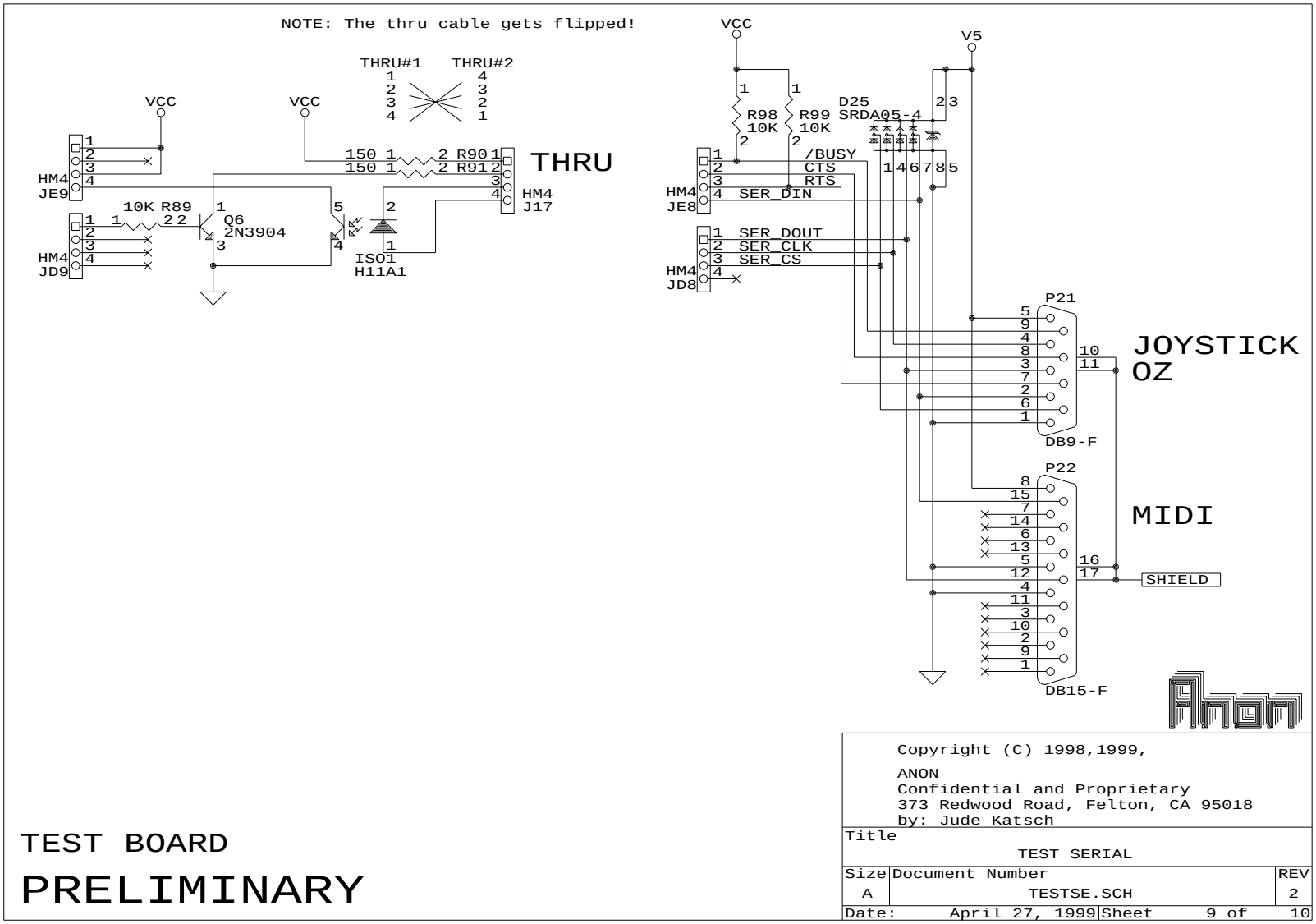
Rohm (tqfp) package (TESTRM)



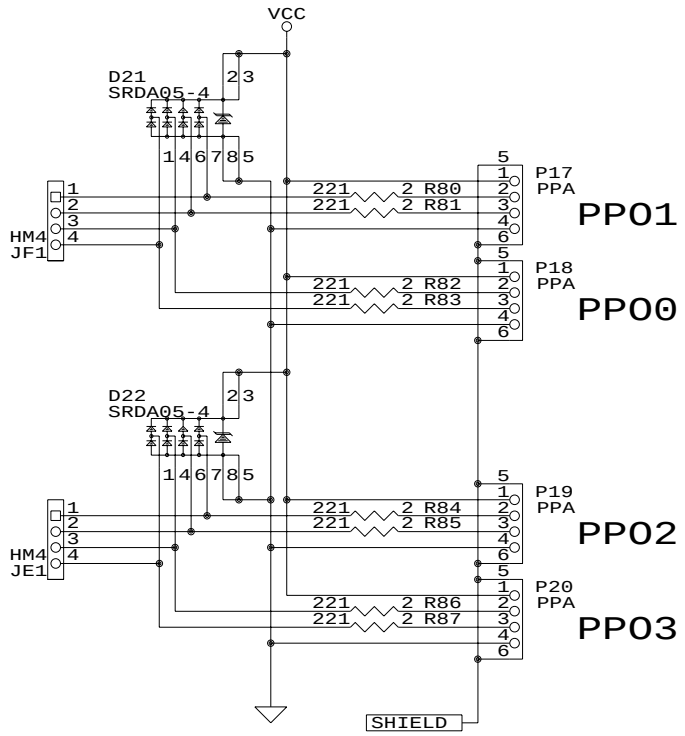
Joystick (TESTJS)



Serial (TESTSE)



Splitter (TESTSP)



TEST BOARD

PRELIMINARY

Copyright (C) 1998,1999,
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
	TEST SPLITTER	
Size A	Document Number TESTSP.SCH	REV 2
Date: April 27, 1999	Sheet 10 of 10	

11.2 MAIN TEST BOARD HACKS V1.0

Hack list for Polyface test board V1.0

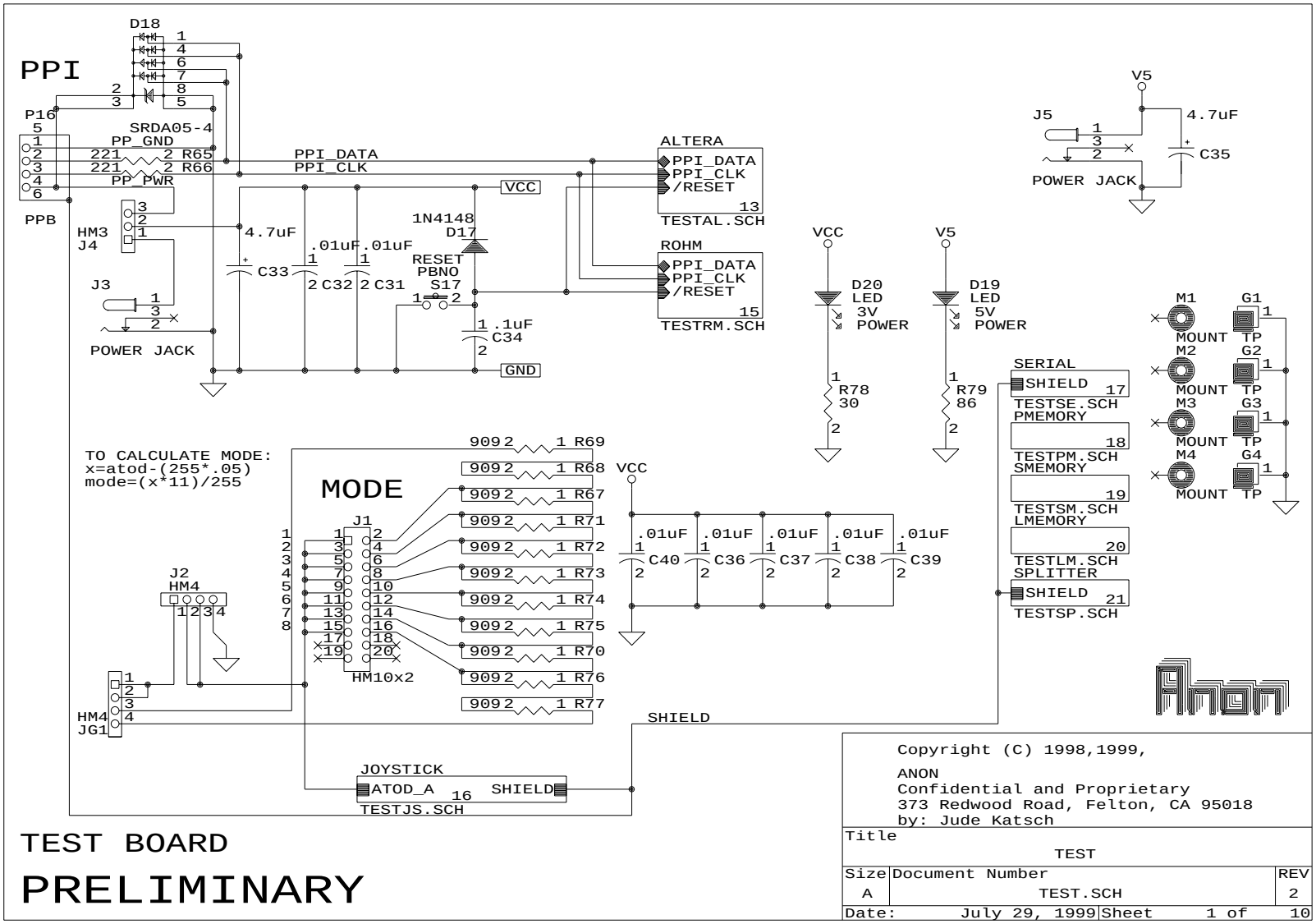
1. a) Cut the trace between U13-8 and JD6-1 on the solder side of the PCB.
2. a) Remove pin 10 of U13. (cut the pin between the IC and the PCB)
   b) Jumper U13 pin 10 and U13 pin 11 on the solder side of the PCB.
3. a) Add a 2 pin male header on the solder side of the PCB on pins 15 & 16 of J13.
   b) Mark pin 16 as pin 1.
4. a) Cut JE9 pin 3. (This pin is on the solder side)
   b) Add a 510 ohm resistor beween JE9 pin 3 & 4 on the component side of the PCB.

11.3 MAIN TEST BOARD PCB V1.0

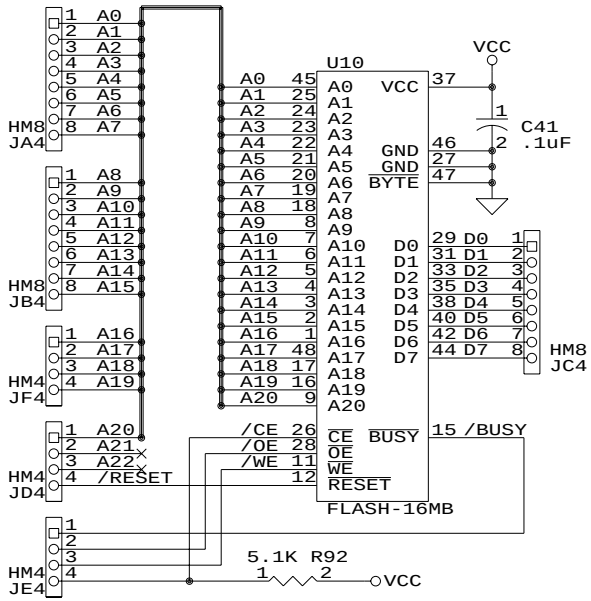
11.4 MAIN TEST BOARD SCHEMATICS V2.0

Description	Section	Page
TOP	11.4	91
Parallel Memory	11.4	92
Serial Memory	11.4	93
Latch Memory	11.4	94
Altera	11.4	95
Altera ROM	11.4	96
Rohm (tqfp) package	11.4	97
Joystick	11.4	98
Serial	11.4	99
Splitter	11.4	100

TOP (TEST)


PPI
D18
SRDA05-4
P16
PP_GND
PP_PWR
PPB
R65
R66
PPI_DATA
PPI_CLK
VCC
GND
HM3
J4
J3
POWER JACK
4.7uF
C33
.01uF
C32
C31
1N4148
D17
RESET
PBNO
S17
.1uF
C34
ALTERA
PPI_DATA
PPI_CLK
/RESET
TESTAL.SCH
ROHM
TESTRM.SCH
V5
J5
C35
D20
LED
3V
POWER
D19
5V
R78
30
R79
86
SERIAL
SHIELD
TESTSE.SCH
PMEMORY
TESTPM.SCH
SMEMORY
TESTSM.SCH
LMEMORY
TESTLM.SCH
SPLITTER
TESTSP.SCH
M1
M2
M3
M4
MOUNT
G1
G2
G3
G4
TP
TO CALCULATE MODE:
x=atod-(255*.05)
mode=(x*11)/255
MODE
J1
HM10x2
J2
HM4
JG1
R69
R68
R67
R71
R72
R73
R74
R75
R70
R76
R77
C40
C36
C37
C38
C39
SHIELD
JOYSTICK
ATOD_A
TESTJS.SCH
TEST BOARD
PRELIMINARY
Copyright (C) 1998,1999,
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch
Title
TEST
Size
A
Document Number
TEST.SCH
REV
2
Date: July 29, 1999
Sheet 1 of 10

Parallel Memory (TESTPM)



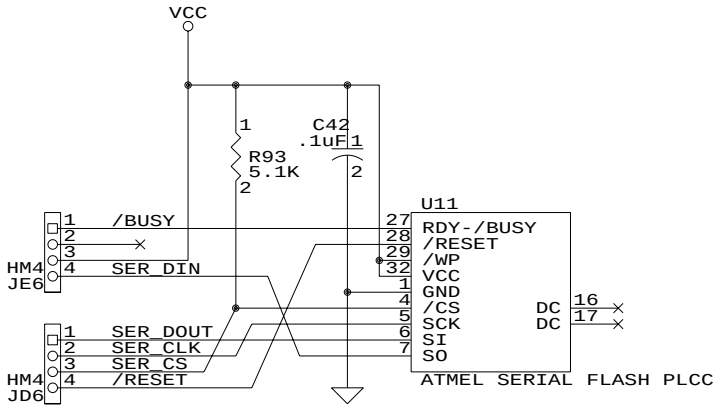
TEST BOARD

PRELIMINARY

Copyright (C) 1998,1999,
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
	TEST PARALLEL MEMORY	
Size A	Document Number TESTPM.SCH	REV 2
Date: April 27, 1999	Sheet 2 of 10	

Serial Memory (TESTSM)



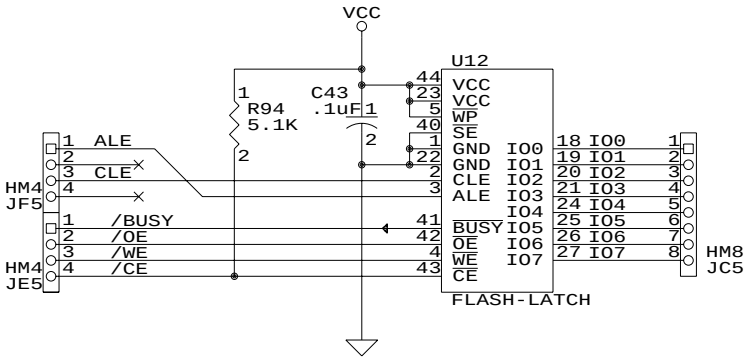
TEST BOARD

PRELIMINARY

Copyright (C) 1998,1999,
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
	TEST SERIAL MEMORY	
Size A	Document Number TESTSM.SCH	REV 2
Date: April 28, 1999	Sheet 3 of 10	

Latch Memory (TESTLM)



TEST BOARD

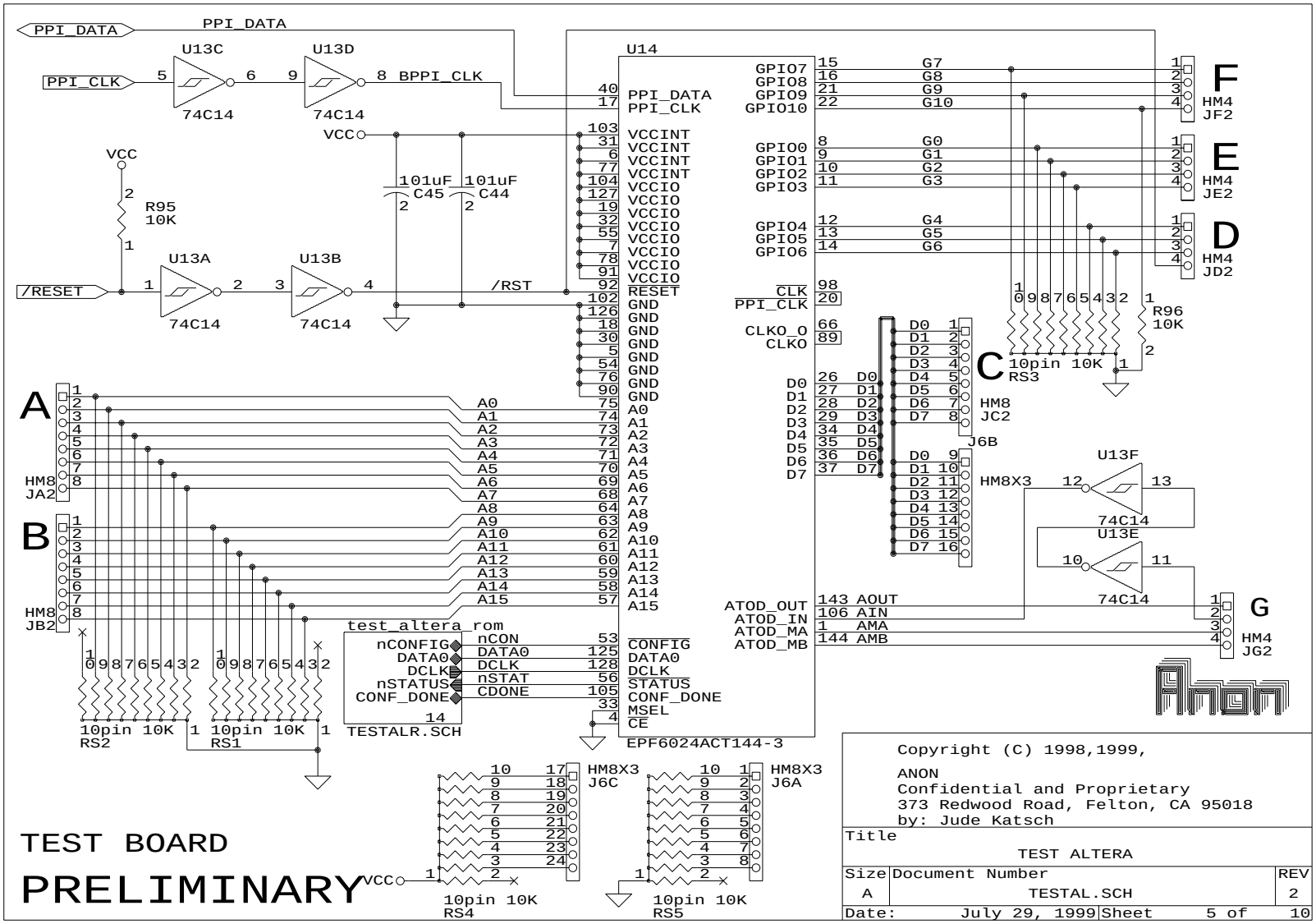
PRELIMINARY

Copyright (C) 1998,1999,

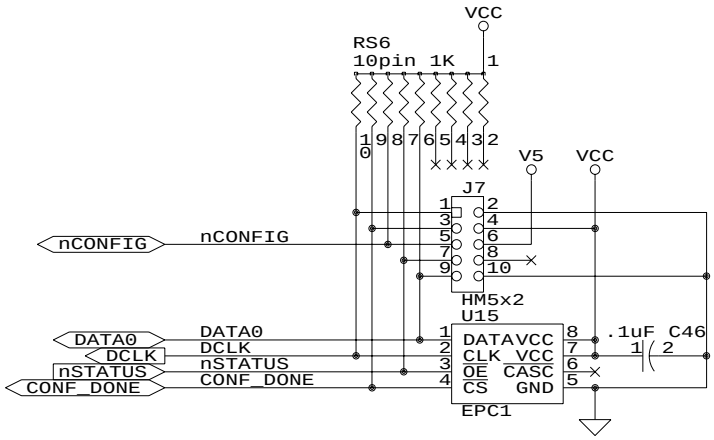
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
	TEST LATCH MEMORY	
Size A	Document Number TESTLM.SCH	REV 2
Date: April 25, 1999	Sheet 4 of 10	

Altera (TESTAL)



Altera ROM (TESTALR)



Copyright (C) 1998,1999,
ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title
TEST ALTERA ROM

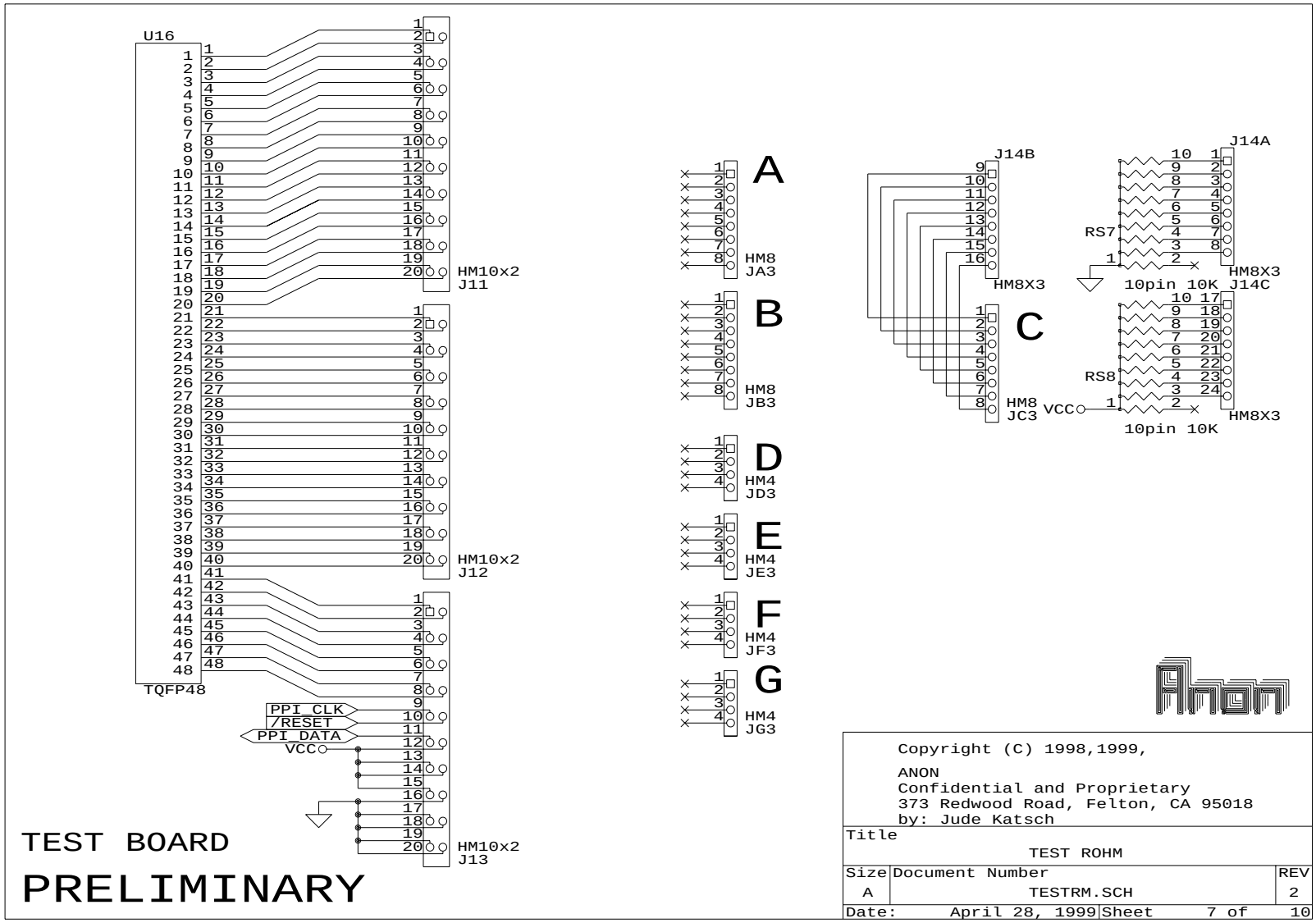
Size	Document Number	REV
A	TESTALR.SCH	2

Date: April 25, 1999 Sheet 6 of 10

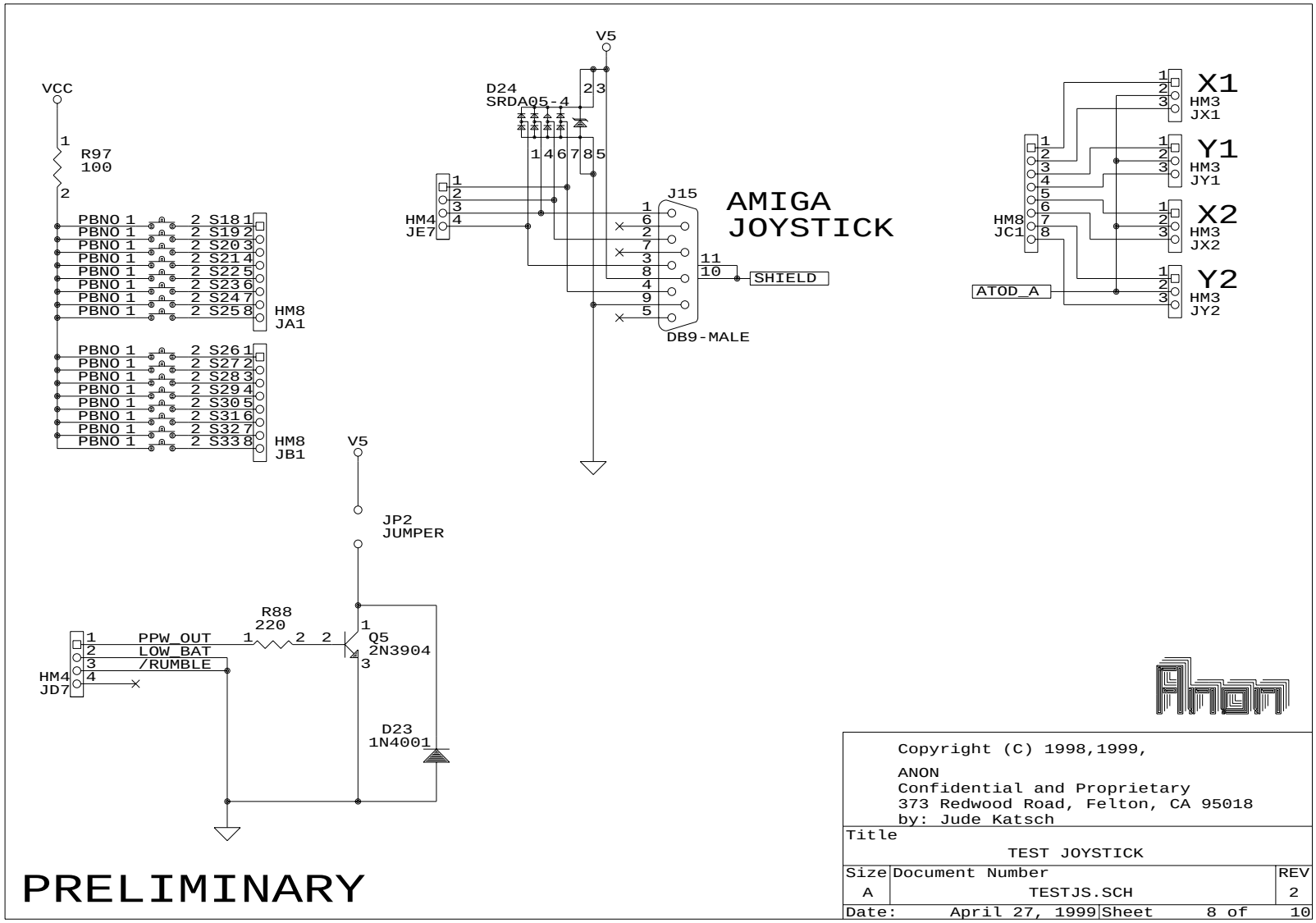
TEST BOARD

PRELIMINARY

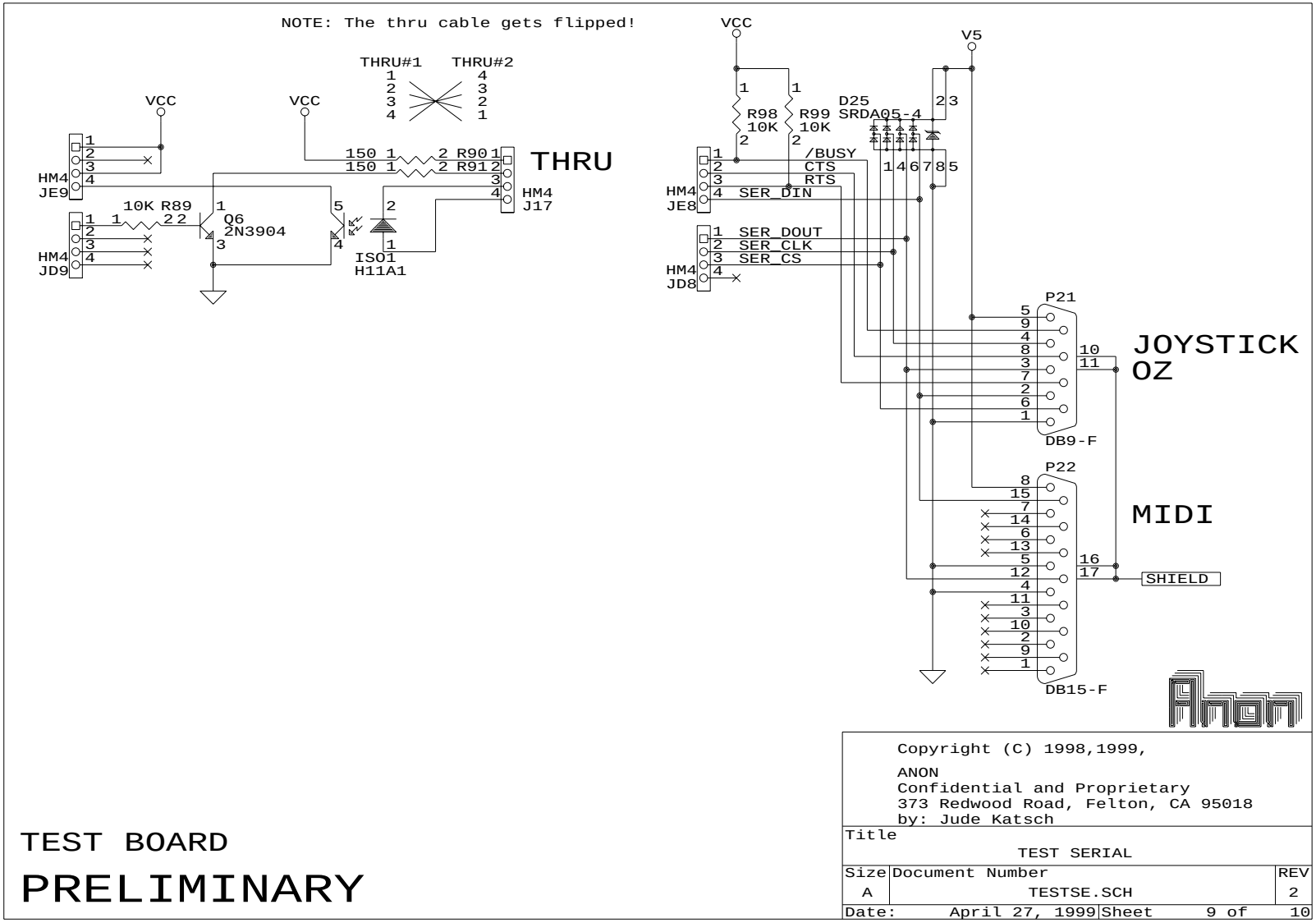
Rohm (tqfp) package (TESTRM)



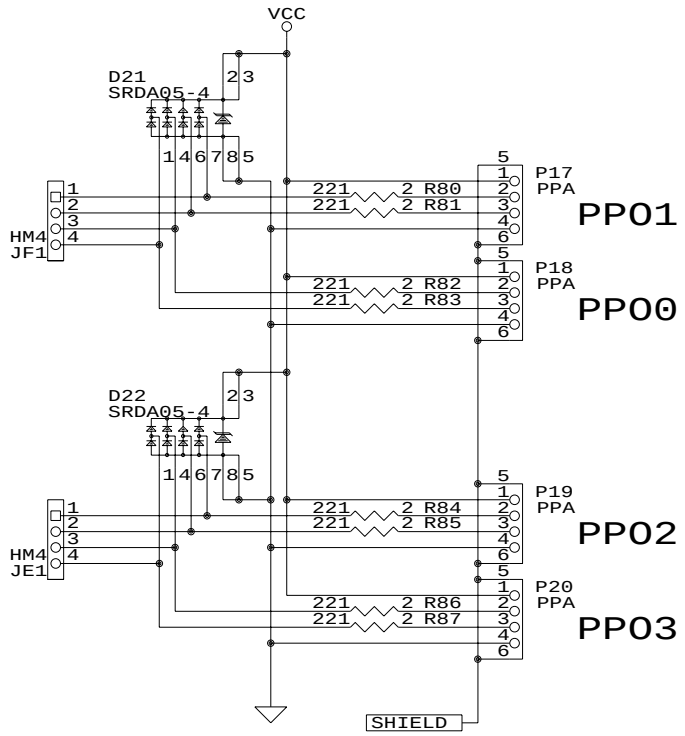
Joystick (TESTJS)



Serial (TESTSE)



Splitter (TESTSP)



TEST BOARD

PRELIMINARY

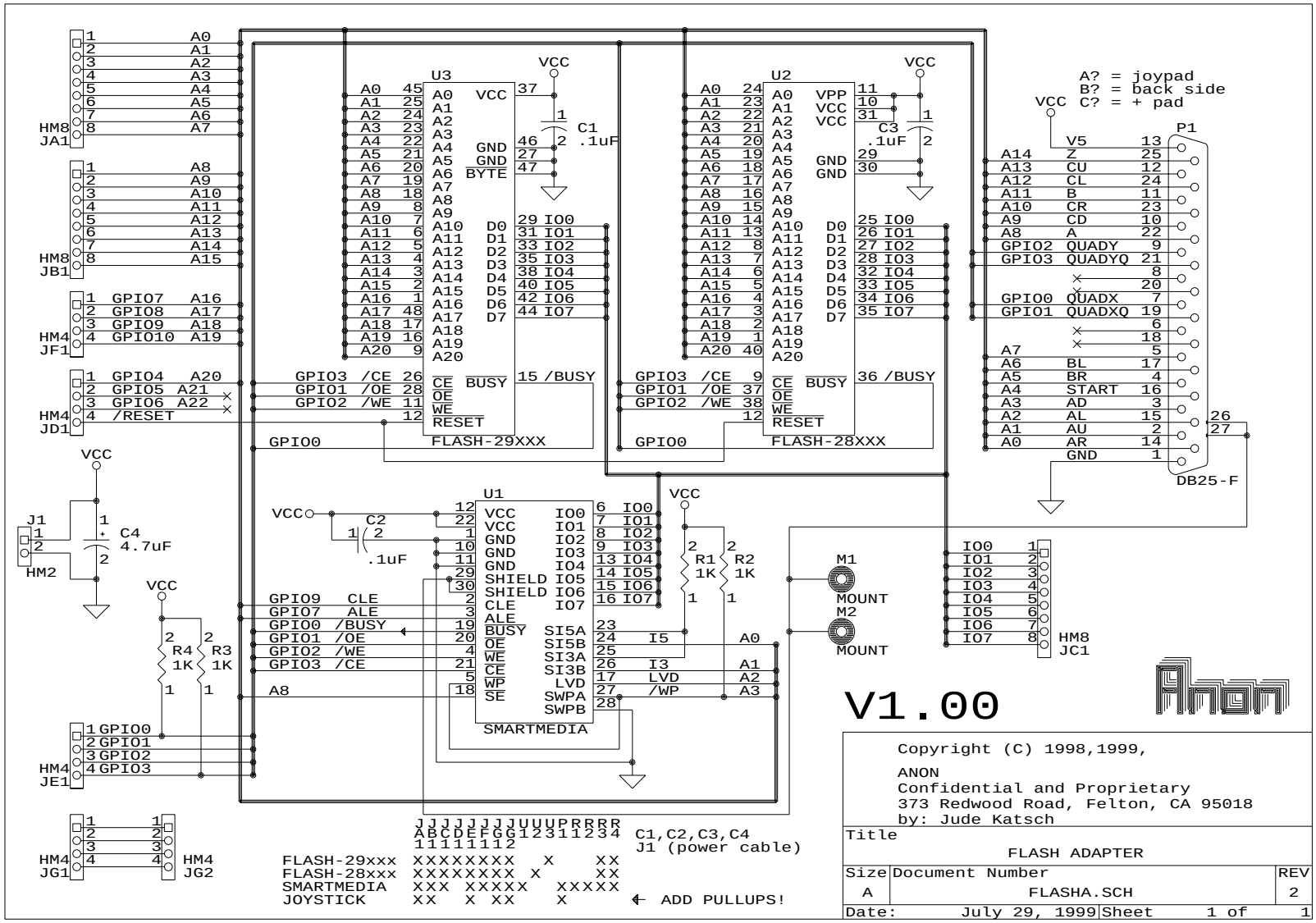
Copyright (C) 1998,1999,

ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
	TEST SPLITTER	
Size A	Document Number TESTSP.SCH	REV 2
Date: April 27, 1999	Sheet 10 of 10	

11.5 FLASH ADAPTER SCHEMATICS V1.0

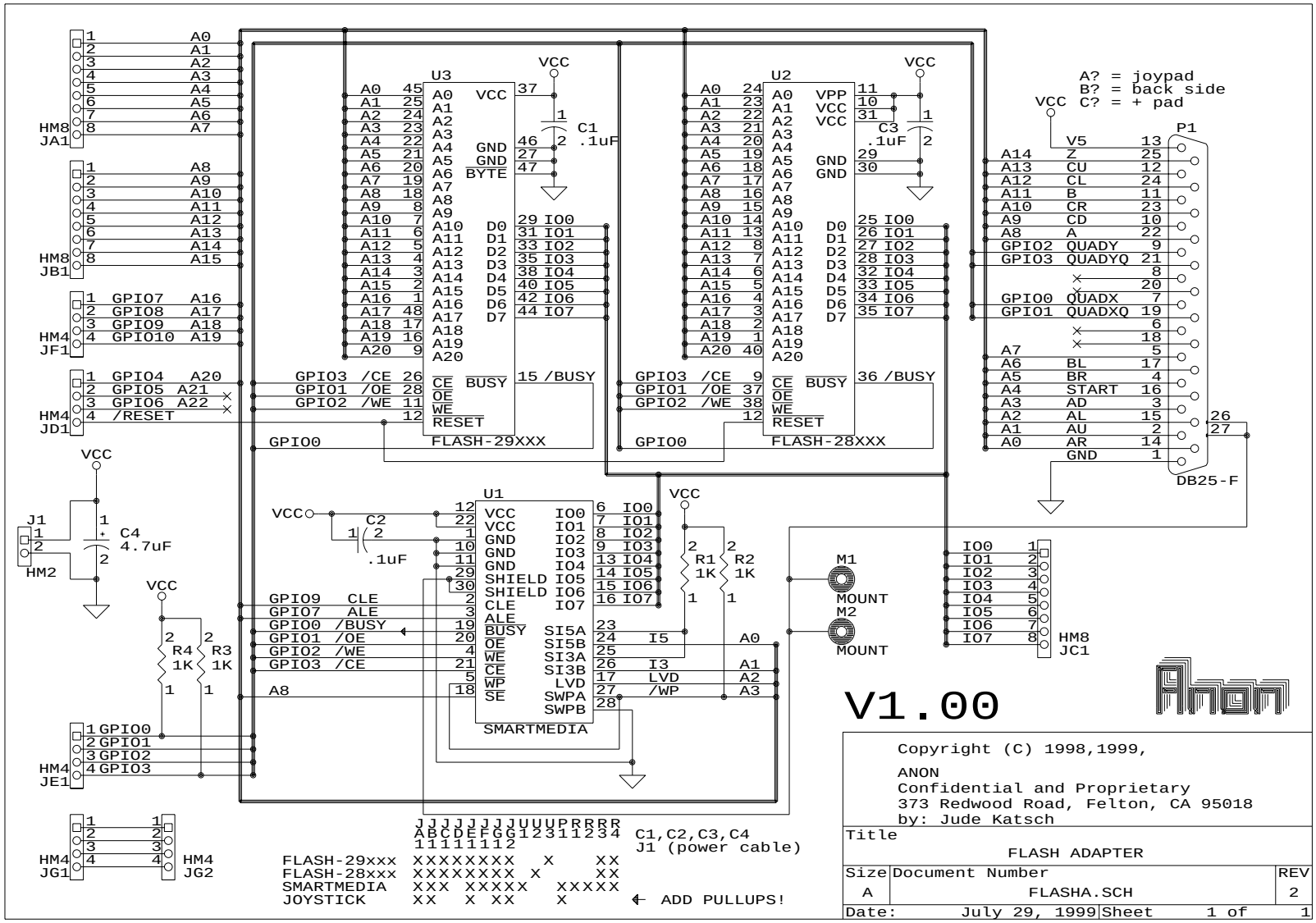
11.5.1 FLASH ADAPTER (FLASHA)



11.6 FLASH ADAPTER PCB V1.0

11.7 FLASH ADAPTER SCHEMATICS V2.0

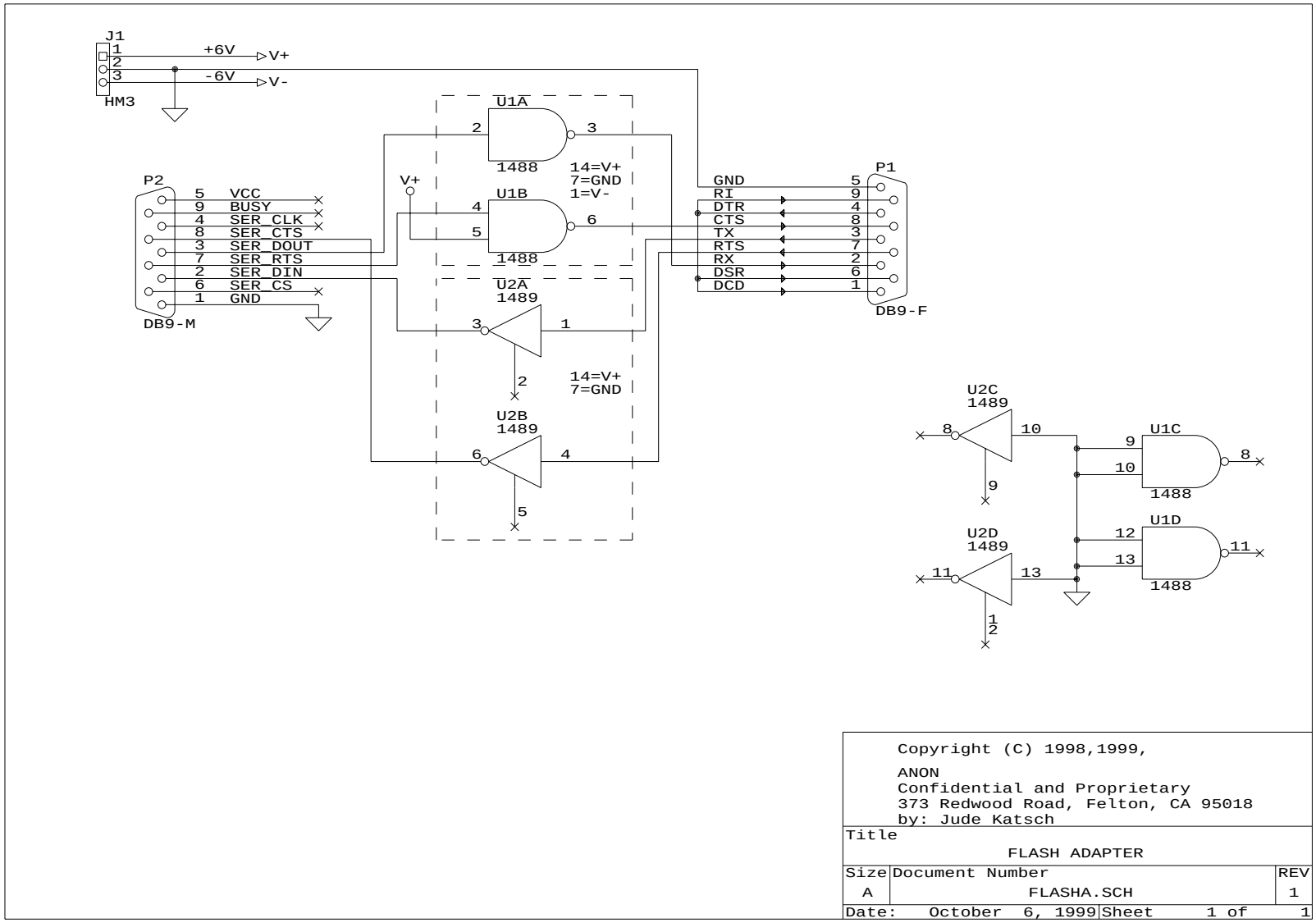
11.7.1 FLASH ADAPTER (FLASHA)



11.8 SERIAL ADAPTER

Description	Section	Page
SERIAL DRIVERS	11.8.1	106
SERIAL MAXIM	11.8.2	107

11.8.1 SERIAL DRIVERS (SERIAL)

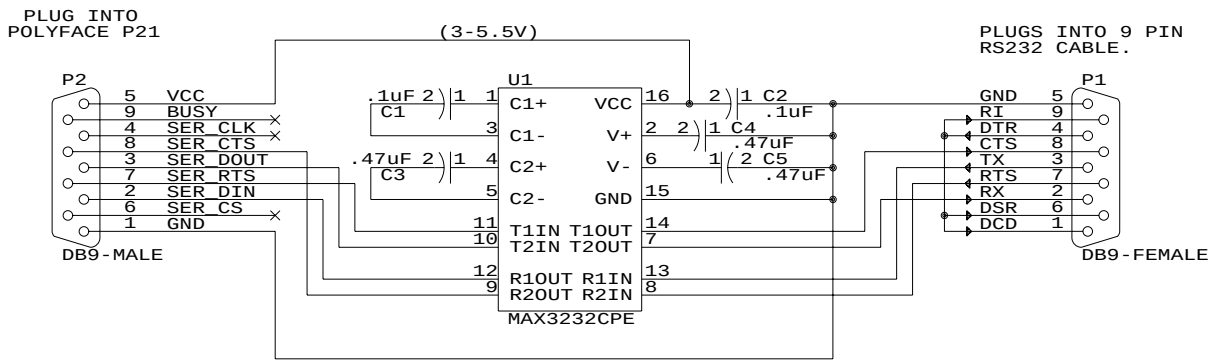


Copyright (C) 1998,1999,

ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
	FLASH ADAPTER	
Size	Document Number	REV
A	FLASHA.SCH	1
Date: October 6, 1999	Sheet 1 of 1	

11.8.2 SERIAL MAXIM (MAXIM)



All capcaitors are Ceramic

If the polyface test board does not have an
external 5V supply, J7 pins 4 and 6 must be jumpered.

Copyright (C) 1998,1999,

ANON
Confidential and Proprietary
373 Redwood Road, Felton, CA 95018
by: Jude Katsch

Title		
MAXIM SERIAL ADAPTER		
Size	Document Number	REV
A	MAXIM.SCH	1
Date: October 6, 1999	Sheet 1 of 1	

Chapter 12

SOFTWARE TEST ENVIRONMENT

Description	Section	Page
Peripheral Port main include	12.1	109
main() loop to interface with pptrace	12.2	113
Configure (map/add/remove)	12.3	117
DEFCFG includes	12.4	122
DEFCFG routines	12.5	122
Peripheral Port includes	12.6	125
Peripheral Port Command constructor	12.7	125
FIFO include	12.8	134
FIFO interface to/from pptrace	12.9	134
Interface between pport and epp	12.10	137
Socket EPP (enhanced parallel port) include	12.11	139
Socket EPP (enhanced parallel port) interface (for verilog environment)	12.12	140
EPP (enhanced parallel port) (for linux environment)	12.13	143
CRC Look Up Table (pre-constructed)	12.14	145
CRC routine	12.15	149
CRC Look Up Table generation routine	12.16	149
Polyface includes	12.17	150
Polyface Driver	12.18	153
Polyface Test includes	12.19	159
Polyface Test routines	12.20	160
Polyface Joystick Driver	12.21	168
Polyface Serial Driver	12.22	172
Polyface Parallel Memory Driver	12.23	176
Generic Memory Driver include	12.24	180
Generic Memory Driver	12.25	183
Generic FLASH CFI interface	12.26	190
AMD FLASH interface	12.27	192
PPTRACE GUI	12.28	196
DIRECT GUI (enter commands directly to polyface)	12.29	204
SERIAL GUI	12.30	212

12.1 Peripheral Port main include (PPORT_H)

```
/*
peripheral port includes (pport.h)
Written by Jude Katsch @ ANON
02/01/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/

#define MIN(a,b) (a<b)?(a):(b)
#define MAX(a,b) (a>b)?(a):(b)
int crc_calc(unsigned char data,int crc);
int crc_block(unsigned char *data,int length);
int pp_status_read(int port, int *status);
int pp_status_write(int port, int status);
char *pp_error(int error);
int pp_data_write_one(int port,int id, int address, int size, int flags, unsigned char data);
int pp_data_write    (int port,int id, int address, int size, int flags, unsigned char *data);
int pp_data_read     (int port,int id, int address, int size, int flags, unsigned char *data);
int pp_wait_txempty(int port);
int pp_command(int port,int id,int command);
int pp_command_read(int port,int id,int command,int *data);
int pp_command_error(int port,int *error);
int pp_command_alive(int port,int *alive);
int pp_command_request(int port,int *request);
int pp_command_reset(int port,int id);
int pp_command_blur(int port,int id);
int pp_command_focus(int port,int id);
int pp_command_tag(int port,int id);
int pp_command_unbrand(int port,int id);
int pp_command_brand(int port,int id,int brand);
int pp_command_probe(int port, int id, unsigned char *probe);
int pp_command_magic(int port, int id);
int pp_rd_state(int port, int id, unsigned char *data);
int pp_rd_error(int port, int id, unsigned char *data);
int pp_wr_state(int port, int id, unsigned char data);
int pp_wr_error(int port, int id, unsigned char data);


#define JS_ENABLED (1<<0)
#define ATOD_ERROR (1<<1)

struct axis
   {
   struct axis * next;
   int data;
   int absolute;
   int address;
   int flags;
   };

struct rumble
   {
   struct rumble * next;
   int speed;
   int pattern;
   int flags;
   };

struct keys
   {
   struct keys * next;
   int data;
   int address;
```

```
64    int flags;
65    };
66
67 struct joystick
68    {
69    struct axis   *joystick;
70    struct axis   *quad;
71    struct keys   *keys;
72    struct rumble *rumble;
73    };
74
75 #define SER_BUFSIZE 1000
76 struct data_buffer
77    {
78    struct data_bufer *next;
79    int topptr;  // take data from here
80    int botptr;  // put data in here (size = botptr-topptr)
81    unsigned char data[SER_BUFSIZE];
82    };
83
84 #define SER_FLAGS_SEND_EN (1<<0)
85 #define SER_FLAGS_RECV_EN (1<<1)
86 struct serial
87    {
88    struct    device *device; // so we can back reference
89    struct    data_buffer *sdbuf;
90    struct    data_buffer *rdbuf;
91    int (* send)(); // call to send data
92    int (* recv)(); // call to recv data
93    int       flags;
94    int       error_send_of;
95    int       error_recv_of;
96    int       error_recv_uf;
97    int       error_recv_p;
98    };
99
100 struct device
101    {
102    struct    peripheral_port *pport;
103    int       id;
104    int       parent;
105    struct    driver *driver;     // downloaded driver pointer
106    int       ports;              // one-hot of selectable ports i.e. 00001001, ports 3&0 are selectable
107    unsigned char  probe[4];      // last device probe
108    int       type;               // type of device i.e. PP_DEV_TYPE_?
109    int       enabled_maxspeed;   // the maximum xfer rate in Hz if enabled
110    int       disabled_maxspeed;  // the maximum xfer rate in Hz if disabled
111    int       enabled_current;    // the amount of current the device is using (if enabled)
112    int       disabled_current;   // the amount of current the device is using (if disabled)
113    int       flags;              // i.e. PP_DEV_FLAGS_?
114    void      *user;              // for use by driver
115    struct    function_config *fcfg; // configuration routines
116    struct    memory   *memory;     // memory info/calls (first in a linked list)
117    struct    joystick *joystick;   // joystick info
118    struct    serial   *serial;     // serial info
119    struct    ll_function_execute *llfexec;
120    };
121
122 // this structure is at the head of downloaded driver
123 // note: it is possible to configure a device without a driver
124 struct driver
125    {
126    int       (*open)();   // call to initilize driver
127    int       (*close)();  // call to initilize driver
128    int       (*config)(struct device *device); // setup function tables and such
129    struct    function_execute *fexec;
```

```
   int       flags;       // PP_DRI_FLAGS_?
   int       version;      // use latest revision driver for all devices with the same version
   int       revision;     // for simple bug fixes
   char      info[120];      // printable information on driver copy-
right/mfg and such. limit to three lines of 40 chars
   };

// the driver header is read first it is a READ ONLY structure and is REMOVED after driver is loaded
struct driver_header
   {
   int       magic;        // a unique 32 bit number for sanitiy purposes
   int       baseaddr;     // base address of driver in rom
   int       size;         // in bytes
   int       version;      // use latest revision driver for all devices with the same version
   int       revision;     // for simple bug fixes
   int       used;         // ++ for each peripheral using this driver. if used==0 then we can remove it!
   char      key [1024/8]; // key for driver validity verification
   char      info[120];      // printable information on driver copy-
right/mfg and such. limit to three lines of 40 chars
   };

struct function_execute
   {
   int (* error   )   (struct device *device);
   int (* request )   (struct device *device);
   int (* release )   (struct device *device);
   int (* execute )   (struct device *device);
   int (* configure ) (struct device *device);
   };

// we'll set things up so we can have a linked list of functions
struct ll_function_execute
   {
   struct ll_function_execute *next;
   struct function_execute *fexec;
   };

struct function_config
   {
   int (* config  )(struct device *device);
   int (* removed )(struct device *device);
   int (* error   )(struct device *device);
   int (* next    )(struct device *device);
   int (* root    )(struct device *device);
   int (* vacant  )(struct device *device);
   int (* enable  )(struct device *device);
   int (* disable )(struct device *device);
   };

struct peripheral_port
   {
   int    port;     // port number (0 or 1)
   int    depth;    // how deep?
   int    alive;    // the current configuration
   struct device *dev[31];
   };

struct peripheral_ports
   {
   struct peripheral_port *pport[2];
   };

#define CMD_WRITE        (0<<25)
#define CMD_READ         (1<<25)
#define CMD_WRITE_DIRECT (2<<25)
#define CMD_COMMAND(cmd) (((cmd)&0xff)<<17)
```

```
#define MAX_XFER_SIZE    0x7f
#define CMD_SIZE(size)   (((size)&MAX_XFER_SIZE)<<9)
#define CMD_XTRA(extra)  (((extra)&0xff)<<1)
#define CMD_ID(id)       (((id)&0x1f)<<27)
#define CMD_IA           (1<<16)
#define CMD_CRC          CMD_XTRA(0x80)

// execute
#define PP_CMD_RESET     (CMD_COMMAND(0xb1))
#define PP_CMD_TAG       (CMD_COMMAND(0xb2))
#define PP_CMD_BRAND     (CMD_COMMAND(0xb4))
#define PP_CMD_UNBRAND   (CMD_COMMAND(0xb3))
#define PP_CMD_BLUR      (CMD_COMMAND(0xb0)|CMD_XTRA(0x02))
#define PP_CMD_FOCUS     (CMD_COMMAND(0xb0)|CMD_XTRA(0x01))
#define PP_CMD_NOP       (CMD_COMMAND(0xa0))

// registers
#define PP_CMD_RERROR    0x98
#define PP_CMD_RSTATE    0x99

// read
#define PP_CMD_PROBE     0x94
#define PP_CMD_MAGIC     0x90
#define PP_CMD_ALIVE     (CMD_COMMAND(0x80))
#define PP_CMD_REQUEST   (CMD_COMMAND(0x84))
#define PP_CMD_ERROR     (CMD_COMMAND(0x88))

//
#define PP_MAGIC3        'E'
#define PP_MAGIC2        'D'
#define PP_MAGIC1        'U'
#define PP_MAGIC0        'J'

// poll register
#define PP_UNBRANDED     (1)
#define PP_BRANDED       (~PP_UNBRANDED)

// errors
#define PP_OK            0
#define PP_ERROR_BASE    0
#define PP_BAD_PARAM          (PP_ERROR_BASE+1)
#define PP_TIMEOUT            (PP_ERROR_BASE+2)
#define PP_BAD_PARITY         (PP_ERROR_BASE+3)
#define PP_INTERNAL           (PP_ERROR_BASE+4)
#define PP_TOO_MANY_DEVICES   (PP_ERROR_BASE+5)
#define PP_DONE               (PP_ERROR_BASE+6)
#define PP_BAD_MAGIC          (PP_ERROR_BASE+7)
#define PP_CRC_ERROR          (PP_ERROR_BASE+8)
#define PP_EMPTY              (PP_ERROR_BASE+9)
#define PP_BAD_DATA           (PP_ERROR_BASE+10)
#define PP_BAD_ADDRESS        (PP_ERROR_BASE+11)
#define PP_NOT_WRITEABLE      (PP_ERROR_BASE+12)
#define PP_NOT_SUPPORTED      (PP_ERROR_BASE+13)
#define PP_DRIVER_UNAVAILABLE (PP_ERROR_BASE+14)
#define PP_BAD_MODE           (PP_ERROR_BASE+15)
#define PP_FLASH_SUSPEND      (PP_ERROR_BASE+16)
#define PP_FLASH_TIMEOUT      (PP_ERROR_BASE+17)
#define PP_FLASH_BUSY         (PP_ERROR_BASE+18)

// bits in status register
#define CTRL_STAT_TXFULL        31
#define CTRL_STAT_RXFULL        30
#define CTRL_STAT_RXNULL        29
#define CTRL_STAT_REFRAME       28
#define CTRL_STAT_RESET         27
#define CTRL_STAT_RXOVERFLOW    15
```

```
#define CTRL_STAT_PRESCALE       0

// probe bits
#define PP_PROBE_ID_GET(probe)  ((probe>>1)&0x1f) // byte 3
#define PP_PROBE_BRANDED         6 // byte 3
#define PP_PROBE_TAGGED          7 // byte 3

//#define PP_PROBE_DOWNLOAD_DRIVER           0xf // byte 0

//#define PP_PROBE_DOWNLOAD_DRIVER_DELAYED   0 // byte 0
//#define PP_PROBE_DOWNLOAD_DRIVER_PARALLEL0 1 // byte 0
//#define PP_PROBE_DOWNLOAD_DRIVER_SERIAL    2 // byte 0
//#define PP_PROBE_DOWNLOAD_DRIVER_LATCHED   3 // byte 0
//#define PP_PROBE_DOWNLOAD_DRIVER_EMBEDED   4 // byte 0
//#define PP_PROBE_EXTENDED        6 // byte 0
#define PP_PROBE_DEFAULT_CONFIG  7 // byte 0

#define PP_PROBE_ID    3
#define PP_PROBE_MFG   2
#define PP_PROBE_TYPE  1
#define PP_PROBE_FLAGS 0

// device flags
#define PP_DEV_FLAGS_ENABLED     (1<<0)
#define PP_DEV_FLAGS_CONFIGED    (1<<1)
#define PP_DEV_FLAGS_ROOT        (1<<2)
#define PP_DEV_FLAGS_SELECT      (1<<3)

// remove when we add real interrupts
#define TIMEOUT 200 // number of time we do a status read to find rxfull
```

12.2 main() loop to interface with pptrace (TEST_C)

```
/*
peripheral port test
Written by Jude Katsch @ ANON
04/01/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/

#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include "pport.h"
#include "epp.h"
#include "sepp.h"
#include "config.h"
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include "fifo.h"
#include <string.h>

struct peripheral_port config_pp[2]; // static initilize to zero
struct peripheral_ports config_pps = {{&config_pp[0],&config_pp[1]}};

char buffer[1024];

char pdata[]="********_********_********_********";

int details=1;
```

```
31 int quiet=0;
32
33 int main(int argc, int *argv[])
34    {
35    int i,j;
36    int data;
37    int retval;
38    int command,port,size,id,mode;
39    unsigned char databuf [128];
40    int idatabuf[16];
41    struct device *dev;
42
43    ctrl_open();
44
45 #ifdef SEPP
46 #define SCALE (0x02)
47 #else
48 #define SCALE (0x06)
49 #endif
50
51    printf("scale=%d\n",SCALE);
52 //   pp_wait_txempty(CTRL_PORT1);
53    ctrl_wr_stat(CTRL_PORT1,SCALE);
54
55    pp_wait_txempty(CTRL_PORT0);
56    ctrl_wr_stat(CTRL_PORT0,SCALE|(1<<CTRL_STAT_REFRAME)); // REFRAME
57
58    pp_wait_txempty(CTRL_PORT0);
59    ctrl_wr_stat(CTRL_PORT0,SCALE|(1<<CTRL_STAT_REFRAME)); // REFRAME
60
61    while((retval=cmd_read(buffer,sizeof(buffer)))>=0)
62      {
63      if(details) printf("cmd=%s\n",buffer);
64      if(strcmp(buffer,"QUIT")==0)
65         {
66         ctrl_close();
67         fifo_close();
68         return(0);
69 }
70      else if(strcmp(buffer,"SYNC")==0)
71         mon_open();
72      else if(strcmp(buffer,"REFRAME")==0)
73         {
74         pp_wait_txempty(CTRL_PORT0);
75         ctrl_wr_stat(CTRL_PORT0,SCALE|(1<<CTRL_STAT_REFRAME));
76 }
77      else if(strcmp(buffer,"END")==0)
78         {
79         mon_write("END\n");
80 }
81      else if(strcmp(buffer,"REND")==0)
82         {
83         mon_write("REND\n");
84 }
85      else if(strcmp(buffer,"TICK")==0)
86         {
87         for(i=1;i<32;i++)
88           {
89           dev = config_pps.pport[0]->dev[i];
90           if((dev)&&(dev->driver))
91              (*dev->driver->fexec->execute)(dev);
92           }
93 }
94      else if(strcmp(buffer,"MAP")==0)
95         {
96         retval=pp_port_map(config_pps.pport[0]);
```

```
 97         if(details) printf("pp_port_map returns %d\n",retval);
 98 }
 99      else if(strcmp(buffer,"ERROR")==0)
100         {
101         if((retval=pp_command_error(config_pps.pport[0]->port,&data))!=PP_OK)
102    {
103            mon_error("polling error\n");
104            mon_write("POLL:ERROR:XXXXXXXX_XXXXXXXX_XXXXXXXX_XXXXXXXX\n");
105            }
106         else
107    {
108            for(i=1;i<32;i++)
109               {
110               dev = config_pps.pport[0]->dev[i];
111               if((data&(1<<i))&&(dev)&&(dev->driver))
112                  (*dev->fcfg->error)(dev);
113               }
114            longtobinary(pdata,data);
115            mon_write("POLL:ERROR:%s\n",pdata);
116            }
117         }
118      else if(strcmp(buffer,"ALIVE")==0)
119         {
120         if((retval=pp_command_alive(config_pps.pport[0]->port,&data))!=PP_OK)
121    {
122            mon_error("polling alive\n");
123            mon_write("POLL:ALIVE:XXXXXXXX_XXXXXXXX_XXXXXXXX_XXXXXXXX\n");
124            }
125         else
126    {
127            longtobinary(pdata,data);
128            mon_write("POLL:ALIVE:%s\n",pdata);
129            }
130 }
131      else if(strcmp(buffer,"REQUEST")==0)
132        {
133        if((retval=pp_command_request(config_pps.pport[0]->port,&data))!=PP_OK)
134           {
135           mon_error("polling request\n");
136           mon_write("POLL:REQUEST:XXXXXXXX_XXXXXXXX_XXXXXXXX_XXXXXXXX\n");
137           }
138        else
139           {
140           for(i=1;i<32;i++)
141      {
142              dev = config_pps.pport[0]->dev[i];
143              if((data&(1<<i))&&(dev)&&(dev->driver))
144                 (*dev->driver->fexec->request)(dev);
145              }
146           longtobinary(pdata,data);
147           mon_write("POLL:REQUEST:%s\n",pdata);
148           }
149        }
150      else if(strcmp(buffer,"RESET")==0)
151         pp_command_reset(CTRL_PORT0,CTRL_BCAST);
152      else if(strncmp(buffer,"SERW",4)==0)
153         {
154         if((retval=sscanf(buffer,"SERW:%d:%d:%d:%n",&port,&id,&size,&data))!=3)
155    {
156            mon_error("SERW invalid number of parammeters expected:3 got:%d\n",retval);
157 //            printf("buffer[%d]=%s\n",data,&buffer[data]);
158            }
159         else
160   {
161           dev = config_pps.pport[0]->dev[id];
162           if( (dev) && (dev->serial) && (dev->serial->send))
```

```
    {if((retval=(*dev->serial->send)(dev,size,&buffer[data]))!=PP_OK) return(retval);}
          else
             mon_error("serial send not supported");
  }
        }
     else if(strncmp(buffer,"EXECW1",6)==0)
        {
        if(sscanf(buffer,"EXECW1:%d:%d:%d:%d",&mode,&port,&id,&command)!=4)
           mon_write("%s:ERROR EXECW1 invalid number of parammeters\n",mode?"EXEC":"");
        else
   {
           retval=pp_data_write_one(port,id,0,0,command,0);
           mon_write("%s:direct write returned '%s'\n",mode?"EXEC":"",pp_error(retval));
           }
        }
     else if(strncmp(buffer,"EXECW",5)==0)
        {

i=sscanf(buffer,"EXECW:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:%d:",&mode,&port,&id,&command,&size,

&idatabuf[0],&idatabuf[1],&idatabuf[2],&idatabuf[3],&idatabuf[4],&idatabuf[5],&idatabuf[6],&idatabuf[7],

&idatabuf[8],&idatabuf[9],&idatabuf[10],&idatabuf[11],&idatabuf[12],&idatabuf[13],&idatabuf[14],&idatabuf[14]);
        if(i!=(size+5))
           mon_write("%s:ERROR EXECW invalid number of parammeters expected:%d was:%d\n",mode?"EXEC":"",size+4,i);
        else
           {
           for(i=0;i<16;i++) databuf[i] = (unsigned char) idatabuf[i];
           retval=pp_data_write(port,id,0,0,command,databuf);
           mon_write("%s:indirect write returned '%s'\n",mode?"EXEC":"",pp_error(retval));
           }
        }
     else if(strncmp(buffer,"EXECR",5)==0)
        {
        if(sscanf(buffer,"EXECR:%d:%d:%d:%d:%d",&mode,&port,&id,&command,&size)!=5)
           mon_write("%s:ERROR EXECR invalid number of parammeters\n",mode?"EXEC":"");
        else
   {
           retval=pp_data_read(port,id,0,0,command,databuf);
           j=sprintf(buffer,"%s:indirect read returned '%s'",mode?"EXEC":"",pp_error(retval));
           if(size>0)
           j+=sprintf(&buffer[j]," data=");
           for(i=0;i<size;i++)
              sprintf(&buffer[j+(i*5)],"0x%02x ",(int)databuf[i]);
           mon_write("%s\n",buffer);
           }
        }
     else
        printf("unknown command = '%s'\n",buffer);
     }


//     retval=pp_command_alive(port,&alive);
//     printf("error=%d\n",retval);

//   pp_command_reset(CTRL_PORT1,CTRL_BCAST);
//   pp_command(CTRL_PORT1,0,PP_CMD_NOP); // so PORT1 is ignored

// to test the epp interface, a quick feedback
#ifdef LOOPTEST
  data = 0x55aa55aa;
  while(1)
{
data1=++data;
if((retval=pp_cmd_test(&data1))) return(retval);
if(data1!=data) mon_error("expected 0x%08x was 0x%08x\n",data,data1);
```

```
//      printf("data=0x%08x\n",data);
if((data&0xff)==0) printf(".");
}
#endif

   ctrl_close();
   fifo_close();
   return(1);
   }
```

12.3 Configure (map/add/remove) (CONFIG_C)

```
/*
peripheral port config
Written by Jude Katsch @ ANON
02/01/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/

/* README README README README
NOTICE: This is not REAL code it is designed to STOP if an error occurs.
NOTICE: This is not REAL code it is designed to get statistical information for debugging purposes
NOTICE: I assume 32 bit ints
NOTICE: This is not portable code! I assume little endin! (intel)
NOTICE: This code was written to debug peripherals! It is NOT efficient or even reasonable for production.
NOTICE: I start counting port numbers at 0, the manual & reference schematics start at 1.
NOTICE: all of these routines use only PORT0. The necessary code to handle PORT1 is not included!!!
Sharing PORT0 & PORT1 is a very messy and time consuming task that is not needed to verify Peripheral hardware.
YOU HAVE BEEN WARNED!!!
*/

extern struct function_config default_config;
extern struct driver polyface_driver;

#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include "pport.h"
#include "epp.h"
#include "sepp.h"
#include "fifo.h"

extern int quiet;

int pp_device_configure(struct peripheral_port *pport,unsigned int *id,int parent);
int pp_port_map(struct peripheral_port *pport);
int pp_device_map(struct peripheral_port *pport,int depth,int parent);
int pp_port_error(struct peripheral_port *pport);
int print_device(struct device *dev)
   {
   printf("struct device * @ 0x%08x\n",(unsigned int)dev);
   printf("\tperipheral port * = 0x%08x\n",(unsigned int)dev->pport);
   printf("\tid = %d\n",dev->id);
   printf("\tparent = %d\n",dev->parent);
   printf("\tdriver * = 0x%08x\n",(unsigned int)dev->driver);
   printf("\tports = 0x%08x\n",dev->ports);
   printf("\tprobe = 0x%02x_%02x_%02x_%02x\n",dev->probe[3],dev->probe[2],dev->probe[1],dev->probe[0]);
   printf("\ttype = 0x%08x\n",dev->type);
   printf("\tflags = 0x%08x\n",dev->flags);
   printf("\tuser * = 0x%08x\n",(unsigned int)dev->user);
```

```
   printf("\tfunction config * = 0x%08x\n",(unsigned int)dev->fcfg);
   printf("\tmemory * = 0x%08x\n",(unsigned int)dev->memory);
   printf("\tjoystick * = 0x%08x\n",(unsigned int)dev->joystick);
   printf("\tserial * = 0x%08x\n",(unsigned int)dev->serial);
   return(0);
   };

// scan port and recover from errors if any
int pp_port_error(struct peripheral_port *pport)
   {
   int error;
   int i;
   int port = pport->port;
   int retval;

   if(!quiet) printf("in pp_error_map PORT:%d\n",port);
   retval=pp_command_error(port,&error);
   if(retval!=PP_OK) return(retval);

   // error returns a 1 in the bit position of a devicewith a pending error. unbranded devices
   // return a 1 in bit 0. All inputs are or'ed.

   if(error) // are there any removed devices?
      {
      for(i=1;i<31;i++)
         if((error>>i)&1)
           {
           if(pport->dev[i]==NULL)
              {
              mon_error("unxepcted dev[%d]==NULL\n",i);
              return(PP_INTERNAL);
              }
           printf("checking errors for device %d\n",i);
           if((retval=(*pport->dev[i]->fcfg->error)(pport->dev[i]))!=PP_OK) return(retval);
           }
      }
   return(PP_OK);
   }

// update map for all devices on a port.
// remove unplugged devices
// add unbranded devices
int pp_port_map(struct peripheral_port *pport)
   {
   int depth=0;
   int alive;
   int i,j;
   int port = pport->port;
   int retval;

   if(!quiet) printf("in pp_port_map PORT:%d\n",port);

   while(1)
     {
     retval=pp_command_alive(port,&alive);
     if(retval==PP_OK) break;
//     if(retval==PP_TIMEOUT) continue; // if nothing is plugged in we will continuously get timeouts
     mon_error("alive() loop failed\n");
     return(retval);
     }

   // alive returns a 1 in the bit position of a branded device. unbranded devices
   // return a 1 in bit 0. All inputs are or'ed.

   j=(pport->alive^alive)&PP_BRANDED;

```

```
116 //   printf("j=0x%08x\n",j);
117
118    if((j&alive)&PP_BRANDED)    // are there any unexpected branded device (sanity check)
119       {
120       mon_error("unxepcted branded device. previous_alive=0x%08x alive=0x%08x\n",pport->alive,alive);
121       return(PP_INTERNAL);
122       }
123
124    if(j) // are there any removed devices?
125       {
126       for(i=1;i<31;i++)
127          if((j>>i)&1)
128            {
129            if(pport->dev[i]==NULL)
130               {
131               mon_error("unxepcted dev[%d]==NULL\n",i);
132               return(PP_INTERNAL);
133               }
134            if(!quiet) printf(" removing unplugged device %d\n",i);
135            if((retval=(*pport->dev[i]->fcfg->removed)(pport->dev[i]))!=PP_OK) return(retval);
136            mon_write("DEV:%d:%d:DEL\n",pport->port,i);
137            free(pport->dev[i]); // deallocate device structure;
138            pport->dev[i]=NULL; // remove device pointer
139            pport->alive &= ~(1<<i); // clear device bit
140            }
141       }
142
143    if(alive&PP_UNBRANDED) // are there any unbranded devices out there?
144       return(pp_device_map(pport,depth,0));
145
146    return(PP_OK);
147    }
148
149 int pp_device_map(struct peripheral_port *pport,int depth,int parent)
150    {
151    int port = pport->port;
152    int retval;
153    unsigned char probe[4];
154    int id=0;
155
156    if(!quiet) printf("in pp_device_configure(0x%08x,%d)\n",(unsigned int)pport,depth);
157
158 // set the focus
159    if(depth==0) // to focus root device
160       { if((retval=pp_command_focus(port,CTRL_BCAST))!=PP_OK) return(retval); }
161    else         // to focus all devices beyond the root
162       { if((retval=pp_command_blur(port,CTRL_BCAST))!=PP_OK) return(retval); }
163
164 // probe the device with focus
165    if((retval=pp_command_probe(port,CTRL_BCAST,probe))!=PP_OK) return(retval);
166
167 // does it return the magic number?
168    if((retval=pp_command_magic(port,CTRL_BCAST))!=PP_OK) return(retval);
169
170    id=PP_PROBE_ID_GET(probe[PP_PROBE_ID]); // id of focused device
171
172 // get an id
173    if((probe[PP_PROBE_ID]&(1<<PP_PROBE_BRANDED))==0)
174       {
175       if((retval=pp_device_configure(pport,&id,parent))!=PP_OK) return(retval);
176
177       if(pport->dev[id]==0)
178          {
179          mon_error("NULL device pointer id=%d\n",id);
180          return(PP_INTERNAL);
181          }
```

```
182
183       print_device(pport->dev[id]);
184       }
185
186    // configure device
187       if((retval=(*pport->dev[id]->fcfg->config)(pport->dev[id]))!=PP_OK) return(retval);
188
189    // set root if root device (depth==0)
190       if((depth++==0)&&((retval=(*pport->dev[id]->fcfg->root)(pport->dev[id]))!=PP_OK)) return(retval);
191
192 // recursively scan all ports
193    while(1)
194       {
195       retval=(*pport->dev[id]->fcfg->next)(pport->dev[id]);
196       if(retval==PP_DONE) return(PP_OK);
197       if(retval!=PP_OK) return(retval);
198       if((retval=pp_command_tag(port,CTRL_BCAST))!=PP_OK) return(retval);
199       if((retval=pp_command_tag(port,id))!=PP_OK) return(retval);
200       retval=pp_device_map(pport,depth,id);
201       if(retval==PP_EMPTY)
202          (*pport->dev[id]->fcfg->vacant)(pport->dev[id]);
203       else if(retval!=PP_OK)
204          return(retval);
205       }
206    }
207
208 int pp_device_configure(struct peripheral_port *pport,unsigned int *id,int parent)
209    {
210    int i;
211    unsigned int port=pport->port;
212    int retval;
213
214    if(!quiet) printf("in pp_device_configure(0x%08x,0x%08x,%d)\n",(unsigned int)pport,(unsigned int)id,parent);
215
216    printf("pport->alive=0x%08x\n",pport->alive);
217
218    for(i=1;(i<32)&&((((pport->alive)>>i)&1)==1);i++);
219
220    printf("id selected=%d\n",i);
221
222    if((*id=i)==32)
223       {
224       mon_error("too many devices on port.\n");
225       return(PP_TOO_MANY_DEVICES);
226       }
227
228 // sanity check, make sure device pointer is zero
229    if(pport->dev[i]!=0)
230       {
231       mon_error("expected null device pointer.\n");
232       return(PP_INTERNAL);
233       }
234
235    if(!quiet) printf("new device to be branded with id=%d\n",i);
236
237 // we now have an ID for the part
238
239   if((retval=pp_command_brand(port,CTRL_BCAST,i))!=PP_OK) return(retval);
240
241 // allocate a device structure
242
243    if((pport->dev[i]=(struct device *)malloc(sizeof(struct device)))==NULL)
244       {
245       mon_error("allocating memory for device structure.\n");
246       return(PP_INTERNAL);
247       }
```

```
248
249 // initilize device structure
250    pport->dev[i]->pport = pport;
251    pport->dev[i]->id = i;
252    pport->dev[i]->parent = parent;
253    pport->dev[i]->driver = NULL;
254    pport->dev[i]->type   = 0;
255    pport->dev[i]->ports  = 0x0f; // enable 3210
256    pport->dev[i]->enabled_maxspeed = 500000;
257    pport->dev[i]->disabled_maxspeed = 500000;
258    pport->dev[i]->enabled_current = 0;
259    pport->dev[i]->disabled_current = 0;
260    pport->dev[i]->flags = 0;
261    pport->dev[i]->user =0;
262    pport->dev[i]->fcfg = NULL;
263    pport->dev[i]->memory=NULL;
264    pport->dev[i]->joystick=NULL;
265    pport->dev[i]->serial=NULL;
266    pport->dev[i]->llfexec=NULL;
267    pport->alive |= 1<<i; // set alive bit
268
269    mon_write("DEV:%d:%d:ADD\n",pport->port,i);
270
271 //while(1)
272 //  {
273 //  if((retval=pp_command_probe(port,i,pport->dev[i]->probe))!=PP_OK) printf("****************\n");
274 //  if((retval=pp_command_magic(port,i))!=PP_OK) printf("****************\n");
275 //  }
276
277 // probe of branded device
278   if((retval=pp_command_probe(port,i,pport->dev[i]->probe))!=PP_OK) return(retval);
279
280 // does it return the magic number?
281    if((retval=pp_command_magic(port,i))!=PP_OK) return(retval);
282
283 // We have a branded device that seems to be working, now we need to see if we can find a driver for it.
284 // if there is a downloadable driver we should download it, verify it, and open it. //???
285 // we also need some revision logic in here to load the latest revision driver. //???
286
287 // note: it is not necessary to have a driver to configure a device if it uses the defualt config.
288 //       This allows drivers to be downloaded from other sources such as DVD after configuration.
289 //   switch((pport->dev[i]->probe[PP_PROBE_FLAGS]&PP_PROBE_DOWNLOAD_DRIVER))
290 //      {
291 //      case PP_PROBE_DOWNLOAD_DRIVER_EMBEDED:
292    pport->dev[i]->driver = &polyface_driver;
293 //         break;
294 //      case PP_PROBE_DOWNLOAD_DRIVER_DELAYED: // driver will be configured later
295 ////???         pport->dev[i]->driver = delayed_driver; NOTE: delayed driver MUST use default config
296 //         break;
297 //      case PP_PROBE_DOWNLOAD_DRIVER_PARALLEL0:
298 //         //???
299 //      case PP_PROBE_DOWNLOAD_DRIVER_LATCHED:
300 //         //???
301 //      case PP_PROBE_DOWNLOAD_DRIVER_SERIAL:
302 //         //???
303 //      default:
304 //         {
305 //         printf(" id=%d downloadable driver not available yet!",i);
306 //         return(PP_BAD_MAGIC);
307 //         }
308 //      }
309
310 // does it use the default config functions or are the config func-
    tions part of the built in driver (only two choices)
311    if(pport->dev[i]->probe[PP_PROBE_FLAGS]&(1<<PP_PROBE_DEFAULT_CONFIG))
312       pport->dev[i]->fcfg = &default_config;
```

```

// make sure we have a config driver.
   if(pport->dev[i]->fcfg==NULL)
      {
      printf(" id=%d no config driver defined!\n",i);
      return(PP_BAD_MAGIC);
      }

   return(PP_OK);
   }
```

12.4 DEFCFG includes (DEFAULT_CONFIG_H)

```
#define PP_STATE_VACANT_OH 0
#define PP_STATE_SELECT    4
#define PP_STATE_ROOT      6
#define PP_STATE_ENABLE    7

#define PP_ERROR_COLLISION0      0
#define PP_ERROR_COLLISION1      1
#define PP_ERROR_COLLISION2      2
#define PP_ERROR_COLLISION3      3
#define PP_ERROR_WRITE_OVERFLOW  4
#define PP_ERROR_READ_UNDERFLOW  5
#define PP_ERROR_INVALID_COMMAND 6
#define PP_ERROR_DEVICE_ERROR    7
```

12.5 DEFCFG routines (DEFAULT_CONFIG_C)

```
/*
polyface default config (default_config.c)
Written by Jude Katsch @ ANON
02/20/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/

#include <stdio.h>
#include "pport.h"
#include "epp.h"
#include "default_config.h"
#include "fifo.h"

// These are the default configuartion routines. Any device that uses the polyface, it's pport.v,
// or is register compatible can use these routines. Devices that extend the current functionality
// or have a different register structure must include a function_config as part of their driver.
// A bit, PP_PROBE_DEFAULT_CONFIG, in the probe register enables the default_config.

// All config routines require device * as the only argument

//==========================================================================================
// called when a device is hot un-plugged
int default_config_removed(struct device *dev)
   {
   int retval;
   printf("in default_config_removed(0x%08x)\n",(unsigned int) dev);
   if((dev)&&(dev->driver))
      if((retval=(*dev->driver->fexec->release)(dev))!=PP_OK) return(retval);
   // remove any memory allocations
   // remove from any request lists
```

```
32    return(PP_OK);
33    }
34
35 //==========================================================================================
36 // called during mapping after device is branded and device structure is initilized
37
38 int default_config_config(struct device *dev)
39    {
40    int retval;
41
42    printf("in default_config_config(0x%08x)\n",(unsigned int)dev);
43
44 // see device->flags & PP_DEV_FLAGS_CONFIGED to see if previously configured
45    dev->flags &= ~PP_DEV_FLAGS_ROOT;
46    dev->flags &= ~PP_DEV_FLAGS_SELECT;
47
48    pp_wr_state(dev->pport->port,dev->id,0); // clear the state flags
49    pp_wr_error(dev->pport->port,dev->id,0xff); // clear the error flags
50
51 // call the specific config for driver (if there is a driver)
52    if((dev->driver!=NULL)&&((dev->flags&PP_DEV_FLAGS_CONFIGED)==0))
53       if((retval=(*dev->driver->config)(dev))!=PP_OK) return(retval);
54
55    dev->flags |= PP_DEV_FLAGS_CONFIGED;
56
57    return(PP_OK);
58    }
59
60 //==========================================================================================
61 // called when a device returns any error
62 int default_config_error(struct device *dev)
63    {
64    int retval;
65    unsigned char error;
66    printf("in default_config_error(0x%08x) ID:%d\n",(unsigned int)dev,(unsigned int)dev->id);
67    pp_rd_error(dev->pport->port,dev->id,&error); // read the error register
68    if(error&(1<<PP_ERROR_COLLISION0))      mon_error("collision on port 0\n");
69    if(error&(1<<PP_ERROR_COLLISION1))      mon_error("collision on port 1\n");
70    if(error&(1<<PP_ERROR_COLLISION2))      mon_error("collision on port 2\n");
71    if(error&(1<<PP_ERROR_COLLISION3))      mon_error("collision on port 3\n");
72    if(error&(1<<PP_ERROR_WRITE_OVERFLOW))  mon_error("write overflow\n");
73    if(error&(1<<PP_ERROR_READ_UNDERFLOW))  mon_error("read underflow\n");
74    if(error&(1<<PP_ERROR_INVALID_COMMAND)) mon_error("invalid command\n");
75    if(error&(1<<PP_ERROR_DEVICE_ERROR))
76       if((retval=(*dev->driver->fexec->error)(dev))!=PP_OK) return(retval);
77    pp_wr_error(dev->pport->port,dev->id,0xff); // clear the error flags
78    return(PP_OK);
79    }
80
81 //==========================================================================================
82 // scan thru ports
83 int default_config_next(struct device *dev)
84    {
85    int select;
86    int retval;
87    unsigned char state;
88
89    printf("in default_config_next(0x%08x)\n",(unsigned int)dev);
90    if((retval=pp_rd_state(dev->pport->port,dev->id,&state))!=PP_OK) return(retval);
91    select = (state>>PP_STATE_SELECT)&0x3;
92
93    if((select==0) && (dev->ports&(1<<0)) && ((dev->flags&PP_DEV_FLAGS_SELECT)==0))
94       select=0;
95    else if((select<1) && (dev->ports&(1<<1)))
96       select=1;
97    else if((select<2) && (dev->ports&(1<<2)))
```

```
      select=2;
   else if((select<3) && (dev->ports&(1<<3)))
      select=3;
   else
      select=4;

   state &= ~(0x3<<PP_STATE_SELECT); // mask state bits
   state |= (select<<PP_STATE_SELECT); // new state bits
   dev->flags |= PP_DEV_FLAGS_SELECT; // we're in the selecting process

   if(select==4)
      {
      printf("select=DONE\n");
      return(PP_DONE);
      }
   else
      {
      printf("select=%d\n",select);
      return(pp_wr_state(dev->pport->port,dev->id,state));
      }
   }

//==========================================================================================
// configure the device as a root device
int default_config_root(struct device *dev)
   {
   int retval;
   unsigned char state;
   printf("in default_config_root(0x%08x)\n",(unsigned int)dev);
   if((retval=pp_rd_state(dev->pport->port,dev->id,&state))!=PP_OK) return(retval);
   dev->flags |= PP_DEV_FLAGS_ROOT; // root flag
   state |= (1<<PP_STATE_ROOT); // root flag in device
   return(pp_wr_state(dev->pport->port,dev->id,state));
   }

//==========================================================================================
// mark the currently selected port as vacant (nothing attached)
int default_config_vacant(struct device *dev)
   {
   unsigned char state;
   int retval;
   printf("in default_config_vacant(0x%08x)\n",(unsigned int)dev);
   if((retval=pp_rd_state(dev->pport->port,dev->id,&state))!=PP_OK) return(retval);
   state |= (1<<((state >>PP_STATE_SELECT)&0x3)); // mark the current selected bit as vacant
   return(pp_wr_state(dev->pport->port,dev->id,state));
   }

//==========================================================================================
// enable the device
int default_config_enable(struct device *dev)
   {
   int retval;
   unsigned char state;
   printf(__FILE__ ": in default_config_enable(0x%08x)\n",(unsigned int)dev);
   if((retval=pp_rd_state(dev->pport->port,dev->id,&state))!=PP_OK) return(retval);
   state |= (1<<PP_STATE_ENABLE); //
   dev->flags |= PP_DEV_FLAGS_ENABLED;
   return(pp_wr_state(dev->pport->port,dev->id,state));
   }

//==========================================================================================
// disable the device
int default_config_disable(struct device *dev)
   {
   int retval;
   unsigned char state;
```

```
   printf(__FILE__ ": in default_config_disable(0x%08x)\n",(unsigned int)dev);
   if((retval=pp_rd_state(dev->pport->port,dev->id,&state))!=PP_OK) return(retval);
   state &= ~(1<<PP_STATE_ENABLE); //
   dev->flags |= PP_DEV_FLAGS_ENABLED;
   return(pp_wr_state(dev->pport->port,dev->id,state));
   }

struct function_config default_config =
   {
   default_config_config,
   default_config_removed,
   default_config_error,
   default_config_next,
   default_config_root,
   default_config_vacant,
   default_config_enable,
   default_config_disable
   };
```

12.6 Peripheral Port includes (PPTEST_H)

```
#define CTRL_PORT0     0
#define CTRL_PORT1     1
#define CTRL_BCAST     0

int ctrl_open();
int ctrl_close();
int ctrl_xfer(int control);
int ctrl_write(int register,int data);
int ctrl_read(int register,int *data);
int ctrl_wr_cmd(int port,int data);
int ctrl_wr_data(int port,int data);
int ctrl_rd_data(int port,int *data);
int ctrl_wr_stat(int port,int data);
int ctrl_rd_stat(int port,int *data);
int ctrl_fifo_read(int size,unsigned char *data);

// much match epp.pv
#define CTRL_WR_CMD    0x10     //00_0_1_0000
#define CTRL_WR_DATA   0x11     //00_0_1_0001
#define CTRL_WR_STAT   0x12     //00_0_1_0010
#define CTRL_WR_TEST   0x13     //00_0_1_0011
#define CTRL_RD_DATA   0x01     //00_0_0_0001
#define CTRL_RD_STAT   0x02     //00_0_0_0010
#define CTRL_RD_TEST   0x03     //00_0_0_0011
#define CTRL_INT       0x13     //00_0_1_0011
#define CTRL_PORT_SHIFT 5

int pp_cmd_test(int *data);
```

12.7 Peripheral Port Command constructor (PPORT_C)

```
/*
peripheral port primitives (pport.c)
Written by Jude Katsch @ ANON
02/01/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/

```

```
 9 // todo :
10 // add interrupts
11
12 #include <stdio.h>
13 #include "pport.h"
14 #include "epp.h"
15 #include "fifo.h"
16
17 extern int quiet;
18
19 //==========================================================================================
20 // errors
21
22 char * pp_error(int error)
23    {
24    char *str="PP_OK";
25    switch(error)
26       {
27       case PP_OK:
28         str="PP_OK";
29         break;
30       case PP_BAD_PARAM:
31         str="PP_BAD_PARAM";
32         break;
33       case PP_TIMEOUT:
34         str="PP_TIMEOUT";
35         break;
36       case PP_BAD_PARITY:
37         str="PP_BAD_PARITY";
38         break;
39       case PP_INTERNAL:
40         str="PP_INTERNAL";
41         break;
42       case PP_TOO_MANY_DEVICES:
43         str="PP_TOO_MANY_DEVICES";
44         break;
45       case PP_DONE:
46         str="PP_DONE";
47         break;
48       case PP_BAD_MAGIC:
49         str="PP_BAD_MAGIC";
50         break;
51       case PP_CRC_ERROR:
52         str="PP_CRC_ERROR";
53         break;
54       case PP_EMPTY:
55         str="PP_EMPTY";
56         break;
57       case PP_BAD_DATA:
58         str="PP_BAD_DATA";
59         break;
60       case PP_BAD_ADDRESS:
61         str="PP_BAD_ADDRESS";
62         break;
63       case PP_NOT_WRITEABLE:
64         str="PP_NOT_WRITEABLE";
65         break;
66       case PP_DRIVER_UNAVAILABLE:
67         str="PP_DRIVER_UNAVAILABLE";
68         break;
69       case PP_BAD_MODE:
70         str="PP_BAD_MODE";
71         break;
72       case PP_FLASH_SUSPEND:
73         str="PP_FLASH_SUSPEND";
74         break;
```

```
      case PP_FLASH_TIMEOUT:
        str="PP_FLASH_TIMEOUT";
        break;
      case PP_FLASH_BUSY:
        str="PP_FLASH_BUSY";
        break;

      default:
        str="unknown";
        break;
      }
   if((error!=PP_OK) && !quiet) printf("pp_error: %s\n",str);
   return(str);
   }

int pp_error_poll(int error) // ignore parity on poll commands
   {
   int error_out;
   error_out = (error==PP_BAD_PARITY) ? PP_OK : error;
   pp_error(error_out);
   return(error_out);
   }

//==========================================================================================
// range checks

int pp_check_id(int id)
   {
   if(id<0||id>31)
      {
      printf("invalid ID should be 0 to 31, was %d\n",id);
      exit(PP_BAD_PARAM);
      }
   return(PP_OK);
   }

int pp_check_port(int port)
   {
   if(port<0||port>1)
      {
      printf("invalid PORT should be 0 or 1, was %d\n",port);
      exit(PP_BAD_PARAM);
      }
   return(PP_OK);
   }

int pp_check_size(int size)
   {
   if(size<0||size>127)
      {
      printf("invalid SIZE should be between 0 or 127, was %d\n",size);
      exit(PP_BAD_PARAM);
      }
   return(PP_OK);
   }

int pp_check_address(int address)
   {
   if(address<0||address>255)
      {
      printf("invalid ADDRESS should be between 0 or 255, was %d\n",address);
      exit(PP_BAD_PARAM);
      }
   return(PP_OK);
   }

```

```
141 // check for matching 4 bytes
142 int pp_check_byte(int data)
143    {
144    if(((data&0xff)!=((data>>8)&0xff))||
145       ((data&0xff)!=((data>>16)&0xff))||
146       ((data&0xff)!=((data>>24)&0xff)))
147       return(PP_BAD_DATA);
148    return(PP_OK);
149    }
150
151 int check_crc(int was,int exp)
152    {
153    if(was!=exp)
154       {
155       printf("ERROR CRC mismatch, expected:0x%04x was:0x%04x\n",exp,was);
156       return(PP_BAD_DATA);
157       }
158    return(PP_OK);
159    }
160
161 //==========================================================================================
162 // read write peripharl port status registers
163
164 int pp_status_write(int port, int status)
165    {
166    ctrl_wr_stat(port,status);
167 #ifdef DEBUG
168    printf("pp_status_write 0x%08x\n",status);
169 #endif
170    return(PP_OK);
171    }
172
173 int pp_status_read(int port, int *status)
174    {
175    ctrl_rd_stat(port,status);
176 #ifdef DEBUG
177    printf("pp_status_read 0x%08x\n",*status);
178 #endif
179    return(PP_OK);
180    }
181
182 //==========================================================================================
183 // check fifo status (depth=1) UGH!
184
185 int pp_wait_txempty(int port)
186    {
187    int data;
188    int dummy;
189 #ifdef DEBUG
190    printf("PORT:%d pp_wait_txempty\n",port);
191 #endif
192    do {
193       pp_status_read(port,&data);
194      } while(data&(1<<CTRL_STAT_TXFULL));
195
196    if(data&(1<<CTRL_STAT_RXFULL))
197      {
198      ctrl_rd_data(port,&dummy);
199      return(pp_wait_txempty(port));
200      }
201
202    return(PP_OK);
203    }
204
205 int pp_wait_rxfull(int port)
206    {
```

```
   int data,i,prev_data;;
#ifdef DEBUG
   printf("pp_wait_rxfull\n");
#endif
   for(i=0;i<TIMEOUT;i++)
     {
     pp_status_read(port,&data);
     if(data&(1<<CTRL_STAT_RXOVERFLOW))
        {
        pp_status_write(port,data|(1<<CTRL_STAT_RXOVERFLOW));
        mon_error("RXOVERFLOW i=%d prev_data=0x%08x data=0x%08x\n",i,prev_data,data);
        }
     if(data&(1<<CTRL_STAT_RXFULL)) return(PP_OK);
     prev_data=data;
     }
   printf("pp_wait_rxfull TIMEOUT\n");
   return(PP_TIMEOUT);
   }

//==========================================================================================
// commands

int eparity(int data)
   {
   int eparity;
   eparity  = (data>>16)^data;
   eparity ^= (eparity>>8);
   eparity ^= (eparity>>4);
   eparity ^= (eparity>>2);
   eparity ^= (eparity>>1);
   return(eparity&0x1);
   }

int pp_cmd(int port,int data)
   {
   pp_wait_txempty(port);
   ctrl_wr_cmd(port,eparity(data)|data);
   return(PP_OK);
   }

int pp_command(int port,int id, int  command)
   {
   int data;
   pp_check_id(id);
   pp_check_port(port);
   data=command|CMD_ID(id);
   pp_cmd(port,data);
   return(PP_OK);
   }

int pp_command_read(int port,int id, int command, int *data)
   {
   int idata,retval;
   pp_check_id(id);
   pp_check_port(port);
   idata=command|CMD_ID(id);
   pp_cmd(port,idata);
   if((retval=pp_wait_rxfull(port))!=PP_OK) return(retval);
   ctrl_rd_data(port,data);
   return((eparity(*data))?PP_BAD_PARITY:PP_OK);
   }

//==========================================================================================
// read/write data

int pp_data_write_one(int port,int id, int address, int size, int flags, unsigned char data)
```

```
   {
   int command;
   int inc_add = CMD_IA&flags;
   pp_check_address(address);
   pp_check_size(size);
   command  = CMD_WRITE;
   command |= flags;
   command |= CMD_SIZE(size);
   command |= CMD_COMMAND(address);
   command |= CMD_XTRA(data);
   command |= CMD_WRITE_DIRECT;
   command |= CMD_ID(id);
   size = (command>>9)&MAX_XFER_SIZE;
   data = (command>>1)&0xff;
   address = (command>>17)&0xff;
   mon_write("DATA:%d:%d:WRITE:%d:%d\n",port,id,address,data);
   if(!quiet) printf("ID:%d writing %d byte%s of 0x%02x %sat address 0x%02x
command:0x%08x\n",id,size,(size==1)?"":"s",data,(inc_add)?"starting ":"",address,command);
   return(pp_command(port,id,command));
   }

int pp_data_write(int port,int id, int address, int size, int flags, unsigned char *data)
   {
   int command;
   int retval;
   int i,j,k;
   int crc=0;
   int bytes;
   int add;
   int inc_add =(flags & CMD_IA);
   int do_crc = (flags & CMD_CRC);
   unsigned char byte_data;
   int long_data;
   pp_check_address(address);
   pp_check_size(size);
   command  = CMD_WRITE;
   command |= flags;
   command |= CMD_SIZE(size);
   command |= CMD_COMMAND(address);
   command |= CMD_ID(id);
   size = (command>>9)&MAX_XFER_SIZE;
   address = (command>>17)&0xff;
   if(!quiet) printf("ID:%d writing %d byte%s at address 0x%02x (%s address)
command:0x%08x\n",id,size,(size==1)?"":"s",address,(inc_add)?"increment":"single",command);
   if((retval=pp_command(port,id,command))!=PP_OK) return(retval);
   for(i=0;i<((size+3)>>2);i++)
      {
      bytes=(size-(i*4));
      if(bytes>4) bytes=4;
      add=address+((inc_add)?i*4:0);
      long_data=0;
      for(j=0;j<bytes;j++)
{
        byte_data=data[(i*4)+j];
        mon_write("DATA:%d:%d:WRITE:%d:%d\n",port,id,add+((inc_add)?j:0),byte_data);
        crc=crc_calc(byte_data,crc);
        if(!quiet) printf("(0x%02x)=0x%02x crc=0x%04x\n",add+((inc_add)?j:0),byte_data,crc);
        long_data |= byte_data<<((3-j)*8); // convert to big endian
}
      for(k=bytes;k<4;k++) // left over bytes in long (set to zero)
        {
        crc=crc_calc(0x00,crc);
        if(!quiet) printf("0x%02x crc=0x%04x\n",0x00,crc);
        }
      if(!quiet) printf("(0x%02x)=0x%02x_%02x_%02x_%02x crc=0x%04x\n",add,
         (long_data>>24)&0xff,(long_data>>16)&0xff,(long_data>>8)&0xff,(long_data>>0)&0xff,crc);
```

```
      pp_wait_txempty(port);
      ctrl_wr_data(port,long_data);
      }
   if(do_crc)
      {
      if((retval=pp_wait_rxfull(port))!=PP_OK) return(retval);
      ctrl_rd_data(port,&long_data);

if(!quiet) printf("0x%02x_%02x_%02x_%02x=(CRC)\n",(long_data>>24)&0xff,(long_data>>16)&0xff,(long_data>>8)&0xff,(long_da
      return(check_crc(crc,(long_data>>16)&0xffff));
      }
   return(PP_OK);
   }

int pp_data_read(int port,int id, int address, int size, int flags, unsigned char *data)
   {
   int command;
   int retval;
   int i,j;
   int crc=0,crc_exp;
   int bytes;
   int add;
   int inc_add = (flags & CMD_IA);
   int do_crc = (flags & CMD_CRC);
   int long_data;
   unsigned char byte_data;
   pp_check_address(address);
   pp_check_size(size);
   command  = CMD_READ;
   command |= flags;
   command |= CMD_SIZE(size);
   command |= CMD_COMMAND(address);
   command |= CMD_ID(id);
   size = (command>>9)&MAX_XFER_SIZE;
   address = (command>>17)&0xff;
   if(!quiet) printf("ID:%d reading %d byte%s at address 0x%02x (%s address)
command:0x%08x\n",id,size,(size==1)?"":"s",address,(inc_add)?"increment":"single",command);
   if((retval=pp_command(port,id,command))!=PP_OK) return(retval);
   for(i=0;i<((size+3)>>2);i++)
      {
      if((retval=pp_wait_rxfull(port))!=PP_OK) return(retval);
      ctrl_rd_data(port,&long_data);
      add=address+((inc_add)?i*4:0);

if(!quiet) printf("0x%02x_%02x_%02x_%02x=(0x%02x)\n",(long_data>>24)&0xff,(long_data>>16)&0xff,(long_data>>8)&0xff,(long_
      bytes=(size-(i*4));
      if(bytes>4) bytes=4;
      for(j=0;j<bytes;j++)
{
        byte_data=data[(i*4)+j]=(unsigned char)(long_data>>((3-j)*8))&0xff;
        crc=crc_calc(byte_data,crc);
        if(!quiet) printf("0x%02x=(0x%02x) crc=0x%04x\n",byte_data,add+((inc_add)?j:0),crc);
        mon_write("DATA:%d:%d:READ:%d:%d\n",port,id,add+((inc_add)?j:0),byte_data);
}
      }
   crc_exp = crc; // if we don't override then crc will match
   switch(size%4)
     {
     case 1:  // dduull00
        crc_exp=(long_data>>8)&0xffff;
        break;
     case 2:  // ddddduull
        crc_exp=(long_data)&0xffff;
        break;
     case 3:  // ddddddduu ll000000
        crc_exp=((long_data<<8)&0xff00)|(crc_exp&0xff);
```

```
      case 0:  // ddddddddd uull0000
       if(!do_crc) break; // don't do extra crc read
       if((retval=pp_wait_rxfull(port))!=PP_OK) return(retval);
       ctrl_rd_data(port,&long_data);

if(!quiet) printf("0x%02x_%02x_%02x_%02x=(CRC)\n",(long_data>>24)&0xff,(long_data>>16)&0xff,(long_data>>8)&0xff,(long_da
       if(size%4)
          crc_exp=((long_data>>24)&0xff)|(crc_exp&0xff00);
       else
          crc_exp=(long_data>>16)&0xffff;
      }
   return(check_crc(crc,crc_exp));
   }

//=================================================================================================
// poll commands

int pp_cmd_poll(int port,int *data,int command,char *name)
   {
   int retval;
   command |= CMD_READ | CMD_XTRA(0x40) | CMD_SIZE(4) | CMD_IA;
   retval  = (pp_command_read(port,CTRL_BCAST,command,data));

   if(!quiet) printf("command:0x%08x %s returned: %02x_%02x_%02x_%02x\n",command,name,
           ((*data>>24)&0xff),((*data>>16)&0xff),((*data>>8)&0xff),((*data>>0)&0xff));

   return(pp_error_poll(retval));
   }

int pp_command_error(int port,int *error)
   { return(pp_cmd_poll(port,error,PP_CMD_ERROR,"ERROR")); }

int pp_command_alive(int port,int *alive)
   { return(pp_cmd_poll(port,alive,PP_CMD_ALIVE,"ALIVE")); }

int pp_command_request(int port,int *request)
   { return(pp_cmd_poll(port,request,PP_CMD_REQUEST,"REQUEST")); }

//=================================================================================================
// execute commands

int pp_cmd_exec(int port,int id,int command,char *name)
   {
   command |= CMD_WRITE | CMD_SIZE(0);
   if(!quiet) printf("ID:%d command:0x%08x %s\n",id,command,name);
   return(pp_command(port,id,command));
   }

int pp_command_reset(int port,int id)
   { return(pp_cmd_exec(port,id,PP_CMD_RESET,"RESET")); }

int pp_command_blur(int port,int id)
   { return(pp_cmd_exec(port,id,PP_CMD_BLUR,"BLUR")); }

int pp_command_focus(int port,int id)
   { return(pp_cmd_exec(port,id,PP_CMD_FOCUS,"FOCUS")); }

int pp_command_tag(int port,int id)
   { return(pp_cmd_exec(port,id,PP_CMD_TAG,"TAG")); }

int pp_command_unbrand(int port,int id)
   { return(pp_cmd_exec(port,id,PP_CMD_UNBRAND,"UNBRAND")); }

int pp_command_brand(int port,int id,int brand)
   { return(pp_cmd_exec(port,id,PP_CMD_BRAND|((0x1f&brand)<<1),"BRAND")); }
```

```
465 //==========================================================================================
466 // read commands
467
468 int pp_cmd_read(int port,int id,unsigned char *data,int command,char *name)
469    {
470    int retval;
471    int long_data;
472    if(!quiet) printf("ID:%d %s\n",id,name);
473    retval=pp_data_read(port,id,command,4,CMD_CRC|CMD_IA,data);
474    long_data = (data[3]<<24)|(data[2]<<16)|(data[1]<<8)|(data[0]<<0);
475    return(((retval==PP_OK)&&eparity(long_data))?PP_BAD_PARITY:retval);
476    }
477
478 int pp_command_probe(int port, int id, unsigned char *probe)
479    {
480    int retval;
481    if((retval=pp_cmd_read(port,id,probe,PP_CMD_PROBE,"PROBE"))!=PP_OK) return(retval);
482    return((probe[3]|probe[2]|probe[1]|probe[0])?PP_OK:PP_EMPTY);
483    }
484
485 int pp_command_magic(int port, int id)
486    {
487    int retval;
488    unsigned char magic[4];
489    if((retval=pp_cmd_read(port,id,magic,PP_CMD_MAGIC,"MAGIC"))!=PP_OK) return(retval);
490    if((magic[0]!=PP_MAGIC0)||(magic[1]!=PP_MAGIC1)||(magic[2]!=PP_MAGIC2)||(magic[3]!=PP_MAGIC3))
491      {
492       printf("id=%d invalid magic number expected:0x%02x_%02x_%02x_%02x (%c%c%c%c) was:0x%02x_%02x_%02x_%02x\n",
493      id,PP_MAGIC0,PP_MAGIC1,PP_MAGIC2,PP_MAGIC3,PP_MAGIC0,PP_MAGIC1,PP_MAGIC2,PP_MAGIC3,
494              magic[0],magic[1],magic[2],magic[3]);
495       return(PP_BAD_MAGIC);
496       }
497    return(PP_OK);
498    }
499
500 //==========================================================================================
501 // read command registers
502
503 int pp_cmd_reg_read(int port,int id,unsigned char *data,int command,char *name)
504    {
505    int retval;
506    if(!quiet) printf("ID:%d %s\n",id,name);
507    retval=pp_data_read(port,id,command,1,CMD_IA|CMD_CRC,data);
508    pp_error(retval);
509    return(retval);
510    }
511
512 int pp_rd_state(int port, int id, unsigned char *data)
513    { return(pp_cmd_reg_read(port,id,data,PP_CMD_RSTATE,"RD-STATE")); }
514
515 int pp_rd_error(int port, int id, unsigned char *data)
516    { return(pp_cmd_reg_read(port,id,data,PP_CMD_RERROR,"RD-ERROR")); }
517
518 //==========================================================================================
519 // direct write commands
520
521 int pp_cmd_direct(int port,int id,int command,unsigned char data,char *name)
522    {
523    int retval;
524    if(!quiet) printf("ID:%d %s\n",id,name);
525    retval=pp_data_write_one(port,id,command,1,0,data);
526    pp_error(retval);
527    return(retval);
528    }
529
530 int pp_wr_state(int port, int id, unsigned char data)
```

```
   { return(pp_cmd_direct(port,id,PP_CMD_RSTATE,data,"WR-STATE")); }

int pp_wr_error(int port, int id, unsigned char data)
   {  return(pp_cmd_direct(port,id,PP_CMD_RERROR,data,"WR-ERROR")); }
```

12.8 FIFO include (FIFO_H)

```


void cmd_close();
void mon_close();
void fifo_close();
int cmd_open();
int mon_open();
int mon_write(char *fmt,...);
int mon_error(char *fmt,...);
int cmd_read(char *buffer,int size);
int bytetobinary(char *string,int num);
int longtobinary(char *string,int num);
```

12.9 FIFO interface to/from pptrace (FIFO_C)

```
/*
peripheral port test
Written by Jude Katsch @ ANON
04/01/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/


#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include "fifo.h"
#include <stdarg.h>

// Open the input fifo and wait for a responce before opening the output fifo

char *mon_namne="pptrace.mon";
int fifo_mon=0;
char *cmd_name="pptrace.cmd";
int fifo_cmd=0;
struct stat sbuf;

void mon_close()
   {
   if(fifo_mon>0) close(fifo_mon);
   fifo_mon=0;
   }

void cmd_close()
   {
   if(fifo_cmd>0) close(fifo_cmd);
   fifo_cmd=0;
   }
```

```
 39
 40 void fifo_close()
 41    {
 42    mon_close();
 43    cmd_close();
 44    }
 45
 46 int cmd_open()
 47    {
 48 //   char buf[2];
 49
 50    if(fifo_cmd) return(0);
 51
 52    cmd_close();
 53
 54 // open input fifo
 55    printf(__FILE__ ": opening '%s' fifo for reading\n",cmd_name);
 56    if((stat(cmd_name,&sbuf)<0)||!(sbuf.st_mode&S_IFIFO))
 57       {
 58       mknod(cmd_name,S_IFIFO|0277,0);
 59       chmod(cmd_name,0600);
 60       }
 61
 62 //   if((int)(fifo_cmd=open(cmd_name,O_RDWR|O_NDELAY))<=0)
 63 //   if((int)(fifo_cmd=open(cmd_name,O_RDWR))<=0)
 64    if((int)(fifo_cmd=open(cmd_name,O_RDWR))<=0)
 65       {
 66       fprintf(stderr,__FILE__ ": could not open '%s' fifo for reading\n",cmd_name);
 67       return(-1);
 68       }
 69
 70 //   while (read(fifo_cmd,buf,1) > 0); // drain the fifo (should not be necessary)
 71 //   fcntl(fifo_cmd,F_SETFL,fcntl(fifo_cmd,F_GETFL)|O_NONBLOCK);
 72    return(0);
 73    }
 74
 75 int mon_open()
 76    {
 77    mon_close();
 78
 79 // open output fifo
 80    printf(__FILE__ ": opening '%s' fifo for writing\n",mon_namne);
 81    if((stat(mon_namne,&sbuf)<0)||!(sbuf.st_mode&S_IFIFO))
 82       {
 83       mknod(mon_namne,S_IFIFO|0277,0);
 84       chmod(mon_namne,0600);
 85       }
 86
 87 //   if((fifo_mon=open(mon_namne,O_RDWR|O_NDELAY))<0)
 88    if((fifo_mon=open(mon_namne,O_RDWR))<0)
 89       {
 90       fprintf(stderr,__FILE__ ": could not open '%s' fifo for writing\n",mon_namne);
 91       return(1);
 92       }
 93
 94    return(0);
 95    }
 96
 97 int fifo_write(char *data)
 98   {
 99   int size,retval;
100   size=strlen(data);
101
102   while((fifo_mon>0)&&(size>0))
103      {
104      if((retval=write(fifo_mon,data,size))<0)
```

```
105 {
106         perror("writing message to fifo");
107         return(-1);
108         }
109      size-=retval;
110      }
111    return(0);
112   }
113
114
115 // NOTE: ok to call when fifo is closed
116 static char buffer[1024]="ERROR:";
117 #define BUF_START 6
118 int mon_write(char *fmt,...)
119    {
120    va_list argp;
121    va_start(argp, fmt);
122    vsprintf(&buffer[BUF_START], fmt, argp);
123    va_end(argp);
124 //   printf("mon_write(%s)\n",&buffer[BUF_START]);
125    return(fifo_write(&buffer[BUF_START]));
126    }
127
128 // NOTE: ok to call when fifo is closed
129 int mon_error(char *fmt,...)
130    {
131    va_list argp;
132    va_start(argp, fmt);
133    vsprintf(&buffer[BUF_START], fmt, argp);
134    va_end(argp);
135    fprintf(stderr,buffer);
136    return(fifo_write(buffer));
137    }
138
139 // returns >=0 numnber of characters read
140 // -1 = error, fifo closed
141 // will try to open fifo_cmd if closed
142 // reads a line of text until \n convert to \0
143 int cmd_read(char *buffer,int size)
144    {
145    static char sbuf[1024];
146    static int cpos=0; // current position
147    int i,j,k,l;
148    int first=1;
149
150    if(size==0) return(0);
151    if(cmd_open()) return(-1);
152
153    i=cpos;
154    cpos=0;
155
156    while(1)
157       {
158       if((i==0)&&((i=read(fifo_cmd,sbuf,sizeof(sbuf)-cpos))<=0))
159          {
160          if(errno==EAGAIN) continue;
161          if(i==0)
162             fprintf(stderr,__FILE__ ": EOF on fifo_cmd\n");
163          else
164             perror(__FILE__ ": reading fifo_cmd");
165          cmd_close();
166          return(-4);
167          }
168       first=0;
169
170 //      printf("i=%d errno=%d sbuf=%s\n",i,errno,sbuf);
```

```

      for(j=0;j<i;j++)
{
        if(sbuf[cpos+j]=='\n')
  {
          if(cpos+j+1>size) return(-2); // too large!
          for(k=0;k<(cpos+j);k++)
             buffer[k]=sbuf[k]; // copy to other buffer
          buffer[k]='\0'; // eos
          for(j++,l=0;j<i;j++)
             sbuf[l++]=sbuf[cpos+j]; // align to [0]
          cpos=l;
          return(++k);
          }
        }
      cpos+=i;
      i=0;
      }
   return(-3); // should never get here
   }



int longtobinary(char *string,int num)
  {
  int i,j;
// 88888888_88888888_88888888_88888888
  for(i=0,j=0;i<32;i++,num<<=1)
    {
//    printf("0x%08x %s",num&(1<<31),string);
    if(i&&(i%8==0)) string[j++]='_';
    string[j++]=(num&(1<<31))?'1':'0';
    }
  return(++j);
  }

int bytetobinary(char *string,int num)
  {
  int i,j;
// 88888888
  for(i=0,j=0;i<8;i++,num<<=1)
    string[j++]=(num&(1<<7))?'1':'0';
  return(++j);
  }
```

12.10 Interface between pport and epp (CTRL_C)

```
 /*
peripheral port socket ctrl.c
Written by Jude Katsch @ ANON
04/01/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/

#include "epp.h"
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
extern FILE *fifo_fd;
```

```
17
18 unsigned char  buffer [256];
19 char  byte_string [8];
20 int pp_read_fifo()
21    {
22    int retval;
23    int data;
24    int depth;
25    int overflow,underflow;
26    int i,j,k;
27    return(0);
28 //   if(fifo_fd<=0) return(0); // fifo not open
29    if((retval=ctrl_read((1<<CTRL_PORT_SHIFT)|CTRL_RD_TEST,&data))) return(retval); // read info reg
30    depth = data&0xffff;
31    underflow = (data>>16)&0x1;
32    overflow = (data>>17)&0x1;
33 //#ifdef DEBUG
34    printf("pp_read_fifo fifo_of=%d fifo_uf=%d fifo_depth=%d\n",overflow,underflow,depth);
35 //#endif
36    if(overflow) printf("pp_read_fifo: ERROR: fifo overflow\n");
37    if(underflow) printf("pp_read_fifo: ERROR: fifo underflow\n");
38    if(depth>4) // wait until we have accumulated some data
39      {
40 //     ctrl_fifo_read(depth,buffer);
41 //     for(j=0;j<depth;j++)
42 //         {
43 //         for(i=0;i<4;i++)
44 //            byte_string[i]='0'+((buffer[j]>>((3-i)*2))&0x3);
45 //         for(k=4;k>0;k-=retval)
46 //            {
47 //            retval=fwrite(byte_string,1,k,fifo_fd);
48 //            fflush(fifo_fd);
49 //            if(retval<0)
50 //       {
51 //       fprintf(stderr,"pp_read_fifo: can't write to fifo, error=%d\n",errno);
52 //       fclose(fifo_fd);
53 //       fifo_fd=(FILE *)retval;
54 //       }
55 //            }
56 //         }
57      }
58     return(0);
59     }
60
61 int ctrl_wr_cmd(int port,int data) // data is in upper 16 bits
62    {
63    pp_read_fifo();
64 #ifdef DEBUG
65    printf("ctrl_wr_cmd 0x
   %02x_%02x_%02x_%02x\n",((data>>24)&0xff),((data>>16)&0xff),((data>>8)&0xff),((data>>0)&0xff));
66 #endif
67    return(ctrl_write((port<<CTRL_PORT_SHIFT)|CTRL_WR_CMD,data));
68    }
69
70 int ctrl_wr_data(int port,int data)
71    {
72    pp_read_fifo();
73 #ifdef DEBUG
74    printf("ctrl_wr_data 0x
   %02x_%02x_%02x_%02x\n",((data>>24)&0xff),((data>>16)&0xff),((data>>8)&0xff),((data>>0)&0xff));
75 #endif
76    return(ctrl_write((port<<CTRL_PORT_SHIFT)|CTRL_WR_DATA,data));
77    }
78
79 int ctrl_rd_data(int port,int *data)
80    {
```

```
   int retval;
   pp_read_fifo();
//   ctrl_write((port<<CTRL_PORT_SHIFT)|CTRL_INT,0);
   retval=ctrl_read((port<<CTRL_PORT_SHIFT)|CTRL_RD_DATA,data);
#ifdef DEBUG
   printf("ctrl_rd_data 0x
%02x_%02x_%02x_%02x\n",((*data>>24)&0xff),((*data>>16)&0xff),((*data>>8)&0xff),((*data>>0)&0xff));
#endif
   return(retval);
   }

int ctrl_wr_stat(int port,int data)
   {
   pp_read_fifo();
#ifdef DEBUG
   printf("ctrl_wr_state 0x
%02x_%02x_%02x_%02x\n",((data>>24)&0xff),((data>>16)&0xff),((data>>8)&0xff),((data>>0)&0xff));
#endif
   return(ctrl_write((port<<CTRL_PORT_SHIFT)|CTRL_WR_STAT,data));
   }

int ctrl_rd_stat(int port,int *data)
   {
   int retval;
   pp_read_fifo();
   retval=ctrl_read((port<<CTRL_PORT_SHIFT)|CTRL_RD_STAT,data);
#ifdef DEBUG
   printf("ctrl_rd_state 0x
%02x_%02x_%02x_%02x\n",((*data>>24)&0xff),((*data>>16)&0xff),((*data>>8)&0xff),((*data>>0)&0xff));
#endif
   return(retval);
   }

//=========================================================================================
// pp_cmd_test

int pp_cmd_test(int *data)
   {
   ctrl_write(CTRL_WR_TEST,*data);
   return(ctrl_read(CTRL_RD_TEST,data));
   }
```

12.11 Socket EPP (enhanced parallel port) include (SEPP_H)

```
#define SEPP_DATA         0   // read/write
#define SEPP_DATA_SIZE    8
#define SEPP_WRITE        8   // write
#define SEPP_ADDRSTB      9   // write
#define SEPP_DATASTB      10  // write
#define SEPP_INTR         11  // read
#define SEPP_UD1          12  // read
#define SEPP_UD2          13  // read
#define SEPP_UD3          14  // read
#define SEPP_CYCLE        15  // read
#define SEPP_TIMEOUT      16  // read
#define SEPP_BUSY         17  // read

#define SEPP_RESET_SET    18  // write
#define SEPP_RESET_CLR    19  // write
#define SEPP_RESET        20  // read
#define SEPP_INTEN_SET    21  // write
#define SEPP_INTEN_CLR    22  // write
#define SEPP_INTEN        23  // read
```

```
#define SEPP_TIMEOUT_CLR  24  // write
```

12.12 Socket EPP (enhanced parallel port) interface (for verilog environment) (SEPP_C)

```
 /*
peripheral port socket epp  (sepp.c)
Written by Jude Katsch @ ANON
02/01/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/

#include <sys/types.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/time.h>
#include <netinet/in.h>
#include <netdb.h>
#include <stdio.h>
#include <errno.h>
#include "epp.h"
#include "sepp.h"
#include "pport.h"

// add interrupts

int server_handle=0;
int client_handle=0;

// these routines NEVER return an interface error, they always exit()

void error_exit(char *str)
   {
   ctrl_close();
   fprintf(stderr,"sepp: error_exit '%s' error=%d\n",str,errno);
   exit(1);
   }

//#define DEBUG

int ctrl_open()
   {
   int i;
   int retval;
   int length;
   FILE *fp;
   struct sockaddr_in server;
   char *filename="port.tmp";

   if((fp=fopen(filename,"w"))==NULL)
      error_exit("can't open port file\n");

   if((server_handle=socket(AF_INET,SOCK_STREAM,0))<0)
      error_exit("socket()");

   server.sin_family = AF_INET;
   server.sin_addr.s_addr = INADDR_ANY;
   server.sin_port = 0;

   if (bind(server_handle,(struct sockaddr *)&server,sizeof(server)))
      error_exit("bind()");
```

```

   length = sizeof(server);
   if (getsockname(server_handle,(struct sockaddr *)&server,&length))
      error_exit("getsockname()");

   printf("sepp: socket has port #%d\n", ntohs(server.sin_port));
   fprintf(fp,"%d", ntohs(server.sin_port));
   fclose(fp);

   printf("sepp: listening...\n");
   if(listen(server_handle,1)<0)
      error_exit("listen()");

   if((client_handle=accept(server_handle,(struct sockaddr *)NULL,(int *)NULL))<0)
      error_exit("accept()");

   printf("sepp: connection accepted...\n");

   printf("sepp: resetting epp...\n");
   ctrl_xfer((1<<SEPP_RESET_SET)|(1<<SEPP_TIMEOUT_CLR)|(1<<SEPP_INTEN_CLR));
   ctrl_xfer( 1<<SEPP_RESET_CLR);
   for(i=0;i<4;i++)
      ctrl_xfer((1<<SEPP_RESET_SET)|(1<<SEPP_TIMEOUT_CLR));
   retval=ctrl_xfer( 1<<SEPP_RESET_CLR);
   return(0);
   }

int ctrl_close()
   {
   if(server_handle>0) close(server_handle);
   server_handle = 0;

   if(client_handle>0) close(client_handle);
   client_handle = 0;

   return(0);
   }

int ctrl_xfer(int control)
   {
   int convert=0;
   char acontrol[33];
   int i;

   if(client_handle==0)
      error_exit("lost client_handle");

   for(i=31;i>=0;i--)
      {
      acontrol[i]='0'+(control&1);
      control>>=1;
      }

   acontrol[32]=0;

#ifdef DEBUG
   printf("sepp: sending '%s'\n",acontrol);
#endif

   if(send(client_handle,acontrol,33,0)<0)
       error_exit("send()");

   if((i=recv(client_handle,acontrol,33,0))<0)
       error_exit("recv()");

   if(i==0)
```

```
124      {
125      printf("sepp: closing client connection\n");
126      exit(0);
127      }
128
129 #ifdef DEBUG
130    printf("sepp: received '%s'\n",acontrol);
131 #endif
132
133    control=0;
134    for(i=0;i<=31;i++)
135       {
136       control<<=1;
137       if(acontrol[i]=='1')
138          control |= 1;
139       else if(acontrol[i]!='0')
140          convert|=1;
141       }
142
143    if(convert)
144      {
145      printf("unexpected bits in 0x%s converted to 0x%08x\n",acontrol,control);
146 //      exit(1); // abort!!!
147      }
148 //   printf("control=0x%08x\n",control);
149
150    if(control&(1<<SEPP_TIMEOUT))
151       error_exit("EPP TIMEOUT");
152
153    return(control);
154    }
155
156 int ctrl_write(int reg,int data)
157    {
158    int i;
159    for(i=0;i<4;i++)
160       {
161       ctrl_xfer((1<<SEPP_WRITE)|(1<<SEPP_DATASTB)|((data&0xff)<<SEPP_DATA));
162       data >>= 8;
163       }
164    ctrl_xfer((1<<SEPP_WRITE)|(1<<SEPP_ADDRSTB)|((reg&0xff)<<SEPP_DATA));
165    return(0);
166    }
167
168 int ctrl_read(int reg,int *data)
169    {
170    int i;
171    unsigned int idata=0;
172    int retval;
173    ctrl_xfer((1<<SEPP_WRITE)|(1<<SEPP_ADDRSTB)|((reg&0xff)<<SEPP_DATA));
174    for(i=0;i<4;i++)
175       {
176       idata>>=8;
177       retval=ctrl_xfer((0<<SEPP_WRITE)|(1<<SEPP_DATASTB));
178       idata |=(retval&0xff)<<24;
179 //      printf("idata=0x%08x\n",idata);
180       }
181    *data=idata;
182    return(0);
183    }
184
185
186 int ctrl_fifo_read(int size, unsigned char * data)
187    {
188    int i;
189    int retval;
```

```
   for(i=0;i<size;i++)
      {
      retval=ctrl_xfer((0<<SEPP_WRITE)|(1<<SEPP_ADDRSTB));
      data[i] = (unsigned char) retval&0xff;
      }
   return(0);
   }
```

12.13 EPP (enhanced parallel port) (for linux environment) (EPP_C)

```
 /*
peripheral port socket epp  (epp.c)
Written by Jude Katsch @ ANON
02/01/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/

#include <sys/types.h>
#include <sys/socket.h>
#include <sys/time.h>
#include <netinet/in.h>
#include <netdb.h>
#include <stdio.h>
#include <errno.h>
#include <epp.h>
#include "pptest.h"
#include "pport.h"
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <sys/types.h>
#include <asm/ptrace.h>
#include <linux/parport.h>
#include <sys/stat.h>


// add interrupts

int server_handle=0;
int client_handle=0;

// these routines NEVER return an interface error, they always exit()

//char *default_port="/dev/epp1";
char *default_port="/dev/pptest1";
int p=0;

void error_exit(char *str)
   {
   ctrl_close();
   fprintf(stderr,"pptest: error_exit '%s' error=%d\n",str,errno);
   exit(1);
   }

int ctrl_open()
   {
   char *port = default_port;

   if ((p = open (port, O_RDWR)) <=0)
     error_exit("could not find port");

   if (ioctl (p, EPPCLAIM, 0))
```

```
      error_exit("could not claim port");

   if(ioctl (p, EPPRESET, 0))
      error_exit("could not reset epp");

//   if(ioctl (p, EPPWCONTROL, EPP_CONTROL_INTEN|EPP_CONTROL_RESET))
//      error_exit("could not enable epp interrupt");

   return(0);
   }

int ctrl_close()
   {
   if(p>0)
      {
      ioctl (p, EPPRELEASE, 0);
      close (p);
      }
   return(0);
   }

int ctrl_write(int reg,int data)
   {
   int i;
   for(i=0;i<4;i++)
      {
      if(ioctl(p,EPPWDATA,(data&0xff)))
         error_exit("writing data");
      data >>= 8;
      }
   if(ioctl(p,EPPWADDRESS,(reg&0xff)))
      error_exit("writing write address");
   return(0);
   }

int ctrl_read(int reg,int *data)
   {
   int i;
   unsigned int idata=0;
   int retval;
   if(ioctl(p,EPPWADDRESS,(reg&0xff)))
      error_exit("writing read address");
   for(i=0;i<4;i++)
      {
      idata>>=8;
      if(ioctl(p,EPPRDATA,&retval))
         error_exit("reading data");
      idata |=(retval&0xff)<<24;
      }
   *data=idata;
   return(0);
   }

int ctrl_fifo_read(int size, unsigned char * data)
   {
   int i;
   int retval;
   for(i=0;i<size;i++)
      {
      if(ioctl(p,EPPRADDRESS,&retval))
         error_exit("reading fifo data");
      data[i] = (unsigned char) retval&0xff;
      }
   return(0);
   }

```

12.14 CRC Look Up Table (pre-constructed) (CRC_LUT_H)

```
// automaticly created crc lookup table
0x0000, // 000
0x8005, // 001
0x800f, // 002
0x000a, // 003
0x801b, // 004
0x001e, // 005
0x0014, // 006
0x8011, // 007
0x8033, // 008
0x0036, // 009
0x003c, // 010
0x8039, // 011
0x0028, // 012
0x802d, // 013
0x8027, // 014
0x0022, // 015
0x8063, // 016
0x0066, // 017
0x006c, // 018
0x8069, // 019
0x0078, // 020
0x807d, // 021
0x8077, // 022
0x0072, // 023
0x0050, // 024
0x8055, // 025
0x805f, // 026
0x005a, // 027
0x804b, // 028
0x004e, // 029
0x0044, // 030
0x8041, // 031
0x80c3, // 032
0x00c6, // 033
0x00cc, // 034
0x80c9, // 035
0x00d8, // 036
0x80dd, // 037
0x80d7, // 038
0x00d2, // 039
0x00f0, // 040
0x80f5, // 041
0x80ff, // 042
0x00fa, // 043
0x80eb, // 044
0x00ee, // 045
0x00e4, // 046
0x80e1, // 047
0x00a0, // 048
0x80a5, // 049
0x80af, // 050
0x00aa, // 051
0x80bb, // 052
0x00be, // 053
0x00b4, // 054
0x80b1, // 055
0x8093, // 056
0x0096, // 057
0x009c, // 058
0x8099, // 059
0x0088, // 060
0x808d, // 061
```

```
0x8087, // 062
0x0082, // 063
0x8183, // 064
0x0186, // 065
0x018c, // 066
0x8189, // 067
0x0198, // 068
0x819d, // 069
0x8197, // 070
0x0192, // 071
0x01b0, // 072
0x81b5, // 073
0x81bf, // 074
0x01ba, // 075
0x81ab, // 076
0x01ae, // 077
0x01a4, // 078
0x81a1, // 079
0x01e0, // 080
0x81e5, // 081
0x81ef, // 082
0x01ea, // 083
0x81fb, // 084
0x01fe, // 085
0x01f4, // 086
0x81f1, // 087
0x81d3, // 088
0x01d6, // 089
0x01dc, // 090
0x81d9, // 091
0x01c8, // 092
0x81cd, // 093
0x81c7, // 094
0x01c2, // 095
0x0140, // 096
0x8145, // 097
0x814f, // 098
0x014a, // 099
0x815b, // 100
0x015e, // 101
0x0154, // 102
0x8151, // 103
0x8173, // 104
0x0176, // 105
0x017c, // 106
0x8179, // 107
0x0168, // 108
0x816d, // 109
0x8167, // 110
0x0162, // 111
0x8123, // 112
0x0126, // 113
0x012c, // 114
0x8129, // 115
0x0138, // 116
0x813d, // 117
0x8137, // 118
0x0132, // 119
0x0110, // 120
0x8115, // 121
0x811f, // 122
0x011a, // 123
0x810b, // 124
0x010e, // 125
0x0104, // 126
0x8101, // 127
```

```
0x8303, // 128
0x0306, // 129
0x030c, // 130
0x8309, // 131
0x0318, // 132
0x831d, // 133
0x8317, // 134
0x0312, // 135
0x0330, // 136
0x8335, // 137
0x833f, // 138
0x033a, // 139
0x832b, // 140
0x032e, // 141
0x0324, // 142
0x8321, // 143
0x0360, // 144
0x8365, // 145
0x836f, // 146
0x036a, // 147
0x837b, // 148
0x037e, // 149
0x0374, // 150
0x8371, // 151
0x8353, // 152
0x0356, // 153
0x035c, // 154
0x8359, // 155
0x0348, // 156
0x834d, // 157
0x8347, // 158
0x0342, // 159
0x03c0, // 160
0x83c5, // 161
0x83cf, // 162
0x03ca, // 163
0x83db, // 164
0x03de, // 165
0x03d4, // 166
0x83d1, // 167
0x83f3, // 168
0x03f6, // 169
0x03fc, // 170
0x83f9, // 171
0x03e8, // 172
0x83ed, // 173
0x83e7, // 174
0x03e2, // 175
0x83a3, // 176
0x03a6, // 177
0x03ac, // 178
0x83a9, // 179
0x03b8, // 180
0x83bd, // 181
0x83b7, // 182
0x03b2, // 183
0x0390, // 184
0x8395, // 185
0x839f, // 186
0x039a, // 187
0x838b, // 188
0x038e, // 189
0x0384, // 190
0x8381, // 191
0x0280, // 192
0x8285, // 193
```

```
0x828f, // 194
0x028a, // 195
0x829b, // 196
0x029e, // 197
0x0294, // 198
0x8291, // 199
0x82b3, // 200
0x02b6, // 201
0x02bc, // 202
0x82b9, // 203
0x02a8, // 204
0x82ad, // 205
0x82a7, // 206
0x02a2, // 207
0x82e3, // 208
0x02e6, // 209
0x02ec, // 210
0x82e9, // 211
0x02f8, // 212
0x82fd, // 213
0x82f7, // 214
0x02f2, // 215
0x02d0, // 216
0x82d5, // 217
0x82df, // 218
0x02da, // 219
0x82cb, // 220
0x02ce, // 221
0x02c4, // 222
0x82c1, // 223
0x8243, // 224
0x0246, // 225
0x024c, // 226
0x8249, // 227
0x0258, // 228
0x825d, // 229
0x8257, // 230
0x0252, // 231
0x0270, // 232
0x8275, // 233
0x827f, // 234
0x027a, // 235
0x826b, // 236
0x026e, // 237
0x0264, // 238
0x8261, // 239
0x0220, // 240
0x8225, // 241
0x822f, // 242
0x022a, // 243
0x823b, // 244
0x023e, // 245
0x0234, // 246
0x8231, // 247
0x8213, // 248
0x0216, // 249
0x021c, // 250
0x8219, // 251
0x0208, // 252
0x820d, // 253
0x8207, // 254
0x0202, // 255
```

12.15 CRC routine (CRC_C)

```c
/*
crc (cyclic redundancy check (crc.c)
Written by Jude Katsch @ ANON
02/20/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/

static int crc_lut[256]; // crc look up table

#define CRC16 0x8005
// x^16+x^15+x^2+1

// This needs to be done once!
// I've also made a program crc_lut.c that generates a lut that can be included in
// c code. Either way works.
int crc_build_lut()
   {
   int i,j,k;
   for(i=0;i<256;i++)
      {
      for(j=i<<8,k=0;k<8;k++)
         j=(j&0x8000) ? (j<<1)^CRC16 : (j<<1);
      crc_lut[i]=j;
      }
   return(0);
   }

int crc_calc(unsigned char data,int crc)
   {
   if(crc_lut[1]==0) crc_build_lut();
   return(((crc_lut[((crc>>8)^data)&0xff])^(crc<<8))&0xffff);
   }

int crc_block(unsigned char *data,int length)
   {
   int crc;
   if(crc_lut[1]==0) crc_build_lut();
   for(crc=0x0000;length--;data++)
      crc=((crc_lut[((crc>>8)^(*data))&0xff])^(crc<<8))&0xffff;
   return(crc);
   }
```

12.16 CRC Look Up Table generation routine (CRC_LUT_C)

```c
/*
crc look up table generator  (cyclic redundancy check)(crc_lut.c)
Written by Jude Katsch @ ANON
02/20/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/


#define CRC16 0x8005
// x^16+x^15+x^2+1

int main()
   {
   int i,j,k;
   printf("// automaticly created crc lookup table\n");
```

```
   for(i=0;i<256;i++)
      {
      for(j=i<<8,k=0;k<8;k++)
         j=(j&0x8000) ? (j<<1)^CRC16 : (j<<1);
      printf("0x%04x, // %03d\n",j&0xffff,i);
      }
   }
```

12.17 Polyface includes (POLYFACE_H)

```

#define SELECT_ADDRESS 0x20
#define ADD_A0        (SELECT_ADDRESS|0x00)
#define ADD_A1        (SELECT_ADDRESS|0x01)
#define ADD_A2        (SELECT_ADDRESS|0x02)
#define ADD_STROBE    (SELECT_ADDRESS|0x03)
#define ADD_PINOUT    (SELECT_ADDRESS|0x04)
#define ADD_CONFIG    (SELECT_ADDRESS|0x05)
#define ADD_INPUTA    (SELECT_ADDRESS|0x06)
#define ADD_REQUEST   (SELECT_ADDRESS|0x07)
#define ADD_INPUTB    (SELECT_ADDRESS|0x08)
#define SELECT_JOYSTICK 0x30
#define JS_SW0        (SELECT_JOYSTICK|0x00)
#define JS_SW1        (SELECT_JOYSTICK|0x01)
#define JS_QUADX      (SELECT_JOYSTICK|0x02)
#define JS_QUADY      (SELECT_JOYSTICK|0x03)
#define JS_CHANNEL    (SELECT_JOYSTICK|0x04)
#define JS_ATOD       (SELECT_JOYSTICK|0x05)

#define ADD_REQUEST_BUSY          (1<<0)
#define ADD_REQUEST_INT           (1<<1)
#define ADD_REQUEST_ATOD          (1<<2)
#define ADD_REQUEST_ATOD_ERROR    (1<<3)
#define ADD_REQUEST_SER_RECV_FOUR (1<<4)
#define ADD_REQUEST_SER_RECV_ANY  (1<<5)
#define ADD_REQUEST_SER_SEND_FOUR (1<<6)
#define ADD_REQUEST_SER_SEND_ANY  (1<<7)

#define ADD_REQUEST_EN_BUSY          (1<<0)
#define ADD_REQUEST_EN_BUSY_N        (1<<1)
#define ADD_REQUEST_EN_ATOD          (1<<2)
#define ADD_REQUEST_EN_INT           (1<<3)
#define ADD_REQUEST_EN_SER_RECV_FOUR (1<<4)
#define ADD_REQUEST_EN_SER_RECV_ANY  (1<<5)
#define ADD_REQUEST_EN_SER_SEND_FOUR (1<<6)
#define ADD_REQUEST_EN_SER_SEND_ANY  (1<<7)

#define SELECT_MEMORY 0x10
#define MEM_DATA      (SELECT_MEMORY|0x00)
#define MEM_DATA_INC  (SELECT_MEMORY|0x01)
#define MEM_PROG      (SELECT_MEMORY|0x02)
#define MEM_PROG_ADD  (SELECT_MEMORY|0x03)
#define MEM_STB_A     (0x1<<2)
#define MEM_STB_B     (0x2<<2)
#define MEM_STB_C     (0x3<<2)
#define SELECT_SERIAL 0x40
#define SER_BAUD      (SELECT_SERIAL|0x00)
#define SER_FLAGS0    (SELECT_SERIAL|0x01)
#define SER_FLAGS1    (SELECT_SERIAL|0x02)
#define SER_DATA      (SELECT_SERIAL|0x03)
#define SER_SSTATUS   (SELECT_SERIAL|0x04)
#define SER_RSTATUS   (SELECT_SERIAL|0x05)

```

```
#define ATOD_CHANNEL_NONE 0x00
#define ATOD_CHANNEL_MODE 0x01
#define ATOD_CHANNEL_X1   0x02
#define ATOD_CHANNEL_Y1   0x03
#define ATOD_CHANNEL_X2   0x04
#define ATOD_CHANNEL_Y2   0x05
//#define ATOD_CHANNEL_NONE 0x07

// serial flags
#define SER_FLAGS0_LSB_FIRST  (1<<0)
#define SER_FLAGS0_SYNC_BITS  (1<<1)
#define SER_FLAGS0_PARITY_BIT (1<<2)
#define SER_FLAGS0_MASKED_CLK (1<<3)
#define SER_FLAGS0_CLK_HIGH   (1<<4)
#define SER_FLAGS0_LEVEL_HIGH (1<<5)
#define SER_FLAGS0_CS_NEG     (1<<6)
#define SER_FLAGS0_ASYNC      (1<<7)

#define SER_FLAGS0_PARITY_EVEN (SER_FLAGS0_PARITY_BIT|SER_FLAGS0_SYNC_BITS)
#define SER_FLAGS0_PARITY_ODD  (SER_FLAGS0_PARITY_BIT)

#define SER_FLAGS1_RECV      (1<<0)
#define SER_FLAGS1_SEND      (1<<1)
#define SER_FLAGS1_STICKY_CS (1<<2)
#define SER_FLAGS1_RUN       (1<<3)
#define SER_FLAGS1_UNUSED    (1<<4)
#define SER_FLAGS1_IO_FLIP   (1<<5)
#define SER_FLAGS1_DIN_FLIP  (1<<6)
#define SER_FLAGS1_MASK_CTS  (1<<7)

#define SER_SSTATUS_VALID 0x1f
#define SER_SSTATUS_OF (1<<7)
#define SER_SSTATUS_ERRORS (SER_SSTATUS_OF)
#define SER_RSTATUS_VALID 0x1f
#define SER_RSTATUS_OF (1<<5)
#define SER_RSTATUS_UF (1<<6)
#define SER_RSTATUS_P  (1<<7)
#define SER_RSTATUS_ERRORS (SER_RSTATUS_OF|SER_RSTATUS_UF|SER_RSTATUS_P)

#define SER_BAUD_STOP  0x00
#define SER_BAUD_MIN   0xfe
#define SER_BAUD_1200  0x2e
#define SER_BAUD_2400  0x98
#define SER_BAUD_4800  0xa6
#define SER_BAUD_9600  0xd5
#define SER_BAUD_MAX   0xff

#define SER_PWM_BAUD   (SER_BAUD_MIN)
#define SER_PWM_FLAGS0 (SER_FLAGS0_ASYNC)
#define SER_PWM_FLAGS1 (SER_FLAGS1_RUN|SER_FLAGS1_STICKY_CS|SER_FLAGS1_MASK_CTS)

#define SER_THRU_BAUD   (SER_BAUD_9600)
#define SER_THRU_FLAGS0 (SER_FLAGS0_ASYNC|SER_FLAGS0_PARITY_EVEN)

#define SER_THRU_FLAGS1 (SER_FLAGS1_RECV|SER_FLAGS1_SEND|SER_FLAGS1_RUN|SER_FLAGS1_DIN_FLIP|SER_FLAGS1_MASK_CTS)

#define SER_ASYNC_BAUD   (SER_BAUD_9600)
#define SER_ASYNC_FLAGS0 (SER_FLAGS0_ASYNC|SER_FLAGS0_LSB_FIRST|SER_FLAGS0_PARITY_EVEN)
#define SER_ASYNC_FLAGS1 (SER_FLAGS1_RECV|SER_FLAGS1_SEND|SER_FLAGS1_RUN|SER_FLAGS1_IO_FLIP)

// mode selector
#define MODE_TEST      0x0
#define MODE_JOYSTICK  0x1
#define MODE_WHEEL     0x2
#define MODE_CONFIG    0x7

```

```
#define CONFIG_SPLITTER   0x00
#define CONFIG_RUMBLE     0x01 // rumble card
#define CONFIG_THRU       0x02
#define CONFIG_UCONT      0x03 // serial microcontroler
#define CONFIG_SERIAL     0x04 // a standard serial device
#define CONFIG_OZJS       0x05 // oz joystick

// memory configurations
// note: for each memory type there are 16 sub-memory types
#define CONFIG_MEMORY_TYPE     0xf0
#define CONFIG_SERIAL_MEMORY   0x10
#define CONFIG_PARALLEL_MEMORY 0x20
#define CONFIG_LATCHED_MEMORY  0x30

#define POA_BITS          (1<<0)
#define POA_SWITCH        (0<<0)
#define POA_ADDRESS       (1<<0)

#define POB_BITS          (1<<1)
#define POB_SWITCH        (0<<1)
#define POB_ADDRESS       (1<<1)

#define POD_BITS          (3<<2)
#define POD_INPUT         (0<<2)
#define POD_ADDRESS       (1<<2)
#define POD_SERIAL        (2<<2)
#define POD_RUMBLE        (3<<2)

#define POE_BITS          (3<<4)
#define POE_QUADRATURE    (0<<4)
#define POE_MEMORY        (1<<4)
#define POE_SPLITTER      (2<<4)
#define POE_SERIAL        (3<<4)

#define POF_BITS          (3<<6)
#define POF_INPUT         (0<<6)
#define POF_ADDRESS       (1<<6)
#define POF_SPLITTER      (2<<6)
#define POF_INT           (3<<6)

//#define POF_SPLITTER      (0<<6)
//#define POF_ADDRESS       (1<<6)
//#define POF_ADDSTB        (2<<6)
//#define POF_LATCH         (3<<6)


#define STROBE_ASTB   6
#define STROBE_ALE    7
#define STROBE_BSTB   2
#define STROBE_BLE    3
#define STROBE_ADD    0


extern struct function_execute polyface_joystick_fexec;
extern struct function_execute polyface_serial_fexec;
extern struct function_execute polyface_test_fexec;

int polyface_error(struct device *dev);
int polyface_serial_init(struct device *dev);

struct polyface_user_joystick
   {
   struct axis *req; // who is the current joystick request for
   int enable; // enable flags for joystick
   };

```

```
#define JS_EN_SW    0
#define JS_EN_SW0   (1<<(JS_EN_SW+0))
#define JS_EN_SW1   (1<<(JS_EN_SW+1))
#define JS_EN_SWX   (JS_EN_SW0|JS_EN_SW1)
#define JS_EN_QUAD  8
#define JS_EN_QUAD0 (1<<(JS_EN_QUAD+0))
#define JS_EN_QUAD1 (1<<(JS_EN_QUAD+1))
#define JS_EN_QUADX (JS_EN_QUAD0|JS_EN_QUAD1)
#define JS_EN_JS    16
#define JS_EN_JS0   (1<<(JS_EN_JS+0))
#define JS_EN_JS1   (1<<(JS_EN_JS+1))
#define JS_EN_JS2   (1<<(JS_EN_JS+2))
#define JS_EN_JS3   (1<<(JS_EN_JS+3))
#define JS_EN_JSX   (JS_EN_JS0|JS_EN_JS1|JS_EN_JS2|JS_EN_JS3)
#define JS_EN_RUMBLE 24

struct polyface_user_serial
   {
   int baud;
   int flags0;
   int flags1;
   };

#define MEMORY_FLAGS_IF               0
#define MEMORY_FLAGS_IF_MASK          (0xf<<MEMORY_FLAGS_IF)
#define MEMORY_FLAGS_IF_SUB           4
#define MEMORY_FLAGS_IF_SUB_MASK      (0xf<<MEMORY_FLAGS_IF_SUB)
#define MEMORY_FLAGS_IF_TYPE          (0x0f<<MEMORY_FLAGS_IF)
#define MEMORY_FLAGS_IF_PARALLEL      (0x01<<MEMORY_FLAGS_IF)
#define MEMORY_FLAGS_IF_SERIAL        (0x02<<MEMORY_FLAGS_IF)
#define MEMORY_FLAGS_IF_LATCHED       (0x03<<MEMORY_FLAGS_IF)
#define MEMORY_FLAGS_IF_PARALLEL_8    ((0x01<<MEMORY_FLAGS_IF_SUB)|(MEMORY_FLAGS_IF_PARALLEL))
#define MEMORY_FLAGS_IF_PARALLEL_16   ((0x02<<MEMORY_FLAGS_IF_SUB)|(MEMORY_FLAGS_IF_PARALLEL))
#define MEMORY_FLAGS_IF_PARALLEL_20   ((0x03<<MEMORY_FLAGS_IF_SUB)|(MEMORY_FLAGS_IF_PARALLEL))
#define MEMORY_FLAGS_IF_PARALLEL_23   ((0x04<<MEMORY_FLAGS_IF_SUB)|(MEMORY_FLAGS_IF_PARALLEL))

struct polyface_user_memory
   {
   int flags;
   int parallel_address; // address on pins for all parallel memories
   };

// pointed to by device.user
struct polyface
   {
   int mode;   // mode register
   int config; // config register
   int strobe; // strobe register
   int pinout; // pinout register
   int request;


   struct polyface_user_joystick js;
   struct polyface_user_serial   ser;
   struct polyface_user_memory   mem;
   };
```

12.18 Polyface Driver (POLYFACE_DRIVER_C)

```
/*
polyface device driver (embeded) (polyface_driver.c)
Written by Jude Katsch @ ANON
02/20/99
```

```
 5 Confidentail and Proprietary
 6 Copyright (c) 1999, ANON
 7 */
 8
 9 #define READ_MODE // read mode from atod
10 #define VERSION 0
11 #define REVISION 0
12
13 #include <stdlib.h>
14 #include <stdio.h>
15 #include "pport.h"
16 #include "default_config.h"
17 #include "fifo.h"
18 #include "memory.h"
19
20 extern int quiet;
21
22 #include "polyface.h"
23
24 struct driver polyface_driver;
25
26 //==========================================================================================
27 int add_fexec(struct device *dev,struct function_execute *fexec)
28    {
29    struct ll_function_execute **llfexec;
30
31    llfexec = &dev->llfexec;
32    while((*llfexec)!=NULL)
33       llfexec=&(*llfexec)->next;
34
35    if(((*llfexec)=(struct ll_function_execute *)malloc(sizeof(struct ll_function_execute)))==NULL)
36       {
37       fprintf(stderr,"can't allocate memory for ll_function_exectue");
38       exit(1);
39       }
40
41    (*llfexec)->next=NULL;
42    (*llfexec)->fexec=fexec;
43
44    return(PP_OK);
45    };
46
47 //==========================================================================================
48 int polyface_release(struct device *dev)
49    {
50    int retval;
51    struct ll_function_execute *llfexec,*llfexec_next;
52    if(!quiet) printf(__FILE__ ": in polyface_release(0x%08x)\n",(unsigned int)dev);
53    llfexec = dev->llfexec;
54    while(llfexec!=NULL)
55       {
56       if((llfexec->fexec!=NULL)&&(llfexec->fexec->release!=NULL))
57          if((retval=(*llfexec->fexec->release)(dev))!=PP_OK) return(retval);
58       llfexec_next=llfexec->next;
59       free(llfexec);
60       llfexec=llfexec_next;
61       }
62
63    if(dev->user)
64       {
65       free(dev->user);
66       dev->user=NULL;
67       }
68
69    return(PP_OK);
70    }
```

```

//==========================================================================================
int polyface_error(struct device *dev)
   {
   int retval;
   struct ll_function_execute *llfexec;
   if(!quiet) printf(__FILE__ ": in polyface_error(0x%08x)\n",(unsigned int)dev);
   llfexec = dev->llfexec;
   while(llfexec!=NULL)
      {
      if((llfexec->fexec!=NULL)&&(llfexec->fexec->error!=NULL))
        if((retval=(*llfexec->fexec->error)(dev))!=PP_OK) return(retval);
      llfexec=llfexec->next;
      }
   return(PP_OK);
   }

//==========================================================================================
int polyface_request(struct device *dev)
   {
   int retval;
   struct ll_function_execute *llfexec;
   if(!quiet) printf(__FILE__ ": in polyface_request(0x%08x)\n",(unsigned int)dev);
   llfexec = dev->llfexec;
   while(llfexec!=NULL)
      {
      if((llfexec->fexec!=NULL)&&(llfexec->fexec->request!=NULL))
         if((retval=(*llfexec->fexec->request)(dev))!=PP_OK) return(retval);
      llfexec=llfexec->next;
      }
   return(PP_OK);
   }

//==========================================================================================
int polyface_execute(struct device *dev)
   {
   int retval;
   struct ll_function_execute *llfexec;
   if(!quiet) printf(__FILE__ ": in polyface_execute(0x%08x)\n",(unsigned int)dev);
   llfexec = dev->llfexec;
   while(llfexec!=NULL)
      {
      if((llfexec->fexec!=NULL)&&(llfexec->fexec->execute!=NULL))
         if((retval=(*llfexec->fexec->execute)(dev))!=PP_OK) return(retval);
      llfexec=llfexec->next;
      }
   return(PP_OK);
   }

//==========================================================================================
int polyface_open()
   {
   if(!quiet) printf(__FILE__ ": in polyface_open\n");
   return(PP_OK);
   }

//==========================================================================================
int polyface_close()
   {
   if(!quiet) printf(__FILE__ ": in polyface_close\n");
   return(PP_OK);
   }

//==========================================================================================
int polyface_configure(struct device *dev)
   {
```

```
   unsigned char data[4];
   char *smode;
   struct polyface *user;
   int retval;
   struct ll_function_execute *llfexec;

   if((retval=dev->fcfg->enable(dev))!=PP_OK) return(retval);

   // init a0,a1,a2 to 0x00
   pp_data_write_one(dev->pport->port,dev->id,ADD_A0,3,CMD_IA,0);

   if((user=dev->user=(struct polyface *)malloc(sizeof(struct polyface)))==NULL)
      {
      mon_error("allocating memory for polyface structure.\n");
      return(PP_INTERNAL);
      }

   user->mode=MODE_CONFIG;
   user->pinout=-1;
   user->config=0;
   user->request=0;
   user->js.req=NULL;
   user->js.enable=0;
   user->ser.baud=0;
   user->ser.flags0=0;
   user->ser.flags1=0;
   user->strobe=STROBE_ADD;
   smode="UNKNOWN";

   if(!quiet) printf(__FILE__ ": in polyface_configure(0x%08x)\n",(unsigned int)dev);

// read the mode
#ifdef READ_MODE
//while(1)
//   {
   pp_data_write_one(dev->pport->port,dev->id,JS_CHANNEL,1,CMD_IA,ATOD_CHANNEL_MODE);

//   do{
//      if((retval=pp_command_request(dev->pport->port,&request))!=PP_OK) return(retval);
//   } while(((request&(1<<dev->id))==0) &&

   do {
      if((retval=pp_data_read(dev->pport->port,dev->id,ADD_REQUEST,1,0,data))!=PP_OK) return(retval);
   } while((data[0]&ADD_REQUEST_ATOD)==0);

   if(data[0]&ADD_REQUEST_ATOD_ERROR)
      {
      mon_error("atod error while reading mode\n");
      user->mode = MODE_JOYSTICK;
      }
   else
      {

if((retval=pp_data_read(dev->pport->port,dev->id,JS_ATOD,1,CMD_IA|CMD_CRC,data))!=PP_OK) return(retval);
      user->mode = ((data[0]-(255/20))*11)/255;
      }


if((retval=pp_data_read(dev->pport->port,dev->id,ADD_REQUEST,1,CMD_CRC,data))!=PP_OK) return(retval);
   if(!quiet) printf("request=0x%02x\n",data[0]);

   pp_data_write_one(dev->pport->port,dev->id,JS_CHANNEL,1,CMD_IA,ATOD_CHANNEL_NONE);

   if(!quiet) printf("mode=%d\n",user->mode);

//   exit(1);
```

```
201 //   }
202
203 #else
204    if(!quiet) printf("********* mode scan bypassed\n");
205    user->mode = 7;
206    if(!quiet) printf("mode=%d\n",user->mode);
207 #endif
208
209    switch(user->mode)
210      {
211      case MODE_TEST:
212         smode="TEST";
213         user->pinout=(POA_ADDRESS|POB_ADDRESS|POE_MEMORY|POD_ADDRESS|POF_ADDRESS);
214         add_fexec(dev,&polyface_test_fexec);
215         break;
216      case MODE_JOYSTICK:
217         user->pinout=(POA_SWITCH|POB_SWITCH|POE_QUADRATURE|POD_RUMBLE|POF_SPLITTER);
218         smode="JOYSTICK";
219         user->js.enable=JS_EN_SWX|JS_EN_QUADX|JS_EN_JSX|JS_EN_RUMBLE;
220         user->ser.baud  =SER_PWM_BAUD;
221         user->ser.flags0=SER_PWM_FLAGS0;
222         user->ser.flags1=SER_PWM_FLAGS1;
223         add_fexec(dev,&polyface_joystick_fexec);
224         add_fexec(dev,&polyface_serial_fexec);
225         break;
226      case MODE_WHEEL  :
227         user->pinout=(POA_SWITCH|POB_SWITCH|POE_QUADRATURE|POD_INPUT|POF_SPLITTER);
228         smode="WHEEL";
229         add_fexec(dev,&polyface_joystick_fexec);
230         add_fexec(dev,&polyface_serial_fexec);
231         break;
232      case MODE_CONFIG :
233
    if((retval=pp_data_read(dev->pport->port,dev->id,ADD_CONFIG,1,CMD_CRC,data))!=PP_OK) return(retval);
234         user->config=data[0]; // &0xff; // use low byte
235         switch(user->config&CONFIG_MEMORY_TYPE)
236            {
237            case CONFIG_SERIAL_MEMORY:
238               user->pinout=(POA_ADDRESS|POB_ADDRESS|POE_SERIAL|POD_SERIAL|POF_SPLITTER);
239               smode="SERIAL MEMORY";
240               add_fexec(dev,&polyface_serial_fexec);
241 //              add_fexec(dev,&polyface_serial_memory_fexec);
242               break;
243            case CONFIG_PARALLEL_MEMORY:
244               user->pinout=(POA_ADDRESS|POB_ADDRESS|POE_MEMORY|POD_ADDRESS|POF_ADDRESS);
245               smode="PARALLEL MEMORY";
246               user->mem.parallel_address=-1; // invalidate
247               user->mem.flags=MEMORY_FLAGS_IF_PARALLEL_23;
248               add_fexec(dev,&memory_fexec);
249               break;
250            case CONFIG_LATCHED_MEMORY:
251               user->pinout=(POA_ADDRESS|POB_ADDRESS|POE_MEMORY|POD_ADDRESS|POF_ADDRESS);
252               user->strobe=STROBE_BLE;
253               smode="LATCHED MEMORY";
254 //              add_fexec(dev,&polyface_latched_memory_fexec);
255               break;
256            default:
257               switch(user->config)
258          {
259          case CONFIG_SPLITTER:
260                     user->pinout=(POA_SWITCH|POB_SWITCH|POE_SPLITTER|POD_INPUT|POF_SPLITTER);
261                     smode="SPLITTER";
262                     add_fexec(dev,&polyface_serial_fexec);
263                     break;
264          case CONFIG_RUMBLE:
265                     user->pinout=(POA_SWITCH|POB_SWITCH|POE_QUADRATURE|POD_RUMBLE|POF_SPLITTER);
```

```
                    user->js.enable=JS_EN_RUMBLE;
            user->ser.baud  =SER_PWM_BAUD;
            user->ser.flags0=SER_PWM_FLAGS0;
            user->ser.flags1=SER_PWM_FLAGS1;
                    smode="RUMBLE";
                    add_fexec(dev,&polyface_serial_fexec);
                    break;
         case CONFIG_THRU:
                    user->pinout=(POA_SWITCH|POB_SWITCH|POE_SERIAL|POD_SERIAL|POF_SPLITTER);
            user->ser.baud  =SER_THRU_BAUD;
            user->ser.flags0=SER_THRU_FLAGS0;
            user->ser.flags1=SER_THRU_FLAGS1;
                    smode="THRU";
                    add_fexec(dev,&polyface_serial_fexec);
                    break;
         case CONFIG_SERIAL:
            user->pinout=(POA_SWITCH|POB_SWITCH|POE_SERIAL|POD_SERIAL|POF_SPLITTER);
            user->ser.baud  =SER_ASYNC_BAUD;
            user->ser.flags0=SER_ASYNC_FLAGS0;
            user->ser.flags1=SER_ASYNC_FLAGS1;
            smode="SERIAL";
            add_fexec(dev,&polyface_serial_fexec);
            break;
         case CONFIG_OZJS:
            user->pinout=(POA_SWITCH|POB_SWITCH|POE_QUADRATURE|POD_INPUT|POF_SPLITTER);
            smode="OZ JOYSTICK";
                    user->js.enable=JS_EN_SWX|JS_EN_QUADX|JS_EN_RUMBLE;
            add_fexec(dev,&polyface_joystick_fexec);
            break;
         }
           }
        mon_write("INFO:%d:%d:CONFIG:0x%02x\n",dev->pport->port,dev->id,user->config);
      }

   if(!quiet) printf(__FILE__ ": mode = %s\n",smode);

// do we need to read the config register
   if(user->pinout!=-1)
      pp_data_write_one(dev->pport->port,dev->id,ADD_PINOUT,1,0,(unsigned char) user->pinout);
   else
      mon_error("could not determine polyface pinout\n");

   pp_data_write_one(dev->pport->port,dev->id,ADD_STROBE,1,0,(unsigned char) user->strobe);

// select the available ports dependent on the pinout
   dev->ports = 0;
   if((((struct polyface *)(dev->user))->pinout&POE_BITS)==POE_SPLITTER)
      dev->ports |= (1<<2)|(1<<3); // ports 2 and 3

   if((((struct polyface *)(dev->user))->pinout&POF_BITS)==POF_SPLITTER)
      dev->ports |= (1<<0)|(1<<1); // ports 0 and 1

   if((((struct polyface *)(dev->user))->pinout&POF_BITS)==POF_INT)
      dev->ports |= (1<<0); // ports 0

   mon_write("DEV:%d:%d:DEVICE:%s\n",dev->pport->port,dev->id,smode);
   mon_write("INFO:%d:%d:DEVICE:%s\n",dev->pport->port,dev->id,smode);
   mon_write("INFO:%d:%d:MODE:%d\n",dev->pport->port,dev->id,user->mode);

   llfexec = dev->llfexec;
   while(llfexec!=NULL)
      {
      if((llfexec->fexec!=NULL)&&(llfexec->fexec->configure!=NULL))
         if((retval=(*llfexec->fexec->configure)(dev))!=PP_OK) return(retval);
      llfexec=llfexec->next;
      }
```

```
   pp_data_write_one(dev->pport->port,dev->id,ADD_REQUEST,1,0,(unsigned char) user->request);
   return(PP_OK);
   }

struct function_execute polyface_default_fexec =
   {
   polyface_error,
   polyface_request,
   polyface_release,
   polyface_execute,
   polyface_configure,
   };

struct driver polyface_driver =
   {
   polyface_open,
   polyface_close,
   polyface_configure,
   &polyface_default_fexec,
   0, // flags
   VERSION,
   REVISION,
   "POLYFACE DRIVER\nWritten by Jude Katsch for VMLabs\nConfidential and Proprietary\n"
   };
```

12.19 Polyface Test includes (POLYFACE_TEST_H)

```
unsigned char strobe_test[8]={0xf0,0x0f,0x56,0xad,0xf0,0x0f,0xa5,0x5a};

unsigned char inc_data[16]={0xaa,0x55,0x02,0x03,0x04,0x05,0x06,0x07,0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f};

unsigned char ser_data[]={0xaa,0x55,0x66,0xdd,0xa5,0x5a,0x00,0xff,
  0x11,0x22,0x33,0x44,0x55,0x66,0x77,0x88,
  0x99,0xaa,0xbb,0xcc,0xdd,0xee,0xff,0x00};

unsigned char strobe_expected[] = {
0xf0,0x0f,0x56,0xad,0xf0,0x0f,0xa5,0x5a, // 0 STB_NONE
0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f, // 0 STB_A
0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17, // 0 STB_B
0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f, // 0 STB_C
0xf0,0x0f,0x56,0xad,0xf0,0x0f,0xa5,0x5a, // 1 STB_NONE
0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f, // 1 STB_A
0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17, // 1 STB_B
0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f, // 1 STB_C
0xaa,0xaa,0xaa,0xaa,0xaa,0xaa,0xaa,0xaa, // 2 STB_NONE
0xaa,0x09,0xaa,0x0b,0xaa,0x0d,0xaa,0x0f, // 2 STB_A
0xaa,0xaa,0x12,0x13,0xaa,0xaa,0x16,0x17, // 2 STB_B
0xaa,0xaa,0xaa,0xaa,0x1c,0x1d,0x1e,0x1f, // 2 STB_C
0xaa,0xaa,0xaa,0xaa,0xaa,0xaa,0xaa,0xaa, // 3 STB_NONE
0xaa,0x09,0xaa,0x0b,0xaa,0x0d,0xaa,0x0f, // 3 STB_A
0xaa,0xaa,0x12,0x13,0xaa,0xaa,0x16,0x17, // 3 STB_B
0xaa,0xaa,0xaa,0xaa,0x1c,0x1d,0x1e,0x1f, // 3 STB_C
0xf0,0x0f,0x56,0xad,0xf0,0x0f,0xa5,0x5a, // 4 STB_NONE
0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f, // 4 STB_A
0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17, // 4 STB_B
0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f, // 4 STB_C
0xf0,0x0f,0x56,0xad,0xf0,0x0f,0xa5,0x5a, // 5 STB_NONE
0x08,0x09,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f, // 5 STB_A
0x10,0x11,0x12,0x13,0x14,0x15,0x16,0x17, // 5 STB_B
0x18,0x19,0x1a,0x1b,0x1c,0x1d,0x1e,0x1f, // 5 STB_C
0xaa,0xaa,0xaa,0xaa,0xaa,0xaa,0xaa,0xaa, // 6 STB_NONE
0xaa,0x09,0xaa,0x0b,0xaa,0x0d,0xaa,0x0f, // 6 STB_A
0xaa,0xaa,0x12,0x13,0xaa,0xaa,0x16,0x17, // 6 STB_B
```

```
0xaa,0xaa,0xaa,0xaa,0x1c,0x1d,0x1e,0x1f, // 6 STB_C
0xaa,0xaa,0xaa,0xaa,0xaa,0xaa,0xaa,0xaa, // 7 STB_NONE
0xaa,0x09,0xaa,0x0b,0xaa,0x0d,0xaa,0x0f, // 7 STB_A
0xaa,0xaa,0x12,0x13,0xaa,0xaa,0x16,0x17, // 7 STB_B
0xaa,0xaa,0xaa,0xaa,0x1c,0x1d,0x1e,0x1f, // 7 STB_C
};

#define TEST_PARALLEL         0x80
#define TEST_PARALLEL_PROG_AA 0xaa
#define TEST_PARALLEL_PROG_55 0x55
#define TEST_FLOAT            0x00
#define TEST_IO               0x10
#define TEST_SERIAL           0x20
#define TEST_QUAD             0x30

unsigned char prog_aa_test[8]={0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08};
unsigned char prog_55_test[8]={0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18};
unsigned char prog_66_test[8]={0x21,0x22,0x23,0x24,0x25,0x26,0x27,0x28};

unsigned char prog_aa_exp[8]={0xaa,0x02,0x33,0xaa,0x05,0x06,0x07,0x08};
unsigned char prog_55_exp[8]={0x11,0x55,0x13,0x14,0x55,0x16,0x17,0x18};
unsigned char prog_66_exp[8]={0x21,0x22,0x23,0x24,0x25,0x77,0x27,0x28};

char quad_data[] = {
   31, -31,
  -31,  31,
   16,  16,
   16,  16,
   16,  16,
   16,  16,
   16,  16,
   16,  16,
   16,  16,
   16,  16,
   16,  16,
   16,  16,
   16,  16,
   16,  16,
   16,  16,
   16,  16,
   16,  16,
   16,  16,
   16,  16,
   0,   0};
```

12.20 Polyface Test routines (POLYFACE_TEST_C)

```
/*
polyface test driver (polyface_test.c)
Written by Jude Katsch @ ANON
10/05/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/

#include "pport.h"
#include "polyface.h"
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include "fifo.h"
#include "polyface_test.h"
extern int quiet;

```

```
18 int compare(int add,int size,unsigned char *data_was,unsigned char *data_exp)
19    {
20    int i,error=0;
21    for(i=0;i<size;i++)
22      if(data_was[i]!=data_exp[i]) {
23          mon_error("REGRESSION DATA MISMATCH- data @ (0x%04x) should be- 0x%02x was-
   0x%02x\n",add+i,data_exp[i],data_was[i]);
24          error++;
25      }
26    return(error);
27    }
28
29 int compare_one(char *who,unsigned char data_was,unsigned char data_exp)
30    {
31    if(data_was!=data_exp) {
32        mon_error("REGRESSION DATA MISMATCH- data @ %s should be- 0x%02x was- 0x%02x\n",who,data_exp,data_was);
33        return(1);
34    }
35    return(0);
36    }
37
38 int polyface_test_execute(struct device *device)
39    {
40    unsigned char data[256];
41    unsigned char data_exp[256];
42    int add;
43    int retval,i,j,k,x;
44    int xstart,ystart;
45    int error=0;
46    int ser_cnt=0;
47    printf(__FILE__ ": in polyface_test_execute\n");
48
49    mon_write("MSG:PROBE VERSION=%d\n",device->probe[PP_PROBE_FLAGS]&0x7f);
50    mon_write("MSG:PROBE MFG=%d\n",device->probe[PP_PROBE_MFG]);
51    mon_write("MSG:PROBE TYPE=%d\n",device->probe[PP_PROBE_TYPE]);
52
53    mon_write("MSG:TESTING QUADRATURE\n");
54
55 #if 1
56
   pp_data_write_one(device->pport->port,device->id,ADD_PINOUT,1,0,POA_ADDRESS|POB_ADDRESS|POE_QUADRATURE);
57
   pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0);  // make sure we start off by not running
58    pp_data_write_one(device->pport->port,device->id,ADD_A1,1,0,TEST_QUAD);
59
60    if((retval=pp_data_read(device->pport->port,device-
   >id,JS_QUADX,2,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
61 //   xstart=(int)(signed char)data[0];
62    xstart=0;
63 //   ystart=(int)(signed char)data[1];
64    ystart=0;
65    printf("x=%d, y=%d // before quadrature\n",xstart,ystart);
66
67    for(x=0;quad_data[x*2]!=0;x++)
68      {
69      data[0]=(1<<7)|((((~quad_data[x*2])&0x80)>>2)|abs(quad_data[x*2]);
70      data[1]=(1<<6)|((((~quad_data[x*2+1])&0x80)>>2)|abs(quad_data[x*2+1]);
71      data[2]=0;
72
73      // load the quadrature counters
74
   if((retval=pp_data_write(device->pport->port,device->id,ADD_A0,3,CMD_CRC,data))!=PP_OK) return(retval);
75
76      do {
77
   pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0);  // make sure we start off by not running
```

```
78        if((retval=pp_data_read(device->pport->port,device->id,MEM_DATA_INC,6,CMD_CRC,data))!=PP_OK) return(retval);
79        } while((data[0]!=0)||(data[1]!=0));
80
81      if((retval=pp_data_read(device->pport->port,device->id,JS_QUADX,2,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
82      printf("x=%d, y=%d // after quadrature\n",data[0],data[1]);
83
84 //      error+=compare_one("QUADX",data[0],xstart+=quad_data[x*2]);
85      error+=compare_one("QUADX",data[0],(xstart+(data[3]<<8|data[2]))&0xff);
86 //      error+=compare_one("QUADY",data[1],ystart+=quad_data[x*2+1]);
87      error+=compare_one("QUADX",data[1],(ystart+(data[5]<<8|data[4]))&0xff);
88      }
89
90 #endif
91
92    pp_data_write_one(device->pport->port,device->id,ADD_PINOUT,1,0,POA_ADDRESS|POB_ADDRESS|POE_MEMORY|POD_ADDRESS|POF_ADDRESS);
93    pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0x00);
94    pp_data_write_one(device->pport->port,device->id,ADD_A1,1,0,TEST_PARALLEL);
95
96 #if 1
97    mon_write("MSG:TESTING AUTO-INCREMENT ADDRESS LINES\n");
98
99 // test address lines
100    for(k=0,j=1;k<256;j++)
101      {
102      for(i=0;(i<j)&&(k<256);i++,k++)
103        data[i]=k;
104      if((retval=pp_data_write(device->pport->port,device->id,MEM_DATA_INC,i,CMD_CRC,data))!=PP_OK) return(retval);
105
if((retval=pp_data_read(device->pport->port,device->id,JS_SW0,1,CMD_CRC,data))!=PP_OK) return(retval);
106      error+=compare_one("SW0",data[0],(k&0xff));
107      }
108
109    for(k=0,j=1;k<256;j++)
110      {
111      add=k;
112      for(i=0;(i<j)&&(k<256);i++,k++)
113        data_exp[i]=k;
114      if((retval=pp_data_read(device->pport->port,device->id,MEM_DATA_INC,i,CMD_CRC,data))!=PP_OK) return(retval);
115      error+=compare(add,i,data,data_exp);
116
117
if((retval=pp_data_read(device->pport->port,device->id,JS_SW0,1,CMD_CRC,data))!=PP_OK) return(retval);
118      error+=compare_one("SW0",data[0],(k&0xff));
119      }
120
121 //       for(j=0;j<32;j++)
122 //          if(data[j]!=(i*32+j))
123 //             {
124 //             mon_error("REGRESSION- data @ (0x%02x) should be- 0x%2x was- 0x%2x\n",(i*32+j),(i*32+j),data[j]);
125 //             error++;
126 //             }
127
128    mon_write("MSG:TESTING STROBES\n");
129
130 // put pattern into memory
131    pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0);
132    pp_data_write_one(device->pport->port,device->id,ADD_A1,1,0,TEST_PARALLEL);
133    pp_data_write_one(device->pport->port,device->id,ADD_A2,1,0,0);
134    if((retval=pp_data_write(device->pport->port,device->id,MEM_DATA_INC,8,CMD_CRC,strobe_test))!=PP_OK) return(retval);
```

```
135
136 // test strobe lines
137    for(i=0;i<8;i++)
138       {
139       pp_data_write_one(device->pport->port,device->id,ADD_STROBE,1,0,i);
140       pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0);
141       pp_data_write_one(device->pport->port,device->id,ADD_A1,1,0,TEST_PARALLEL|i);
142       if((retval=pp_data_read(device->pport->port,device-
    >id,MEM_DATA_INC          ,8,CMD_CRC,data))!=PP_OK) return(retval);
143       add=(i*(8*4))+(8*0);
144       error+=compare(add,8,data,&strobe_expected[add]);
145       printf("0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x, // %d
    STB_NONE\n",data[0],data[1],data[2],data[3],data[4],data[5],data[6],data[7],i);
146       if((retval=pp_data_read(device->pport->port,device-
    >id,MEM_DATA_INC|MEM_STB_A,8,CMD_CRC,data))!=PP_OK) return(retval);
147       add=(i*(8*4))+(8*1);
148       error+=compare(add,8,data,&strobe_expected[add]);
149       printf("0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x, // %d
    STB_A\n",data[0],data[1],data[2],data[3],data[4],data[5],data[6],data[7],i);
150       if((retval=pp_data_read(device->pport->port,device-
    >id,MEM_DATA_INC|MEM_STB_B,8,CMD_CRC,data))!=PP_OK) return(retval);
151       add=(i*(8*4))+(8*2);
152       error+=compare(add,8,data,&strobe_expected[add]);
153       printf("0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x, // %d
    STB_B\n",data[0],data[1],data[2],data[3],data[4],data[5],data[6],data[7],i);
154       if((retval=pp_data_read(device->pport->port,device-
    >id,MEM_DATA_INC|MEM_STB_C,8,CMD_CRC,data))!=PP_OK) return(retval);
155       add=(i*(8*4))+(8*3);
156       error+=compare(add,8,data,&strobe_expected[add]);
157       printf("0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x, // %d
    STB_C\n",data[0],data[1],data[2],data[3],data[4],data[5],data[6],data[7],i);
158       }
159
160    mon_write("MSG:TESTING PROGRAM\n");
161
162    pp_data_write_one(device->pport->port,device->id,ADD_STROBE,1,0,0);
163
164
165 // put pattern into memory
166    pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0x00); // reset offset counter
167    pp_data_write_one(device->pport->port,device->id,MEM_DATA,1,0,0);
168    pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0xaa);
169    pp_data_write_one(device->pport->port,device->id,ADD_A1,1,0,TEST_PARALLEL_PROG_AA);
170    pp_data_write_one(device->pport->port,device->id,ADD_A2,1,0,0);
171    if((retval=pp_data_write(device->pport->port,device-
    >id,MEM_DATA,8,CMD_CRC,prog_aa_test))!=PP_OK) return(retval);
172
173    pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0x00); // reset offset counter
174    pp_data_write_one(device->pport->port,device->id,MEM_DATA,1,0,0);
175    pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0x55);
176    pp_data_write_one(device->pport->port,device->id,ADD_A1,1,0,TEST_PARALLEL_PROG_55);
177    pp_data_write_one(device->pport->port,device->id,ADD_A2,1,0,0);
178    if((retval=pp_data_write(device->pport->port,device-
    >id,MEM_DATA,8,CMD_CRC,prog_55_test))!=PP_OK) return(retval);
179
180    pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0x00); // reset offset counter
181    pp_data_write_one(device->pport->port,device->id,MEM_DATA,1,0,0);
182    pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0x66);
183    pp_data_write_one(device->pport->port,device->id,ADD_A1,1,0,TEST_PARALLEL_PROG_AA);
184    pp_data_write_one(device->pport->port,device->id,ADD_A2,1,0,0);
185    if((retval=pp_data_write(device->pport->port,device-
    >id,MEM_DATA,8,CMD_CRC,prog_66_test))!=PP_OK) return(retval);
186
187
188 // put pattern into memory
189    pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0x00); // reset offset counter
```

```
190    pp_data_write_one(device->pport->port,device->id,MEM_DATA,1,0,0);
191
192    pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0x66);
193    pp_data_write_one(device->pport->port,device->id,ADD_A1,1,0,TEST_PARALLEL_PROG_AA);
194    pp_data_write_one(device->pport->port,device->id,ADD_A2,1,0,0x5d);
195    pp_data_write_one(device->pport->port,device->id,MEM_PROG,1,0,0x33);
196    pp_data_write_one(device->pport->port,device->id,MEM_PROG_ADD,1,0,0x77);
197
198    pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0x00); // reset offset counter
199    pp_data_write_one(device->pport->port,device->id,MEM_DATA,1,0,0);
200
201    pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0xaa);
202    pp_data_write_one(device->pport->port,device->id,ADD_A1,1,0,TEST_PARALLEL_PROG_AA);
203    if((retval=pp_data_read(device->pport->port,device-
    >id,MEM_DATA_INC,8,CMD_CRC,data))!=PP_OK) return(retval);
204    add=0xaaaa;
205    error+=compare(add,8,data,prog_aa_exp);
206    printf("0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x, //
    PROG_AA\n",data[0],data[1],data[2],data[3],data[4],data[5],data[6],data[7]);
207    pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0x55);
208    pp_data_write_one(device->pport->port,device->id,ADD_A1,1,0,TEST_PARALLEL_PROG_55);
209    if((retval=pp_data_read(device->pport->port,device-
    >id,MEM_DATA_INC,8,CMD_CRC,data))!=PP_OK) return(retval);
210    add=0x5555;
211    error+=compare(add,8,data,prog_55_exp);
212    printf("0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x, //
    PROG_55\n",data[0],data[1],data[2],data[3],data[4],data[5],data[6],data[7]);
213    pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0x66);
214    pp_data_write_one(device->pport->port,device->id,ADD_A1,1,0,TEST_PARALLEL_PROG_AA);
215    if((retval=pp_data_read(device->pport->port,device-
    >id,MEM_DATA_INC,8,CMD_CRC,data))!=PP_OK) return(retval);
216    add=0xaa66;
217    error+=compare(add,8,data,prog_66_exp);
218    printf("0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x,0x%02x, //
    PROG_66\n",data[0],data[1],data[2],data[3],data[4],data[5],data[6],data[7]);
219
220    mon_write("MSG:TESTING STICKY_CS\n");
221 //???
222
223 #endif
224    mon_write("MSG:TESTING RESISTORS\n");
225    pp_data_write_one(device->pport->port,device->id,ADD_PINOUT,1,0,0); // power-up all tristate!
226    pp_data_write_one(device->pport->port,device->id,ADD_A1,1,0,TEST_FLOAT);
227    if((retval=pp_data_read(device->pport->port,device-
    >id,ADD_INPUTA,1,CMD_CRC,data))!=PP_OK) return(retval);
228    error+=compare_one("INPUTA",data[0],0x7a);
229    if((retval=pp_data_read(device->pport->port,device-
    >id,ADD_INPUTB,1,CMD_CRC,data))!=PP_OK) return(retval);
230    error+=compare_one("INPUTB",data[0],0x01);
231    if((retval=pp_data_read(device->pport->port,device->id,JS_SW0,1,CMD_CRC,data))!=PP_OK) return(retval);
232    error+=compare_one("SW0",data[0],0x00);
233    if((retval=pp_data_read(device->pport->port,device->id,JS_SW1,1,CMD_CRC,data))!=PP_OK) return(retval);
234    error+=compare_one("SW1",data[0],0x00);
235
236    mon_write("MSG:TESTING CFG, INPUTA, INPUTB, and A0\n");
237
238    pp_data_write_one(device->pport->port,device->id,ADD_PINOUT,1,0,POA_ADDRESS|POB_ADDRESS);
239
240    pp_data_write_one(device->pport->port,device->id,ADD_A1,1,0,TEST_IO);
241
242    for(i=1;i<256;i<<=1) {
243       pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,i);
244       if((retval=pp_data_read(device->pport->port,device-
    >id,ADD_INPUTA,1,CMD_CRC,data))!=PP_OK) return(retval);
245       error+=compare_one("INPUTA",data[0],i);
246       if((retval=pp_data_read(device->pport->port,device-
```

```
>id,ADD_INPUTB,1,CMD_CRC,data))!=PP_OK) return(retval);
      error+=compare_one("INPUTB",data[0],(i&0x7));

if((retval=pp_data_read(device->pport->port,device->id,JS_SW0,1,CMD_CRC,data))!=PP_OK) return(retval);
      error+=compare_one("SW0",data[0],i);
      if((retval=pp_data_read(device->pport->port,device-
>id,ADD_CONFIG,1,CMD_CRC,data))!=PP_OK) return(retval);
      error+=compare_one("CFG",data[0],i);
      }

   mon_write("MSG:TESTING BUSY\n");
//???

   mon_write("MSG:TESTING INT\n");
//???

   mon_write("MSG:TESTING BAUD\n");
     // a[5]==0 rts==a[4], else rts=cts
     // ser_din=ser_dout
     // a[3:0]==0 ser_data_a[7:0]
     // a[3:0]==1 ser_data_a[15:8]
     // a[3:0]==2 baud[7:0]
     // a[3:0]==3 baud[15:8]

   pp_data_write_one(device->pport->port,device-
>id,ADD_PINOUT,1,0,POA_ADDRESS|POB_ADDRESS|POD_SERIAL|POE_SERIAL);
   pp_data_write_one(device->pport->port,device->id,ADD_A1,1,0,TEST_SERIAL); // start baud
   data[0]=0; data[1]=SER_FLAGS1_SEND|SER_FLAGS1_RUN|SER_FLAGS1_RECV;
   if((retval=pp_data_write(device->pport->port,device-
>id,SER_FLAGS0,2,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
   if((retval=pp_data_read(device->pport->port,device-
>id,SER_SSTATUS,2,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
   error+=compare_one("SER_SSTATUS",data[0],0x08);
   error+=compare_one("SER_RSTATUS",data[1],0x00);

   if((retval=pp_data_read(device->pport->port,device-
>id,ADD_REQUEST,1,CMD_CRC,data))!=PP_OK) return(retval);

data[0]&=(ADD_REQUEST_SER_RECV_FOUR|ADD_REQUEST_SER_RECV_ANY|ADD_REQUEST_SER_SEND_FOUR|ADD_REQUEST_SER_SEND_ANY);
   error+=compare_one("ADD_REQUEST",data[0],ADD_REQUEST_SER_SEND_FOUR|ADD_REQUEST_SER_SEND_ANY);

   pp_data_write_one(device->pport->port,device->id,SER_BAUD,1,0,0xfe); // baud=/512
   pp_data_write_one(device->pport->port,device->id,SER_DATA,1,0,ser_data[ser_cnt++]); // start baud

   do {
      pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0x2);
      if((retval=pp_data_read(device->pport->port,device-
>id,MEM_DATA_INC,2,CMD_CRC,data))!=PP_OK) return(retval);
      } while (((data[1]&0x80)!=0)|((data[1]|data[0])==0));

   pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0x2);
   if((retval=pp_data_read(device->pport->port,device-
>id,MEM_DATA_INC,2,CMD_CRC,data))!=PP_OK) return(retval);

   i=(data[1]<<8)|data[0];

   printf("baud_count=%d\n",i);
   if(i!=510) {
      mon_error("REGRESSION BAUD INVALID- should be- 510 was- %d\n",i);
      error++;
   }

   mon_write("MSG:TESTING SERIAL FIFOS\n");

   pp_data_write_one(device->pport->port,device->id,SER_BAUD,1,0,0x7f); // baud=/4

```

```
   if((retval=pp_data_read(device->pport->port,device->id,SER_SSTATUS,2,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
   error+=compare_one("SER_SSTATUS",data[0],0x07);
   error+=compare_one("SER_RSTATUS",data[1],0x00);

   // transfer to recv fifo (en cts)
   pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,1<<5);

   do {
     if((retval=pp_data_read(device->pport->port,device->id,SER_SSTATUS,1,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
     } while((data[0]&SER_SSTATUS_VALID)!=8);

   if((retval=pp_data_read(device->pport->port,device->id,ADD_REQUEST,1,CMD_CRC,data))!=PP_OK) return(retval);
   data[0]&=(ADD_REQUEST_SER_RECV_FOUR|ADD_REQUEST_SER_RECV_ANY|ADD_REQUEST_SER_SEND_FOUR|ADD_REQUEST_SER_SEND_ANY);
   error+=compare_one("ADD_REQUEST",data[0],ADD_REQUEST_SER_RECV_ANY|ADD_REQUEST_SER_SEND_FOUR|ADD_REQUEST_SER_SEND_ANY);

   if((retval=pp_data_read(device->pport->port,device->id,SER_SSTATUS,2,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
   error+=compare_one("SER_SSTATUS",data[0],0x08);
   error+=compare_one("SER_RSTATUS",data[1],0x01);

   // put in three more
   if((retval=pp_data_write(device->pport->port,device->id,SER_DATA,3,CMD_CRC,&ser_data[ser_cnt]))!=PP_OK) return(retval);
   ser_cnt+=3;

   do {
     if((retval=pp_data_read(device->pport->port,device->id,SER_SSTATUS,1,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
     } while((data[0]&SER_SSTATUS_VALID)!=8);

   if((retval=pp_data_read(device->pport->port,device->id,ADD_REQUEST,1,CMD_CRC,data))!=PP_OK) return(retval);
   data[0]&=(ADD_REQUEST_SER_RECV_FOUR|ADD_REQUEST_SER_RECV_ANY|ADD_REQUEST_SER_SEND_FOUR|ADD_REQUEST_SER_SEND_ANY);
   error+=compare_one("ADD_REQUEST",data[0],ADD_REQUEST_SER_RECV_ANY|ADD_REQUEST_SER_RECV_FOUR|ADD_REQUEST_SER_SEND_FOUR|

   if((retval=pp_data_read(device->pport->port,device->id,SER_SSTATUS,2,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
   error+=compare_one("SER_SSTATUS",data[0],0x08);
   error+=compare_one("SER_RSTATUS",data[1],0x04);

   // disable transfer to recv fifo (en cts)
   pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,0);

   // fill all but three
   if((retval=pp_data_write(device->pport->port,device->id,SER_DATA,5,CMD_CRC,&ser_data[ser_cnt]))!=PP_OK) return(retval);
   ser_cnt+=5;

   if((retval=pp_data_read(device->pport->port,device->id,ADD_REQUEST,1,CMD_CRC,data))!=PP_OK) return(retval);
   data[0]&=(ADD_REQUEST_SER_RECV_FOUR|ADD_REQUEST_SER_RECV_ANY|ADD_REQUEST_SER_SEND_FOUR|ADD_REQUEST_SER_SEND_ANY);
   error+=compare_one("ADD_REQUEST",data[0],ADD_REQUEST_SER_RECV_FOUR|ADD_REQUEST_SER_RECV_ANY|ADD_REQUEST_SER_SEND_ANY);

   if((retval=pp_data_read(device->pport->port,device->id,SER_SSTATUS,1,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
   error+=compare_one("SER_SSTATUS",data[0],0x03);

```

```
   // fill up to end
   if((retval=pp_data_write(device->pport->port,device->id,SER_DATA,data[0]&SER_SSTATUS_VALID,CMD_CRC,&ser_data[ser_cnt]))!=PP_OK) return(retval);
   ser_cnt+=data[0]&SER_SSTATUS_VALID;

   if((retval=pp_data_read(device->pport->port,device->id,ADD_REQUEST,1,CMD_CRC,data))!=PP_OK) return(retval);

data[0]&=(ADD_REQUEST_SER_RECV_FOUR|ADD_REQUEST_SER_RECV_ANY|ADD_REQUEST_SER_SEND_FOUR|ADD_REQUEST_SER_SEND_ANY);
   error+=compare_one("ADD_REQUEST",data[0],ADD_REQUEST_SER_RECV_FOUR|ADD_REQUEST_SER_RECV_ANY);

   if((retval=pp_data_read(device->pport->port,device->id,SER_SSTATUS,2,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
   error+=compare_one("SER_SSTATUS",data[0],0x00);
   error+=compare_one("SER_RSTATUS",data[1],0x04);

   // transfer to recv fifo (en cts)
   pp_data_write_one(device->pport->port,device->id,ADD_A0,1,0,1<<5);

   do {
     if((retval=pp_data_read(device->pport->port,device->id,SER_SSTATUS,1,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
     } while((data[0]&SER_SSTATUS_VALID)!=8);

   if((retval=pp_data_read(device->pport->port,device->id,SER_SSTATUS,2,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
   error+=compare_one("SER_SSTATUS",data[0],0x08);
   error+=compare_one("SER_RSTATUS",data[1],0x0c);

   // fill up to end
   if((retval=pp_data_write(device->pport->port,device->id,SER_DATA,data[0]&SER_SSTATUS_VALID,CMD_CRC,&ser_data[ser_cnt]))!=PP_OK) return(retval);
   ser_cnt+=data[0]&SER_SSTATUS_VALID;

   do {
     if((retval=pp_data_read(device->pport->port,device->id,SER_RSTATUS,1,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
     } while((data[0]&SER_RSTATUS_VALID)!=(0x10&SER_RSTATUS_VALID));

   if((retval=pp_data_read(device->pport->port,device->id,SER_SSTATUS,2,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
   error+=compare_one("SER_SSTATUS",data[0],0x04);
   error+=compare_one("SER_RSTATUS",data[1],0x10);

   if((retval=pp_data_read(device->pport->port,device->id,SER_DATA,0x10,CMD_CRC,data))!=PP_OK) return(retval);

   error+=compare(0x00,0x10,data,ser_data);
   printf("0x%02x 0x%02x 0x%02x 0x%02x 0x%02x 0x%02x 0x%02x 0x%02x 0x%02x 0x%02x 0x%02x 0x%02x 0x%02x 0x%02x 0x%02x 0x%02x 0x%02x 0x%02x // serport\n",
  data[0],data[1],data[2],data[3],data[4],data[5],data[6],data[7],
  data[8],data[9],data[10],data[11],data[12],data[13],data[14],data[15]);

   do {
     if((retval=pp_data_read(device->pport->port,device->id,SER_RSTATUS,1,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
     } while((data[0]&SER_RSTATUS_VALID)!=(0x04&SER_RSTATUS_VALID));

   if((retval=pp_data_read(device->pport->port,device->id,SER_SSTATUS,2,CMD_CRC|CMD_IA,data))!=PP_OK) return(retval);
   error+=compare_one("SER_SSTATUS",data[0],0x08);
   error+=compare_one("SER_RSTATUS",data[1],0x04);

   if((retval=pp_data_read(device->pport->port,device->id,SER_DATA,0x4,CMD_CRC,data))!=PP_OK) return(retval);

```

```
   error+=compare(0x10,4,data,&ser_data[0x10]);
   printf("0x%02x 0x%02x 0x%02x 0x%02x // serport\n", data[0],data[1],data[2],data[3]);


   if(error>0) mon_error("regression failed!!!! %d error(s)\n",error);

   exit(0);
   return(PP_OK);
   }

int polyface_test_request(struct device *device)
   {
   printf(__FILE__ ": in polyface_test_request\n");
   return(PP_OK);
   }

int polyface_test_configure(struct device *device)
   {
   printf(__FILE__ ": in polyface_test_configure\n");
   return(PP_OK);
   }

int polyface_test_release(struct device *device)
   {
   printf(__FILE__ ": in polyface_test_release\n");
   return(PP_OK);
   }

struct function_execute polyface_test_fexec =
   {
   NULL,
   polyface_test_request,
   polyface_test_release,
   polyface_test_execute,
   polyface_test_configure,
   };
```

12.21 Polyface Joystick Driver (POLYFACE_JOYSTICK_C)

```
/*
polyface joystick driver (polyface_joystick.c)
Written by Jude Katsch @ ANON
05/15/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/

#include "pport.h"
#include "polyface.h"
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include "fifo.h"

extern int quiet;

char byte[]="76543210";

int atod_channel[]={ATOD_CHANNEL_X1,ATOD_CHANNEL_X2,ATOD_CHANNEL_Y1,ATOD_CHANNEL_Y2};

int polyface_joystick_joystick_read(struct device *device,int request)
   {
   unsigned char data[4];
```

```
25    int retval;
26    struct axis *joystick;
27    struct polyface_user_joystick *js = &((struct polyface *)device->user)->js;
28    joystick = js->req;
29
30    if(request||(joystick!=NULL))
31       {
32       if(joystick==NULL) return(0); // nothing to do
33
34 //      if(request==0) mon_error("atod did not complete between ticks.\n");
35
36       // is it a atod request?
37
   if((retval=pp_data_read(device->pport->port,device->id,ADD_REQUEST,1,0,data))!=PP_OK) return(retval);
38       if((data[0]&ADD_REQUEST_ATOD)==0) return(0); // the request is not for the atod
39
40       if(data[0]&ADD_REQUEST_ATOD_ERROR)
41          {
42          joystick->flags |= ATOD_ERROR;
43          data[0]=0x00;
44          mon_write("VARIABLE:%d:%d:JOYS%d:ERROR\n",device->pport->port,device->id,joystick->address);
45          }
46       else
47          {
48          joystick->flags &= ~ATOD_ERROR;
49          if((retval=pp_data_read(device->pport->port,device-
   >id,JS_ATOD,1,CMD_IA|CMD_CRC,data))!=PP_OK) return(retval);
50
   mon_write("VARIABLE:%d:%d:JOYS%d:0x%02x\n",device->pport->port,device->id,joystick->address,data[0]);
51          }
52
53       joystick->data = data[0];
54       joystick=joystick->next;
55       }
56    else
57       joystick=device->joystick->joystick;
58
59    while(joystick!=NULL)
60       {
61       if((joystick->flags & JS_ENABLED) && (js->enable & (1<<(joystick->address+JS_EN_JS))))
62  {
63          js->req = joystick;
64
   pp_data_write_one(device->pport->port,device->id,JS_CHANNEL,1,CMD_IA,atod_channel[joystick->address]);
65          return(0);
66          }
67       joystick=joystick->next;
68       }
69
70    js->req = NULL;
71    pp_data_write_one(device->pport->port,device->id,JS_CHANNEL,1,CMD_IA,ATOD_CHANNEL_NONE);
72    return(0);
73    }
74
75 int polyface_joystick_quad_read(struct device *device)
76    {
77    unsigned char data[4];
78    char previous;
79    char delta;
80    int i=0;
81    int retval;
82    struct axis *quad=device->joystick->quad;
83    struct polyface_user_joystick *js = &((struct polyface *)device->user)->js;
84    while(quad)
85       {
86       previous=quad->data;
```

```
      if((quad->flags & JS_ENABLED)&&(js->enable&(1<<(JS_EN_QUAD+i))))
 {

if((retval=pp_data_read(device->pport->port,device->id,quad->address,1,CMD_IA|CMD_CRC,data))!=PP_OK)
            return(retval);
         quad->data = data[0];
         delta = (int)(quad->data-previous);
         quad->absolute+=(int)delta;
//         printf("p:0x%02x c:0x%02x d:0x%02x a:0x%02x\n",previous,quad->data,delta,quad->absolute);
         mon_write("VARIABLE:%d:%d:QUAD%d:%+
8d/0x%02x\n",device->pport->port,device->id,i,quad->absolute,quad->data);
         }
      quad=quad->next;
      i++;
      }
   return(0);
   }

int polyface_joystick_keys_read(struct device *device)
   {
   unsigned char data[4];
   int i=0;
   int retval;
   struct keys *keys=device->joystick->keys;
   struct polyface_user_joystick *js = &((struct polyface *)device->user)->js;
   while(keys)
      {
      if((keys->flags & JS_ENABLED)&&(js->enable&(1<<(JS_EN_SW+i))))
 {

if((retval=pp_data_read(device->pport->port,device->id,keys->address,1,CMD_IA|CMD_CRC,data))!=PP_OK)
            return(retval);
         keys->data = data[0];
         bytetobinary(byte,keys->data);
         mon_write("VARIABLE:%d:%d:KEYS%d:%s\n",device->pport->port,device->id,i,byte);
         }
      keys=keys->next;
      i++;
      }
   return(0);
   }

int polyface_joystick_rumble(struct device *device)
   {
   struct polyface_user_joystick *js = &((struct polyface *)device->user)->js;
   if((device->joystick->rumble->flags&JS_ENABLED)&&(js->enable&JS_EN_RUMBLE))
      {
//      pp_data_write_one(device->pport->port,device->id,SER_FLAGS0,1,0,SER_FLAGS0_ASYNC);
//      pp_data_write_one(device->pport->port,device-
>id,SER_FLAGS1,1,0,SER_FLAGS1_STICKY_CS|SER_FLAGS1_SEND|SER_FLAGS1_MASK_CTS|SER_FLAGS1_RUN);
//      pp_data_write_one(device->pport->port,device->id,SER_BAUD,1,0,device->joystick->rumble->speed);
//      pp_data_write_one(device->pport->port,device->id,SER_DATA,1,0,device->joystick->rumble->pattern);
      }
   return(0);
   }

// call to update all the joystick registers
int polyface_joystick_execute(struct device *device)
   {
   if(!quiet) printf(__FILE__ ": in polyface_joystick_execute\n");
   polyface_joystick_keys_read(device);
   polyface_joystick_quad_read(device);
   polyface_joystick_joystick_read(device,0);
   return(PP_OK);
   }

```

```
int polyface_joystick_request(struct device *device)
   {
   polyface_joystick_joystick_read(device,1);
   if(!quiet) printf(__FILE__ ": in polyface_joystick_request\n");
   return(PP_OK);
   }

int polyface_joystick_configure(struct device *device)
   {
   int i;
   printf(__FILE__ ": in polyface_joystick_configure\n");
   mon_write("JOYSTICK:%d:%d:ADD\n",device->pport->port,device->id);
   i=sizeof(struct joystick)+(6*sizeof(struct axis))+(2*sizeof(struct keys))+sizeof(struct rumble);
   if((device->joystick=(struct joystick *)calloc(i,1))==NULL)
      {
      mon_error("allocating memory for joystick_info structure.\n");
      return(PP_INTERNAL);
      }

   device->joystick->joystick                    = (struct axis     *) &device->joystick[1];
   device->joystick->joystick->next              = (struct axis     *) &device->joystick->joystick[1];
   device->joystick->joystick->next->next        = (struct axis     *) &device->joystick->joystick[2];
   device->joystick->joystick->next->next->next  = (struct axis     *) &device->joystick->joystick[3];
   device->joystick->quad                        = (struct axis     *) &device->joystick->joystick[4];
   device->joystick->quad->next                  = (struct axis     *) &device->joystick->quad[1];
   device->joystick->keys                        = (struct keys     *) &device->joystick->quad[2];
   device->joystick->keys->next                  = (struct keys     *) &device->joystick->keys[1];
   device->joystick->rumble                      = (struct rumble   *) &device->joystick->keys[2];

   device->joystick->keys->address               = JS_SW0;
   device->joystick->keys->next->flags           = JS_ENABLED;
   device->joystick->keys->next->address         = JS_SW1;
   device->joystick->keys->flags                 = JS_ENABLED;

   device->joystick->quad->address               = JS_QUADX;
   device->joystick->quad->flags                 = JS_ENABLED;
   device->joystick->quad->data                  = 0; // data read
   device->joystick->quad->absolute              = 0; // current position

   device->joystick->quad->next->address         = JS_QUADY;
   device->joystick->quad->next->flags           = JS_ENABLED;
   device->joystick->quad->next->data            = 0; // data read
   device->joystick->quad->next->absolute        = 0; // current position

   device->joystick->rumble->speed               = 0xfe; // slowest
   device->joystick->rumble->pattern             = 0x0f;
   device->joystick->rumble->flags               = JS_ENABLED;

   device->joystick->joystick->address                   = 0;
   device->joystick->joystick->flags                     = JS_ENABLED;
   device->joystick->joystick->next->address             = 1;
   device->joystick->joystick->next->flags               = JS_ENABLED;
   device->joystick->joystick->next->next->address       = 2;
   device->joystick->joystick->next->next->flags         = JS_ENABLED;
   device->joystick->joystick->next->next->next->address = 3;
   device->joystick->joystick->next->next->next->flags   = JS_ENABLED;

   ((struct polyface *)device->user)->request |= ADD_REQUEST_EN_ATOD;

   return(0);
   }

int polyface_joystick_release(struct device *device)
   {
   if(device->joystick)
      {
```

```
      free(device->joystick);
      device->joystick=NULL;
      }
   mon_write("JOYSTICK:%d:%d:DEL\n",device->pport->port,device->id);
   return(0);
   }

struct function_execute polyface_joystick_fexec =
   {
   NULL,
   polyface_joystick_request,
   polyface_joystick_release,
   polyface_joystick_execute,
   polyface_joystick_configure,
   };
```

12.22 Polyface Serial Driver (POLYFACE_SERIAL_C)

```
/*
polyface serial port interface polyface_serial.c
Written by Jude Katsch @ ANON
04/10/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/

#include "pport.h"
#include "polyface.h"
#include <stdlib.h>
#include <stdio.h>
#include "fifo.h"
#include <ctype.h>

extern int quiet;

int serial_test(struct device *device,unsigned char flags0,unsigned char flags1)
   {
   unsigned char data[4];
   int retval;
   unsigned char test_data[]={0xff,0x00,0xff,0x00};
   printf(__FILE__ ": in serial_test\n");
   pp_data_write_one(device->pport->port,device->id,SER_FLAGS0,1,0,flags0);
   pp_data_write_one(device->pport->port,device->id,SER_FLAGS1,1,0,flags1);
   pp_data_write_one(device->pport->port,device->id,SER_BAUD,1,0,0x7f);
   pp_data_write_one(device->pport->port,device->id,SER_DATA,1,0,0xaa);
   pp_data_write_one(device->pport->port,device->id,SER_DATA,1,0,0x55);
   pp_data_write(device->pport->port,device->id,SER_DATA,2,0,test_data);
   do
     {
     if((retval=pp_data_read(device->pport->port,device->id,SER_SSTATUS,2,CMD_IA|CMD_CRC,data))!=PP_OK) return(retval);
     printf("sstatus=0x%02x rstatus=0x%02x\n",data[0],data[1]);
     } while((data[1]&0xf)!=0x4);

if((retval=pp_data_read(device->pport->port,device->id,SER_DATA,4,CMD_CRC,data))!=PP_OK) return(retval);
   printf("recieved 0x%02x 0x%02x 0x%02x 0x%02x\n",data[0],data[1],data[2],data[3]);
   if(data[0]!=0xaa||data[1]!=0x55||data[2]!=0xff||data[3]!=0x00)
   printf("ERROR Expected: 0xaa 0x55 0xff 0x00\n");
   return(PP_OK);
   }

int polyface_serial_test(struct device *device)
   {
```

```
   unsigned char flags=SER_FLAGS1_SEND|SER_FLAGS1_RUN|SER_FLAGS1_RECV;
   printf(__FILE__ ": in polyface_serial_test\n");

   pp_data_write_one(device->pport->port,device->id,SER_FLAGS0,1,0,SER_FLAGS0_MASKED_CLK);
   pp_data_write_one(device->pport->port,device->id,SER_FLAGS1,1,0,flags);
   pp_data_write_one(device->pport->port,device->id,SER_BAUD,1,0,0x7f);

   serial_test(device,SER_FLAGS0_MASKED_CLK                                          ,flags);
   serial_test(device,SER_FLAGS0_MASKED_CLK|                     SER_FLAGS0_LEVEL_HIGH,flags);
   serial_test(device,SER_FLAGS0_MASKED_CLK|SER_FLAGS0_CLK_HIGH                      ,flags);
   serial_test(device,SER_FLAGS0_MASKED_CLK|SER_FLAGS0_CLK_HIGH|SER_FLAGS0_LEVEL_HIGH ,flags);
   serial_test(device,SER_FLAGS0_MASKED_CLK|SER_FLAGS0_CLK_HIGH|SER_FLAGS0_LEVEL_HIGH|SER_FLAGS0_CS_NEG,flags);

   serial_test(device,SER_FLAGS0_MASKED_CLK|SER_FLAGS0_CLK_HIGH|SER_FLAGS0_LEVEL_HIGH,flags|SER_FLAGS1_STICKY_CS);

   serial_test(device,SER_FLAGS0_LSB_FIRST|SER_FLAGS0_SYNC_BITS,flags);
   serial_test(device,SER_FLAGS0_LSB_FIRST|SER_FLAGS0_PARITY_BIT,flags);
   serial_test(device,SER_FLAGS0_LSB_FIRST|SER_FLAGS0_SYNC_BITS|SER_FLAGS0_PARITY_BIT,flags);

   serial_test(device,SER_FLAGS0_ASYNC|SER_FLAGS0_LSB_FIRST|SER_FLAGS0_SYNC_BITS,flags);
   serial_test(device,SER_FLAGS0_ASYNC|SER_FLAGS0_LSB_FIRST|SER_FLAGS0_PARITY_BIT,flags);
   serial_test(device,SER_FLAGS0_ASYNC|SER_FLAGS0_LSB_FIRST|SER_FLAGS0_SYNC_BITS|SER_FLAGS0_PARITY_BIT,flags);
//   serial_test(device,SER_FLAGS0_ASYNC|SER_FLAGS0_LSB_FIRST|SER_FLAGS0_SYNC_BITS|SER_FLAGS0_PARITY_BIT,flags|SER_FLAGS1_IC

   return(PP_OK);
   }


// this is called to release a device
int polyface_serial_release(struct device *device)
   {
   if(!quiet) printf(__FILE__ ": in polyface_serial_release\n");
   mon_write("SERIAL:%d:%d:DEL\n",device->pport->port,device->id);
   free(device->serial);
   device->serial=NULL;
   return(PP_OK);
   }

int polyface_serial_buffer_cleanup(struct device *device)
   {
   int i,j;
   struct data_buffer *dbuf = device->serial->sdbuf;
   if(dbuf->topptr>(SER_BUFSIZE/4)||( (dbuf->topptr!=0)&&(dbuf->botptr>(SER_BUFSIZE-(SER_BUFSIZE/8))) ))
      {
      for(j=0,i=dbuf->topptr;i<dbuf->botptr;i++,j++)
         dbuf->data[j]=dbuf->data[i];
      dbuf->topptr = 0;
      dbuf->botptr = j;
      }
   return(PP_OK);
   }

int polyface_serial_error_send(struct device *device,char data)
   {
   if(data&SER_SSTATUS_OF)
      {
      device->serial->error_send_of++;
      mon_error("serial receive overflow error\n");
      pp_data_write_one(device->pport->port,device->id,SER_SSTATUS,1,0,SER_SSTATUS_ERRORS);
      }
   return(PP_OK);
   }
```

```
106
107 int polyface_serial_error_recv(struct device *device,char data)
108    {
109    if(data&SER_RSTATUS_ERRORS)
110       {
111 if(data&SER_RSTATUS_OF) { device->serial->error_recv_of++; mon_error("serial receive overflow error\n"); }
112 if(data&SER_RSTATUS_UF) { device->serial->error_recv_uf++; mon_error("serial receive underflow error\n"); }
113 if(data&SER_RSTATUS_P)  { device->serial->error_recv_p++;  mon_error("serial receive parity error\n"); }
114       pp_data_write_one(device->pport->port,device->id,SER_RSTATUS,1,0,SER_RSTATUS_ERRORS&data);
115       }
116    return(PP_OK);
117    }
118
119 int polyface_serial_error(struct device *device)
120    {
121    int retval;
122    unsigned char data[4];
123    if (!quiet) printf(__FILE__ ": in polyface_serial_error\n");
124    if((retval=pp_data_read(device->pport->port,device->id,ADD_REQUEST,1,0,data))!=PP_OK) return(retval);
125 //   if((data[0]&ADD_REQUEST_SER_ERROR)==0) return(PP_OK);
126    if((retval=pp_data_read(device->pport->port,device->id,SER_SSTATUS,2,0,data))!=PP_OK) return(retval);
127    polyface_serial_error_send(device,data[0]);
128    polyface_serial_error_recv(device,data[1]);
129    return(PP_OK);
130    }
131
132 int polyface_serial_request(struct device *device)
133    {
134    int retval;
135    struct data_buffer *dbuf = device->serial->sdbuf;
136    unsigned char data[8],data1[4];
137    int avail,size,xfer_size;
138    if (!quiet) printf(__FILE__ ": in polyface_serial_request\n");
139    if((retval=pp_data_read(device->pport->port,device->id,ADD_REQUEST,1,0,data1))!=PP_OK) return(retval);
140    if((data1[0]&ADD_REQUEST_SER_RECV_ANY)&&(device->serial->flags&SER_FLAGS_RECV_EN))
141       {
142
    if((retval=pp_data_read(device->pport->port,device->id,SER_RSTATUS,1,0,data))!=PP_OK) return(retval);
143       polyface_serial_error_recv(device,data[0]);
144       avail = data[0]&0xf;
145       if(avail>0)
146 {
147         if((retval=pp_data_read(device->pport->port,device-
    >id,SER_DATA,avail,CMD_CRC,data))!=PP_OK) return(retval);
148         mon_write("SERIAL:%d:%d:DATA:%d:%d,%d,%d,%d,%d,%d,%d,%d\n",device->pport->port,device-
    >id,avail,data[0],data[1],data[2],data[3],data[4],data[5],data[6],data[7]);
149         }
150       }
151    size = (dbuf) ? (dbuf->botptr-dbuf->topptr) : 0;
152    if((data1[0]&ADD_REQUEST_SER_SEND_ANY)&&(size>0)&&(device->serial->flags&SER_FLAGS_SEND_EN))
153       {
154
    if((retval=pp_data_read(device->pport->port,device->id,SER_SSTATUS,1,0,data))!=PP_OK) return(retval);
155       polyface_serial_error_send(device,data[0]);
156       avail = data[0]&0xf;
157       if(avail>0)
158 {
159         xfer_size = (size>avail) ? avail : size;
160         if((retval=pp_data_write(device->pport->port,device->id,SER_DATA,xfer_size,CMD_CRC,&dbuf-
    >data[dbuf->topptr]))!=PP_OK) return(retval);
161 //         printf("sending %d bytes\n",xfer_size);
162         dbuf->topptr+=xfer_size;
163         polyface_serial_buffer_cleanup(device);
164         }
165       }
166    return(PP_OK);
```

```
   }

int polyface_serial_send(struct device *device,int size,char *data)
   {
   int i;
   char dchar;
   struct data_buffer *dbuf = device->serial->sdbuf;
   if(!quiet) printf(__FILE__ ": in polyface_serial_send size=%d data=%s\n",size,data);
   if(device->serial==NULL) return(PP_INTERNAL);
   polyface_serial_buffer_cleanup(device);
   if((dbuf->botptr+size)>SER_BUFSIZE)
      {
      mon_error("serial send buffer exceeds %d bytes\n",SER_BUFSIZE);
      return(PP_INTERNAL);
      }
   for(i=0;i<size;i++)
     {
     dchar=data[i];
     if (dchar=='\\')
       {
       if (++i==size)
          {
          mon_error("serial string ends with a '\\', ignored\n");
          break;
          }
       switch(data[i])
 {
         case '0' : dchar=0x00; break;
         case 'a' : dchar=0x07; break;
         case 'b' : dchar=0x08; break;
         case 'f' : dchar=0x0c; break;
         case 'n' : dchar=0x0a; break;
         case 'r' : dchar=0x0d; break;
         case 't' : dchar=0x09; break;
         case 'v' : dchar=0x0b; break;
         case 'x' :
           if((i+2)>=size||(!isxdigit(data[i+1]))||(!isxdigit(data[i+2])))
      {
              mon_error("invalid hex number, use '\\x??' where ? is [0-9A-Fa-f]\n");
              break;
              }
           dchar =((isdigit(data[++i]))?data[i]-'0':(tolower(data[i])-'a'+10))*0x10;
           dchar+=((isdigit(data[++i]))?data[i]-'0':(tolower(data[i])-'a'+10));
           break;
         default  : dchar=data[i]; break;
         }
       }
     dbuf->data[dbuf->botptr++]=dchar;
     }
//   printf("serial_data=%s\n",&dbuf->data[dbuf->topptr]);
   return(polyface_serial_request(device));
   }

int polyface_serial_execute(struct device *device)
   {
   if (!quiet) printf(__FILE__ ": in polyface_serial_execute\n");
   return(PP_OK);
   }

int polyface_serial_configure(struct device *device)
   {
   int retval;
   unsigned char data[4];
   struct polyface_user_serial *ser = &((struct polyface *)device->user)->ser;
//   return(polyface_serial_test(device));
   if (!quiet) printf(__FILE__ ": in polyface_serial_configure\n");
```

```
// clear errors (if any)
   if(device->serial)
      {
      fprintf(stderr,"ERROR: expected device->serial to be NULL!\n");
      exit(1);
      }
   if((device->serial=(struct serial *)malloc(sizeof(struct serial)))==NULL)
      {
      fprintf(stderr,"can't allocate memory for serial");
      exit(1);
      }
   device->serial->device=device;
   device->serial->flags=SER_FLAGS_SEND_EN|SER_FLAGS_RECV_EN;
   device->serial->rdbuf=NULL; //we dont support a recieve buffer yet!
   device->serial->send=&polyface_serial_send;
   device->serial->recv=NULL;
   device->serial->error_send_of=0;
   device->serial->error_recv_of=0;
   device->serial->error_recv_uf=0;
   device->serial->error_recv_p=0;
   if((device->serial->sdbuf=(struct data_buffer *)malloc(sizeof(struct data_buffer)))==NULL)
      {
      fprintf(stderr,"can't allocate memory for data_buffer");
      exit(1);
      }
   device->serial->sdbuf->next=NULL;
   device->serial->sdbuf->topptr=0;
   device->serial->sdbuf->botptr=0;
   mon_write("SERIAL:%d:%d:ADD\n",device->pport->port,device->id);
   pp_data_write_one(device->pport->port,device->id,SER_FLAGS0,1,0,(unsigned char)ser->flags0);
   pp_data_write_one(device->pport->port,device->id,SER_FLAGS1,1,0,(unsigned char)ser->flags1);
   pp_data_write_one(device->pport->port,device->id,SER_BAUD,1,0,(unsigned char)ser->baud);
   pp_data_write_one(device->pport->port,device->id,SER_SSTATUS,1,0,SER_SSTATUS_ERRORS);
   if((retval=pp_data_read(device->pport->port,device-
>id,SER_SSTATUS,1,CMD_CRC,data))!=PP_OK) return(retval);
   pp_data_write_one(device->pport->port,device->id,SER_RSTATUS,1,0,SER_RSTATUS_ERRORS);
   if((retval=pp_data_read(device->pport->port,device-
>id,SER_RSTATUS,1,CMD_CRC,data))!=PP_OK) return(retval);
   ((struct polyface *)device->user)->request |= ADD_REQUEST_EN_SER_SEND_ANY|ADD_REQUEST_EN_SER_RECV_ANY;
   return(PP_OK);
   }

struct function_execute polyface_serial_fexec =
   {
   polyface_serial_error,
   polyface_serial_request,
   polyface_serial_release,
   polyface_serial_execute,
   polyface_serial_configure,
   };
```

12.23 Polyface Parallel Memory Driver (POLYFACE_MEMORY_PARALLEL_C)

```
/*
polyface memory driver (polyface_memory_parallel.c)
Written by Jude Katsch @ ANON
02/20/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/

#include "pport.h"
#include "memory.h"
```

```
11 #include "polyface.h"
12 #include <stdlib.h>
13 #include <stdio.h>
14 extern int quiet;
15
16 void pmp_increment_address(struct memory *memory, int size)
17    {
18    ((struct polyface *)memory->device->user)->mem.parallel_address=
19       ((((struct polyface *)memory->device->user)->mem.parallel_address+size)&0xff)|
20       ((((struct polyface *)memory->device->user)->mem.parallel_address)&~0xff);
21    }
22
23 void pmp_invalidate_address(struct memory *memory)
24    {
25    ((struct polyface *)memory->device->user)->mem.parallel_address|=~0&0xffffff;
26    }
27
28 //=========================================================================================
29 // Write address lines for parallel memory. NOTE: this is a smart write, only the address lines used
30 // are updated and only if necessary. The bottom byte can be automaticly incremented after each
31 // write.
32
33 // put the data on the pins (this should work with all parallel devices)
34 // NOTE: sticky_cs state a2[7] is probigated
35 // NOTE: size (*size_ptr) will be updated to contain the largest block transfer possible, up to
36 // it's initial value.
37 int pmp_address(struct memory *memory,int address,int *size_ptr)
38    {
39    int *aptr = &(((struct polyface *)memory->device->user)->mem.parallel_address);
40    int data;
41    if(!quiet) printf(__FILE__ ": in pmp_address add=0x%08x size=%d\n",address,size_ptr?(*size_ptr):0);
42
43    if(address>=memory->info->size) // past end of memory
44       return(PP_BAD_ADDRESS);
45
46 // you can only write below the fence if the MEMORY_CONFIGURE flag is set
47    if((address<memory->info->fence_address)&&((memory->flags&MEMORY_FLAGS_CONFIGURE)==0))
48       return(PP_BAD_ADDRESS);
49
50 // adjust size to byte boundary
51    if(size_ptr!=NULL)
52       {
53 // this assumes there is no internal sram buffer to keep track of
54 // this assumes that there are no suspened operations i.e. erase_suspend
55       if(*size_ptr==0) return(PP_DONE);
56       if(((address%0x100)+(*size_ptr))>0x100)
57          *size_ptr=0x100-(address%0x100);
58       if(address+*size_ptr>=memory->info->size) // let's write to the very end
59          *size_ptr=memory->info->size-address;
60       if(*size_ptr>0x7f) *size_ptr=0x7f; // largest number in command
61       }
62
63    address += memory->base_address;
64
65    switch(((struct polyface *)memory->device->user)-
   >mem.flags&(MEMORY_FLAGS_IF_MASK|MEMORY_FLAGS_IF_SUB_MASK))
66      {
67      case MEMORY_FLAGS_IF_PARALLEL_23: // use all three address bytes
68      case MEMORY_FLAGS_IF_PARALLEL_20:
69 // set address2
70          if((address&0x7f0000)!=((*aptr)&0x7f0000)||((*aptr)<0)) // ignore upper bit of byte
71             {
72             data = ((address>>16)&0x7f)|(((*aptr)>>16)&0x80);
73             pp_data_write_one(memory->device->pport->port,memory->device->id,ADD_A2,1,0,data);
74             (*aptr) = ((*aptr)&0xff00ffff)|(data<<16);
75      }
```

```

     case MEMORY_FLAGS_IF_PARALLEL_16: // use two lower address bytes
// set address1
          if((address&0x00ff00)!=((*aptr)&0x00ff00)||((*aptr)<0))
             {
             data = ((address>>8)&0xff);
             pp_data_write_one(memory->device->pport->port,memory->device->id,ADD_A1,1,0,data);
             (*aptr) = ((*aptr)&0xffff00ff)|(data<<8);
     }

     case MEMORY_FLAGS_IF_PARALLEL_8: // use lower address byte
// set address0
          if((address&0x0000ff)!=((*aptr)&0x0000ff)||((*aptr)<0))
             {
             data = ((address>>0)&0xff);
             pp_data_write_one(memory->device->pport->port,memory->device->id,ADD_A0,1,0,data);
             (*aptr) = ((*aptr)&0xffffff00)|(data<<0);
     }
          break;
    default:
       printf(__FILE__ ": unrecognized parallel address mode\n");
       return(PP_BAD_PARAM);
     }
   (*aptr)&=0x00ffffff; // mask upper byte
   return(PP_OK);
   }

// just read, nothing special
int pmp_read(struct memory *memory,int address,int size,unsigned char *data)
   {
   int retval;
   int xfer_size=MIN(size,memory->maximum_read_size);
   int xfer_address=address;
   if(!quiet) printf(__FILE__ ": in pmp_read\n");
   while((retval=(*memory->iface->address)(memory,xfer_address,&xfer_size))==PP_OK)
     {
     if(!quiet) printf(__FILE__ "ram_read read %d byte%s starting at address 0x%08x\n",
    xfer_size,(xfer_size==1)?"":"s",xfer_address);
     if((retval=pp_data_read(memory->device->pport->port,memory->device->id,
MEM_DATA_INC|memory->strobe,xfer_size,CMD_CRC,data))!=PP_OK)
        {
        pmp_invalidate_address(memory);
        return(retval);
        }
     pmp_increment_address(memory,xfer_size);
     data+=xfer_size;
     xfer_address+=xfer_size;
     size-=xfer_size;
     memory->count_read+=xfer_size;
     xfer_size=MIN(size,memory->maximum_read_size);
     }
   return((retval==PP_DONE)?PP_OK:retval);
   }

// just write, nothing special
int pmp_write(struct memory *memory,int address,int size,unsigned char *data)
   {
   int retval;
   int xfer_size=MIN(size,memory->maximum_write_size);
   int xfer_address=address;

   if(!quiet) printf(__FILE__ ": in pmp_write\n");

   if((memory->flags&MEMORY_FLAGS_WRITE_ENABLED)==0) return(PP_NOT_WRITEABLE);

   while((retval=(*memory->iface->address)(memory,xfer_address,&xfer_size))==PP_OK)
```

```
      {
      if((retval=pp_data_write(memory->device->pport->port,memory->device->id,MEM_DATA_INC|memory->strobe,xfer_size,CMD_CRC,data))!=PP_OK)
         {
         pmp_invalidate_address(memory);
         return(retval);
         }
      pmp_increment_address(memory,xfer_size);
      data+=xfer_size;
      xfer_address+=xfer_size;
      size-=xfer_size;
      memory->count_write+=xfer_size;
      xfer_size=MIN(size,memory->maximum_write_size);
      }
   return((retval==PP_DONE)?PP_OK:retval);
   }

// read don't increment address between reads
int pmp_read_hold(struct memory *memory,int address,int size,unsigned char *data)
   {
   int retval;
   int xfer_size=1;
   if(!quiet) printf(__FILE__ ": in pmp_read_hold\n");
   if((retval=(*memory->iface->address)(memory,address,&xfer_size))!=PP_OK) return(retval);

   while(size>0)
      {
      xfer_size=MIN(size,memory->maximum_read_size);
      if((retval=pp_data_read(memory->device->pport->port,memory->device->id,MEM_DATA|memory->strobe,xfer_size,CMD_CRC,data))!=PP_OK)
         return(retval);
      data+=xfer_size;
      size-=xfer_size;
      memory->count_read+=xfer_size;
      }
   return((retval==PP_DONE)?PP_OK:retval);
   }

// write don't increment address between writes
int pmp_write_hold(struct memory *memory,int address,int size,unsigned char *data)
   {
   int retval;
   int xfer_size=1;

   if(!quiet) printf(__FILE__ ": in pmp_write_hold\n");

   if((memory->flags&MEMORY_FLAGS_WRITE_ENABLED)==0) return(PP_NOT_WRITEABLE);
   if((retval=(*memory->iface->address)(memory,address,&xfer_size))!=PP_OK) return(retval);
   while(size>0)
      {
      xfer_size=MIN(size,memory->maximum_write_size);
      if((retval=pp_data_write(memory->device->pport->port,memory->device->id,MEM_DATA|memory->strobe,xfer_size,CMD_CRC,data))!=PP_OK)
         return(retval);
      data+=xfer_size;
      size-=xfer_size;
      memory->count_write+=xfer_size;
      }
   return((retval==PP_DONE)?PP_OK:retval);
   }

// just fill starting at <address>, <size> bytes with <data>, nothing special
int pmp_write_fill(struct memory *memory,int address,int size,unsigned char data)
   {
   int retval;
   int xfer_size=MIN(size,memory->maximum_write_size);
```

```
   int xfer_address=address;
   if(!quiet) printf(__FILE__ ": in pmp_write_fill\n");

   if((memory->flags&MEMORY_FLAGS_WRITE_ENABLED)==0) return(PP_NOT_WRITEABLE);

   while((retval=(*memory->iface->address)(memory,xfer_address,&xfer_size))==PP_OK)
     {
     if((retval=pp_data_write_one(memory->device->pport->port,memory->device->id,MEM_DATA|memory->strobe,xfer_size,0,data))!=PP_OK)
        {
        pmp_invalidate_address(memory);
        return(retval);
        }
     pmp_increment_address(memory,xfer_size);
     xfer_address+=xfer_size;
     size-=xfer_size;
     memory->count_write+=xfer_size;
     xfer_size=MIN(size,memory->maximum_write_size);
     }
   return((retval==PP_DONE)?PP_OK:retval);
   }

// output (0x00aaaa)=0xaa (0x005555)=0x55 (0x00aaaa)=data
int pmp_write_unlock(struct memory *memory,unsigned char data)
   {
   if(!quiet) printf(__FILE__ ": in pmp_write_unlock\n");
   if((memory->flags&MEMORY_FLAGS_WRITE_ENABLED)==0) return(PP_NOT_WRITEABLE);
   return(pp_data_write_one(memory->device->pport->port,memory->device->id,MEM_PROG|memory->strobe,1,0,data));
   }

// output (0x00aaaa)=0xaa (0x005555)=0x55 (address)=data
int pmp_write_unlock_add(struct memory *memory,int address,unsigned char data)
   {
   int retval;
   int xfer_size=1;
   if(!quiet) printf(__FILE__ ": in pmp_write_unlock_address\n");
   if((memory->flags&MEMORY_FLAGS_WRITE_ENABLED)==0) return(PP_NOT_WRITEABLE);
   if((retval=(*memory->iface->address)(memory,address,&xfer_size))!=PP_OK) return(retval);
   return(pp_data_write_one(memory->device->pport->port,memory->device->id,MEM_PROG_ADD|memory->strobe,1,0,data));
   }

struct memory_interface polyface_parallel_memory_interface =
   {
   &pmp_address,
   &pmp_read,
   &pmp_write,
   &pmp_read_hold,
   &pmp_write_hold,
   &pmp_write_fill,
   &pmp_write_unlock,
   &pmp_write_unlock_add,
   };
```

12.24 Generic Memory Driver include (MEMORY_H)

```
/*
peripheral port memory includes (memory.h)
Written by Jude Katsch @ ANON
07/29/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
```

```
 7 */
 8
 9
10 extern struct memory *memory;
11
12 int read_flash_cfi(struct memory *memory);
13
14 // NOTE: all addressing is done relative to base!
15 struct function_memory
16    {
17    int (*reset)             (struct memory *memory);
18    int (*read_status)       (struct memory *memory,int address,int write_mode,unsigned char data);
19    int (*read_id)           (struct memory *memory);
20    int (*write)             (struct memory *memory,int address,int size,unsigned char *data);
21    int (*read)              (struct memory *memory,int address,int size,unsigned char *data);
22    int (*write_flush)       (struct memory *memory);
23    int (*erase_page)        (struct memory *memory,int address);
24    int (*erase_block)       (struct memory *memory,int address);
25    int (*erase_chip)        (struct memory *memory);
26    int (*erase_suspend)     (struct memory *memory);
27    int (*erase_resume)      (struct memory *memory);
28    int (*read_block_status) (struct memory *memory,int address);
29    int (*lock_block)        (struct memory *memory,int address);
30    int (*unlock_block)      (struct memory *memory,int address);
31    };
32
33 // memory_info is either constructed from cfi, a lut, or by testing, or
34 // retrieved from the base of memory.
35 // flash -- cfi or lut via jedic or memory_info
36 // rom   -- memory_info
37 // ram   -- memory_info (if the ram is unconfigured we will write memory_info)
38
39 #define MEMORY_INFO_NONVOLITLE        (1<<0)
40 #define MEMORY_INFO_WRITEABLE         (1<<1)
41
42 #define MEMORY_INFO_RETRY_CRC_READS       (1<<4) // ok to retry reads if there is a crc failure
43 #define MEMORY_INFO_RETRY_CRC_WRITES      (1<<5) // ok to retry writes if there is a crc failure
44 #define MEMORY_INFO_RETRY_TIMEOUT_READS   (1<<6) // ok to retry reads if there is a timeout failure
45 #define MEMORY_INFO_RETRY_TIMEOUT_WRITES  (1<<7) // ok to retry writes if there is a timeout failure
46
47 #define MEMORY_INFO_DEVICE_WIDTH       8
48 #define MEMORY_INFO_DEVICE_WIDTH_MASK (3<<MEMORY_INFO_DEVICE_WIDTH)
49 #define MEMORY_INFO_DEVICE_WIDTH_8    (0<<MEMORY_INFO_DEVICE_WIDTH)
50 #define MEMORY_INFO_DEVICE_WIDTH_16   (1<<MEMORY_INFO_DEVICE_WIDTH)
51 #define MEMORY_INFO_DEVICE_WIDTH_32   (2<<MEMORY_INFO_DEVICE_WIDTH)
52
53 #define MEMORY_INFO_TYPE              12
54 #define MEMORY_INFO_TYPE_MASK         (0xF<<MEMORY_INFO_TYPE)
55 #define MEMORY_INFO_TYPE_ROM          (0<<MEMORY_INFO_TYPE)
56 #define MEMORY_INFO_TYPE_RAM          (1<<MEMORY_INFO_TYPE)
57 #define MEMORY_INFO_TYPE_EEPROM       (2<<MEMORY_INFO_TYPE)
58 #define MEMORY_INFO_TYPE_FLASH        (3<<MEMORY_INFO_TYPE)
59
60 #define MEMORY_INFO_CFI               (1<<16)
61 #define MEMORY_INFO_ID                (1<<17)
62 #define MEMORY_INFO_INFO              (1<<18)
63 #define MEMORY_INFO_RAM               (1<<19)
64
65
66 // what does an erased memory look like?
67 #define MEMORY_INFO_ERASED
68 #define MEMORY_INFO_ERASED_0         (0x0<<MEMORY_INFO_ERASED)
69 #define MEMORY_INFO_ERASED_1         (0x1<<MEMORY_INFO_ERASED)
70 #define MEMORY_INFO_ERASED_ANY       (0x2<<MEMORY_INFO_ERASED)
71 #define MEMORY_INFO_ERASED_UNKNOWN   (0x3<<MEMORY_INFO_ERASED)
72
```

```
#define MEMORY_INFO_MAX_REGIONS 16
#define MEMORY_INFO_MAGIC 0x5aa56996

struct memory_region
   {
   int sectors;
   int blocks_per_sector;
   int blocks;
   int pages_per_block;
   int page_size;
   };

struct memory_algorithm
   {
   int idcode;    // id code see CFI publication 100
   int address;   // offset of table
   void *table;   // data structure for algorithm
   };

#define MEMORY_FLAGS_CONFIGURE              (1<<0)  // must be set to write below fence
#define MEMORY_FLAGS_WRITE_ENABLED          (1<<1)  // must be set to write to memory

struct memory_info
   {
   int       magic;
   int       flags;
   int       mfg_code;
   int       device_code;
   int       size; // size in bytes
   int       interface;
   struct    memory_algorithm primary;
   struct    memory_algorithm alternate;
// regions
   int       regions;
   struct memory_region region[MEMORY_INFO_MAX_REGIONS];
// all timeouts are in useconds
   int       to_write_byte;
   int       to_write_buffer;
   int       to_erase_block;
   int       to_erase_chip;
// maximum allowable transfer size
   int       maximum_read_size;
   int       maximum_write_size;
// addresses below the fence should not be modified unless the MEMORY_FLAGS_CONFIGURE
// flag is set (the fence should reserve room the info block)
   int       next_base_address;
   int       fence_address;
   int       crc;
   };

// NOTE: all addressing is done relative to base!
struct memory_interface
   {
   int (*address)            (struct memory *memory, int address, int *size_ptr);
   int (*read)               (struct memory *memory, int address, int size, unsigned char *data);
   int (*write)              (struct memory *memory, int address, int size, unsigned char *data);
   int (*read_hold)          (struct memory *memory, int address, int size, unsigned char *data);
   int (*write_hold)         (struct memory *memory, int address, int size, unsigned char *data);
   int (*write_fill)         (struct memory *memory, int address, int size, unsigned char data);
   int (*write_unlock)       (struct memory *memory, unsigned char data);
   int (*write_unlock_add)   (struct memory *memory, int address, unsigned char data);
   };

struct memory
   {
   struct    device *device; // so we can back reference
```

```
   struct    memory *next; // there can be more than one memory
   struct    function_memory *function;
   struct    memory_info *info;
   struct    memory_interface *iface;
   int       flags;
   int       strobe; // all xfers addresses will be or'ed with this
   int       base_address; // offset of device (2^n boundary)
// init maximum xfer sizes to sizes in memory_info, then, if necessary, reduce
// if crc or timeout errors force too many retries.
   int       maximum_read_size;
   int       maximum_write_size;
// for debugging
   int       count_read;
   int       count_write;
   int       error_write_timeout;
   int       error_erase_timeout;
   };


//extern struct function_memory function_memory_parallel_sram;
//extern struct function_memory function_memory_parallel_rom;

int mem_write(struct memory *memory,int address,int size,unsigned char *data);
int mem_read(struct memory *memory,int address,int size,unsigned char *data);
int mem_read_width(struct memory *mem, int offset, int size, unsigned char *data);
int mem_write_verify(struct memory *memory,int address,int size,unsigned char *data);
int mem_dummy_nw();
int mem_dummy_ns();
int mem_dummy_ok();

extern struct function_execute memory_fexec;
```

12.25 Generic Memory Driver (MEMORY_C)

```
/*
memory driver memory.c
Written by Jude Katsch @ ANON
06/21/99
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/

#include "pport.h"
#include "memory.h"
#include <stdlib.h>
#include <stdio.h>
#include "fifo.h"

extern int quiet;
int free_memory(struct memory *memory);

extern struct function_memory function_memory_amd_scs;
// generic type static ram
struct function_memory function_memory_parallel_sram =
   {
   &mem_dummy_ok,   // reset
   &mem_dummy_ns,   // read status
   NULL,            // read_id
   NULL,            // write
   NULL,            // read
   &mem_dummy_ok,   // write flush
   &mem_dummy_ns,   // erase page
   &mem_dummy_ns,   // erase block
```

```
30    &mem_dummy_ns,   // erase chip
31    &mem_dummy_ns,   // erase suspend
32    &mem_dummy_ns,   // erase resume
33    &mem_dummy_ns,   // read block status
34    &mem_dummy_ns,   // lock block
35    &mem_dummy_ns,   // unlock_block
36    };
37
38 // generic type static rom
39 struct function_memory function_memory_parallel_rom =
40    {
41    &mem_dummy_ok,   // reset
42    &mem_dummy_ns,   // read status
43    NULL,            // read_id
44    &mem_dummy_ns,   // write
45    NULL,            // read
46    &mem_dummy_ok,   // write flush
47    &mem_dummy_ns,   // erase page
48    &mem_dummy_ns,   // erase block
49    &mem_dummy_ns,   // erase chip
50    &mem_dummy_ns,   // erase suspend
51    &mem_dummy_ns,   // erase resume
52    &mem_dummy_ns,   // read block status
53    &mem_dummy_ns,   // lock block
54    &mem_dummy_ns,   // unlock_block
55    };
56
57 extern struct memory_interface polyface_parallel_memory_interface;
58
59 unsigned char mem_buffer[MAX_XFER_SIZE];
60
61 // write - verify
62
63 int mem_erase_block(struct memory *memory,int address)
64    {
65    int retval;
66    if(memory->function&&memory->function->erase_block)
67       retval=(*memory->function->erase_block)(memory,address);
68    else
69       retval=PP_NOT_SUPPORTED;
70    if((retval==PP_OK)||(retval==PP_FLASH_BUSY))
71       while((retval=memory->function->read_status(memory,address,0,0))==PP_FLASH_BUSY);
72    if(retval!=PP_OK)
73       mon_error("mem_erase_block- %s\n",pp_error(retval));
74    return(retval);
75    }
76
77 int mem_write(struct memory *memory,int address,int size,unsigned char *data)
78    {
79    int retval;
80    if(memory->function&&memory->function->write)
81       retval=(*memory->function->write)(memory,address,size,data);
82    else
83       retval=(*memory->iface->write)(memory,address,size,data);
84    if(retval!=PP_OK)
85       mon_error("mem_write- %s\n",pp_error(retval));
86    return(retval);
87    }
88
89 int mem_read_id(struct memory *memory)
90    {
91    int retval;
92    if(memory->function&&memory->function->read_id)
93       retval=(*memory->function->read_id)(memory);
94    else
95       retval=PP_NOT_SUPPORTED;
```

```
96    if(retval!=PP_OK)
97       mon_error("mem_read_id- %s\n",pp_error(retval));
98    return(retval);
99    }
100
101 int mem_read(struct memory *memory,int address,int size,unsigned char *data)
102    {
103    int retval;
104    if(memory->function&&memory->function->write)
105       retval=(*memory->function->read)(memory,address,size,data);
106    else
107       retval=(*memory->iface->read)(memory,address,size,data);
108    if(retval!=PP_OK)
109       mon_error("mem_read- %s\n",pp_error(retval));
110    return(retval);
111    }
112
113 // this is a special read route that takes width into account and tries to make it efficient
114 int mem_read_width(struct memory *memory, int offset, int size, unsigned char *data)
115    {
116    int i,j=0;
117    int xfer_size;
118    int retval;
119    int width=(memory->info->flags&MEMORY_INFO_DEVICE_WIDTH_MASK)>>MEMORY_INFO_DEVICE_WIDTH;
120    size<<=width;
121    offset<<=width;
122
123    while(size>0)
124       {
125       xfer_size=(size>MAX_XFER_SIZE)?MAX_XFER_SIZE:size;
126       if((retval=memory->iface->read(memory,offset,xfer_size,mem_buffer))!=PP_OK) return(retval);
127       for(i=0;i<xfer_size;i+=(1<<width))
128          data[j++]=mem_buffer[i];
129       size-=xfer_size;
130       offset+=xfer_size;
131       }
132    return(PP_OK);
133    }
134
135 int mem_write_verify(struct memory *memory,int address,int size,unsigned char *data)
136    {
137    int retval;
138    int i,j=0;
139    int xfer_size;
140    int xfer_address;
141
142    if(!quiet) printf(__FILE__ ": in write_verify\n");
143
144    if((retval=mem_write(memory,address,size,data))!=PP_OK)  return(retval);
145    xfer_address = address;
146    while(size>0)
147       {
148       xfer_size = MIN(size,MAX_XFER_SIZE);
149       if((retval=mem_read(memory,xfer_address,xfer_size,mem_buffer))!=PP_OK) return(retval);
150       for(i=0;i<xfer_size;i++)
151          if(data[j++] != mem_buffer[i]) return(PP_BAD_DATA);
152       xfer_address+=xfer_size;
153       size-=xfer_size;
154       }
155    return(PP_OK);
156    }
157
158 int memory_write_enable(struct memory *memory)
159    {
160    if(!quiet) printf(__FILE__ ": in memory_sram_write_enable\n");
161    memory->flags |= MEMORY_FLAGS_WRITE_ENABLED;
```

```
   return(PP_OK);
   }

int memory_write_disable(struct memory *memory)
   {
   if(!quiet) printf(__FILE__ ": in memory_sram_write_disable\n");
   memory->flags &= ~MEMORY_FLAGS_WRITE_ENABLED;
   return(PP_OK);
   }

// do nothing but return success
int mem_dummy_ok()
   {
   return(PP_OK);
   }

// do nothing but return failure
int mem_dummy_nw()
   {
   return(PP_NOT_WRITEABLE);
   }

// do nothing but return failure
int mem_dummy_ns()
   {
   return(PP_NOT_SUPPORTED);
   }

unsigned char test_ptr[]={0x00,0xff,0x00,0xff,0xaa,0xaa,0xaa,0xaa,
        0x55,0x55,0x55,0x55,0x66,0xdd,0x66,0xdd,0x81,0x42,0x24,0x18};

int memory_parallel_configure(struct device *device,int base_address)
   {
   struct memory **mptrptr;
   struct memory *memory;
   int retval;
   struct memory_info mem_info;
   int port,id;

   if(!quiet) printf(__FILE__ ": in polyface_memory_configure\n");

// first we need to initilize a memory structure at the end of the linked list
   // scan end of linked list
   for(mptrptr=&(device->memory);(*mptrptr);mptrptr=&((*mptrptr)->next));

   if(((*mptrptr)=(struct memory *)calloc(sizeof(struct memory),1))==NULL)
      {
      mon_error("allocating memory for memory structure.\n");
      retval=PP_INTERNAL;
      goto memory_parallel_configure_error;
      }

   memory = *mptrptr; // to make things just a tad clearer

   mon_write("MEM:%d:%d:%d:ADD\n",port,id,memory);

// setup defaults
   memory->device = device;
   memory->next = NULL;
   memory->function=&function_memory_parallel_sram;

// allocate memory_info
   if((memory->info=(struct memory_info *)calloc(sizeof(struct memory_info),1))==NULL)
      {
      mon_error("allocating memory for memory_info structure.\n");
      retval=PP_INTERNAL;
```

```
      goto memory_parallel_configure_error;
      }

   memory->iface=&polyface_parallel_memory_interface;
   memory->flags=MEMORY_FLAGS_CONFIGURE|MEMORY_FLAGS_WRITE_ENABLED;

//???? need to get interrupts to work!
//   memory->maximum_read_size=MAX_XFER_SIZE;  // limited by bits in command
   memory->maximum_write_size=MAX_XFER_SIZE; // limited by bits in command
   memory->maximum_read_size=16;

// now we need to figure out what type of memory is attached
/// first we will fill in a fake 64k byte memory memory_info so we can do some reads/writes
   memory->info->size=1<<16;
   memory->function=&function_memory_parallel_sram;

// read memory_info structure from base of memory

if((retval=memory->iface->read(memory,0,sizeof(struct memory_info),(unsigned char *)&mem_info))!=PP_OK)
      goto memory_parallel_configure_error;

// DANGER: data structure in memory is stored in little-endian (intel) mode
// this must be modified for big-endian systems!!!!!!

// Any part may have a memory_info block at the base of memory but it is not required
// for known flash parts and may be missing on unconfigured sram parts.
// THIS WILL DETECT ALL ROMS + ALL PARTS WITH A INFO BLOCK
// Is the memory_info structure there and valid?
   if((mem_info.magic==MEMORY_INFO_MAGIC) &&
      (crc_block((unsigned char *)&mem_info,(sizeof(struct memory_info)-4))==mem_info.crc))
      {
      if(!quiet)
         printf(__FILE__ "we have a CRC match for an embeded memory_info structure\n");
      (*memory->info)=mem_info;
      if(memory->info->next_base_address==base_address) // we've wrapped
         goto no_more_memory;
      goto we_have_info;
      }

// if we get here then there is no memory_info block at the base of memory. Let's see if
// it is a known flash part

// THIS WILL DETECT ALL FLASH PARTS WITH A CFI
// read cfi - We can get all the information we need about the device if it supports cfi

   memory->info->flags=MEMORY_INFO_DEVICE_WIDTH_8;
   retval=read_flash_cfi(memory);
   if(retval==PP_OK)
      goto we_have_info;
   if(retval!=PP_BAD_DATA)
      goto memory_parallel_configure_error;

   memory->info->flags=MEMORY_INFO_DEVICE_WIDTH_16;
   retval=read_flash_cfi(memory);
   if(retval==PP_OK)
      goto we_have_info;
   if(retval!=PP_BAD_DATA)
      goto memory_parallel_configure_error;


   memory->info->flags=MEMORY_INFO_DEVICE_WIDTH_32;
   retval=read_flash_cfi(memory);
   if(retval==PP_OK)
      goto we_have_info;
   if(retval!=PP_BAD_DATA)
      goto memory_parallel_configure_error;
```

```
293
294    goto no_more_memory;
295
296 // THIS WILL DETECT FLASH PARTS WITH MFG/DEVICE REGISTERS (we may not have info in lut)
297 // read jedec information (we can have configuration tables for known devices)
298
299 // LETS SEE IF IT IS AN UNCONFIGURED RAM
300 // it doesn't look like a flash part, lets see if it is an unconfigrued RAM
301 // if rams are continuous in memory then they look like one big ram
302 //   memory->info=mem_info;
303 //   memory->info->blocks_per_page=1;
304 //   memory->info->flags=MEMORY_INFO_TYPE_RAM;
305 //   memory->function->write_enable(memory);
306 //   memory->info->magic=MEMORY_INFO_MAGIC;
307 //   memory->info->maximum_read_size=MAX_XFER_SIZE;
308 //   memory->info->maximum_write_size=MAX_XFER_SIZE;
309 //   memory->info->fence_address=0;
310 //   memory->info->page_count=(1<<23)/memory->info-
    >page_size;  // maximum size ??? should be limited by max address
311 //
312 //// NOTE: this test is destructive!
313 //   for(i=0;i<24;i++)
314 //      {
315 //      address = memory->info->page_size*(1<<(i-1));
316 //      test_ptr[0]=i;
317 //      if((retval=ram_write_verify(memory,address,sizeof(test_ptr),test_ptr))!=PP_OK)
318 //         {
319 //         if(retval==PP_BAD_DATA)
320 //            break; // past end of sram
321 //         if(retval!=PP_OK)
322 //            return(retval); // error in xfer
323 //         }
324 //      if((retval=ram_read(memory,0,1,test_ptr))!=PP_OK)
325 //         return(retval); // error in xfer
326 //      if(test_ptr[0]!=0)  break; // wrap to zero
327 //      }
328 //
329 //   memory->info->page_count=(address>>1)/memory->info->page_size;
330 //
331 // the memory size will be (2^(page_count>>1))*page_size
332 //   if(memory->info->page_count==0)
333 //      {    // there is no ram
334 //
335 //// write out the memory_info structure for the next time thru (if it's sram maybe its bat-
    tery backed up)
336 //// check to see if there is any memory beyond this one
337 //// we need to be carefull of parts that wrap!
338 //   memory->info->next_base_address = base_address + (memory->info->page_size*memory->info->page_count);
339 //   memory->info->crc=crc_block((unsigned char *)memory->info,(sizeof(struct memory_info)-4));
340 //   if((retval=ram_write_verify(memory,0,sizeof(struct memory_info),(unsigned char *)memory-
    >info))!=PP_OK) return(retval);
341 //   memory->function->write_disable(memory);
342 //// put fence on page address
343 //   memory->info->fence_address=((sizeof(struct memory_info)+memory->info->page_size-1)/memory->info-
    >page_size)*memory->info->page_size;
344 //
345 we_have_info:
346    switch(memory->info->flags & MEMORY_INFO_TYPE_MASK)
347       {
348       case MEMORY_INFO_TYPE_ROM:
349          memory->function = &function_memory_parallel_rom;
350          break;
351       case MEMORY_INFO_TYPE_RAM:
352          memory->function = &function_memory_parallel_sram;
353          break;
354       case MEMORY_INFO_TYPE_FLASH:
```

```
355          if((memory->info->flags&MEMORY_INFO_CFI)&&memory->info->primary.idcode==2)
356    {
357            memory->function = &function_memory_amd_scs;
358            if((retval=mem_read_id(memory))!=PP_OK)
359               goto memory_parallel_configure_error;
360 // lets see if the flash part works
361            if((retval=mem_erase_block(memory,0))!=PP_OK)
362               goto memory_parallel_configure_error;
363            if((retval=mem_write_verify(memory,0,sizeof(test_ptr),test_ptr))!=PP_OK)
364               goto memory_parallel_configure_error;
365            break;
366            }
367       default:
368          mon_error("memory type not supported\n");
369          retval=PP_DRIVER_UNAVAILABLE;
370          goto memory_parallel_configure_error;
371       }
372
373    port = memory->device->pport->port;
374    id   = memory->device->id;
375    mon_write("MEM:%d:%d:%d:INFO:MEMORY_FLAGS:0x%08x\n",port,id,memory,memory->flags);
376    mon_write("MEM:%d:%d:%d:INFO:MEMORY_STROBE:0x%08x\n",port,id,memory,memory->strobe);
377    mon_write("MEM:%d:%d:%d:INFO:MEMORY_BASEADDRESS:0x%08x\n",port,id,memory,memory->base_address);
378    mon_write("MEM:%d:%d:%d:INFO:INFO_FLAGS:0x%08x\n",port,id,memory,memory->info->flags);
379    if(memory->info->flags&MEMORY_INFO_ID)
380       {
381       mon_write("MEM:%d:%d:%d:INFO:INFO_TYPE:ID\n",port,id,memory);
382       mon_write("MEM:%d:%d:%d:INFO:INFO_MFG_CODE:0x%08x\n",port,id,memory,memory->info->mfg_code);
383       mon_write("MEM:%d:%d:%d:INFO:INFO_DEVICE_CODE:0x%08x\n",port,id,memory,memory->info->device_code);
384       }
385    if(memory->info->flags&MEMORY_INFO_CFI)
386       {
387       mon_write("MEM:%d:%d:%d:INFO:INFO_TYPE:CFI\n",port,id,memory);
388
    mon_write("MEM:%d:%d:%d:INFO:INFO_PRIMARY_ID_CODE:0x%08x\n",port,id,memory,memory->info->primary.idcode);
389
    mon_write("MEM:%d:%d:%d:INFO:INFO_PRIMARY_ADDRESS:0x%08x\n",port,id,memory,memory->info->primary.address);
390       if(memory->info->alternate.idcode)
391          {
392          mon_write("MEM:%d:%d:%d:INFO:INFO_ALTERNATE_ID_CODE:0x%08x\n",port,id,memory,memory->info-
    >alternate.idcode);
393          mon_write("MEM:%d:%d:%d:INFO:INFO_ALTERNATE_ADDRESS:0x%08x\n",port,id,memory,memory->info-
    >alternate.address);
394          }
395       }
396
397    mon_write("MEM:%d:%d:%d:INFO:INFO_SIZE:0x%08x\n",port,id,memory,memory->info->size);
398    {
399    int ba=0;
400    int i,j,k=0;
401    for(i=0;i<memory->info->regions;i++)
402       for(j=0;j<memory->info->region[i].blocks;j++,ba+=memory->info->region[i].page_size)
403          mon_write("MEM:%d:%d:%d:INFO:INFO_BLOCK_%02d:% 6dbytes @
    0x%08x\n",port,id,memory,k++,memory->info->region[i].page_size,ba);
404    }
405
406    return(PP_OK);
407
408 no_more_memory:
409    retval=PP_OK;
410 memory_parallel_configure_error:
411    free_memory(memory);
412    (*mptrptr)=NULL;
413    return(retval);
414    }
415
```

```
416 int free_memory(struct memory *memory)
417    {
418    if(!memory) return(PP_OK);
419    if(!quiet) printf(__FILE__ ": in free_memory\n");
420    mon_write("MEM:%d:%d:%d:DEL\n",memory->device->pport->port,memory->device->id,memory);
421    if(memory->info) free(memory->info);
422    free(memory);
423    return(PP_OK);
424    }
425
426 // this is called to release a device that has been unplugged
427 int memory_release(struct device *device)
428    {
429    struct memory *memory,*next;
430    if(!quiet) printf(__FILE__ ": in memory_release\n");
431    for(memory=device->memory;memory;memory=next)
432       {
433       next=memory->next;
434       free_memory(memory);
435       }
436    device->memory=NULL;
437    return(PP_OK);
438    }
439
440
441 int memory_configure(struct device *device)
442    {
443    return(memory_parallel_configure(device,0));
444    };
445
446 struct function_execute memory_fexec =
447    {
448    NULL, // ERROR
449    NULL, // REQUEST
450    &memory_release, // RELEASE
451    NULL, // EXECUTE
452    &memory_configure, // CONFIGURE
453    };
```

12.26 Generic FLASH CFI interface (FLASH_CFI_C)

```
 1 /*
 2 memory driver flash_cfi.c
 3 Written by Jude Katsch @ ANON
 4 06/21/99
 5 Confidentail and Proprietary
 6 Copyright (c) 1999, ANON
 7 */
 8
 9 #include "memory.h"
10 #include "pport.h"
11 #include <stdlib.h>
12 #include <stdio.h>
13
14 extern int quiet;
15
16 // this code should be implementation independent (yea right!)
17
18 #define CFI_ADDRESS 0x55
19 #define CFI_DATA    0x98
20
21 #define CFI_BYTE(offset)    (cfi_data[(offset)])
22 #define CFI_WORD(offset)    ((CFI_BYTE(offset+1)<<8)|(CFI_BYTE(offset)))
```

```
23 #define CFI_VOLT(offset)    (CFI_BYTE(offset))
24 #define CFI_SIZE(offset)    (1<<CFI_BYTE(offset))
25 #define CFI_TYPU(offset)    (CFI_SIZE(offset)&~1)
26 #define CFI_TYPM(offset)    (CFI_TYPU(offset)*1000)
27 #define CFI_MAX(offset,typ) (CFI_SIZE(offset)*(typ))
28 #define CFI_BCOUNT(offset)  (CFI_WORD(offset)+1)
29 #define CFI_BSIZE(offset)   ((CFI_WORD(offset))?(CFI_WORD(offset)*256):128)
30
31 #define CFI_BASE 0x10
32
33 #define CFI_Q            (0x10-CFI_BASE)
34 #define CFI_R            (0x11-CFI_BASE)
35 #define CFI_Y            (0x12-CFI_BASE)
36 #define CFI_PRIMARY_IDCODE       (0x13-CFI_BASE)
37 #define CFI_PRIMARY_TABLE        (0x15-CFI_BASE)
38 #define CFI_ALTERNATE_IDCODE     (0x17-CFI_BASE)
39 #define CFI_ALTERNATE_TABLE      (0x19-CFI_BASE)
40 #define CFI_TYP_WRITE_BYTE       (0x1f-CFI_BASE)
41 #define CFI_TYP_WRITE_BUFFER     (0x20-CFI_BASE)
42 #define CFI_TYP_ERASE_BLOCK      (0x21-CFI_BASE)
43 #define CFI_TYP_ERASE_CHIP       (0x22-CFI_BASE)
44 #define CFI_MAX_WRITE_BYTE       (0x23-CFI_BASE)
45 #define CFI_MAX_WRITE_BUFFER     (0x24-CFI_BASE)
46 #define CFI_MAX_ERASE_BLOCK      (0x25-CFI_BASE)
47 #define CFI_MAX_ERASE_CHIP       (0x26-CFI_BASE)
48 #define CFI_DEVICE_SIZE          (0x27-CFI_BASE)
49 #define CFI_DEVICE_INTERFACE     (0x28-CFI_BASE)
50 #define CFI_DEVICE_WRITE_BUFFER  (0x2a-CFI_BASE)
51 #define CFI_DEVICE_ERASE_REGIONS (0x2c-CFI_BASE)
52 #define CFI_DEVICE_FIRST_ERASE_REGION (0x2d-CFI_BASE)
53
54 #define CFI_TOP  CFI_DEVICE_ERASE_REGIONS+(MEMORY_INFO_MAX_REGIONS*4)
55 #define CFI_XFER_SIZE (CFI_TOP-CFI_BASE+1)
56
57
58 // the cfi information is taken from "Common Flash Memory Interface Specification" Re-
   lease 1.1 May 30, 1997
59 // see Intel, AMD or Fujitsu web sites for a copy of the spec
60
61 // DANGER! this can be distruc-
   tive it write CFI_DATA to the CFI_ADDRESS!!!!!!! Make sure it is not RAM or other
62 // writeable media before calling!
63
64 unsigned char cfi_data[CFI_XFER_SIZE];
65
66 int read_flash_cfi(struct memory *memory)
67    {
68    int retval,i;
69    struct memory_info *info = memory->info;
70    int width=(memory->info->flags&MEMORY_INFO_DEVICE_WIDTH_MASK)>>MEMORY_INFO_DEVICE_WIDTH;
71
72    if(!quiet) printf(__FILE__ ": in read_flash_cfi\n");
73
74 // put the device in cfi query mode
75    cfi_data[0]=CFI_DATA;
76    if((retval=memory->iface->write(memory,CFI_ADDRESS<<width,1,cfi_data))!=PP_OK) return(retval);
77
78 // read cfi data
79    if((retval=mem_read_width(memory,CFI_BASE,CFI_XFER_SIZE,cfi_data))!=PP_OK) return(retval);
80
81    if((cfi_data[CFI_Q]!='Q')||(cfi_data[CFI_R]!='R')||(cfi_data[CFI_Y]!='Y')) return(PP_BAD_DATA);
82
83 // at this point it looks like we have valid cfi data so lets fill the structure
84
85    info->flags                |= MEMORY_INFO_CFI|MEMORY_INFO_NONVOLITLE|MEMORY_INFO_TYPE_FLASH;
86    info->magic                 = MEMORY_INFO_MAGIC;
```

```
   info->primary.idcode        = CFI_WORD(CFI_PRIMARY_IDCODE);
   info->primary.address       = CFI_WORD(CFI_PRIMARY_TABLE);
   info->alternate.idcode      = CFI_WORD(CFI_ALTERNATE_IDCODE);
   info->alternate.address     = CFI_WORD(CFI_ALTERNATE_TABLE);
   info->to_write_byte         = CFI_MAX(CFI_MAX_WRITE_BYTE,  CFI_TYPU(CFI_TYP_WRITE_BYTE));
   info->to_write_buffer       = CFI_MAX(CFI_MAX_WRITE_BUFFER,CFI_TYPU(CFI_TYP_WRITE_BUFFER));
   info->to_erase_block        = CFI_MAX(CFI_MAX_ERASE_BLOCK, CFI_TYPU(CFI_TYP_ERASE_BLOCK));
   info->to_erase_chip         = CFI_MAX(CFI_MAX_ERASE_CHIP,  CFI_TYPU(CFI_TYP_ERASE_CHIP));
   info->size                  = CFI_SIZE(CFI_DEVICE_SIZE);
   info->interface             = CFI_WORD(CFI_DEVICE_INTERFACE);
   info->maximum_write_size    = CFI_WORD(CFI_DEVICE_WRITE_BUFFER);
   info->maximum_read_size     = MAX_XFER_SIZE;
   info->regions               = CFI_BYTE(CFI_DEVICE_ERASE_REGIONS);

   if(info->regions<=MEMORY_INFO_MAX_REGIONS)
      {
      if(info->regions==0) // denotes entire device
         {
         info->regions=1;
         info->region[0].blocks            = 1;
         info->region[0].pages_per_block   = 1;
         info->region[0].page_size         = info->size;
         }
      else
         {
         for(i=0;i<info->regions;i++)
    {
            info->region[i].blocks            = CFI_BCOUNT(CFI_DEVICE_FIRST_ERASE_REGION+(i*4));
            info->region[i].pages_per_block   = 1;
            info->region[i].page_size         = CFI_BSIZE(CFI_DEVICE_FIRST_ERASE_REGION+(i*4)+2);
            }
         }
      }
   else
      return(PP_BAD_DATA);
   return(PP_OK);
   }
```

12.27 AMD FLASH interface (FLASH_AMD_SCS_C)

```
/*
polyface flash amd standard command set (flash_amd_scs)
Written by Jude Katsch @ ANON
8/9/1999
Confidentail and Proprietary
Copyright (c) 1999, ANON
*/

#include "pport.h"
#include "memory.h"
#include "polyface.h"
#include "fifo.h"
#include <stdio.h>
#include <stdlib.h>

extern int quiet;
int amd_scs_reset(struct memory *memory)
   {
   unsigned char data=0xf0;
   if(!quiet) printf(__FILE__ ": in amd_scs_reset\n");
   return(memory->iface->write(memory,0,1,&data));
   };

```

```
// check a write: amd_scs_read_status(memory,address,1,data);
// check if busy: amd_scs_read_status(memory,address,0,0);

int amd_scs_read_status(struct memory *memory,int address,int write_mode,unsigned char data)
   {
   int retval;
   unsigned char sdata[2],bdata,tdata;
   if(!quiet) printf(__FILE__ ": in amd_scs_read_status\n");
   if((retval=memory->iface->read_hold(memory,address,2,sdata))!=PP_OK) return(retval);
   tdata=sdata[0]^sdata[1];
   if(write_mode)
      {
      if(((data^sdata[1])&(1<<7))==0)
         {
         if(data==sdata[1])
            return(PP_OK);
         else
            return(PP_INTERNAL); // should get timeout never bad data!
         }
      if(tdata&(1<<6))
        {
        if(sdata[0]&(1<<5))
           {
           amd_scs_reset(memory);
           memory->error_write_timeout++;
           return(PP_TIMEOUT);
           }
        }
      else
        return(PP_INTERNAL); // should get timeout never bad data!
      return(PP_FLASH_BUSY);
      }
   if((retval=pp_data_read(memory->device->pport->port,memory->device->id,ADD_REQUEST,1,CMD_CRC,&bdata))!=PP_OK) return(retval);
   if((bdata&ADD_REQUEST_BUSY)==1) // is not busy
      {
      if((retval=memory->iface->read_hold(memory,address,2,sdata))!=PP_OK) return(retval);
      tdata=sdata[0]^sdata[1];

if((tdata&(1<<6))==0) return(PP_OK); // ok to read/program this address (another block may be suspended)
      return(PP_FLASH_SUSPEND); // the block is suspended do not read or program
      }
   if(sdata[0]&(1<<5))
      {
      amd_scs_reset(memory);
      memory->error_erase_timeout++;
      return(PP_FLASH_TIMEOUT);
      }
   if((tdata&((1<<6)|(1<<2)))==((1<<6)|(1<<2))) // dq6 and dq2 toggle
      return(PP_FLASH_BUSY);
   return(PP_INTERNAL); // we should never get here
   };

int amd_scs_read_id(struct memory *memory)
   {
   unsigned char data[3];
   int retval,retval2;
   if(!quiet) printf(__FILE__ ": in amd_scs_read_id\n");
   if((retval=amd_scs_read_status(memory,0,0,0))!=PP_OK) return(retval);
   if((retval=memory->iface->write_unlock(memory,0x90))==PP_OK)
     {
     if((retval=memory->iface->read(memory,0x0,0x3,data))==PP_OK)
       {
       if(data[0]==0x01)
          {
          memory->info->mfg_code=(int)data[0];
```

```
          memory->info->device_code=(int)data[2];
          memory->info->flags|=MEMORY_INFO_ID;
          retval=PP_OK;
          }
        else
          retval=PP_BAD_DATA;
       }
     }
   retval2=amd_scs_reset(memory);
   if(retval==PP_OK) retval=retval2;
   return(retval);
   };

int amd_scs_write_one(struct memory *memory,int address,unsigned char data)
   {
   int retval;
   if(!quiet) printf(__FILE__ ": in amd_scs_writet_one\n");
   if((retval=amd_scs_read_status(memory,address,0,0))!=PP_OK) return(retval);
   if((retval=memory->iface->write_unlock(memory,0xa0))!=PP_OK)
      goto amd_scs_write_one_error;
   if((retval=memory->iface->write_hold(memory,address,1,&data))!=PP_OK)
      goto amd_scs_write_one_error;
   while((retval=amd_scs_read_status(memory,address,1,data))==PP_FLASH_BUSY);
amd_scs_write_one_error:
   return(retval);
   };

int amd_scs_write(struct memory *memory,int address,int size,unsigned char *data)
   {
   int retval,retval1;
   unsigned char wdata[]={0xa0,0x00};
   int initial_size=size;
   if(!quiet) printf(__FILE__ ": in amd_scs_write\n");
   if(size==1) return(amd_scs_write_one(memory,address,*data));
   if((retval=amd_scs_read_status(memory,address,0,0))!=PP_OK) return(retval);
   if((retval=memory->iface->write_unlock(memory,0x20))!=PP_OK) return(retval); // unlock bypass
   while(size>0)
      {
      mon_write("MEM:%d:%d:%d:BUSY:WRITE:%d\n",memory->device->pport->port,memory->device->id,memory,((initial_size-size)*100)/initial_size);
      wdata[1]=(*data);

if((retval=memory->iface->write_hold(memory,address,2,wdata))!=PP_OK) break; // unlock bypass program
      while((retval=amd_scs_read_status(memory,address,1,wdata[1]))==PP_FLASH_BUSY);
      if(retval!=PP_OK) break;
      data++;
      address++;
      size--;
      }
   mon_write("MEM:%d:%d:%d:BUSY:WRITE:DONE\n",memory->device->pport->port,memory->device->id,memory);
   // this may not work if we got errors. How do we reover?
   wdata[0]=0x90; // unlock bypass reset
   wdata[1]=0x00;
   retval1=memory->iface->write_hold(memory,0,2,wdata);
   if(retval==PP_OK) retval=retval1;
   return(retval);
   };

int amd_scs_read(struct memory *memory,int address,int size,unsigned char *data)
   {
   int retval;
   if(!quiet) printf(__FILE__ ": in amd_scs_read\n");
// DANGER! this is only valid within block

if((retval=amd_scs_read_status(memory,address,0,0))!=PP_OK) return(retval); // make sure were not busy!
   return(retval=memory->iface->read(memory,address,size,data));
```

```
   };

int amd_scs_write_flush(struct memory *memory)
   {
// there is no buffer so there is no write flush!
   if(!quiet) printf(__FILE__ ": in amd_scs_write_flush\n");
   return(PP_OK);
   };

int amd_scs_erase_page(struct memory *memory,int address)
   {
   if(!quiet) printf(__FILE__ ": in amd_scs_erase_page\n");
   return(PP_NOT_SUPPORTED);
   };

int amd_scs_erase_block(struct memory *memory,int address)
   {
   unsigned char sdata[2],tdata;
   int retval;
   if(!quiet) printf(__FILE__ ": in amd_scs_erase_block\n");
   if((retval=memory->iface->read_hold(memory,address,2,sdata))!=PP_OK) return(retval);
   tdata=sdata[0]^sdata[1];
   if(((tdata&((1<<6)|(1<<2)))==((1<<6)|(1<<2)))&&((sdata[0]&(1<<3))==1)) // dq6 and dq2 toggle and dq3=1
      return(PP_FLASH_BUSY);
   if((retval=memory->iface->write_unlock(memory,0x80))!=PP_OK) return(retval);
   if((retval=memory->iface->write_unlock_add(memory,address,0x30))!=PP_OK) return(retval);
   if((retval=memory->iface->read_hold(memory,address,2,sdata))!=PP_OK) return(retval);
   tdata=sdata[0]^sdata[1];
   if((tdata&((1<<6)|(1<<2)))==((1<<6)|(1<<2))) // dq6 and dq2 toggle
      {
      if((sdata[0]&(1<<3))==0) //and dq3=0
         return(PP_OK);
      else
         return(PP_FLASH_BUSY);
      }
   return(PP_OK);
   };

int amd_scs_erase_chip(struct memory *memory)
   {
   int retval;
   unsigned char bdata;
   if(!quiet) printf(__FILE__ ": in amd_scs_erase_chip\n");
   if((retval=pp_data_read(memory->device->pport->port,memory->device-
>id,ADD_REQUEST,1,CMD_CRC,&bdata))!=PP_OK) return(retval);
   if((bdata&ADD_REQUEST_BUSY)!=0) return(PP_FLASH_BUSY);
   if((retval=memory->iface->write_unlock(memory,0x80))!=PP_OK) return(retval);
   if((retval=memory->iface->write_unlock(memory,0x10))!=PP_OK) return(retval);
   return(PP_OK);
   };

int amd_scs_erase_suspend(struct memory *memory)
   {
   if(!quiet) printf(__FILE__ ": in amd_scs_erase_suspend\n");
   return(PP_NOT_SUPPORTED);
   };

int amd_scs_erase_resume(struct memory *memory)
   {
   if(!quiet) printf(__FILE__ ": in amd_scs_erase_resume\n");
   return(PP_NOT_SUPPORTED);
   };

int amd_scs_read_block_status(struct memory *memory,int address)
   {
   if(!quiet) printf(__FILE__ ": in amd_scs_read_block_status\n");
```

```
   return(PP_NOT_SUPPORTED);
   };

int amd_scs_lock_block(struct memory *memory,int address)
   {
   if(!quiet) printf(__FILE__ ": in amd_scs_lock_block\n");
   return(PP_NOT_SUPPORTED);
   };

int amd_scs_unlock_block(struct memory *memory,int address)
   {
   if(!quiet) printf(__FILE__ ": in amd_scs_unlock_block\n");
   return(PP_NOT_SUPPORTED);
   };

struct function_memory function_memory_amd_scs=
   {
   &amd_scs_reset,
   &amd_scs_read_status,
   &amd_scs_read_id,
   &amd_scs_write,
   &amd_scs_read,
   &amd_scs_write_flush,
   &amd_scs_erase_page,
   &amd_scs_erase_block,
   &amd_scs_erase_chip,
   &amd_scs_erase_suspend,
   &amd_scs_erase_resume,
   &amd_scs_read_block_status,
   &amd_scs_lock_block,
   &amd_scs_unlock_block,
   };
```

12.28 PPTRACE GUI (PPTRACE_TCL)

```
#!/bin/sh
# the next line restarts using wish \
exec wish "$0" "$@"

set monfd ""
set monname "pptrace.mon" ; # in to pptest
set cmdfd ""
set cmdname "pptrace.cmd"  ; # out of pptest

wm title . "PPTRACE"
wm iconname . "PPTRACE"
wm minsize . 1 1

set cmdfd [open $cmdname w ]

set lucmd(0x10) "MDATA"
set lucmd(0x11) "MDATA_INC"
set lucmd(0x12) "PROG"
set lucmd(0x13) "PROG_ADD"
set lucmd(0x14) "STB A MDATA"
set lucmd(0x15) "STB A MDATA_INC"
set lucmd(0x16) "STB A PROG"
set lucmd(0x17) "STB A PROG_ADD"
set lucmd(0x18) "STB B MDATA"
set lucmd(0x19) "STB B MDATA_INC"
set lucmd(0x1a) "STB B PROG"
set lucmd(0x1b) "STB B PROG_ADD"
set lucmd(0x1c) "STB C MDATA"
```

```
29 set lucmd(0x1d) "STB C MDATA_INC"
30 set lucmd(0x1e) "STB C PROG"
31 set lucmd(0x1f) "STB C PROG_ADD"
32 set lucmd(0x20) {A[7:0]}
33 set lucmd(0x21) {A[15:8]}
34 set lucmd(0x22) {S_CS,A[22:16]}
35 set lucmd(0x23) STROBE
36 set lucmd(0x24) PINOUT
37 set lucmd(0x25) CONFIG
38 set lucmd(0x26) INPUTA
39 set lucmd(0x27) REQUEST
40 set lucmd(0x28) INPUTB
41 set lucmd(0x30) {SWITCH[8:1]}
42 set lucmd(0x31) {SWITCH[16:9]}
43 set lucmd(0x32) QUADX
44 set lucmd(0x33) QUADY
45 set lucmd(0x34) CHANNEL
46 set lucmd(0x35) ANALOG
47 set lucmd(0x40) BAUD
48 set lucmd(0x41) FLAGS0
49 set lucmd(0x42) FLAGS1
50 set lucmd(0x43) SDATA
51 set lucmd(0x44) SSTATUS
52 set lucmd(0x45) RSTATUS
53 set lucmd(0x80) ALIVE
54 set lucmd(0x84) REQUEST
55 set lucmd(0x88) ERROR
56 set lucmd(0x90) MAGIC
57 set lucmd(0x94) PROBE
58 set lucmd(0x98) ERROR
59 set lucmd(0x99) STATE
60 set lucmd(0x9a) CRC
61 set lucmd(0xb0) FOCUS
62 set lucmd(0xb1) RESET
63 set lucmd(0xb2) TAG
64 set lucmd(0xb3) UNBRAND
65 set lucmd(0xb4) BRAND
66
67
68 frame .scroll
69 text .scroll.text -width 80 -height 10 -setgrid true -wrap none -
   xscrollcommand [list .scroll.xscroll set] -yscrollcommand [list .scroll.yscroll set]
70 scrollbar .scroll.xscroll -orient horizontal -command [list .scroll.text xview]
71 scrollbar .scroll.yscroll -orient vertical   -command [list .scroll.text yview]
72 pack .scroll.xscroll -side bottom -fill x
73 pack .scroll.yscroll -side right -fill y
74 pack .scroll.text -side left -fill both -expand true
75
76 .scroll.text tag configure writeTag -foreground blue
77 .scroll.text tag configure readTag -foreground red
78
79 proc addScroll { message tag } {
80    .scroll.text insert end $message\n $tag
81    .scroll.text see end
82    if [.scroll.text compare end > 1000.0 ] {.scroll.text delete 1.0 2.0}
83 }
84
85
86 proc writePipe {cmd} {
87     global cmdfd
88     if {$cmdfd!=""} {
89        addScroll $cmd writeTag
90 #        puts "writePipe $cmd"
91         puts $cmdfd $cmd
92         catch {flush $cmdfd}
93     }
```

```
 94 }
 95
 96 frame .menu -relief raised -bd 2
 97 pack .menu -side top -fill x -expand 0
 98 frame .spacer -width 480 -height 2
 99 pack .spacer -side top
100 menubutton .menu.device -text "Device" -menu .menu.device.m -underline 0
101 menu .menu.device.m -tearoff 0
102 #for {set i 0} {$i<=31} {incr i} {
103 #   .menu.device.m add cascade -label [format "%02d" $i] -state disabled -menu .menu.device.m.$i
104 #}
105
106 frame .menubar
107 checkbutton .menubar.details -text DETAILS -variable showScroll -relief raised -padx 4 -pady 4 -
    command { repack }
108 button .menubar.init -text INIT -command {writePipe REFRAME\nREFRAME\nRESET\nALIVE\nERROR}
109 button .menubar.map -text MAP -command {writePipe MAP\nALIVE\nERROR\nREQUEST}
110 button .menubar.direct -text DIRECT -command { wm deiconify .command }
111 button .menubar.quit -text QUIT -command {writePipe QUIT; exit}
112 pack .menubar.details .menubar.init .menubar.map .menubar.direct .menubar.quit -side left
113
114
115 set running 0
116 set hit 0
117 set miss 0
118 set tps 0
119
120 proc timerStart {} {
121    global running
122    set running 1
123    }
124
125 proc timerEnd {} {
126    global running
127    set running 0
128    }
129
130 set req_running 0
131 proc reqStart {} {
132    global req_running
133    set req_running 1
134    }
135
136 proc reqEnd {} {
137    global req_running
138    set req_running 0
139    }
140
141 proc forceTick {} {
142     global running hit miss
143    if {$running == 0} {
144       incr hit
145 #   set data "abcd"
146 #   writePipe "SERW:0:1:[string length $data]:$data\nEND"
147       writePipe MAP\nALIVE\nERROR\nREQUEST\nTICK\nEND
148       timerStart
149    } else {
150       incr miss
151    }
152 }
153 proc tick {} {
154     global tps
155     if {$tps==0} {
156        after 1000 tick
157        .tickbar.tick config -state normal
158     } else {
```

```
159         forceTick
160         after [expr 1000 / $tps] tick
161         .tickbar.tick config -state disabled
162     }
163 }
164
165 set requestEn 0
166
167 frame .tickbar
168 checkbutton .tickbar.request -text REQUEST\nENABLE -variable requestEn -relief raised  -padx 8 -pady 8
169 button .tickbar.tick -text TICK\nFORCE -command {forceTick} -padx 7 -pady 7
170 frame  .tickbar.tps -relief sunken -bd 2
171 label  .tickbar.tps.label -text TICKS/\nSEC:
172 scale  .tickbar.tps.scale -from 0 -to 60 -length 120 -variable tps -orient horizontal
173 pack   .tickbar.tps.label .tickbar.tps.scale -side left
174 frame  .tickbar.miss -relief sunken -bd 2
175 label  .tickbar.miss.label -text MISS: -width 6
176 label  .tickbar.miss.data  -textvariable miss -width 8 -font fixed
177 pack   .tickbar.miss.label -side top -padx 2 -pady 2
178 pack   .tickbar.miss.data -side top
179 frame  .tickbar.hit -relief sunken -bd 2
180 label  .tickbar.hit.label -text HIT: -width 6
181 label  .tickbar.hit.data  -textvariable hit -width 8 -font fixed
182 pack   .tickbar.hit.label -side top -padx 2 -pady 2
183 pack   .tickbar.hit.data -side top
184 pack   .tickbar.request .tickbar.tick .tickbar.tps .tickbar.miss .tickbar.hit -side left -fill x
185
186 set showError 0
187 frame .error
188 frame .error.bar -bor 1 -rel sunken -height 2
189 checkbutton .error.showError -text ERROR: -command repack
190 label .error.message -textvariable errorMessage  -foreground red
191 pack .error.showError -side left
192 pack .error.message -side left -fill x
193
194 proc repack {} {
195    global showScroll showError
196    pack forget .menu.device
197    pack forget .menubar
198    pack forget .tickbar
199    pack forget .scroll
200    pack forget .pollbar
201    pack forget .error
202
203    pack .menu.device -side left
204    pack .menubar -side top -expand 0
205    pack .pollbar -side top -expand 0
206    pack .tickbar -side top -expand 0
207    if {$showError}  {pack .error.bar .error -side top -expand 0 -fill x}
208    if {$showScroll} {pack .scroll -side top -expand 1 -fill both}
209 }
210
211 proc req {} {
212     global running requestEn req_running
213     update
214     if {$requestEn==1} {after idle req} { after [expr 1000 / (30*8)] req }
215     if { ($running==0) && ($req_running==0) && ($requestEn==1) } { writePipe REQUEST\nREND }
216 }
217
218 proc openMon {} {
219     global  monfd monname
220     catch {exec rm -f $monname}
221     catch {exec mkfifo -m 0600 $monname}
222     writePipe SYNC
223     set monfd [open $monname r]
224     fileevent $monfd readable {traceFetch}
```

```
225 }
226
227 openMon
228 writePipe RESET
229
230 frame .pollbar
231 proc pollbar {name} {
232    set window .pollbar.[string tolower $name]
233    frame  $window
234    button $window.button -text $name -width 8 -command "writePipe $name"
235    label  $window.label  -textvariable poll($name) -width 36 -font fixed
236    pack   $window.button -side left
237    pack   $window.label -side left
238    pack   $window -side top
239    writePipe $name
240 }
241 pollbar ALIVE
242 pollbar ERROR
243 pollbar REQUEST
244
245 .menubar.init invoke
246 .menubar.map  invoke
247
248 proc add_item {item name type} {
249    global ${name}${item}
250 #   puts "item=$item name=$name type=$type"
251    if {[string compare $name "reg"]==0} {set side {}} {set side right.}
252    set window .${name}info.$side$item.[string tolower $type]
253    if { ![winfo exists $window] } {
254       frame $window
255       label $window.label -width 15 -text $type
256       label $window.data  -width 15 -textvariable ${name}${item}($type) -font fixed
257       pack  $window.label -side left
258       pack  $window.data  -side right
259       pack  $window    -side top
260    }
261 }
262
263 proc write1Reg { port id address data } {
264    set cmd 0
265    set cmd [expr ($cmd & ~(0xff<<1 )) | ($data<<1)]
266    set cmd [expr ($cmd & ~(0x7f<<9 )) | (1<<9 )]
267    set cmd [expr ($cmd & ~(0x01<<16)) | (1<<16)]
268    set cmd [expr ($cmd & ~(0xff<<17)) | ($address<<17)]
269    set cmd [expr ($cmd & ~(0x01<<25)) | (0<<25)]
270    set cmd [expr ($cmd & ~(0x01<<26)) | (1<<26)]
271    set cmd [expr ($cmd & ~(0x1f<<27)) | ($id<<27)]
272    set parity [expr ($cmd ^ ($cmd>>16)) & 0xffff]
273    set parity [expr ($parity ^ ($parity>>8)) & 0xff]
274    set parity [expr ($parity ^ ($parity>>4)) & 0x0f]
275    set parity [expr ($parity ^ ($parity>>2)) & 0x03]
276    set parity [expr ($parity ^ ($parity>>1)) & 0x01]
277    set cmd [expr $cmd | $parity]
278    writePipe EXECW1:0:$port:$id:$cmd:
279 }
280
281 proc read1Reg { port id address } {
282    set cmd 0
283    set cmd [expr ($cmd & ~(0xff<<1 )) | (1<<8)]
284    set cmd [expr ($cmd & ~(0x7f<<9 )) | (1<<9 )]
285    set cmd [expr ($cmd & ~(0x01<<16)) | (1<<16)]
286    set cmd [expr ($cmd & ~(0xff<<17)) | ($address<<17)]
287    set cmd [expr ($cmd & ~(0x01<<25)) | (1<<25)]
288    set cmd [expr ($cmd & ~(0x01<<26)) | (0<<26)]
289    set cmd [expr ($cmd & ~(0x1f<<27)) | ($id<<27)]
290    set parity [expr ($cmd ^ ($cmd>>16)) & 0xffff]
```

```
291    set parity [expr ($parity ^ ($parity>>8)) & 0xff]
292    set parity [expr ($parity ^ ($parity>>4)) & 0x0f]
293    set parity [expr ($parity ^ ($parity>>2)) & 0x03]
294    set parity [expr ($parity ^ ($parity>>1)) & 0x01]
295    set cmd [expr $cmd | $parity]
296    writePipe EXECR:0:$port:$id:$cmd:1:
297 }
298
299 proc enable_window { name } {
300    global ${name}:menu
301 #   puts "${name}:menu = [set ${name}:menu]"
302    if { [set ${name}:menu] == 0 } {
303       wm withdraw .$name
304    } else {
305       wm deiconify .$name
306    }
307 }
308
309 proc getInfo {port id} {
310    set name $port:$id:info
311    global ${name}:menu global ${name}
312    if ![winfo exists .$name] {
313       toplevel .$name
314       wm withdraw .$name
315       .menu.device.m add cascade -label "DEV_[format "%02d" $id]" -state normal -menu ".menu.device.m.$port:$id"
316       set m [menu ".menu.device.m.$port:$id" -tearoff 0]
317       $m add check -label INFO -variable "${name}:menu" -command "enable_window $name"
318       set ${name}:menu 0
319       wm title .$name "PPTRACE $port $id INFO"
320       frame .$name.reg -relief sunken -bd 2
321       pack  .$name.reg -side left -padx 2 -pady 2 -fill y
322       if { $id >0} {
323          set name1 [frame .$name.right]
324          frame $name1.info -relief sunken -bd 2
325          pack  $name1.info -side top -padx 2 -pady 2
326          frame $name1.variable -relief sunken -bd 2
327          pack  $name1.variable -side top -padx 2 -pady 2
328          set but [frame $name1.buttons -relief sunken -bd 2]
329          pack  $name1.buttons -side top -padx 2 -pady 2 -fill both
330          pack $name1 -side left  -fill y
331          buttonRead $but CONFIG  0x25 $port $id
332          buttonRead $but INPUTA  0x26 $port $id
333          buttonRead $but INPUTB  0x28 $port $id
334          buttonRead $but REQUEST 0x27 $port $id
335          buttonRead $but ERROR   0x98 $port $id
336          button $name1.quit -text QUIT -command "wm withdraw .$name; set ${name}:menu 0"
337          pack $name1.quit -side bottom -padx 2 -pady 2 -fill both
338       } else {
339          button .$name.quit -text QUIT -command "wm withdraw .$name; set ${name}:menu 0"
340          pack .$name.quit -side bottom -padx 2 -pady 2 -fill both
341       }
342    }
343 }
344
345 proc buttonRead { window name address port id} {
346    set var $port:$id:info
347    set reg $port:$id:reg
348    global $var $reg
349    set num [format "%d" $address]
350    set ${var}(${num},t) $name
351    button $window.$address -textvariable ${var}(${num}) -command "read1Reg $port $id $num"
352    $window.$address invoke
353    trace variable ${reg}(${num},R) uw buttonUpdate
354    pack $window.$address -side top -padx 2 -pady 2 -fill x
355    $window.$address invoke
356 }
```

```
357
358 proc buttonUpdate {name1 name2 op} {
359    set data [split $name1 :]
360    set name [lindex $data 0]:[lindex $data 1]:info
361    global $name $name1
362    set num [lindex [split $name2 ,] 0]
363    if {[string compare $op "u"]==0} { trace vdelete ${name1}(${name2}) uw buttonUpdate; return}
364    set ${name}(${num}) "[set ${name}(${num},t)] : [format "0x%02x" [set ${name1}(${name2})]]"
365 #   puts "[set ${name}(${num},t)] : [format "0x%02x" [set ${name1}(${name2})]]"
366 }
367
368 proc ungetInfo {port id} {
369    set name $port:$id
370    global ${name}info ${name}variable ${name}reg ${name}menu
371    .menu.device.m.$port:$id delete INFO
372    .menu.device.m delete DEV_[format "%02d" $id]*
373    wm withdraw .${name}:info
374    destroy .menu.device.m.$port:$id
375    destroy .${name}:info
376    array set ${name}info {};
377    array set ${name}variable {};
378    array set ${name}reg {};
379 }
380
381 proc traceFetch {} {
382    global monfd poll showError errorMessage data
383    if [eof $monfd] {
384       catch {close $monfd}
385    } else {
386       gets $monfd message
387 #      puts $message
388       addScroll $message readTag
389       set command [split $message :]
390       set cmd0 [lindex $command 0]
391       set cmd1 [lindex $command 1]
392       set cmd2 [lindex $command 2]
393       set cmd3 [lindex $command 3]
394       set cmd4 [lindex $command 4]
395       set cmd5 [lindex $command 5]
396       set name $cmd1:$cmd2:
397       switch $cmd0 {
398          POLL {
399          set poll($cmd1) $cmd2
400          }
401          DEV {
402             global $name
403             switch $cmd3 {
404             ADD {
405                getInfo $cmd1 $cmd2
406                }
407             DEL {
408 #               ungetSerial $cmd1 $cmd2
409                ungetInfo $cmd1 $cmd2
410                }
411             DEVICE {
412 #               set info${name}(DEVICE) $cmd4
413               .menu.device.m entryconfigure "DEV_[format "%02d" $cmd2]*" -label "DEV_[format "%02d" $cmd2] $cmd4"
414               }
415             }
416          }
417          JOYSTICK {
418             switch $cmd3 {
419 #               ADD { getSerial $cmd1 $cmd2 }
420 #               DEL { ungetSerial $cmd1 $cmd2 }
421              }
422            }
```

```
423         SERIAL {
424            switch $cmd3 {
425               ADD { getSerial $cmd1 $cmd2 }
426               DEL { ungetSerial $cmd1 $cmd2 }
427               DATA { recvData $cmd1 $cmd2 $cmd4 $cmd5 }
428             }
429           }
430         MEM {
431            switch $cmd3 {
432               ADD { getMemory $cmd1 $cmd2 $cmd3}
433               DEL { ungetMemory $cmd1 $cmd2 $cmd3}
434             }
435           }
436         INFO {
437            global ${name}info
438            set ${name}info($cmd3) $cmd4
439            add_item info $name $cmd3
440            }
441         END {
442            timerEnd
443            }
444         REND {
445            reqEnd
446            }
447         ERROR {
448            set showError 1
449            set errorMessage $cmd1
450            repack
451            }
452         EXEC {
453            addExecute execReadTag $cmd1
454            }
455         DATA {
456               getInfo $cmd1 $cmd2
457               global ${name}info lucmd
458               set var "$cmd4,[string index $cmd3 0]"
459               global ${name}reg
460               set ${name}reg($var) [format "0x%02x" $cmd5]
461 #              puts "variable name=${name}reg($var)"
462               set window .${name}info.reg.$var
463               set order {}
464               if {![winfo exists $window] } {
465                  foreach index [pack slaves .${name}info.reg] {
466                     set data [split $index ,:.]
467                     if { [lindex $data 5] > $cmd4 } {set order {-before $index}; break }
468                     if { [lindex $data 5] == $cmd4 } {
469                        if { [string compare [lindex $data 6] R] == 0 } {
470                           set order {-after $index}
471                           break
472                        } else {
473                           set order {-before $index}
474                           break
475                        }
476                     }
477                  }
478                  frame $window
479                  if [info exists lucmd([format "0x%02x" $cmd4])] {set luname $lucmd([format
    "0x%02x" $cmd4])} {set luname ""}
480                  label $window.label -width 22 -text "[format "0x%02x" $cmd4] [string index $cmd3 0]
    $luname" -anchor w -font fixed
481                  label $window.data  -textvariable ${name}reg($var) -font fixed
482                  pack  $window.label -side left
483                  pack  $window.data  -side right
484                  eval pack  $window  $order -side top -fill x
485               }
486            }
```

```
         VARIABLE {
            global ${name}variable
            set ${name}variable($cmd3) $cmd4
            add_item variable $name $cmd3
            }
         }
      }
   }

repack

set executeEn 1
source direct
source serial
source memory

tick
req
writePipe TICK
```

12.29 DIRECT GUI (enter commands directly to polyface) (DIRECT_TCL)

```
#!/bin/sh
# the next line restarts using wish \
exec wish "$0" "$@"

if ![info exists executeEn] {wm withdraw .}

#set abank(0) CONFIG
set abank(1) MEMORY
set abank(2) ADDRESS
set abank(3) JOYSTICK
set abank(4) SERIAL
set abank(8) POLL
set abank(9) REGISTER
set abank(11) EXEC

set acmd(1,0)  { RW " DATA" }
set acmd(1,1)  { RW " DATA_INC" }
set acmd(1,2)  {  W " PROG" }
set acmd(1,3)  {  W " PROG_ADD" }
set acmd(1,4)  { RW "STB A DATA" }
set acmd(1,5)  { RW "STB A DATA_INC" }
```

```
set acmd(1,6)  {  W "STB A PROG" }
set acmd(1,7)  {  W "STB A PROG_ADD" }
set acmd(1,8)  { RW "STB B DATA" }
set acmd(1,9)  { RW "STB B DATA_INC" }
set acmd(1,10) {  W "STB B PROG" }
set acmd(1,11) {  W "STB B PROG_ADD" }
set acmd(1,12) { RW "STB C DATA" }
set acmd(1,13) { RW "STB C DATA_INC" }
set acmd(1,14) {  W "STB C PROG" }
set acmd(1,15) {  W "STB C PROG_ADD" }
set acmd(2,0)  {  W A[7:0] }
set acmd(2,1)  {  W A[15:8] }
set acmd(2,2)  {  W STICKY_CS,A[22:16] }
set acmd(2,3)  {  W STROBE  }
set acmd(2,4)  {  W PINOUT  }
set acmd(2,5)  { R  CONFIG }
set acmd(2,6)  { R  INPUTA }
set acmd(2,7)  { R  REQUEST }
set acmd(2,8)  { R  INPUTB }
set acmd(3,0)  { R  SWITCH[8:1] }
set acmd(3,1)  { R  SWITCH[16:9] }
set acmd(3,2)  { R  QUADX }
set acmd(3,3)  { R  QUADY }
set acmd(3,4)  {  W CHANNEL }
set acmd(3,5)  { R  ANALOG }
set acmd(4,0)  {  W BAUD  }
set acmd(4,1)  {  W FLAGS0 }
set acmd(4,2)  {  W FLAGS1 }
set acmd(4,3)  { RW DATA }
set acmd(4,4)  { RW SSTATUS }
set acmd(4,5)  { RW RSTATUS }
set acmd(8,0)  { R  ALIVE }
set acmd(8,4)  { R  REQUEST }
set acmd(8,8)  { R  ERROR }
set acmd(9,0)  { R  MAGIC }
set acmd(9,4)  { R  PROBE }
set acmd(9,8)  { RW ERROR }
set acmd(9,9)  { RW STATE }
set acmd(9,10) { R  CRC }
set acmd(11,0) {  W FOCUS  }
set acmd(11,1) {  W RESET  }
set acmd(11,2) {  W TAG  }
set acmd(11,3) {  W UNBRAND  }
set acmd(11,4) {  W BRAND  }

set scmd(0,0)  1
set scmd(0,1)  1
set scmd(2,0)  1
set scmd(2,1)  1
set scmd(2,2)  1
set scmd(2,3)  1
set scmd(2,4)  1
set scmd(2,5)  1
set scmd(2,6)  1
set scmd(2,7)  1
set scmd(2,8)  1
set scmd(3,0)  1
set scmd(3,1)  1
set scmd(3,2)  1
set scmd(3,3)  1
set scmd(3,4)  1
set scmd(3,5)  1
set scmd(4,0)  1
set scmd(4,1)  1
set scmd(4,2)  1
set scmd(4,3)  1
```

```
 88 set scmd(4,4)  1
 89 set scmd(4,5)  1
 90 set scmd(8,0)  4
 91 set scmd(8,4)  4
 92 set scmd(8,8)  4
 93 set scmd(9,0)  4
 94 set scmd(9,4)  4
 95 set scmd(9,8)  1
 96 set scmd(9,9)  1
 97 set scmd(9,10) 0
 98 set scmd(11,0) 0
 99 set scmd(11,1) 0
100 set scmd(11,2) 0
101 set scmd(11,3) 0
102 set scmd(11,4) 0
103
104 set ccmd(8,0)  [expr  1<<6]
105 set ccmd(8,4)  [expr  1<<6]
106 set ccmd(8,)   [expr  1<<6]
107 set ccmd(9,0)  0
108 set ccmd(9,4)  0
109 set ccmd(9,8)  C
110 set ccmd(9,9)  C
111 set ccmd(9,10) [expr  1<<7]
112 set ccmd(11,0) FOCUS
113 set ccmd(11,1) 0
114 set ccmd(11,2) 0
115 set ccmd(11,3) 0
116 set ccmd(11,4) ID
117
118
119 set c [toplevel .command ]
120 wm title $c "PPTRACE DIRECT"
121 if [info exists executeEn] {wm withdraw $c }
122 set type [frame $c.decode -borderwidth 2 -relief sunken]
123 label $type.clabel -text "COMMAND:"
124 entry $type.centry -font fixed -width 10
125 bind  $type.centry <Return> commandDo
126 label $type.alabel -text "ADDRESS:"
127 entry $type.aentry -font fixed -width 4
128 bind  $type.aentry <Return> addressDo
129 menubutton $type.rmenubutton  -text "REGISTER:" -menu $type.rmenubutton.menu -relief raised
130 label $type.rsym   -textvariable registerName
131 pack  $type.clabel $type.centry $type.alabel $type.aentry $type.rmenubutton $type.rsym -side left -
    padx 2 -pady 2
132 set m3 [menu $type.rmenubutton.menu -tearoff 0]
133 for {set i 0} {$i < 16} {incr i} {
134    for {set j 0} {$j < 16} {incr j} {
135       if [info exists abank($i)] {
136          if {$j==0} {
137             set m4 [menu $m3.[string tolower $abank($i)] -tearoff 0]
138             $m3 add cascade -label $abank($i) -menu $m4
139          }
140          if [info exists acmd($i,$j)] {
141             set k [expr ($i*16)+$j]
142             set tcmd [lindex $acmd($i,$j) 0]
143             set ncmd [lindex $acmd($i,$j) 1]
144             $m4 add radio -label [format "0x%02x %-20s
    $tcmd" $k $ncmd] -variable command(a) -value $k -font fixed
145          }
146       }
147    }
148 }
149
150 proc commandExit {} {
151 global executeEn c
```

```
   if [info exists executeEn] {
      wm withdraw $c
   } else {
      exit
   }
}
#      destroy $c

set f0 [frame $c.f0]
#scale $f0.port -from 0 -to 1 -variable port -orient vertical -label port -relief sunken
scale $f0.id  -from 0 -to 31 -variable command(id) -orient vertical -label "ID " -relief sunken
set f1 [frame $f0.f1 -borderwidth 2 -relief sunken]
radiobutton $f1.direct   -text "DIRECT\n(byte repeated\n 0-127 times)"  -variable command(dir) -value 1 -width 14 -anchor w -justify left
radiobutton $f1.indirect -text "INDIRECT\n(0-127 bytes)"        -variable command(dir) -value 0 -width 14 -anchor w -justify left
pack $f1.direct $f1.indirect -side top -padx 2 -pady 2
set f2 [frame $f0.f2 ]
set f3 [frame $f2.f3 -borderwidth 2 -relief sunken]
radiobutton $f3.read     -text "READ from\nID (0-31)"  -variable command(rd) -value 1 -width 14 -anchor w -justify left
radiobutton $f3.write    -text "WRITE to\nID (0-31)"   -variable command(rd) -value 0 -width 14 -anchor w -justify left
pack $f3.read $f3.write -side top -anchor w -expand true -fill x -padx 2 -pady 2
checkbutton $f2.inc      -text "++ADDRESS" -variable command(ia)  -width 15 -anchor w -relief sunken
checkbutton $f2.crc      -text CRC          -variable command(crc) -width 15 -anchor w -relief sunken
pack  $f3  -side top
pack  $f2.inc $f2.crc -side bottom
scale $f0.size  -from 0 -to 127 -length 128 -variable command(s) -orient vertical -label "SIZE " -relief sunken
set f4 [frame $f0.f4 -borderwidth 2 -relief sunken ]
label $f4.label -textvariable cinfo -width 14 -justify center
entry $f4.entry -width 4 -font fixed
bind  $f4.entry <Return> cDo
button $f4.clr -text CLEAR -command { set command(c) 0}
scale $f4.id  -from 0 -to 31 -variable command(cid) -orient vertical -label "ID " -relief sunken
set focus [frame $f4.focus -borderwidth 2 -relief sunken]
radiobutton $focus.focus  -text "FOCUS" -variable command(c) -value 1 -width 8 -anchor w -justify left
radiobutton $focus.blur   -text "BLUR"  -variable command(c) -value 2 -width 8 -anchor w -justify left
pack $focus.focus $focus.blur -side top -padx 2 -pady 2
pack  $f0.id $f1 $f0.size $f2 $f4  -side left  -expand true -fill both -padx 2 -pady 2
pack $f4.label  -side top -padx 2 -pady 2
set bb [frame $c.bb]; # button bar
pack [checkbutton $bb.smart -text "SMART" -relief raised -variable smart(en)] -side left -padx 10 -pady 2 -fill y
if [info exists executeEn] {pack [button $bb.execute -text EXECUTE -command commandExecute -relief raised ] -side left -padx 10 -pady 2 }
pack [button $bb.quit -text QUIT -command commandExit -relief raised] -side left -padx 10 -pady 2
set mb [frame $c.mb]; # message bar
label $mb.message -textvariable message -relief sunken -justify left
pack $mb.message -side left -expand 1 -fill x -padx 2 -pady 2
set wdb [frame $c.wdb]; # write data bar
pack [label $wdb.label -text "WRITE DATA:" -justify left]  -side left -padx 2 -pady 2
pack [entry $wdb.entry  -font fixed] -side left -expand 1 -fill x -padx 2 -pady 2

if [info exists executeEn] {
   set fs [frame $c.scroll]
   pack [scrollbar $fs.xscroll -orient horizontal -command [list $fs.text xview]] -side bottom -fill x
   pack [scrollbar $fs.yscroll -orient vertical   -command [list $fs.text yview]] -side right -fill y
   pack [text $fs.text -width 80 -height 10 -setgrid true -wrap none -xscrollcommand [list $fs.xscroll set] -yscrollcommand [list $fs.yscroll set]] -side left -fill both -expand true
   $fs.text tag configure execWriteTag -foreground blue
   $fs.text tag configure execErrorTag -foreground red
   $fs.text tag configure execReadTag  -foreground black
```

```
   proc addExecute { tag message } {
   global fs
      $fs.text insert end $message\n $tag
      $fs.text see end
      if [$fs.text compare end > 50.0 ] {$fs.text delete 1.0 2.0}
   }

  set writeData {}

  proc commandExecute {} {
  global command writeData registerName
     if {$command(dir)==1} {
        writePipe EXECW1:1:0:$command(id):[format "%d" $command(cmd)]:
        addExecute execWriteTag "Direct Write ID:$command(id) add:0x[format "%02x" $command(a)] ($registerName) data:0x[format "%02x" $command(c)] ia:$command(ia) size:$command(s)"
     } elseif { $command(rd)==1 } {
        writePipe EXECR:1:0:$command(id):[format "%d" $command(cmd)]:$command(s):
        addExecute execWriteTag "Indirect read from device $command(id) at address 0x[format "%02x" $command(a)] ($registerName) ia=$command(ia) size=$command(s)"
     } elseif { [readWriteData]==0 } {
        writePipe EXECW:1:0:$command(id):[format "%d" $command(cmd)]:$command(s):$writeData
        addExecute execWriteTag "Indirect write to device $command(id) at address 0x[format "%02x" $command(a)] ($registerName) ia=$command(ia) size=$command(s)"
     }
  }

  proc readWriteData {} {
  global command wdb writeData message
     set writeData {}
     set data [split [$wdb.entry get] \ ,]
     if { [llength $data] < $command(s) } {
       set message "ERROR: $command(s) bytes of data are required but only [llength $data] are avaialbale"
       return 1
     }
     if { [llength $data] > $command(s) } {
       set message "ERROR: $command(s) bytes of data are required but [llength data] are avaialbale, truncating"
     }
     for {set i 0} {$i < $command(s)} {incr i} {
        if [regexp {^(0x[0-9a-fA-F]+)|([0-9]+)$} [lindex $data $i]] {
           append writeData "[format "%d" [lindex $data $i]]:"
        } else {
           set message "ERROR: data\[$i\]=\"[lindex $data $i]\", NOT a valid number"
           return 1;
        }
     }
  return 0
  }
}


pack  $type -side top -fill x
pack  $f0 -side top -fill x
if [info exists executeEn] { pack  $wdb -side top -fill x }
pack  $mb -side top -fill x
pack  $bb -side top
if [info exists executeEn] { pack  $fs -side top -expand 1 -fill both }

proc commandDo {} {
   global command type message
   set cmd [$type.centry get]
   set message ""
   if [regexp {^(0x[0-9a-fA-F]+)|([0-9]+)$} $cmd ] {
      set command(cmd)  $cmd
      set command(c)    [expr ($cmd >> 1 ) & 0xff]
      set command(s)    [expr ($cmd >> 9 ) & 0x7f]
      set command(a)    [expr ($cmd >> 17) & 0xff]
```

```
      set command(id)   [expr ($cmd >> 27) & 0x1f]
      set command(type) [expr ($cmd >> 25) & 0x03]
      set command(ia)   [expr ($cmd >> 16) & 0x01]
      set command(dir)  [expr ($command(type) >> 1) & 0x1]
      set command(crc)  [expr ($cmd >> 8 ) & 0x01 ]
      set command(rd)   [expr $command(type) & 0x1]
      if { $command(cmd) != $cmd } {
         if { [expr $command(cmd) ^ $cmd ] == 1 } {
            set message "ERROR: Invalid COMMAND parity"
         } else {
            set message [format "ERROR: Invalid COMMAND entered was:0x%08x now:0x%08x" $cmd $command(cmd)]
         }
      }
   } else {
      set message "ERROR: COMMAND must be either a hex '0xn' or a decimal 'n' number"
   }
}

proc commandUpdate {name1 name2 op} {
   global command type message
   set message ""
   set command(cmd) [expr ($command(cmd) & ~(0xff<<1 )) | ($command(c)  <<1 )]
   set command(cmd) [expr ($command(cmd) & ~(0x7f<<9 )) | ($command(s)  <<9 )]
   set command(cmd) [expr ($command(cmd) & ~(0x01<<16)) | ($command(ia) <<16)]
   set command(cmd) [expr ($command(cmd) & ~(0xff<<17)) | ($command(a)  <<17)]
   set command(cmd) [expr ($command(cmd) & ~(0x01<<25)) | ($command(rd) <<25)]
   set command(cmd) [expr ($command(cmd) & ~(0x01<<26)) | ($command(dir)<<26)]
   set command(cmd) [expr ($command(cmd) & ~(0x1f<<27)) | ($command(id) <<27)]
   if { [expr ($command(cmd)>>26)&0x1] == 0 } { set com-
mand(cmd) [expr ($command(cmd) & ~(0x1<<8)) | ($command(crc)<<8)] }
   set command(cmd) [expr ($command(cmd) & ~1) ] ; # erase parity
   set parity [expr ($command(cmd) ^ ($command(cmd)>>16)) & 0xffff]
   set parity [expr ($parity ^ ($parity>>8)) & 0xff]
   set parity [expr ($parity ^ ($parity>>4)) & 0x0f]
   set parity [expr ($parity ^ ($parity>>2)) & 0x03]
   set parity [expr ($parity ^ ($parity>>1)) & 0x01]
   set command(cmd) [expr $command(cmd) | $parity]
   $type.centry delete 0 end
   $type.centry insert 0 [format "0x%08x" $command(cmd)]
}

proc addressDo {} {
   global command type message
   set cmd [$type.aentry get]
   set message ""
   if { $cmd == "" } {
      aUpdate 0 0 0
   } elseif { [regexp {^(0x[0-9a-fA-F]+)|([0-9]+)$} $cmd ] && $cmd>=0 &&  $cmd<=0xff } {
      set command(a) $cmd
   } else {
      set message "ERROR: ADDRESS must be either a hex '0x00-0xff' or a decimal '0-255' number"
   }
}
proc aUpdate {name1 name2 op} {
   global command registerName type acmd smart ccmd scmd
   set message ""
   set i [expr ($command(a)>>4)&0xf]
   set j [expr $command(a)&0xf ]
   set smart(c) C
   set smart(s) -1
   set smart(rw) RW
   set registerName UNKNOWN

   if [info exists acmd($i,$j)] {
      set registerName [lindex $acmd($i,$j) 1]
      if [info exists ccmd($i,$j)] {set smart(c) $ccmd($i,$j)}
```

```
      if [info exists scmd($i,$j)] {set smart(s) $scmd($i,$j)}
      set smart(rw) [lindex $acmd($i,$j) 0]
   }
   $type.aentry delete 0 end
   $type.aentry insert 0 [format "0x%02x" $command(a)]
   commandUpdate 0 0 0
}

proc cDo {} {
   global command type message f4
   set cmd [$f4.entry get]
   set message ""
   if { $command(dir) == 1 } { set max 0xff } { set max 0x7f }
   if { $cmd == "" } {
      cUpdate 0 0 0
   } elseif { [regexp {^(0x[0-9a-fA-F]+)|([0-9]+)$} $cmd ] && $cmd>=0 &&  $cmd<=$max } {
      set command(c) $cmd
   } else {
      if { $command(dir) == 1 } {
          set message "ERROR: C-DATA must be either a hex '0x00-0xff' or a decimal '0-255' number"
      } else {
          set message "ERROR: C must be either a hex '0x00-0x7f' or a decimal '0-127' number"
      }
   }
}

proc cUpdate {name1 name2 op} {
   global command f4
   set message ""
   $f4.entry delete 0 end
   if { $command(dir) == 1 } {
      $f4.entry insert 0 [format "0x%02x" $command(c)]
   } else {
      $f4.entry insert 0 [format "0x%02x" [expr $command(c) & 0x7f] ]
   }
   commandUpdate 0 0 0
}

proc cidUpdate {name1 name2 op} {
   global command
   set message ""
   set command(c) $command(cid)
}

proc dirUpdate {name1 name2 op} {
   smartUpdate 0 0 0
   cUpdate 0 0 0
}

proc smartUpdate {name1 name2 op} {
   global smart command f0 f1 f2 f3 f4 cinfo
   $f3.write    configure -state normal
   $f3.read     configure -state normal
   $f2.crc      configure -state normal
   $f1.indirect configure -state normal
   $f1.direct   configure -state normal
   $f0.size     configure -state normal
   $f2.inc      configure -state normal
   set c C

   if { $smart(en) == 1 } {
      if { [string compare $smart(rw) R] == 0 } {
         $f3.read   invoke
         $f1.indirect invoke
         $f1.direct   configure -state disabled
         $f3.write    configure -state disabled
```

```
      }
      if { [string compare $smart(rw) W ] == 0 } {
         $f3.write invoke
         $f3.read  configure -state disabled
      }
      if { $smart(s) != -1 } {
         set command(s) $smart(s)
         $f0.size configure -state disabled
         set command(ia) 1
         $f2.inc configure -state disabled
         if { $smart(s) == 0 } {
            set command(crc) 0
            $f2.crc configure -state disabled
            $f1.indirect invoke
            $f1.direct   configure -state disabled
         }
      }
      set c $smart(c)
   }
   if { $command(dir) == 1} {
      $f3.write invoke
      $f3.read  configure -state disabled
      $f2.crc   configure -state disabled
      if { [string compare $c C] == 0} { set c DATA }
   }
   pack forget $f4.entry
   pack forget $f4.id
   pack forget $f4.focus
   pack forget $f4.clr
   switch $c {
      DATA    { set cinfo "C-DATA\n(0x00-0xFF)\n(0-255)";  pack $f4.entry $f4.clr -side top -padx 2 -pady 2}
      C       { set cinfo "C\n(0x00-0x7f)\n(0-127)";       pack $f4.entry $f4.clr -side top -padx 2 -pady 2}
      ID      { set cinfo "C-ID";                 pack $f4.id    -side top -padx 2 -pady 2}
      FOCUS   { set cinfo "C-FOCUS";               pack $f4.focus -side top -padx 2 -pady 2}
      default { set cinfo [format "C=0x%02x" $c]                 }
   }
}

$type.centry delete 0 end
$type.centry insert 0 0x00000000
commandDo
trace variable command(c)   w cUpdate
trace variable command(s)   w commandUpdate
trace variable command(ia)  w commandUpdate
trace variable command(a)   w aUpdate
trace variable command(rd)  w commandUpdate
trace variable command(dir) w dirUpdate
trace variable command(id)  w commandUpdate
trace variable command(crc) w commandUpdate
trace variable smart(en)    w smartUpdate
trace variable smart(s)     w smartUpdate
trace variable smart(c)     w smartUpdate
trace variable smart(rw)    w smartUpdate
trace variable command(cid) w cidUpdate
commandDo
```

12.30 SERIAL GUI (SERIAL_TCL)

```
set clock(frequency) [expr (28.63636e6/4)]
set clock(divider)   6
set clock(period)    [expr (1/$clock(frequency))*($clock(divider)+1)*2]

set baud(STOP) 0
set baud(MIN)  [expr 1/($clock(period) * 510)]
set baud(1200) 1200
set baud(2400) 2400
set baud(4800) 4800
set baud(9600) 9600
set baud(MAX)  [expr 1/($clock(period) * 2)]

set j 0xff
for {set i 0 } { $i < 255} { incr i } {
   set j [expr (($j<<1)|( (($j>>7)&1) ^ (($j>>5)&1) ^ (($j>>4)&1) ^ (($j>>3)&1))) & 0xff ]
   set lfsr([expr (255-$i)*2]) $j
   set rlfsr($j) [expr (255-$i)*2]
#   puts "lfsr($i) /[expr (255-$i)*2] = 0x[format "%02x" $j]";
}
set lfsr(0) 0
set rlfsr(0) 0

set baudMask 0
#port:id:serialbaud
proc baudUpdate {name1 name2 op} {
   global $name1 lfsr clock baudMask
   if {[string compare $op "u"]==0} { trace vdelete ${name1}(${name2}) uw baudUpdate; return}
   set div [set ${name1}(divider)]
   set ${name1}(tdivider) "DIVIDER: $div"
   if { $div==0 } {set baud 0} else {set baud [expr 1/($clock(period)*$div)]}
   set ${name1}(tlfsr) "LFSR: 0x[format "%02x" $lfsr($div)]"
   set ${name1}(tbaud) "BAUD: [format "%9.2f" $baud]"
   set data [split $name1 :]
   if { $baudMask > 0 } {incr baudMask -1} {write1Reg [lindex $data 0] [lindex $data 1] 0x40 $lfsr($div)}
}

proc baudUpdateReg {name1 name2 op} {
   set data [split $name1 :]
   set name "[lindex $data 0]:[lindex $data 1]:serialbaud"
   global $name baudMask $name1 rlfsr
   if {[string compare $op "u"]==0} { trace vdelete ${name1}(${name2}) uw baudUpdateReg; return}

#   puts "name=$name ${name1}(${name2})=[set ${name1}(${name2})] $rlfsr([format "%d" [set ${name1}(${name2})]])"
   incr baudMask
   set ${name}(divider) $rlfsr([format "%d" [set ${name1}(${name2})]])
}

set flags0names { LSB_FIRST SYNC_BITS PARITY_BIT MASKED_CLK CLK_HIGH LEVEL_HIGH CS_NEG ASYNC }
set flags1names { RECV SEND STICK_CS RUN UNUSED IO_FLIP DIN_FLIP MASK_CTS }

set ser-
Mode { PWM:510:0x80:0x8e RS232:0:0x00:0x00 SPI_MODE0:0:0x00:0x00 SPI_MODE1:0:0x00:0x00 SPI_MODE2:0:0x00:0x00 SPI_MODE

proc flagsBitUpdate {name1 name2 op} {
   set data [split $name1 :]; # port:id:
   set var [string range [lindex $data 2] 6 end]
   global ${var}names $name1 flagsMask
   if {[string compare $op "u"]==0} { trace vdelete ${name1}(${name2}) uw flagsBitUpdate; return}
#   if {[string compare $name2 0]} return; # sometimes name2 is invalid, why? ???
#   puts "updating ${name1}(${name2})"; #=[set ${name1}(${name2})]"
   set reg 0
   set i 0
   foreach index  [set ${var}names] {
```

```
      if { [set ${name1}($index)]==1} {set reg [expr $reg | (1<<$i) ] }
      incr i
   }

if { [string compare flags0 $var]==0 } { set add 0x41; set flag flags0 } {set add 0x42; set flag flags1}
#   puts "writing $reg to $add"
   if { [set flagsMask(${flag})]>0 } {incr flagsMask(${flag}) -1 } else {write1Reg [lindex $data 0] [lindex $data 1] $add $reg}
   set ${name1}(treg) "[string toupper $var]:0x[format "%02x" $reg]"
}

#port:id:flags(0|1)
set flagsMask(flags0) 0
set flagsMask(flags1) 0
proc flagsUpdate {name reg} {
   global flagsMask
   set data [split $name :]; # port:id:
   set var [string range [lindex $data 2] 6 end]
   global ${var}names $name
   set i 0
   foreach index  [set ${var}names] {
      if { [expr ($reg &  (1<<$i))] } {
         if ![set ${name}($index)] { incr flagsMask(${var}); set ${name}($index) 1 }
      } else {
         if  [set ${name}($index)] { incr flagsMask(${var}); set ${name}($index) 0 }
      }
#      puts "[set flagsMask(${var})] $index"
      incr i
   }
}

proc flagsUpdateReg { name1 name2 op } {
# port:id:reg:(0x41,w)
   global $name1 flagsMask
   if {[string compare $op "u"]==0} { trace vdelete ${name1}(${name2}) uw flagsUpdateReg; return}
#   puts "in flagsUpdateReg ${name1}(${name2}) op=$op trace=[trace vinfo ${name1}(${name2})]"
   set data [split $name1 :]
   if { [lindex [split $name2 ,] 0] == 0x41 } { set flag flags0 } { set flag flags1 }
   set name "[lindex $data 0]:[lindex $data 1]:serial${flag}"
#   if { [set flagsMask(${flag})]>0 } {incr flagsMask(${flag}) -1 } else { }
   flagsUpdate $name [set ${name1}(${name2})]
}

#puts "$name1=0x[format "%02x" [set ${name1}(${name2})]];
proc sstatusUpdate {name1 name2 op} {
   global $name1
   if {[string compare $op "u"]==0} { trace vdelete ${name1}(${name2}) uw sstatusUpdate; return}
   set data [split $name1 : ]
   set name "[lindex $data 0]:[lindex $data 1]:serialstatus"
   global $name
   set num [format "%d"  [set ${name1}(${name2})]]
   set ${name}(sreg)    "SSTATUS:0x[format "%02x" $num]"
   set ${name}(svalid) "AVAIL = [expr $num&0x1f]"
   set ${name}(serrof) "ERR OF = [expr ($num>>7)&0x1]"
}

proc rstatusUpdate {name1 name2 op} {
   global $name1
   if {[string compare $op "u"]==0} { trace vdelete ${name1}(${name2}) uw rstatusUpdate; return}
   set data [split $name1 : ]
   set name "[lindex $data 0]:[lindex $data 1]:serialstatus"
   global $name
   set num [format "%d"  [set ${name1}(${name2})]]
   set ${name}(rreg)    "RSTATUS:0x[format "%02x" $num]"
   set ${name}(rvalid) "VALID = [expr $num&0x1f]"
   set ${name}(rerrof) "ERR OF = [expr ($num>>5)&0x1]"
```

```
126    set ${name}(rerruf) "ERR UF = [expr ($num>>6)&0x1]"
127    set ${name}(rerrp)  "ERR PARITY = [expr ($num>>7)&0x1]"
128 }
129
130 # add menu index
131 proc getSerial { port id } {
132    set name $port:$id:serial
133    global baud clock ${name}baud ${name}flags0 ${name}flags1 flags0names flags1names ser-
    Mode ${name}:menu ${name}:menuindex rlfsr
134    set window .${name}
135    set m .menu.device.m.$port:$id
136    if { ![winfo exists $window] } {
137       global $port:$id:reg
138       toplevel $window
139       wm withdraw $window
140       set wt [frame $window.top  -relief sunken -bd 2]
141       $m add check -label SERIAL -variable "${name}:menu" -command "enable_window $name"
142       set ${name}:menu 0
143       wm title $window "PPTRACE $port $id SERIAL"
144       frame $wt.baud -relief sunken
145       label $wt.baud.period -text "CLK PERIOD: [format "%.4f" [expr ($clock(period) * 1e6)] ]uS" -relief sunken -anchor w
146       pack $wt.baud.period -side top -fill x -padx 2 -pady 2
147       label $wt.baud.divider -textvariable ${name}baud(tdivider) -relief sunken -anchor w
148       pack $wt.baud.divider -side top -fill x -padx 2 -pady 2
149       label $wt.baud.baud -textvariable ${name}baud(tbaud) -relief sunken -anchor w
150       pack $wt.baud.baud -side top -fill x -padx 2 -pady 2
151       label $wt.baud.lfsr -textvariable ${name}baud(tlfsr) -relief sunken -anchor w
152       pack $wt.baud.lfsr -side top -fill x -padx 2 -pady 2
153       foreach index { STOP MAX 9600 4800 2400 1200 MIN} {
154          set bw $wt.baud.w[string tolower $index]
155          if { $baud($index) == 0 } {
156             set div 0
157             set rate 0
158           } else {
159             set div [expr int((1/($clock(period)*$baud($index)))+1)&0x1fe]; # an even divider 0-510
160             set rate [expr 1/($clock(period)*$div)]
161           }
162           set bl "[format "%5s  /%03d %9.2f" $index $div $rate]"
163           radiobutton $bw -text $bl -variable ${name}baud(divider) -value $div -font fixed
164           pack $bw -side top -fill x -padx 2 -pady 2
165       }
166       pack $wt.baud -side left -padx 2 -pady 2
167       set var $port:$id:reg([format "%d" 0x40],W)
168 #      puts "var=$var"
169       if [info exists [set var]] {set reg [set reg [set $var]] } {set reg 0x00}
170 #      puts "var=$var rlfsr($reg)=$rlfsr([format "%d" $reg])"
171 #      puts "reg=$reg"
172       set ${name}baud(divider $rlfsr([format "%d" $reg])
173       trace variable $var uw baudUpdateReg
174 #      set ${name}baud(divider) 0
175
    scale $wt.baudscale  -from 0 -to 510 -resolution 2 -variable ${name}baud(divider) -orient vertical -label
    "DIVIDER" -relief sunken
176       pack $wt.baudscale -side left -padx 2 -pady 2 -fill y
177       frame $wt.flags0
178       label $wt.flags0.label -textvariable ${name}flags0(treg) -relief sunken -width 18
179       pack $wt.flags0.label -side top -padx 2 -pady 2 -fill x
180       set i 0
181       foreach index  $flags0names {
182           set bw $wt.flags0.[string tolower $index]
183           checkbutton $bw -text "$i $index" -variable ${name}flags0($index) -anchor w
184           pack $bw -side top -fill x -padx 2 -pady 2
185           trace variable ${name}flags0($index) wu flagsBitUpdate
186           incr i
187       }
188 #      flagsBitUpdate ${name}flags0 0 0
```

```
189        pack $wt.flags0 -side left -padx 2 -pady 2 -fill y
190
191        frame $wt.flags1
192        label $wt.flags1.label -textvariable ${name}flags1(treg) -relief sunken  -width 18
193        pack $wt.flags1.label -side top -padx 2 -pady 2 -fill x
194        set i 0
195        foreach index  $flags1names {
196            set bw $wt.flags1.[string tolower $index]
197            checkbutton $bw -text "$i $index" -variable ${name}flags1($index) -anchor w
198            pack $bw -side top -fill x -padx 2 -pady 2
199            trace variable ${name}flags1($index) wu flagsBitUpdate
200            incr i
201        }
202 #       flagsBitUpdate ${name}flags1 0 0
203        pack $wt.flags1 -side left -padx 2 -pady 2 -fill y
204
205 #       baudUpdate "$port:$id:serialbaud" 0 0
206
207        set wss [frame $wt.sstatus]
208        label  $wss.label  -textvariable ${name}status(sreg) -relief sunken -width 18
209        button $wss.read   -text "READ" -command "read1Reg $port $id [format "%d" 0x44] "
210        button $wss.clr    -text "ERROR CLEAR" -command "write1Reg $port $id [format "%d" 0x44] [format "%d" 0x80]; $wss.read
    invoke"
211        label  $wss.valid  -textvariable ${name}status(svalid)  -relief sunken -anchor w
212        label  $wss.errof  -textvariable ${name}status(serrof)  -relief sunken -anchor w
213        pack $wss.label $wss.read $wss.clr $wss.valid $wss.errof -side top -fill x -padx 2 -pady 2
214        pack $wss -side left -padx 2 -pady 2 -fill y
215
216        set var $port:$id:reg([format "%d" 0x44],R)
217        global $var
218        trace variable $var uw sstatusUpdate
219        $wss.read invoke
220
221        set wsr [frame $wt.rstatus]
222        label  $wsr.label  -textvariable ${name}status(rreg) -relief sunken -width 18
223        button $wsr.read   -text "READ" -command " read1Reg $port $id [format "%d" 0x45] "
224        button $wsr.clr    -text "ERROR CLEAR" -command "write1Reg $port $id [format "%d" 0x45] [format "%d" 0xe0]; $wsr.read
    invoke"
225        label  $wsr.valid  -textvariable ${name}status(rvalid) -relief sunken -anchor w
226        label  $wsr.errof  -textvariable ${name}status(rerrof) -relief sunken -anchor w
227        label  $wsr.erruf  -textvariable ${name}status(rerruf) -relief sunken -anchor w
228        label  $wsr.errp   -textvariable ${name}status(rerrp)  -relief sunken -anchor w
229        pack $wsr.label $wsr.read $wsr.clr $wsr.valid $wsr.errof $wsr.erruf $wsr.errp -side top -fill x -
    padx 2 -pady 2
230        pack $wsr -side left -padx 2 -pady 2 -fill y
231
232        set var $port:$id:reg([format "%d" 0x45],R)
233        global $var
234        trace variable $var uw rstatusUpdate
235        $wsr.read invoke
236
237        frame $wt.mode
238        label $wt.mode.label -text MODES -relief sunken -width 18
239        pack $wt.mode.label -side top -padx 2 -pady 2 -fill x
240        foreach index  $serMode {
241            set mode [split $index :]
242            set bw $wt.mode.[string tolower [lindex $mode 0]]
243            button $bw -text [lindex $mode 0] -command "flagsUpdate ${name}flags0 [lindex $mode 2]; flagsUpdate ${name}flags1
    [lindex $mode 3]; set ${name}baud(divider) [lindex $mode 1] "
244            pack $bw -side top -fill x -padx 2 -pady 2
245        }
246        button $wt.mode.quit -text QUIT -command "set ${name}:menu 0; enable_window $name"
247        pack $wt.mode.quit -fill x -padx 2 -pady 2
248        pack $wt.mode -side left -padx 2 -pady 2 -fill y
249
250        set var $port:$id:reg([format "%d" 0x41],W)
```

```
251       trace variable $var uw flagsUpdateReg
252       if [info exists [set var]] { set reg [set reg [set $var]] } {set reg 0x00}
253 #      puts "flags0=$reg"
254       flagsUpdate ${name}flags0 $reg
255
256       set var $port:$id:reg([format "%d" 0x42],W)
257       trace variable $var uw flagsUpdateReg
258       if [info exists [set var]] { set reg [set reg [set $var]] } {set reg 0x00}
259 #      puts "flags1=$reg"
260       flagsUpdate ${name}flags1 $reg
261
262       pack $wt -side top -padx 2 -pady 2
263
264       set sd [frame $window.sd -relief sunken -bd 2]
265       button $sd.label -text "SEND DATA:" -command "sendData $port $id"
266       pack $sd.label -side left -padx 2 -pady 2
267       entry $sd.entry -font fixed
268       bind  $sd.entry <Return> "sendData $port $id"
269       pack $sd.entry -side left -padx 2 -pady 2 -fill x -expand true
270       pack $sd -side top -padx 2 -pady 2 -fill x -expand true
271
272       set rd [frame $window.rd -relief sunken -bd 2]
273       label $rd.label -text "RECV DATA:"
274       pack $rd.label -side top -padx 2 -pady 2
275       frame $rd.scroll
276       text      $rd.scroll.text -width 80 -height 10 -setgrid true -
    xscrollcommand [list $rd.scroll.xscroll set] -yscrollcommand [list $rd.scroll.yscroll set] -font fixed
277       scrollbar $rd.scroll.xscroll -orient horizontal -command [list $rd.scroll.text xview]
278       scrollbar $rd.scroll.yscroll -orient vertical   -command [list $rd.scroll.text yview]
279       pack      $rd.scroll.xscroll -side bottom -fill x
280       pack      $rd.scroll.yscroll -side right -fill y
281       pack      $rd.scroll.text -side left -fill both -expand true
282       pack      $rd.scroll -side top  -fill both -expand true
283       pack $rd -side top -padx 2 -pady 2 -fill both -expand true
284
285       trace variable ${name}baud(divider) wu baudUpdate
286    }
287 }
288
289 proc sendData { port id } {
290    set entry .$port:$id:serial.sd.entry
291    set data [string trimright [$entry get] \n]
292    $entry delete 0 end
293    writePipe "SERW:$port:$id:[string length $data]:$data"
294 }
295
296 proc recvData { port id size data } {
297 #   puts "port=$port id=$id size=$size data=$data"
298    set bytes [split $data ,]
299    set tdata {}
300    for { set i 0 } { $i < $size } {incr i} { append tdata [format "%c" [lindex $bytes $i]] }
301    .$port:$id:serial.rd.scroll.text insert end $tdata
302    .$port:$id:serial.rd.scroll.text see end
303 #   puts "port=$port id=$id size=$size data=$tdata"
304 }
305
306
307 proc ungetSerial { port id } {
308    set name $port:$id:serial
309    if { [winfo exists .$name ] } {
310       global $name ${name}flags0 ${name}flags1 ${name}baud $port:$id:reg
311       wm withdraw .$name
312       destroy .$name
313       .menu.device.m.$port:$id delete SERIAL
314       trace vdelete $port:$id:reg([format "%d" 0x40],W) uw baudUpdateReg
315       trace vdelete $port:$id:reg([format "%d" 0x41],W) uw flagsUpdateReg
```

```
316        trace vdelete $port:$id:reg([format "%d" 0x42],W) uw flagsUpdateReg
317        trace vdelete $port:$id:reg([format "%d" 0x44],R) uw sstatusUpdate
318        trace vdelete $port:$id:reg([format "%d" 0x45],R) uw rstatusUpdate
319        unset $port:$id:serialflags0
320        unset $port:$id:serialflags1
321        unset $port:$id:serialbaud
322     }
323 }
324
325
326 #      if [regexp {[0-9A-Za-z]} $tdata_char] {
327 #      } else {
328 #          append tdata [format "\\x%02x" [lindex $bytes $i]]
329 #      }
330 #   }
331
332
333
334
335
336
337
338
339
340
341
342
```

Chapter 13

CONFIGURATION

The test software reads this register if the MODE is 0x7. The following configurations are currently defined. Keep in mind that the MODE and the CONFIG register are only passed thru the chip. They can be defined to accommodate the software.

FUNCTION	bits
SPLITTER	0000_0000
RUMBLE	0000_0001
THRU	0000_0010
UCONT	0000_0011
	0000_0100
	0000_0101
	0000_0110
	0000_0111
SERIAL_MEM	0001_xxxx
PARALLEL_MEM	0010_xxxx
LATCHED_MEM	0011_xxxx
TEST	1010_1010

Polyface is configured by the driver. To determine how to configure Polyface, the driver reads the mode resistor.

Chapter 14

Nintendo 64 info

The information on the Nintendo 64 joystick interface was gathered by dissecting and measuring an off the shelf unit, serial #NS135525676, and its accompanying joystick.

The Nintendo 64 joystick interface has a three conductor cable (GND, POWER, and SIGNAL), no shielding, and a custom connector.

The exotic design of the connector ensures that nothing can be inadvertently connected to either the jack or the plug. It also provides ESD protection by mechanically preventing direct contact with any conductors. The large connector shell houses a ferrite for EMI suppression.

The clock and data are encoded on the SIGNAL conductor. Figure 14.1 shows the falling edges at 4uS intervals, the rising edges are either 1uS or 3uS, if the are a 1 or a 0 respectively, from the falling edge. The maximum transfer rate is thus 250KHz. The signals levels are 3.3V CMOS.

The data is sent in bursts of 42 bits, the first 8 are sent from the chassis the remaining 36 are sent from the joystick at 30uS intervals, see Figure 14.2 . There are no error detection bits. Each of the four joystick connectors has an independent SIGNAL pin and the bursts between them are offset by 30uS/4. There are additional bursts of data, not shown in Figure 14.2, when reading or writing to the memory cards on the joystick. This timing was gathered while running Super 64 Mario.

The input protection circuit for the SIGNAL pin on the Nintendo 64 joystick is shown in Figure 14.3 . I haven't been able to trace what D1 is, it is probably either a diode or a TVS. I also have no idea what the value for C1 is, but it could be calculated. I don't know why it has what seems to be a fuse in the input path, the same device is also in series with POWER and GND. As a side note, the after market joystick from High Frequency, it has no model or serial numbers anywhere, doesn't seem to have any input protection at all. The SIGNAL input goes right into an ASIC. They may have special ESD inputs on the ASIC but I doubt it. I measured 3.45 Volts between the POWER and GND pins at the chassis.

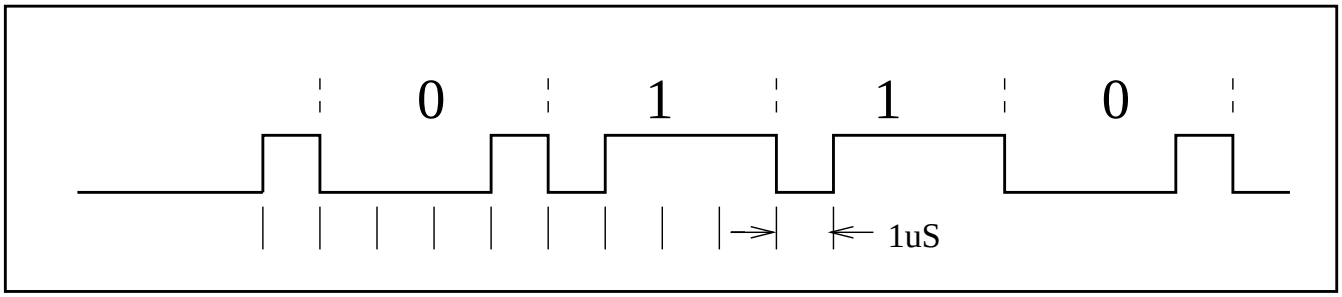


Figure 14.1: Nintendo SIGNAL encoding

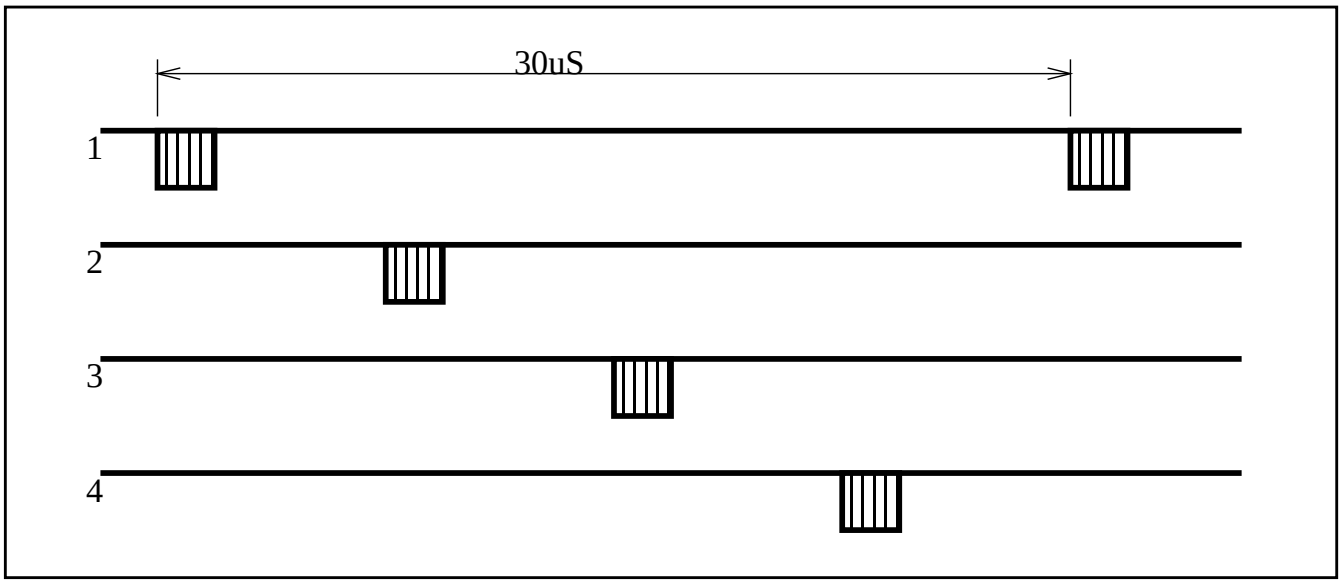


Figure 14.2: Nintendo burst timing

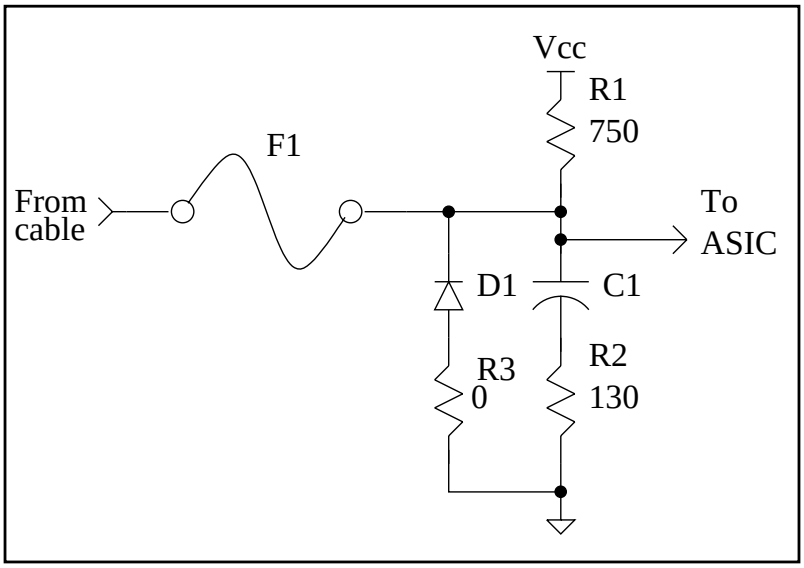


Figure 14.3: Nintendo input protection circuit